

سلام الغفران



دانشکده‌گان علوم
دانشکده ریاضی، آمار و علوم کامپیوتر

صوری سازی ترجمه حذف وابستگی در سیستم‌های نوع محض لایه‌ای

نگارش
حسین مددی

استاد راهنما
مجید علی زاده

پایان نامه برای دریافت درجه کارشناسی ارشد
در رشته علوم کامپیوتر

تابستان ۱۴۰۵

تعهدنامه اصالت اثر

اینجانب حسین مددی متعهد می شوم که مطالب مندرج در این پایان نامه/رساله حاصل کار پژوهشی اینجانب است و به دستاوردهای پژوهشی دیگران که در این پژوهش از آنها استفاده شده است، مطابق مقررات ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان نامه/رساله قبلاً برای احراز هیچ مدرک هم سطح یا بالاتر ارائه نشده است. در صورت اثبات تخلف (در هر زمان) مدرک تحصیلی صادر شده توسط دانشگاه از اعتبار ساقط خواهد شد. کلیه حقوق مادی و معنوی این اثر متعلق به دانشکده ریاضی، آمار و علوم کامپیوتر دانشکدگان علوم دانشگاه تهران می باشد.

حسین مددی



حق مالکیت معنوی

حق مالکیت معنوی این اثر متعلق به دانشگاه تهران می باشد. استفاده از مطالب این پایان نامه در فعالیتهای تحقیقاتی با ذکر منبع بلامانع می باشد. در صورت استفاده تجاری، مانند چاپ این پایان نامه، هماهنگی لازم و اجازه کتبی از دانشگاه و نگارنده پایان نامه الزامی می باشد.

خروجی‌های پایان‌نامه

چکیده

در این پایان نامه، ترجمه حذف وابستگی معرفی شده توسط Nathan Mull برای سیستم‌های نوع محض لایه‌ای [۱۰] به طور کامل در اثبات‌یار Coq صوری‌سازی و اثبات ماشینی شد. توابع ترجمه به همراه تمام لم‌های واسطه شامل لم جابه‌جایی جانشینی، لم حفظ مسیرهای کاهش و لم حفظ نوع، به صورت ماشینی اثبات گردیدند. قضیه اصلی Mull مبنی بر اینکه در تمام سیستم‌های نوع محض لایه‌ای با مجموعه قوانینی خاص، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی است، برای اولین بار به صورت کاملاً ماشینی بررسی شد. کتابخانه‌ای باز و قابل گسترش تولید شد که بستری مناسب برای تعمیم این ترجمه به سیستم‌های وابسته‌تر مانند حساب ساختمان‌ها فراهم می‌آورد. نتایج این پژوهش گامی مهم در جهت پیشرفت اثبات حدس BGK^۱ [۷] در سیستم‌های وابسته به شمار می‌رود.

کلمات کلیدی: نظریه انواع، سیستم‌های نوع محض لایه‌ای، ترجمه حذف وابستگی، نرمال‌سازی قوی، اثبات ماشینی، Coq، حدس BGK

پیشگفتار

”نظریه انواع” یکی از مهم‌ترین شاخه‌های علوم کامپیوتر نظری و منطق ریاضی در قرن بیستم به شمار می‌رود. این نظریه نه تنها پایه و اساس زبان‌های برنامه‌نویسی تابعی پیشرفته مانند OCaml، Haskell و Agda را تشکیل می‌دهد، بلکه هسته اصلی اثبات‌یارهای مطرحی نظیر Coq و Lean و نیز بر مبنای آن طراحی شده است.

یکی از مسائل کلاسیک و همچنان باز در این حوزه، رابطه میان نرمال‌سازی ضعیف و نرمال‌سازی قوی در سیستم‌های نوع وابسته است. حدس معروف BGK که بیش از سه دهه پیش مطرح شد، بیان می‌کند که در هر سیستم نوع محض، اگر نرمال‌سازی ضعیف برقرار باشد، آنگاه نرمال‌سازی قوی نیز برقرار خواهد بود [۷].

این حدس برای سیستم‌های غیروابسته از دهه ۱۹۹۰ اثبات شده است [۸]، اما در سیستم‌های وابسته مانند حساب ساختمان‌ها^۱ همچنان یک مسئله باز باقی مانده است.

Nathan Mull در سال ۲۰۲۳، در رساله دکتری خود در دانشگاه شیکاگو [۱۰]، خانواده جدیدی از سیستم‌های نوع به نام «سیستم‌های نوع محض لایه‌ای^۲» را معرفی کرد و با ارائه ترجمه‌ای هوشمندانه که وابستگی‌های غیرضروری را حذف می‌کند، توانست نشان دهد که در این سیستم‌ها، حدس BGK برقرار است. این نتیجه یکی از بزرگ‌ترین پیشرفت‌های اخیر در مسیر اثبات این حدس به شمار می‌رود.

این پایان‌نامه به صورتی‌سازی کامل اثبات Mull در اثبات‌یار Coq اختصاص دارد. هدف، تبدیل این اثبات نظری قابل توجه، به یک اثبات ماشینی قابل اتکا و استفاده مجدد است که می‌توان از آن برای تعمیم به سیستم‌های پیچیده‌تر نیز استفاده کرد.

محتوا پایان‌نامه در پنج فصل تنظیم شده است. فصل اول مفاهیم و تعاریف اولیه را بیان می‌کند. فصل دوم سیستم‌های نوع محض لایه‌ای و ترجمه حذف وابستگی در آنان را معرفی می‌کند. فصل سوم به طراحی و پیاده‌سازی سیستم‌ها و توابع ترجمه در Coq می‌پردازد. فصل چهارم اثبات‌های صورتی لم‌ها و قضیه اصلی را در Coq ارائه می‌دهد و فصل پنجم به جمع‌بندی و پیشنهادهایی برای پژوهش‌های آینده اختصاص دارد.

^۱ هسته اثبات‌یار Coq

^۲ Tiered Pure Type Systems

امید است این کار بتواند گامی کوچک در جهت پیشرفت یکی از مسائل کلاسیک نظریه انواع بردارد.

فهرست مطالب

یک	چکیده
دو	پیشگفتار
پنج	فهرست مطالب
شش	فهرست شکل‌ها
هفت	فهرست جداول

اول مفاهیم و تعاریف مقدماتی

۱	۱ مفاهیم مقدماتی
۱	۱.۱ مقدمه
۲	۲.۱ حساب لامبدا بدون نوع
۲	۱.۲.۱ نحو
۲	۲.۲.۱ متغیرهای آزاد و بسته
۳	۳.۲.۱ جانشینی
۴	۴.۲.۱ تبدیل α
۴	۵.۲.۱ کاهش β
۵	۶.۲.۱ مسیرهای کاهش و فرم نرمال
۶	۷.۲.۱ نرمال‌سازی ضعیف و قوی
۷	۳.۱ سیستم‌های کاهش انتزاعی
۷	۱.۳.۱ تعریف سیستم کاهش انتزاعی
۷	۲.۳.۱ دنباله و بستار کاهش
۷	۳.۳.۱ فرم نرمال و نرمال‌سازی

۸	 همگرایی	۴.۳.۱
۹	 حساب لامبدا نوع دار ساده	۴.۱
۹	 نحو	۱.۴.۱
۱۰	 زمینه و قضاوت نوع	۲.۴.۱
۱۱	 خواص	۳.۴.۱
۱۲	 سیستم‌های نوع محض	۵.۱
۱۲	 مشخصه و نحو	۱.۵.۱
۱۳	 قضاوت نوع	۲.۵.۱
۱۴	 مکعب لامبدا	۳.۵.۱
۱۶	 اهمیت	۴.۵.۱
۱۶	 خواص فرانظری سیستم‌های نوع محض	۵.۵.۱

دوم سیستم‌های نوع محض لایه‌ای و ترجمه حذف وابستگی ۱۹

۲۰	سیستم‌های نوع محض لایه‌ای	۲
۲۰	 انگیزه و ایده اصلی	۱.۲
۲۰	 انگیزه	۱.۱.۲
۲۰	 ایده اصلی	۲.۱.۲
۲۱	 سیستم‌های نوع محض لایه‌ای	۲.۲
۲۱	 لایه‌ها و شهود آن‌ها	۱.۲.۲
۲۲	 تعریف رسمی سیستم	۲.۲.۲
۲۳	 خواص	۳.۲.۲
۲۷	 گسترش سیستم	۴.۲.۲
۲۸	 درجه	۵.۲.۲

۳۰	ترجمه حذف وابستگی	۳
۳۰	 مقدمه	۱.۳
۳۱	 هدف	۲.۳
۳۳	 ترجمه سطح نوع	۳.۳
۴۳	 ترجمه سطح عبارت	۴.۳
۴۸	 نتیجه اصلی	۵.۳

۵۰ سوم نتایج و پیشنهادات آینده

۴ نتیجه‌گیری و جمع‌بندی ۵۱

۵۱ ۱.۴ جمع‌بندی

۵۱ ۲.۴ پیشنهادات

۵۴ مراجع

۵۵ پیوست: کدهای COQ

۵۵ مشخصه سیستم‌های نوع محض

۵۵ مشخصه سیستم‌های نوع محض لایه‌ای

۵۵ سیستم نوع محض و لم‌ها

۱۲۸ سیستم نوع محض لایه‌ای و ترجمه حذف وابستگی

فهرست تصاویر

۱۵	 ۱.۱ مکعب لامبدا
----	-----------------------

فهرست جداول

۱۶	۱.۱ جدول بیان سیستم‌های نوع در مکعب لامبدا به کمک سیستم‌های نوع محض
----	---

بخش اول

مفاهيم و تعاريف مقدماتي

فصل ۱

مفاهیم مقدماتی

۱.۱ مقدمه

در این فصل، مفاهیم و تعاریف پایه‌ای مورد نیاز برای درک مباحث اصلی پایان‌نامه معرفی می‌شوند. این مفاهیم شامل حساب لامبدا بدون نوع، سیستم‌های کاهش انتزاعی، حساب لامبدا نوع‌دار ساده و سیستم‌های نوع محض هستند که به ترتیب از ساده به پیچیده ارائه می‌گردند. هدف از این فصل، ایجاد یک زبان مشترک و نمادگذاری واحد برای خواننده است تا زمینه لازم برای مطالعه فصل‌های بعدی، که به معرفی سیستم‌های نوع محض لایه‌ای، ترجمه حذف وابستگی و صوری‌سازی آن‌ها در اثبات‌یار Coq اختصاص دارد، فراهم آید.

فصل با بررسی حساب لامبدا بدون نوع آغاز می‌شود که پایه تاریخی و مفهومی تمام سیستم‌های بعدی را تشکیل می‌دهد. سپس چارچوب کلی سیستم‌های کاهش انتزاعی معرفی می‌گردد که ابزار ریاضی برای مطالعه دقیق خواص نرمال‌سازی و همگرایی را فراهم می‌کند. این چارچوب به دلیل عمومیت خود، امکان بیان یکپارچه نتایج مربوط به نرمال‌سازی ضعیف، قوی و همگرایی را فراهم می‌کند. در ادامه، حساب لامبدا نوع‌دار ساده به عنوان نخستین سیستم نوع‌دار وابسته بررسی می‌شود. این سیستم نه تنها مقدمه‌ای طبیعی برای ورود به سیستم‌های پیچیده‌تر است، بلکه نمونه‌ای کلاسیک از چگونگی جلوگیری انواع از حلقه‌های محاسباتی را نشان می‌دهد. در نهایت، به سیستم‌های نوع محض می‌پردازیم که چارچوب کلی و یکپارچه‌ای را برای بیان طیف وسیعی از سیستم‌های نوع، از حساب لامبدا نوع‌دار ساده تا حساب ساختمان‌ها، ارائه می‌کنند.

اثبات‌های تفصیلی بسیاری از قضایا و لم‌های این فصل به منابع استاندارد مانند [۲] و [۷] ارجاع داده شده و تنها صورت تعاریف و قضایای کلیدی همراه با توضیحات ضروری ارائه می‌گردد. این رویکرد از تکرار مطالب جلوگیری می‌کند و خواننده را به مطالعه منابع اصلی ترغیب می‌نماید.

با پایان این فصل، خواننده با ابزارهای نظری لازم برای درک سیستم‌های نوع محض لایه‌ای و ترجمه حذف وابستگی آشنا خواهد شد و آماده ورود به مباحث تخصصی فصل‌های بعدی می‌شود.

۲.۱ حساب لامبدا بدون نوع

حساب لامبدا بدون نوع که نخستین بار توسط Alonzo Church در [۴] معرفی شد، پایه تاریخی و مفهومی تمام سیستم‌های محاسباتی تابعی و نظریه انواع مدرن به شمار می‌رود. این دستگاه، محاسبه را به عنوان کاربرد توابع بر آرگومان‌ها مدل می‌کند و قدرت بیان معادلی با ماشین تورینگ دارد. این حساب، با وجود سادگی نحوی، مسائل عمیقی مانند حلقه‌های نامتناهی و تصمیم‌ناپذیری را نشان می‌دهد؛ که این موارد در سیستم‌های نوع‌دار با استفاده از انواع کنترل می‌شوند.

۱.۲.۱ نحو

تعریف ۱.۱ (نحو عبارات در حساب لامبدا بدون نوع). مجموعه عبارات حساب لامبدا بدون نوع، که معمولاً با Λ نشان داده می‌شود، به صورت بازگشتی زیر تعریف می‌گردد

$$M ::= V \mid \lambda V.M \mid MM$$

که در آن

- $V = \{x, y, z, \dots\}$ مجموعه متغیرهاست.
- $\lambda V.M$ یک انتزاع و بیانگر تعریف یک تابع است.
- MM یک کاربرد و بیانگر صدا زدن یک تابع است.

۲.۲.۱ متغیرهای آزاد و بسته

برای درک دقیق مفاهیم جانشینی و کاهش، معرفی متغیرهای آزاد و بسته ضروری است.

تعریف ۲.۱ (متغیرهای آزاد یک عبارت). برای یک عبارت M ، مجموعه متغیرهای آزاد

آن که با $FV(M)$ نشان داده می‌شود، به صورت بازگشتی زیر تعریف می‌گردد

$$FV(x) = \{x\}$$

$$FV(\lambda x.M) = FV(M) - \{x\}$$

$$FV(MN) = FV(M) \cup FV(N)$$

گزاره ۳.۰۱. گزاره‌های زیر با یکدیگر معادل هستند:

- $x \in FV(M)$

- متغیر x در عبارت M آزاد است.

- متغیر x در عبارت M دارای رخداد آزاد است.

گزاره ۴.۰۱. گزاره‌های زیر با یکدیگر معادل هستند:

- $x \notin FV(M)$

- متغیر x در عبارت M بسته است.

- متغیر x در عبارت M دارای رخداد بسته است.

گزاره ۵.۰۱. گزاره‌های زیر با یکدیگر معادل هستند:

- $FV(M) = \emptyset$

- عبارت M بسته است.

- عبارت M یک ترکیب‌گر است.

۳.۲.۱ جانشینی

جانشینی با نماد $M[N/x]$ نشان داده می‌شود و معنای آن جایگزینی همه رخداد های آزاد x در M با عبارت N است.

تذکره ۶.۰۱. در برخی منابع مانند [۲]، جانشینی با نماد

$$M[x := N]$$

نشان داده می‌شود. ما در این پایان‌نامه از نماد متداول دیگری استفاده می‌کنیم که Mull نیز در تر دکتري خود استفاده کرده است، یعنی

$$M[N/x]$$

تعریف ۷.۱. جانشینی به صورت بازگشتی زیر تعریف می شود

$$\begin{aligned}
 x[N/x] &= N \\
 y[N/x] &= y & (x \neq y) \\
 PQ[N/x] &= (P[N/x])(Q[N/x]) \\
 (\lambda x.M)[N/x] &= \lambda x.M \\
 (\lambda y.M)[N/x] &= \lambda y.(M[N/x]) & (x \neq y)
 \end{aligned}$$

۴.۲.۱ تبدیل α

تبدیل α به ما اجازه می دهد نام متغیرهای بسته را، بدون تغییر معنای عبارت، تغییر دهیم.

تعریف ۸.۱. اگر M یک عبارت باشد و $x \notin FV(M)$ ، آنگاه تبدیل α به صورت زیر بیان می شود

$$\lambda x.M \equiv_{\alpha} \lambda y.M[y/x]$$

۵.۲.۱ کاهش β

تعریف ۹.۱. کاهش β قاعده اصلی محاسبه در حساب لامبدا است و به صورت زیر تعریف می شود

$$(\lambda x.M)N \rightarrow_{\beta} M[N/x]$$

که بستار بازتاب-تعدی آن با نماد $\twoheadrightarrow_{\beta}$ نشان داده می شود.

مثال

۱.

$$\begin{aligned}
 \lambda x.(\lambda xy.y)x &\equiv_{\alpha} \lambda x.(\lambda ry.y)x \\
 &\rightarrow_{\beta} \lambda x.(\lambda y.y)[x/r] \\
 &= \lambda x.(\lambda y.y)
 \end{aligned}$$

۲. اگر $M \equiv \lambda x.xx$ ، آنگاه

$$\begin{aligned} MM &\equiv (\lambda x.xx)M \\ &\rightarrow_{\beta} xx[M/x] \\ &= MM \\ &\equiv (\lambda x.xx)M \\ &\rightarrow_{\beta} xx[M/x] \\ &= \dots \end{aligned}$$

پس داریم

$$MM \rightarrow_{\beta} MM \rightarrow_{\beta} MM \rightarrow_{\beta} \dots$$

همانطور که در مثال ۲ مشاهده می‌کنید، دنباله کاهش عبارت MM نامتناهی است. این مثال ما را به مفاهیم بعدی سوق می‌دهد.

۶.۲.۱ مسیرهای کاهش و فرم نرمال

تعریف ۱۰.۱. یک مسیر کاهش، دنباله‌ای به شکل زیر است که می‌تواند متناهی یا نامتناهی باشد.

$$M_0 \rightarrow_{\beta} M_1 \rightarrow_{\beta} \dots \rightarrow_{\beta} M_n$$

تعریف ۱۱.۱. عبارت M در فرم نرمال (یا یک فرم نرمال) است، اگر هیچ گامی از کاهش β بر آن قابل اعمال نباشد. یعنی

$$M \in \text{NF} \iff \nexists N.M \rightarrow_{\beta} N$$

مثال

عبارات زیر در فرم نرمال هستند:

$$\begin{aligned} x \\ x(\lambda a.a) \\ \lambda x.x \\ \lambda y.x \end{aligned}$$

عبارات زیر در فرم نرمال نیستند:

$$(\lambda x.x)x$$

$$(\lambda y.xy)(\lambda z.r)$$

زیرا به ترتیب داریم:

$$(\lambda x.x)x \equiv_{\alpha} (\lambda y.y)x \rightarrow_{\beta} y[x/y] = x$$

$$(\lambda y.xy)(\lambda z.r) \rightarrow_{\beta} xy[(\lambda z.r)/y] = x(\lambda z.r)$$

۷.۲.۱ نرمال سازی ضعیف و قوی

تعریف ۱۲.۱ (نرمال سازی ضعیف). عبارت M دارای خاصیت نرمال سازی ضعیف است، اگر حداقل یک مسیر کاهش به یک فرم نرمال داشته باشد. یعنی

$$\exists N \in \text{NF}. M \twoheadrightarrow_{\beta} N$$

تعریف ۱۳.۱ (نرمال سازی قوی). عبارت M دارای خاصیت نرمال سازی قوی است، اگر هیچ مسیر کاهش نامتناهی ای از آن آغاز نشود.

قضیه ۱۴.۱. اگر یک عبارت دارای خاصیت نرمال سازی قوی باشد، آنگاه دارای خاصیت نرمال سازی ضعیف نیز است.

عکس قضیه فوق همان حدس BGK است که برای سیستم های وابسته، هنوز اثبات نشده است.

حدس ۱۵.۱ (BGK). اگر یک عبارت دارای خاصیت نرمال سازی ضعیف باشد، آنگاه دارای خاصیت نرمال سازی قوی نیز است.

قضیه ۱۶.۱. در حساب لامبدا بدون نوع، نرمال سازی ضعیف و قوی معادل هستند؛ به بیان دیگر، حدس ۱۵.۱ در حساب لامبدا بدون نوع، برقرار است.

مثال

همانطور که در ۲ دیدیم، عبارت زیر دارای خاصیت نرمال سازی قوی نیست

$$(\lambda x.xx)(\lambda x.xx)$$

۳.۱ سیستم‌های کاهش انتزاعی

سیستم‌های کاهش انتزاعی^۱ که نخستین بار توسط Max Newman در [۱۱] به منظور مطالعه همگرایی روابط کاهش معرفی شدند، چارچوبی ریاضی برای مطالعه روابط کاهش در سیستم‌های محاسباتی هستند. بسیاری از مفاهیم نظریه انواع (از جمله نرمال‌سازی ضعیف و قوی، همگرایی و یکتایی فرم نرمال) به‌طور ضمنی در این چارچوب بیان می‌شوند. در این بخش تعریف رسمی و ویژگی‌های بنیادی این چارچوب را بیان می‌کنیم.

۱.۳.۱ تعریف سیستم کاهش انتزاعی

تعریف ۱.۷.۱. یک سیستم کاهش انتزاعی، یک زوج $(A, \rightarrow)$ است که در آن:

- A مجموعه اشیا است.

- $\rightarrow \subseteq A \times A$ یک رابطه دودویی روی A است که رابطه کاهش نام دارد.

برای مثال در حساب لامبدا، مجموع عبارات Λ همراه با رابطه کاهش β ، یک ARS را تشکیل می‌دهند.

۲.۳.۱ دنباله و بستار کاهش

تعریف ۲.۱۸.۱. دنباله کاهش، زنجیره‌ای از کاهش‌ها به شکل زیر است که می‌تواند متناهی یا نامتناهی باشد.

$$a_0 \rightarrow a_1 \rightarrow \dots \rightarrow a_n$$

تعریف ۲.۱۹.۱. بستار تعدی رابطه $\rightarrow$ را با $a \rightarrow^+ b$ نشان می‌دهیم که به این معناست که حداقل یک گام کاهش از b به a وجود دارد.

بستار بازتابی-تعدی رابطه $\rightarrow$ را با $a \rightarrow^* b$ نشان می‌دهیم که یعنی $b = a$ یا یک دنباله کاهش از b به a وجود دارد.

۳.۳.۱ فرم نرمال و نرمال‌سازی

تعریف ۳.۲۰.۱ (فرم نرمال). شی a در فرم نرمال (یا یک فرم نرمال) است اگر هیچ کاهش بر آن قابل اعمال نباشد. یعنی

$$a \in \text{NF} \iff \nexists b. a \rightarrow b$$

Abstract Reduction Systems^۱ یا به اختصار ARS

تعریف ۲۱.۱ (نرمال سازی ضعیف). شی a دارای خاصیت نرمال سازی ضعیف است، اگر حداقل یک دنباله کاهش به یک فرم نرمال داشته باشد. یعنی

$$\exists b \in \text{NF}. a \rightarrow^* b$$

تعریف ۲۲.۱ (نرمال سازی قوی). شی a دارای خاصیت نرمال سازی قوی است، اگر هیچ دنباله کاهش نامتناهی از آن آغاز نشود.

قضیه ۲۳.۱. اگر یک شی دارای خاصیت نرمال سازی قوی باشد، آنگاه دارای خاصیت نرمال سازی ضعیف نیز است.

۴.۳.۱ همگرایی

تعریف ۲۴.۱ (همگرایی). سیستم کاهش انتزاعی $(A, \rightarrow)$ همگراست، هرگاه هر دو دنباله کاهش که از یک شی منشعب می شوند، دوباره به شی مشترکی برسند. یعنی

$$\forall a, b_1, b_2. (a \rightarrow^* b_1 \wedge a \rightarrow^* b_2) \implies (\exists c. (b_1 \rightarrow^* c \wedge b_2 \rightarrow^* c))$$

نتایجی مانند یکتایی فرم نرمال از قضیه پیش رو ثابت می شوند.

قضیه ۲۵.۱ (قضیه چرچ-راسر). سیستم کاهش انتزاعی $(A, \rightarrow)$ همگراست، اگر و تنها اگر دارای خاصیت چرچ-راسر باشد؛ یعنی

$$\forall b_1, b_2. (b_1 = b_2) \implies (\exists c. (b_1 \rightarrow^* c \wedge b_2 \rightarrow^* c))$$

نتیجه ۲۶.۱ (یکتایی فرم نرمال). فرم نرمال یک شی، در صورت وجود، یکتاست. یعنی

$$\forall M, N_1 \in \text{NF}, N_2 \in \text{NF}. (M \rightarrow^* N_1 \wedge M \rightarrow^* N_2) \implies N_1 = N_2$$

اثبات همگرایی معمولاً دشوار است، اما یک ویژگی محلی ساده تر وجود دارد.

تعریف ۲۷.۱ (همگرایی محلی). سیستم کاهش انتزاعی $(A, \rightarrow)$ همگرای محلی است اگر

$$\forall a, b_1, b_2. (a \rightarrow b_1 \wedge a \rightarrow b_2) \implies (\exists c. (b_1 \rightarrow^* c \wedge b_2 \rightarrow^* c))$$

لم ۲۸.۱ (لم نیومن). اگر یک سیستم کاهش انتزاعی، همگرای محلی و دارای خاصیت نرمال سازی قوی باشد، آنگاه همگراست.

تذکر ۲۹.۱. در برخی سیستم‌ها (مانند حساب لامبدا بدون نوع)، به دلیل همگرایی، نرمال‌سازی ضعیف معادل نرمال‌سازی قوی است. اما در سیستم‌های نوع‌دار وابسته، ممکن است عبارتی دارای خاصیت نرمال‌سازی ضعیف باشد اما در حداقل یک دنباله کاهش نامتناهی ظاهر شود. این دقیقاً مسئله‌ای است که حدس ۱۵.۱ به آن می‌پردازد و Mull در [۱۰] با ارائه سیستم‌های نوع محض لایه‌ای می‌کوشد که آن را حل کند.

۴.۱ حساب لامبدا نوع‌دار ساده

حساب لامبدا نوع‌دار ساده^۱ که نخستین بار توسط Alonzo Church در [۵] به عنوان پایه‌ای برای منطق معرفی شد، یکی از مهم‌ترین و کلاسیک‌ترین سیستم‌های نوع است. این سیستم پایه بسیاری از زبان‌های تابعی، سیستم‌های نوع پیشرفته و بخش مهمی از نظریه انواع به شمار می‌رود. همچنین ساده‌ترین نمونه از «سیستم‌های نوع محض» است و مقدمه‌ای طبیعی برای ورود به مباحث بخش بعدی به حساب می‌آید. سیستم مذکور نشان می‌دهد که افزودن انواع، باعث تضمین نرمال‌سازی قوی می‌شود و از کاهش‌های نامتناهی جلوگیری می‌کند.

۱.۴.۱ نحو

تعریف ۳۰.۱ (انواع). مجموعه انواع به صورت بازگشتی زیر تعریف می‌شود

$$T ::= B \mid T \rightarrow T$$

که در آن

• $B = \{\alpha, \beta, \gamma, \dots\}$ مجموعه انواع پایه است.

• $T \rightarrow T$ بیانگر نوع توابع است.

تعریف ۳۱.۱ (عبارات). مجموعه عبارات به صورت بازگشتی زیر تعریف می‌شود

$$M ::= V \mid \lambda V^T. M \mid MM$$

که در آن

• $V = \{x, y, z, \dots\}$ مجموعه متغیرهاست.

^۱ Simply Typed Lambda Calculus یا به اختصار STLC یا $\lambda \rightarrow$

- $\lambda V^T.M$ یک انتزاع و بیانگر تعریف یک تابع است.

- MM یک کاربرد و بیانگر صدا زدن یک تابع است.

۲.۴.۱ زمینه و قضاوت نوع

تعریف ۳۲.۱ (زمینه). یک زمینه مانند Γ ، دنباله‌ای متناهی از مفروضات به شکل زیر است

$$x_1 : A_1, \dots, x_n : A_n$$

که

- $x_i : A_i$ به این معناست که x_i از نوع A_i است.

- $\Gamma, x : A$ به معنای افزودن فرض $x : A$ به Γ است.

تعریف ۳۳.۱ (قضاوت نوع). یک قضاوت نوع، گزاره‌ای به شکل زیر است

$$\Gamma \vdash M : A$$

که در آن Γ یک زمینه، M یک عبارت و A یک نوع است و معنای آن این است که ”در زمینه Γ ، عبارت M از نوع A است”.

تعریف ۳۴.۱ (قواعد استنتاج نوع). قواعد استنتاج نوع در $STLC$ شامل سه قاعده زیر است:

- قاعده متغیر

$$\frac{}{\Gamma, x : A \vdash x : A}$$

- قاعده انتزاع

$$\frac{\Gamma, x : A \vdash M : B}{\Gamma \vdash \lambda x^A.M : A \rightarrow B}$$

- قاعده کاربرد

$$\frac{\Gamma \vdash M : A \rightarrow B \quad \Gamma \vdash N : A}{\Gamma \vdash MB : B}$$

تعریف ۳۵.۱ (نوع‌پذیری). یک عبارت نوع‌پذیر است هرگاه بتوان طبق قواعد **۳۴.۱** یک نوع برای آن یافت.
به بیان دیگر؛ عبارت M نوع‌پذیر است هرگاه

$$\exists \Gamma, A. \Gamma \vdash M : A$$

مثال

عبارات زیر نوع‌پذیر هستند؛

$$\begin{aligned} \lambda x. x \\ \lambda xy. x \\ \lambda f gx. (f(gx)) \end{aligned}$$

زیرا به ترتیب داریم؛

$$\begin{aligned} \lambda x^A. x : A \rightarrow A \\ \lambda x^A y^B. x : A \rightarrow B \rightarrow A \\ \lambda f^{B \rightarrow C} g^{A \rightarrow B} x^A. (f(gx)) : (B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C \end{aligned}$$

عبارت زیر، که قبل‌تر در **۲** دیدیم، در STLC نوع‌پذیر نیست؛

$$(\lambda x. (xx))(\lambda x. (xx))$$

مثال فوق‌ما را به سمت قضیه مهم بعدی سوق می‌دهد.

۳.۴.۱ خواص

در این بخش، برخی خواص مهم STLC را مطالعه می‌کنیم.

قضیه ۳۶.۱ (حفظ نوع در کاهش). کاهش دادن یک عبارت، نوع آن را تغییر نمی‌دهد.
یعنی

$$(\Gamma \vdash M : A \wedge M \rightarrow_{\beta} N) \implies \Gamma \vdash N : A$$

قضیه بعدی بیان می‌کند که وجود انواع، از ایجاد عباراتی که دارای خاصیت نرمال‌سازی قوی نیستند، جلوگیری می‌کند.

قضیه ۳۷.۱ (قضیه نرمال‌سازی قوی). هر عبارت نوع‌پذیر در STLC دارای خاصیت نرمال‌سازی قوی است.

۵.۱ سیستم‌های نوع محض

سیستم‌های نوع محض^۱ که نخستین بار توسط [۳] و [۱۴] معرفی شدند و سپس توسط [۱] تعمیم یافتند، یک چارچوب یکپارچه برای توصیف طیف وسیعی از سیستم‌های نوع مبتنی بر حساب لامبدا را فراهم می‌کنند. بسیاری از سیستم‌های مهم از جمله حساب لامبدا نوع‌دار ساده، سیستم‌های چندریخت و حساب ساختمان‌ها را می‌توان تنها با انتخاب مناسب مجموعه‌ای از ”رده‌ها”، ”اصول” و ”روابط” در قالب یک PTS بیان کرد. این چارچوب به دلیل سادگی نحوی و همچنین انعطاف بسیار بالا، نقش محوری در نظریه انواع دارد. از طرفی سیستمی که Mull در [۱۰] معرفی می‌کند که اساس کار این پایان‌نامه است، نسخه تغییر یافته همین سیستم است؛ لذا آشنایی با این سیستم برای ما ضروری است.

۱.۵.۱ مشخصه و نحو

تعریف ۳۸.۱ (مشخصه سیستم نوع محض). یک سیستم نوع محض با سه تایی (S, A, R) مشخص می‌شود که در آن

$$\bullet S = \{s_1, s_2, \dots\} \text{ مجموعه رده‌ها}$$

$$\bullet A \subseteq S \times S \text{ مجموعه اصول}$$

$$\bullet R \subseteq S \times S \times S \text{ مجموعه قوانین}$$

هستند.

عبارات و انواع در PTS از منظر نحو یکسان هستند؛ یعنی در این چارچوب، ”نوع” و ”عبارت” یک ساختار واحد دارند و فقط جایگاه آن‌ها در قضاوت نوع است که نقش متفاوتی به آن‌ها می‌دهد.

^۱ Pure Type Systems یا به اختصار PTS

تعریف ۳۹.۱ (نحو سیستم نوع محض). نحو سیستم نوع محض به صورت بازگشتی زیر تعریف می‌شود

$$T ::= S \mid V \mid \lambda V : T.T \mid \Pi V : T.T \mid TT$$

که در آن

• $S = \{s_1, s_2, \dots\}$ مجموعه رده‌ها

• $V = \{x, y, z, \dots\}$ مجموعه متغیرها

• $\lambda V : T.T$ انتزاع

• $\Pi V : T.T$ نوع تابع

• TT کاربرد

هستند.

در این پایان‌نامه، مطابق با [۱۰]، متغیرها را با رده خودشان نشانه‌گذاری می‌شوند.

تعریف ۴۰.۱ (نشانه‌گذاری متغیرها). برای هر $s \in S$ ، V_s را مجموعه شمارشی متغیرهای مربوط به رده s در نظر می‌گیریم. $s v_i$ به معنای i مین متغیر در V_s است و $V = \bigcup_{s \in S} V_s$.

۲.۵.۱ قضاوت نوع

تعریف ۴۱.۱ (زمینه). یک زمینه مانند Γ ، دنباله‌ای متناهی از مفروضات به شکل زیر است

$$x_1 : A_1, \dots, x_n : A_n$$

که در آن $x_i : A_i$ به این معناست که " x_i از نوع A_i است". همچنین $\Gamma, x : A$ به معنای افزودن فرض $x : A$ به Γ است.

تعریف ۴۲.۱ (قواعد استنتاج نوع در سیستم نوع محض). در یک سیستم نوع محض با مشخصه (S, A, R) قواعد زیر موجود است:

• قاعده اصل

$$\frac{}{\vdash s : t} \quad (s, t) \in \mathcal{A}$$

• قاعده شروع

$$\frac{\Gamma \vdash A : s}{\Gamma, {}^s x : A \vdash {}^s x : A} \quad {}^s x \notin \Gamma$$

• قاعده تضعیف

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash B : s}{\Gamma, {}^s x : B \vdash M : A} \quad {}^s x \notin \Gamma$$

• قاعده تولید

$$\frac{\Gamma \vdash A : s_1 \quad \Gamma, {}^{s_1} x : A \vdash B : s_2}{\Gamma \vdash (\Pi^{s_1} x^A. B) : s_3} \quad (s_1, s_2, s_3) \in \mathcal{R}$$

• قاعده انتزاع

$$\frac{\Gamma, {}^s x : A \vdash M : B \quad \Gamma \vdash \Pi^s x^A. B : s}{\Gamma \vdash (\lambda^s x^A. M) : (\Pi^s x^A. B)}$$

• قاعده کاربرد

$$\frac{\Gamma \vdash M : (\Pi^s x^A. B) \quad \Gamma \vdash N : A}{\Gamma \vdash MN : B[N/{}^s x]}$$

• قاعده تبدیل

$$\frac{\Gamma \vdash M : A \quad A \equiv_\beta B \quad \Gamma \vdash B : s}{\Gamma \vdash M : B}$$

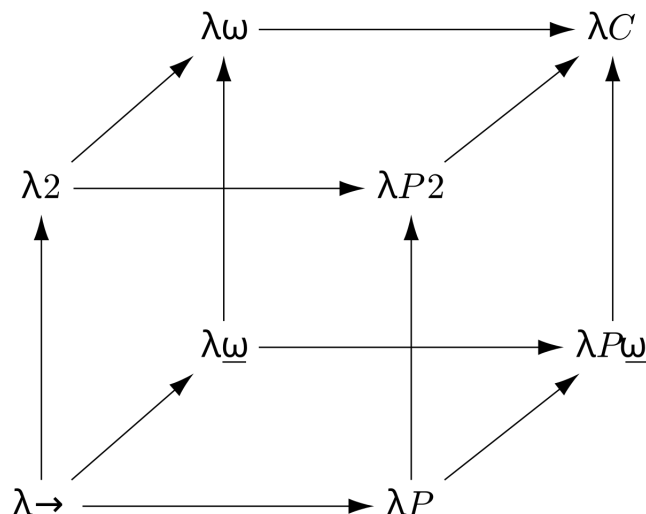
۳.۵.۱ مکعب لامبدا

توصیف

مکعب لامبدا یک چارچوب مفهومی و هندسی برای دسته‌بندی خانواده‌ای از سیستم‌های نوع است. این مکعب نشان می‌دهد که چگونه با افزودن یا حذف برخی قابلیت‌های نوعی، می‌توان به سیستم‌های مختلفی از حساب لامبدا دست یافت. مکعب لامبدا دارای سه بُعد مستقل است که هر بُعد متناظر با افزودن یک قابلیت به سیستم پایه است:

۱. چندریختی

این بُعد نشان‌دهنده امکان کمی‌سازی روی انواع است؛ به بیان دیگر، اجازه می‌دهد توابعی تعریف شوند که برای همه انواع معتبر باشند. وجود این قابلیت منجر به سیستم‌هایی مانند λ_2 می‌شود.



شکل ۱.۱: مکعب لامبدا

۲. اپراتورهای نوع

این بُعد امکان تعریف توابعی را فراهم می‌کند که روی انواع عمل می‌کنند، نه روی عبارات. در این حالت، انواع می‌توانند به عنوان ورودی توابع به کار روند و این امر منجر به سیستم‌هایی مانند $\lambda\omega$ می‌شود.

۳. انواع وابسته

این بُعد اجازه می‌دهد نوع یک عبارت به مقدار یک عبارت دیگر وابسته باشد. به این ترتیب می‌توان انواعی تعریف کرد که اطلاعاتی درباره مقدار عبارات در خود جای می‌دهند؛ این قابلیت، پایه سیستم‌هایی مانند حساب ساختمان‌هاست.

ترکیب یا عدم ترکیب هر یک از این سه قابلیت، هشت رأس مکعب را تشکیل می‌دهد که هر رأس متناظر با یک سیستم نوع مشهور است. برای مثال، حساب لامبدای نوع دار ساده در رأسی قرار دارد که هیچ‌یک از این قابلیت‌ها را ندارد، در حالی که حساب ساختمان‌ها در رأسی قرار می‌گیرد که هر سه قابلیت را به طور هم‌زمان داراست.

تعریف به کمک سیستم‌های نوع محض

حال با معرفی کامل سیستم‌های نوع محض، بسیاری از سیستم‌های مشهور تنها با انتخاب مناسب (S, A, R) قابل بیان‌اند. جدول ۱.۱ بیانگر تعریف هر هشت رأس مکعب لامبدا به کمک سیستم‌های نوع محض است.

	$\mathcal{S}$	$\mathcal{A}$	$\mathcal{R}$
$\lambda \rightarrow$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *)\}$
$\lambda 2$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, *, *)\}$
$\lambda \omega$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (*, \square, \square)\}$
λP	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square)\}$
$\lambda P 2$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square), (\square, *, *)\}$
$\lambda \omega$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (*, \square, \square), (\square, *, *)\}$
$\lambda P \omega$	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square), (*, \square, \square)\}$
λC	$\{(*, \square)\}$	$\{(* : \square)\}$	$\{(*, *, *), (\square, \square, \square), (*, \square, \square), (\square, *, *)\}$

جدول ۱.۱: جدول بیان سیستم‌های نوع در مکعب لامبدا به کمک سیستم‌های نوع محض

۴.۵.۱ اهمیت

در ادامه این پایان‌نامه، سیستم‌های نوع محض نقطه آغاز برای تعریف «سیستم‌های نوع محض لایه‌ای^۱» سپس معرفی ترجمه حذف وابستگی Mull هستند. در واقع:

- TPTS گسترشی ساختاریافته از PTS است.
 - قواعد نوع TPTS نسخه تقویت‌شده همین قواعداند.
 - اثبات‌های نرمال‌سازی وابسته به ساختار PTS بیان می‌شوند.
- در نتیجه، این بخش چارچوب مفهومی لازم برای تعریف سیستم‌های لایه‌ای و صورتی‌سازی آن‌ها در Coq را فراهم می‌کند.

۵.۵.۱ خواص فرانظری سیستم‌های نوع محض

در این بخش برخی خواص فرانظری سیستم‌های نوع محض را مورد بررسی قرار می‌دهیم.

تذکره ۴۳.۱. از تکرار تعاریفی که قبلاً مشاهده کرده‌ایم (برای مثال فرم نرمال و نرمال‌سازی برای یک عبارت که در ۶.۲.۱ دیدیم) خودداری می‌کنیم.

تعریف ۴۴.۱ (نرمال‌سازی ضعیف و قوی در سیستم). یک سیستم نوع محض دارای خاصیت نرمال‌سازی ضعیف یا قوی است، هرگاه هر عبارت نوع‌پذیر در آن دارای این خاصیت باشد.

^۱TPTS یا Tiered Pure Type Systems

لم ۴۵.۱ (تولید).

• رده:

$$\forall s \in \mathcal{S}. \quad \Gamma \vdash s : A \implies \exists s' \in \mathcal{S}. (A =_{\beta} s') \wedge ((s, s') \in \mathcal{A})$$

• متغیر:

$$\forall {}^s x \in V. \quad \Gamma \vdash {}^s x : A \implies \exists B. (A =_{\beta} B) \wedge (({}^s x : B) \in \Gamma)$$

• عبارت Π :

$$\forall s \in \mathcal{S}, B, C. \quad \Gamma \vdash (\Pi {}^s x : B. C) : A \implies$$

$$\exists s', s'' \in \mathcal{S}. (\Gamma \vdash B : s) \wedge (\Gamma, {}^s x : B \vdash C : s') \wedge ((s, s', s'') \in \mathcal{R}) \wedge (A =_{\beta} s'')$$

• عبارت λ :

$$\forall s \in \mathcal{S}, B, M. \quad \Gamma \vdash (\lambda {}^s x : B. M) : A \implies$$

$$\exists s' \in \mathcal{S}, C. (\Gamma \vdash (\Pi {}^s x : B. C) : s') \wedge (\Gamma, {}^s x : B \vdash M : C) \wedge (A =_{\beta} \Pi {}^s x : B. C)$$

• کاربرد:

$$\forall M, N. \quad \Gamma \vdash MN : A \implies$$

$$\exists s \in \mathcal{S}, B, C. (\Gamma \vdash M : (\Pi {}^s x : B. C)) \wedge (\Gamma \vdash N : B) \wedge (A =_{\beta} C[N/{}^s x])$$

لم ۴۶.۱ (جانشینی).

$$\forall \Gamma, \Delta, M, N, A, B. \quad (\Gamma, x : A, \Delta \vdash M : B) \wedge (\Gamma \vdash N : A) \implies \Gamma, \Delta[N/x] \vdash M[N/x] : B[N/x]$$

لم ۴۷.۱ (صحت نوع).

$$\forall \Gamma, M, A. \quad \Gamma \vdash M : A \implies (A \in \mathcal{S}) \vee (\exists s \in \mathcal{S}. \Gamma \vdash A : s)$$

لم ۴۸.۱ (تضعیف).

$$\forall \Gamma, \Delta, M, A. \quad (\Gamma \subset \Delta) \wedge (\Gamma \vdash M : A) \implies \Delta \vdash M : A$$

لم ۴۹.۱ (جابه جایی).

$$\begin{aligned} \forall \Gamma, \Delta, x, y, A, B, C, M. \quad & (x \notin FV(B)) \wedge (\Gamma, x : A, y : B, \Delta \vdash M : C) \\ & \implies \Gamma, y : B, x : A, \Delta \vdash M : C \end{aligned}$$

بخش دوم

سیستم‌های نوع محض لایه‌ای و ترجمه
حذف وابستگی

فصل ۲

سیستم‌های نوع محض لایه‌ای

۱.۲ انگیزه و ایده اصلی

۱.۱.۲ انگیزه

در فصل پیشین، مفاهیم پایه‌ای حساب لامبدا، سیستم‌های کاهش انتزاعی، حساب لامبدا نوع‌دار ساده و سیستم‌های نوع محض را بررسی کردیم. این چارچوب‌ها زمینه لازم برای درک مسائل پیشرفته‌تر نظریه انواع، به‌ویژه رابطه میان نرمال‌سازی ضعیف و قوی را فراهم آوردند.

سیستم‌های نوع محض، به‌عنوان چارچوبی یکپارچه برای توصیف سیستم‌های نوع مبتنی بر حساب لامبدا، بستری مناسب برای مطالعه خواص فرانظری نظیر نرمال‌سازی، یکتایی نوع و سازگاری منطقی فراهم می‌کنند. با این حال، بررسی رابطه میان نرمال‌سازی ضعیف و نرمال‌سازی قوی در این سیستم‌ها، که در قالب حدس ۱۵.۱ بیان می‌شود، در حضور انواع وابسته با دشواری‌های اساسی مواجهه است.

مسئله اصلی از آنجا ناشی می‌شود که در سیستم‌های نوع وابسته، ساختار انواع می‌تواند به‌طور دلخواه به مقادیر عبارات وابسته باشد. این وابستگی باعث می‌شود که رفتار مسیرهای کاهش β به‌شدت پیچیده شده و تحلیل آن‌ها، به‌ویژه در اثبات نرمال‌سازی قوی، با موانع جدی روبه‌رو گردد. در چنین شرایطی، روش‌های کلاسیک که برای سیستم‌های غیروابسته به کار می‌رود، مانند تکنیک‌های مبتنی بر کاهش‌پذیری یا استدلال‌های ترتیبی، دیگر به‌سادگی قابل اعمال نیستند.

۲.۱.۲ ایده اصلی

Mull در رساله دکتری خود [۱۰] بر این اصل استوار است که وابستگی‌های موجود در

سیستم‌های نوع وابسته، لزوماً همگی برای بیان توان محاسباتی سیستم ضروری نیستند. به بیان دیگر، بخشی از این وابستگی‌ها را می‌توان به صورت کنترل‌شده حذف یا محدود کرد، بی‌آنکه به خوش‌نوعی یا قابلیت بیان سیستم آسیبی وارد شود.

بر همین اساس، Mull خانواده جدیدی از سیستم‌های نوع را تحت عنوان «سیستم‌های نوع محض لایه‌ای»^۱ معرفی می‌کند. در این سیستم‌ها، وابستگی میان انواع و عبارات به وسیله مفهوم «لایه» به طور صریح ساختاربندی می‌شود. لایه‌ها امکان تفکیک سطوح مختلف وابستگی را فراهم می‌کنند و از بروز وابستگی‌های چرخه‌ای یا کنترل‌نشده جلوگیری می‌نمایند.

مزیت کلیدی این لایه‌بندی آن است که امکان تعریف یک ترجمه حذف وابستگی از سیستم‌های نوع محض لایه‌ای به یک سیستم نوع محض فراهم می‌شود؛ به گونه‌ای که تحلیل نرمال‌سازی قوی در سیستم مقصد ساده‌تر بوده و می‌توان از آن برای استنتاج نرمال‌سازی قوی در سیستم مبدأ بهره گرفت.

در ادامه این فصل، سیستم‌های نوع محض لایه‌ای به‌طور رسمی معرفی می‌شوند، سپس در فصل‌های بعدی، ترجمه حذف وابستگی و خواص اساسی آن مورد بررسی قرار می‌گیرد. این مباحث، زمینه لازم برای صوری‌سازی کامل این ترجمه و اثبات‌های مرتبط با آن در اثبات‌یار Coq را که در فصل‌های بعدی ارائه می‌شوند، فراهم می‌سازند.

۲.۲ سیستم‌های نوع محض لایه‌ای

۱.۲.۲ لایه‌ها و شهود آن‌ها

ایده اصلی در سیستم‌های نوع محض لایه‌ای، افزودن ساختاری صریح برای کنترل وابستگی‌های نوعی است. این ساختار از طریق مفهوم «لایه» معرفی می‌شود. لایه‌ها سطوحی انتزاعی هستند که به متغیرها و عبارات نسبت داده می‌شوند و میزان وابستگی مجاز میان آن‌ها را محدود می‌کنند.

به‌طور شهودی، لایه‌ها را می‌توان به‌عنوان ابزاری برای مرتب‌سازی وابستگی‌ها در نظر گرفت؛ به گونه‌ای که یک عبارت در یک لایه بالاتر مجاز نیست به‌طور دلخواه به عبارات لایه‌های پایین‌تر وابسته باشد. این محدودیت مانع از شکل‌گیری وابستگی‌های چرخه‌ای و کنترل‌نشده در سطح انواع می‌شود، که یکی از موانع اصلی در تحلیل نرمال‌سازی قوی در سیستم‌های وابسته است.

لازم به تأکید است که لایه‌ها با «رده‌ها» متفاوت‌اند. رده‌ها نقش طبقه‌بندی نحوی انواع

^۱ Tiered Pure Type Systems یا به اختصار TPTS

و عبارات را ایفا می‌کنند، در حالی که لایه‌ها اطلاعاتی فرانظری درباره سطح وابستگی یک عبارت حمل می‌نمایند. به بیان دیگر، رده‌ها بخشی از ساختار نوعی سیستم هستند، اما لایه‌ها ابزاری ساختاری برای کنترل وابستگی‌ها محسوب می‌شوند.

در سیستم‌های نوع محض لایه‌ای، هر متغیر به صورت صریح به یک لایه نسبت داده می‌شود. این نشانه‌گذاری لایه‌ای، مشابه نشانه‌گذاری رده‌ای که پیش‌تر معرفی شد، صرفاً جنبه تزئینی ندارد، بلکه در قواعد نوع‌دهی نقش اساسی ایفا می‌کند. قیود لایه‌ای تعیین می‌کنند که در چه شرایطی تشکیل انواع تابع وابسته، انتزاع و کاربرد مجاز است.

هدف از معرفی لایه‌ها، کاهش توان بیان سیستم نیست، بلکه فراهم‌سازی بستری است که در آن بتوان وابستگی‌های نوعی را به گونه‌ای کنترل‌شده بازآرایی کرد. این کنترل دقیق، امکان تعریف ترجمه حذف وابستگی را فراهم می‌کند؛ ترجمه‌ای که نقش محوری در انتقال خواص نرمال‌سازی از سیستم‌های ساده‌تر به سیستم‌های نوع محض لایه‌ای ایفا می‌نماید.

۲.۲.۲ تعریف رسمی سیستم

تعریف ۱.۲ (سیستم نوع محض لایه‌ای). فرض کنید n یک عدد صحیح نامنفی باشد. یک سیستم نوع محض، n -لایه‌ای است؛ هرگاه مشخصات آن به شکل زیر باشد

$$\begin{aligned}\mathcal{S} &= \{s_i \mid i \in [n]\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \in [n-1]\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\}\end{aligned}$$

که در آن

$$[n] = \{1, \dots, n\}$$

مثال

۱. تنها سیستم نوع محض ۰-لایه‌ای، سیستم نوع محض خالی است؛

$$\mathcal{S} = \mathcal{A} = \mathcal{R} = \emptyset$$

۲. دو سیستم نوع محض ۱-لایه‌ای وجود دارد:

$$\begin{aligned}\mathcal{S} &= \{s_1\} & \mathcal{A} &= \emptyset & \mathcal{R} &= \emptyset \\ \mathcal{S} &= \{s_1\} & \mathcal{A} &= \emptyset & \mathcal{R} &= \{(s_1, s_1)\}\end{aligned}$$

۳. سیستم‌های نوع محض ۲- لایه‌ای، معادل کل مکعب لامبدا هستند.

۳.۲.۲ خواص

در این بخش برخی خواص مهم فرانظری سیستم‌های نوع محض را بررسی می‌کنیم، و خواهیم دید که زیرکلاسی که در بخش قبل معرفی کردیم، چه ویژگی‌های کاربردی‌ای را در اختیار ما قرار می‌دهد.

تعریف ۲.۲ (سیستم نوع محض تابعی). یک سیستم نوع محض را تابعی گوییم، هرگاه

• اگر $(s, t) \in A$ و $(s, t') \in A$ آنگاه $t = t'$.

• اگر $(s, t, u) \in R$ و $(s, t, u') \in A$ آنگاه $u = u'$.

لم ۳.۲ (یکتایی نوع). در یک سیستم نوع محض تابعی، نوع هر عبارت نوع‌پذیر، یکتاست. یعنی

$$\forall \Gamma, M, A, B. \quad (\Gamma \vdash M : A) \wedge (\Gamma \vdash M : B) \implies A =_{\beta} B$$

تعریف ۴.۲ (رده‌های بالا و پایین).

• رده s یک رده بالا است هرگاه

$$\nexists s' \in S. (s, s') \in A$$

• رده s یک رده پایین است هرگاه

$$\nexists s' \in S. (s', s) \in A$$

لم ۵.۲ (رده بالا). برای هر زمینه Γ ، متغیر x ، عبارات A و B و رده بالا s ، گزاره‌های زیر برقرار هستند:

$$\Gamma \not\vdash s : A$$

$$\Gamma \not\vdash x : s$$

$$\Gamma \not\vdash AB : s$$

تعریف ۶.۲ (سیستم نوع محض پایدار). یک سیستم نوع محض، پایدار است؛ هرگاه هر سه گزاره زیر برای آن برقرار باشد:

• تابعی باشد.

• اگر $(s, t) \in A$ و $(s', t) \in A$ آنگاه $s = s'$.

• $\mathcal{R} \subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\}$

تذکر ۷.۲. از اینجا به بعد، از نماد (s, s') به جای قانون (s, s', s') استفاده می‌کنیم.

تعریف ۸.۲ (اجتماع مجزا دو سیستم نوع محض). اجتماع مجزا دو سیستم نوع محض $\lambda\mathcal{S}$ و $\lambda\mathcal{S}'$ ، که با نماد $\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'$ نشان داده می‌شود، یک سیستم نوع محض با مشخصات زیر است

$$\mathcal{S}_{\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'} = \mathcal{S}_{\lambda\mathcal{S}} \sqcup \mathcal{S}_{\lambda\mathcal{S}'}$$

$$\mathcal{A}_{\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'} = \mathcal{A}_{\lambda\mathcal{S}} \cup \mathcal{A}_{\lambda\mathcal{S}'}$$

$$\mathcal{R}_{\lambda\mathcal{S} \sqcup \lambda\mathcal{S}'} = \mathcal{R}_{\lambda\mathcal{S}} \cup \mathcal{R}_{\lambda\mathcal{S}'}$$

در تعریف بعد می‌خواهیم به $A \subset \mathcal{S} \times \mathcal{S}$ ، به عنوان یک رابطه نگاه کنیم.

تعریف ۹.۲ (رابطه A). در یک سیستم نوع محض، مجموعه $A(s)$ که به معنای اصول‌هایی است که s در آنان ظاهر شده، به صورت زیر تعریف می‌شود

$$A(s) = \{(s', t') \in A \mid s' = s \vee t' = s\}$$

نکات

• اگر A را به عنوان یک گراف تفسیر کنیم، $A(s)$ به معنای مجموعه یال‌هایی است که به s متصل هستند.

• در یک سیستم نوع محض پایدار، برای هر رده s داریم: $|A(s)| \leq 2$.

تعریف ۱۰.۲ (بستار رابطه A).

• بستار تعدی: $<_A$

• بستار بازتابی-تعدی: $\leq_A$

• بستار هم‌رزی: $\approx_A$

تعریف ۱۱.۲ (دنباله رده). طبق تعریف قبل، می‌نویسیم $s \leq_A t$ اگر و تنها اگر دنباله‌ای از رده‌ها به صورت زیر موجود باشد

$$\exists t_1, \dots, t_k \in \mathcal{S}. \quad (t_1 = s) \wedge (t_k = t) \wedge (\forall i \in [k-1]. (t_i, t_{i+1}) \in A)$$

دنباله مذکور را یک دنباله به طول $k-1$ از s به t می‌نامیم.

نکته: دقت شود که در تعریف فوق، لزومی ندارد که اعضای یک دنباله رده، منحصر به فرد باشند.

گزاره ۱۲.۲. یک سیستم نوع محض پایدار را در نظر بگیرید. برای هر رده $s \in S$ و هر k ؛

• حداکثر یک دنباله رده با طول k یافت می شود که از s شروع می شود.

• حداکثر یک دنباله رده با طول k یافت می شود که به s خاتمه می یابد.

تعریف ۱۳.۲ (سیستم نوع محض تفکیک پذیر). یک سیستم نوع محض را تفکیک پذیر گوئیم، هرگاه

$$\forall s, s' \in S. \quad (s, s') \in \mathcal{R} \implies s \approx_{\mathcal{A}} s'$$

تعریف ۱۴.۲ (سیستم نوع محض اتمی). یک سیستم نوع محض را اتمی گوئیم، هرگاه

$$\forall s, s' \in S. \quad s \approx_{\mathcal{A}} s'$$

تعریف ۱۵.۲ (سیستم نوع محض کراندار).

• یک سیستم نوع محض، شرط زنجیره صعودی را ارضا می کند، هرگاه دنباله نامتناهی

$$s, s', s'', \dots$$

یافت نشود به طوری که

$$s < s' < s'' < \dots$$

• یک سیستم نوع محض، شرط زنجیره نزولی را ارضا می کند، هرگاه دنباله نامتناهی

$$s, s', s'', \dots$$

یافت نشود به طوری که

$$s > s' > s'' > \dots$$

یک سیستم نوع محض کراندار است، اگر هر دو شرط زنجیره صعودی و نزولی را ارضا کند.

تعریف ۱۶.۲ (سیستم نوع محض ضعیفاً غیروابسته). یک سیستم نوع محض، ضعیفاً (یا به طور ضعیف) غیروابسته است؛ هرگاه

$$(s, s', s'') \in \mathcal{R} \implies s \geq s' \geq s''$$

اصطلاح «سلسله مراتبی» در ادامه به معنای دارا بودن ساختار ترتیبی روی وابستگی‌هاست و با مفهوم «لایه‌ای» که قبل‌تر دیدیم مرتبط است، اما عیناً یکسان نیست.

تعریف ۱۷.۲ (سیستم نوع محض سلسله مراتبی). یک سیستم نوع محض را سلسله مراتبی می‌نامیم هرگاه شرط زنجیره صعودی را ارضا کند و ضعیفاً غیروابسته باشد.

تعریف ۱۸.۲ (سیستم نوع محض غیروابسته تعمیم‌یافته). یک سیستم نوع محض، غیروابسته تعمیم‌یافته است؛ اگر سلسله مراتبی و پایدار باشد.

تعریف ۱۹.۲ (سیستم نوع محض غیروابسته کراندار). یک سیستم نوع محض، غیروابسته کراندار است؛ اگر سلسله مراتبی، پایدار و کراندار باشد.

تعریف ۲۰.۲ (سیستم نوع محض غیروابسته). یک سیستم نوع محض، غیروابسته است؛ اگر سلسله مراتبی، پایدار، کراندار و لایه‌ای باشد.

اکنون می‌توانیم سیستم‌های نوع محض لایه‌ای را به کمک خواصی که بررسی کردیم، مطالعه کنیم.

لم ۲۱.۲. یک سیستم نوع محض، لایه‌ای است؛ اگر و تنها اگر پایدار، کراندار و اتمی باشد.

لم ۲۲.۲. یک سیستم نوع محض، پایدار، کراندار و تفکیک‌پذیر است؛ اگر و تنها اگر اجتماع مجزا سیستم‌های نوع محض لایه‌ای باشد.

نتیجه ۲۳.۲. یک سیستم نوع محض، غیروابسته کراندار است؛ اگر و تنها اگر اجتماع مجزا سیستم‌های نوع محض لایه‌ای باشد.

Doorn و Roux در [۱۳] نشان دادند که نرمال‌سازی (قوی) اجتماع مجزا سیستم‌های نوع محض، معادل نرمال‌سازی (قوی) تک تک سیستم‌هاست. بنابراین برای بررسی خاصیت نرمال‌سازی در سیستم‌های نوع محض پایدار، کراندار و تفکیک‌پذیر، کافی است سیستم‌های نوع محض لایه‌ای را در نظر بگیریم.

گزاره ۲۴.۲. اگر برای همه سیستم‌های نوع محض لایه‌ای، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض پایدار، کراندار و تفکیک‌پذیر نیز چنین است.

به طور خاص، اگر برای همه سیستم‌های نوع محض غیروابسته، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض غیروابسته کراندار نیز چنین است.

۴.۲.۲ گسترش سیستم

در این بخش می‌خواهیم گسترش‌هایی از سیستم‌های نوع محض لایه‌ای را مطالعه کنیم.

گسترش نامتناهی

تعریف ۲۵.۲ (سیستم نوع محض $\mathbb{Z}^+$ -لایه‌ای). سیستم نوع محض $\mathbb{Z}^+$ -لایه‌ای، یک سیستم نوع محض با مشخصات زیر است

$$\begin{aligned} \mathcal{S} &= \{s_i \mid i \in \mathbb{Z}^+\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \in \mathbb{Z}^+\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\} \end{aligned}$$

تعریف ۲۶.۲ (سیستم نوع محض $\mathbb{Z}^-$ -لایه‌ای). سیستم نوع محض $\mathbb{Z}^-$ -لایه‌ای، یک سیستم نوع محض با مشخصات زیر است

$$\begin{aligned} \mathcal{S} &= \{s_i \mid i \in \mathbb{Z}^-\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \leq -2\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\} \end{aligned}$$

تعریف ۲۷.۲ (سیستم نوع محض $\mathbb{Z}$ -لایه‌ای). سیستم نوع محض $\mathbb{Z}$ -لایه‌ای، یک سیستم نوع محض با مشخصات زیر است

$$\begin{aligned} \mathcal{S} &= \{s_i \mid i \in \mathbb{Z}\} \\ \mathcal{A} &= \{(s_i, s_{i+1}) \mid i \in \mathbb{Z}\} \\ \mathcal{R} &\subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\} \end{aligned}$$

گزاره ۲۸.۲. فرض کنید C مجموعه سیستم‌های نوع محض با ویژگی پیش‌رو باشد؛ اگر λS یک سیستم نوع محض لایه‌ای نامتناهی در C باشد، برای هر زیرسیستم نوع محض لایه‌ای متناهی $\lambda S'$ از λS ، یک زیرسیستم نوع محض لایه‌ای متناهی $\lambda S''$ یافت می‌شود به طوری که

$$(\lambda S'' \in C) \wedge (\lambda S' \subset \lambda S'' \subset \lambda S)$$

بنابراین هر سیستم در C دارای خاصیت نرمال‌سازی (قوی) است؛ اگر و تنها اگر هر سیستم متناهی در آن دارای این خاصیت باشد.

نتیجه ۲۹.۲. اگر برای همه سیستم‌های نوع محض لایه‌ای غیروابسته، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض غیروابسته تعمیم‌یافته نیز چنین است.

گسترش دوردار

تعریف ۳۰.۲ (سیستم نوع محض لایه‌ای دوردار). یک سیستم نوع محض لایه‌ای دوردار، یک سیستم نوع محض لایه‌ای با مشخصات زیر است

$$\mathcal{S} = \{s_i \mid i \in [n]\}$$

$$\mathcal{A} = \{(s_i, s_{i+1}) \mid i \in [n-1]\} \cup \{(s_n, s_1)\}$$

$$\mathcal{R} \subset \{(s, s', s') \mid (s, s') \in \mathcal{S} \times \mathcal{S}\}$$

در واقع اصل (s_n, s_1) به مجموعه اصول یک سیستم نوع محض لایه‌ای اضافه شده است و گراف $\mathcal{A}$ به یک گراف دوردار تبدیل شده است.

گزاره ۳۱.۲. اگر برای همه سیستم‌های نوع محض لایه‌ای متناهی بدون دور و دوردار، نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی باشد؛ آنگاه برای همه سیستم‌های نوع محض تفکیک‌پذیر پایدار نیز چنین است.

تذکره ۳۲.۲. در ادامه این پایان‌نامه؛ منظور از لایه‌ای، حالت متناهی بدون دور است.

۵.۲.۲ درجه

یکی از مزایای اصلی مطالعه سیستم‌های پایدار (به طور خاص سیستم‌های لایه‌ای) این است که عبارات را می‌توان بر اساس سطحی در سیستم که در آن قابل‌دستیابی هستند، طبقه‌بندی کرد. این ویژگی با تعریف یک معیار درجه برای عبارات و طبقه‌بندی عبارات بر اساس درجه آنها نشان داده می‌شود.

تعریف ۳۳.۲ (درجه عبارت). درجه یک عبارت با استفاده از تابع $\deg : T \rightarrow \mathbb{N}$ و به

صورت زیر تعریف می شود

$$\deg(s_i) = i + 1$$

$$\deg(s_i x) = i - 1$$

$$\deg(\Pi x^A . B) = \deg(B)$$

$$\deg(\lambda x^A . M) = \deg(M)$$

$$\deg(MN) = \deg(M)$$

تعریف ۳۴.۲.

$$T_j = \{M \in T \mid \deg(M) = j\}$$

$$T_{\geq j} = \{M \in T \mid \deg(M) \geq j\}$$

لم ۳۵.۲ (طبقه بندی). فرض کنید λS یک سیستم نوع محض n -لایه ای باشد. برای هر عبارت A ، گزاره های زیر برقرارند:

$$\deg(A) = n + 1 \iff A = s_n$$

$$\deg(A) = n \iff \exists \Gamma. \Gamma \vdash A : s_n$$

$$\forall i \in [n - 1]. \quad \deg(A) = i \iff \exists \Gamma, B. (\Gamma \vdash A : B) \wedge (\Gamma \vdash B : s_{i+1})$$

کاربرد خاص لم فوق چنین است که

$$\forall \Gamma, M, A. \quad \Gamma \vdash M : A \implies \deg(A) = \deg(M) + 1$$

لم ۳۶.۲.

$$\deg(B) = j - 1 \implies \deg(A[B/s_j x]) = \deg(A)$$

$$A \twoheadrightarrow_\beta B \implies \deg(A) = \deg(B)$$

فصل ۳

ترجمه حذف وابستگی

۱.۳ مقدمه

در چند دهه اخیر، روش‌های متعددی برای اثبات نرمال‌سازی سیستم‌های موجود در مکعب λ و گسترش آنان ارائه شده است، اما بسیاری از این روش‌ها به سیستم‌های نوع محض تعمیم داده نشده‌اند. هدف ما در این بخش، تعمیم یکی از این روش‌ها به نام ترجمه حذف وابستگی است.

ایده کلی چنین است که به منظور نشان دادن آنکه یک سیستم دارای خاصیت نرمال‌سازی قوی است، کافی است یک ترجمه، با خواص حفظ نوع و حفظ مسیرهای کاهش نامتناهی، از آن سیستم به سیستم دیگری که می‌دانیم دارای خاصیت نرمال‌سازی قوی است، ارائه کنیم. Harper و همکارانش [۱۲] چنین ترجمه‌ای از λP به $\lambda \rightarrow$ را ارائه کردند و Guevers و Nederhof [۹] این ترجمه را به ترجمه‌ای از λC به $\lambda \omega$ تعمیم دادند. هر دو این ترجمه‌ها را می‌توان به عنوان حذف قانون وابسته در سیستم مربوطه در نظر گرفت، به طور دقیق‌تر قانون $(*, \square)$ که اجازه می‌دهد نوع به عبارت وابسته باشد.

برای سیستم‌های نوع محض که به اندازه کافی خوش ساختار هستند، این مفهوم وابستگی را می‌توان گسترش داد، همانطور که Barthe و همکارانش [۸] برای تعریف سیستم‌های نوع محض غیروابسته تعمیم‌یافته‌شان انجام داده‌اند (که مشابه تعریف سیستم‌های نوع محض غیروابسته منطقی است که توسط Coquand و Herbelin [۶] ارائه شده است). در ادامه قصد داریم ترجمه‌های مذکور را به سیستم‌های نوع محض به گونه‌ای گسترش دهیم که ویژگی حذف قاعده وابسته را حفظ کند. این ترجمه را می‌توان با ترجمه [۸] ترکیب کرد تا شواهدی را برای حدس BGK ارائه داد.

۲.۳ هدف

طبق ۱۶.۲، می‌دانیم یک سیستم نوع محض لایه‌ای، غیروابسته است؛ هرگاه قوانین آن غیروابسته باشد. یعنی

$$(s, s') \in \mathcal{R} \implies s \geq_A s'$$

ترجمه معرفی شده در این بخش، بخش وابسته یک سیستم نوع محض لایه‌ای را حذف می‌کند. این مورد ما را به سمت تعریف پیش‌رو سوق می‌دهد.

تعریف ۱.۳ (نسخه غیروابسته سیستم نوع محض لایه‌ای). نسخه غیروابسته یک سیستم نوع محض لایه‌ای λS که با λS^* نشان داده می‌شود، سیستمی با مشخصات زیر است

$$S_{\lambda S^*} = S_{\lambda S}$$

$$\mathcal{A}_{\lambda S^*} = \mathcal{A}_{\lambda S}$$

$$\mathcal{R}_{\lambda S^*} = \{(s, s', s') \in \mathcal{R}_{\lambda S} \mid (s \geq s')\}$$

می‌خواهیم یک ترجمه از سیستم نوع محض غیروابسته λS به نسخه غیروابسته آن، یعنی λS^* ارائه کنیم؛ به طوری که خواص حفظ نوع و حفظ مسیرهای کاهش نامتناهی را دارا باشد. الزامات روی ساختار غیروابسته کاملاً قوی خواهند بود.

ترجمه در سطح بالا، استفاده‌های وابسته از قاعده تولید در ۴۲.۱ را حذف می‌کند. اما چون باید مسیرهای کاهش نامتناهی را نیز حفظ کند، نمی‌تواند اطلاعات زیرعبارات را بیش از حد حذف کند؛ زیرا حذف این زیرعبارات ممکن است باعث حذف انتزاع‌هایی شود که مسئول مسیرهای کاهش در عبارت پیش از ترجمه هستند. ایده کلی این است که تمام رده‌ها را به پایین‌ترین رده (s_1) ترجمه کنیم، به طوری که وقتی عبارت M از نوع $\Pi x^A.B$ وجود دارد، شکل ترجمه‌شده آن، که اکنون با $\rho(B)$ نشان داده می‌شود، از رده s_1 است، که تضمین می‌کند هر تشکیل نوع با $\rho(B)$ بعنوان نوع هدف، قطعاً غیروابسته است (زیرا هیچ رده‌ای پایین‌تر از آن وجود ندارد).

مسئله اول این است که عبارات ترجمه‌شده، باید با استفاده از انواع ترجمه‌شده، نوع‌پذیر باشند؛ که نشان می‌دهد نیاز به دو ترجمه جداگانه است، یک ترجمه برای عبارات و یک ترجمه برای انواع. مسئله دوم این است که برخی از انواع، شامل عبارت هستند (مثل $(\lambda x^{s_i}.x).A$ زمانی که $\Gamma \vdash A : s_i$) که نشان می‌دهد به ترجمه‌ای مجزا برای این عبارات نیاز داریم. همچنین فرآیند مذکور باید تا بالاترین رده تکرار شود،

بنابراین ترجمه در دو گام تعریف می‌شود. اول، دنباله‌ای از ترجمه‌ها را تعریف می‌کنیم که در ترجمه ρ_0 برای انواع به پایان می‌رسد. هر ترجمه، رده‌ها را به پایین‌تر نگاشت

می‌کند و نهایتاً به یک نوع 0 (یک متغیر منحصر به فرد) از رده s_1 می‌رسیم. سپس ترجمه γ را برای عبارات تعریف می‌کنیم، به طوری که

$$\forall \Gamma, M, A. \quad \Gamma \vdash M : A \implies \exists \Gamma^*. \Gamma^* \vdash \gamma(M) : \rho_0(A)$$

الزام قوی روی بخش غیروابسته λS از این واقعیت ناشی می‌شود که هر ترجمه بعدی متغیر جدیدی را در زمینه اضافه می‌کند، و با اضافه شدن متغیرهای بیشتر در زمینه، متغیرهای بیشتری نیز باید در انتزاع دخیل شوند تا اطلاعات زیرعبارات از دست نرود. این امر تعریف پیش‌رو را توجیح می‌کند.

تعریف ۲.۳ (سیستم نوع محض لایه‌ای کامل). یک سیستم نوع محض لایه‌ای، (i, j) -کامل^۱ است، اگر قوانین آن شامل

$$\{(s_l, s_k) \mid (l \leq i) \wedge (l \leq k \leq j)\}$$

باشد؛ و کامل است، هرگاه ویژگی بسته بودن زیر را برآورده کند

$$(s_i, s_j) \in \mathcal{R}_{\lambda S} \implies \lambda S \text{ is } (j, i) - full$$

می‌توانیم ببینیم که این تعریف چقدر محدودکننده است. سیستمی که برای $i \geq 2$ و $j \geq 3$ ، (i, j) -کامل است، شامل λU می‌شود و بنابراین ناسازگار خواهد بود. قوی‌ترین سیستم نوع محض n -لایه‌ای کامل سازگار، دارای قوانین زیر است

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_1, s_k) \mid k \in [n]\} \cup \{(s_2, s_k) \mid k \in [n]\}$$

برای باقی این بخش، یک سیستم نوع محض n -لایه‌ای کامل λS را ثابت فرض کنید.

^۱ $(i, j) - full$

۳.۳ ترجمه سطح نوع

تعریف ۳.۳ (تابع ترجمه سطح نوع). برای هر $0 \leq i \leq n$ ، تابع $\rho_i : T_{\geq i+1} \rightarrow T_{i+1}$ به صورت استقرایی زیر تعریف می‌شود.

$$\rho_i(s_j) = \begin{cases} 0 & i = 0 \\ s_i & o.w. \end{cases}$$

$$\rho_i(s^j x) = s^{i+2} x \quad (j \geq i + 2)$$

$$\rho_i(\Pi x^A . B) = \Pi x^{\rho_{\deg A-1}(A)} \dots \Pi x^{\rho_i(A)} . \rho_i(B)$$

$$\rho_i(\lambda x^A . M) = \lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_{i+1}(A)} . \rho_i(M)$$

$$\rho_i(MN) = \rho_i(M) \rho_{\deg N-1}(N) \dots \rho_i(N)$$

که در آن 0 یک متغیر منحصر به فرد است.

تعریف هر ترجمه محدود به مواردی است که برای آن‌ها عباراتی در دامنه مورد نظر وجود دارد. برای مثال، پایه‌ای‌ترین ترجمه $\rho_n : T_{\geq n+1} \rightarrow T_{n+1}$ فقط بر روی $\{s_n\}$ با ضابطه $\rho_n(s_n) = s_n$ تعریف می‌شود و بقیه حالات نادیده گرفته می‌شوند. این ترجمه‌ها به زمینه‌ها نیز به شرح زیر گسترش می‌یابند.

$$\rho_i(\emptyset) = (0 : s_1)$$

$$\rho_i(\Gamma, s^j x : A) = \rho_i(\Gamma), s^j x : \rho_{j-1}(A), \dots, s^{i+1} x : \rho_i(A) \quad (j \geq i + 1)$$

توجه کنید که اگر $i < j$ آنگاه $\rho_j(\Gamma) \subset \rho_i(\Gamma)$ ، این مورد اغلب در ترکیب با لم تضعیف ۴۸.۱ استفاده می‌شود.

سه لم زیر ویژگی‌های کلیدی این ترجمه را توصیف می‌کنند.

نخست اینکه، ترجمه و عمل جانشینی، به نوعی دارای خاصیت جابه‌جایی هستند.

لم ۴.۳ (ترجمه با جانشینی). برای هر $0 \leq i \leq n$ و هر $1 \leq j \leq n$ و هر عبارت A و B ، اگر $A \in T_{\geq i+1}$ و $\deg B = j - 1$ ، آنگاه

$$\rho_i(A[B/s^j x]) = \rho_i(A)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]$$

اثبات. روی i استقرای معکوس می‌زنیم. حکم به وضوح برای ρ_n برقرار است.

برای یک i ثابت، روی ساختار A استقرا می‌زنیم.

• رده

فرض کنید A به فرم s_k باشد ($k \geq i$). از آنجایی که s_k متغیر آزاد ندارد و 0 یک متغیر منحصر به فرد است، آنگاه

$$\begin{aligned}\rho_i(s_k[B/s^j x]) &= \rho_i(s_k) \\ &= \rho_i(s_k)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

• متغیر

فرض کنید A به فرم $s_k y$ باشد ($k \geq i+2$). اگر $j < i+2$ ، آنگاه

$$\rho_i(s_k y[B/s^j x]) = \rho_i(s_k y)$$

اگر $j \geq i+2$ و $s_k y = s^j x$ ، آنگاه

$$\begin{aligned}\rho_i(s^j x[B/s^j x]) &= \rho_i(B) \\ &= s^{i+2} x[\rho_i(B)/s^{i+2} x] \\ &= \rho_i(s^j x)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

اگر $j \geq i+2$ و $s_k y \neq s^j x$ ، آنگاه

$$\begin{aligned}\rho_i(s_k y[B/s^j x]) &= \rho_i(s_k y) \\ &= \rho_i(s_k y)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

• عبارت Π

فرض کنید A به فرم $\Pi y^C.D$ باشد.

(حالت اول)

اگر $\deg C < i+1$ ، آنگاه

$$\begin{aligned}\rho_i((\Pi y^C.D)[B/s^j x]) &= \rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \rho_i(D[B/s^j x])\end{aligned}$$

اگر $j < i+2$ ، آنگاه طبق فرض استقرا داریم

$$\rho_i(D[B/s^j x]) = \rho_i(D) = \rho_i(\Pi y^C.D)$$

و اگر $j \geq i + 2$ ، به طور مشابه داریم

$$\begin{aligned}\rho_i(D[B/s^j x]) &= \rho_i(D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x] \\ &= \rho_i(\Pi y^C.D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

(حالت دوم)

اگر $\deg C \geq i + 1$ ، آنگاه

$$\begin{aligned}\rho_i((\Pi y^C.D)[B/s^j x]) \\ &= \rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C[B/s^j x])} \dots \Pi y^{\rho_i(C[B/s^j x])}.\rho_i(D[B/s^j x])\end{aligned}$$

اگر $j < i + 2$ ، آنگاه طبق فرض استقرا داریم

$$\begin{aligned}\rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C[B/s^j x])} \dots \Pi y^{\rho_i(C[B/s^j x])}.\rho_i(D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C)} \dots \Pi y^{\rho_i(C)}.\rho_i(D) \\ &= \rho_i(\Pi y^C.D)\end{aligned}$$

و اگر $j \geq i + 2$ ، به طور مشابه داریم

$$\begin{aligned}\rho_i(\Pi y^{C[B/s^j x]}.D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C[B/s^j x])} \dots \Pi y^{\rho_i(C[B/s^j x])}.\rho_i(D[B/s^j x]) \\ &= \Pi y^{\rho_{\deg C-1}(C)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]} \dots \Pi y^{\rho_i(C)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]} \\ &\quad .(\rho_i(D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]) \\ &= (\Pi y^{\rho_{\deg C-1}(C)} \dots \Pi y^{\rho_i(C)}.\rho_i(D))[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x] \\ &= \rho_i(\Pi y^C.D)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]\end{aligned}$$

توجه داشته باشید که در اینجا ما به طور ضمنی از این لم استفاده کردیم که اگر

$$\rho_k(C[B/s^j x]) = \rho_k(C)$$

$$\rho_k(C) = \rho_k(C)[\rho_{j-2}(B)/s^j x] \dots [\rho_i(B)/s^{i+2} x]$$

• عبارت λ

فرض کنید A به فرم $\lambda y^C.D$ باشد.

(حالت اول)

اگر $\deg C < i + 2$ ، آنگاه

$$\begin{aligned}\rho_i((\lambda y^C.D)[B/s_j x]) &= \rho_i(\lambda y^{C[B/s_j x]}.D[B/s_j x]) \\ &= \rho_i(D[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

(حالت دوم)

اگر $\deg C \geq i + 2$ ، آنگاه

$$\begin{aligned}\rho_i((\lambda y^C.D)[B/s_j x]) &= \rho_i(\lambda y^{C[B/s_j x]}.D[B/s_j x]) \\ &= \lambda y^{\rho_{\deg C-1}(C[B/s_j x])} \dots \lambda y^{\rho_i(C[B/s_j x])}.\rho_i(D[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

• کاربرد

فرض کنید A به فرم MN باشد.

(حالت اول)

اگر $\deg N < i + 1$ ، آنگاه

$$\begin{aligned}\rho_i((MN)[B/s_j x]) &= \rho_i((M[B/s_j x])(N[B/s_j x])) \\ &= \rho_i(M[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

(حالت دوم)

اگر $\deg N \geq i + 1$ ، آنگاه

$$\begin{aligned}\rho_i((MN)[B/s_j x]) &= \rho_i((M[B/s_j x])(N[B/s_j x])) \\ &= \rho_i(M[B/s_j x])\rho_{\deg N-1}(N[B/s_j x]) \dots \rho_i(N[B/s_j x])\end{aligned}$$

باقی اثبات این حالت، مشابه اثبات مربوط به عبارت Π است.

□

لم بعدی بیان می‌کند که این ترجمه، کاهش‌های β را حفظ می‌کند. این ویژگی برای ترجمه قواعد تبدیل ضروری است؛ چرا که باید بتوانیم پس از اعمال ترجمه، نوع‌های β -هم‌ارز را جایگزین یکدیگر کنیم.

شایان ذکر است که این لم لزوماً به معنای حفظ مسیرهای کاهش نامتناهی توسط ترجمه نیست. دلیل این امر آن است که در جریان ترجمه، اطلاعات مربوط به زیرعبارت‌ها از دست می‌رود؛ برای مثال در مورد یک کاربرد، ممکن است داشته باشیم $\rho_i(MN) = \rho_i(M)$ ، و در نتیجه امیدی به حفظ یک دنباله کاهش نامتناهی مربوط به زیرعبارت N وجود نخواهد داشت.

لم ۵.۳ (حفظ مسیر کاهش). برای هر $0 \leq i \leq n$ و هر عبارت A و B ، اگر $A \in T_{\geq i+1}$ و $A \rightarrow_{\beta} B$ ، آنگاه

$$\rho_i(A) \rightarrow_{\beta} \rho_i(B)$$

اثبات. روی i استقرای معکوس می‌زنیم. حکم به وضوح برای ρ_n برقرار است. برای یک i ثابت، با استفاده از استقرا روی تعریف رابطه کاهش β چندگانه، پیش می‌رویم. در مورد کاهش به فرم

$$\rho_i((\lambda x^A.M)N) \rightarrow_{\beta} \rho_i(M[N/s_{\deg A} x])$$

اگر $\deg N < i+1$ (و $\deg A < i+2$)، آنگاه

$$\begin{aligned} \rho_i((\lambda x^A.M)N) &= \rho_i(\lambda x^A.M) \\ &= \rho_i(M) \\ &= \rho_i(M[N/s_{\deg A} x]) \end{aligned}$$

که تساوی آخر از لم ۴.۳ حاصل شده است.

و اگر $\deg N \geq i+1$ (و $\deg A \geq i+2$)، آنگاه

$$\begin{aligned} \rho_i((\lambda x^A.M)N) &= \rho_i(\lambda x^A.M) \rho_{\deg N-1}(N) \dots \rho_i(N) \\ &= (\lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_{i+1}(A)}. \rho_i(M)) \rho_{\deg N-1}(N) \dots \rho_i(N) \\ &\rightarrow_{\beta} \rho_i(M) [\rho_{\deg N-1}(N)/s_{\deg A} x] \dots [\rho_i(N)/s_{i+2} x] \end{aligned}$$

توجه داشته باشید که در اینجا ما به طور ضمنی از این لم استفاده کردیم که $s_{\deg A} x$ در A ظاهر نمی‌شود، پس

$$\rho_k(A) [\rho_l(N)/s_{l+2} x] = \rho_k(A)$$

به طوری که $i + 1 \leq k \leq \deg A - 1$ و $i \leq l \leq \deg N - 1$. بنابراین هیچیک از کاهش‌های β بعدی، نوع‌ها را در عبارات λ مرتبط تغییر نمی‌دهند.

به سادگی می‌توان بررسی کرد که این ترجمه، کاهش‌های β را نسبت به سازگاری حفظ می‌کند. برای مثال، فرض کنید A به فرم $\Pi x^C.D$ و B به فرم $\Pi x^{C'}.D$ هستند، به طوری که $C \rightarrow_{\beta} C'$.
اگر $\deg C < i + 1$ ، آنگاه

$$\rho_i(\Pi x^C.D) = \rho_i(D) = \rho_i(\Pi x^{C'}.D)$$

اگر $\deg C \geq i + 1$ ، آنگاه

$$\begin{aligned} \rho_i(\Pi x^C.D) &= \Pi x^{\rho_{\deg C - 1}(C)} \dots \Pi x^{\rho_i(C)}. \rho_i(D) \\ &\rightarrow_{\beta} \Pi x^{\rho_{\deg C - 1}(C')} \dots \Pi x^{\rho_i(C')}. \rho_i(D) \\ &= \rho_i(\Pi x^{C'}.D) \end{aligned}$$

که

$$\rho_i(C) \rightarrow_{\beta} \rho_i(C')$$

از طریق استقرا روی تعریف رابطه کاهش β چندگانه و همچنین

$$\rho_k(C) \rightarrow_{\beta} \rho_k(C')$$

برای $k > i$ از طریق استقرا روی i . توجه داشته باشید که به طور ضمنی از این نکته استفاده کردیم که $\deg C = \deg C'$ (۳۶.۲).

□

در نهایت، ترجمه باید خاصیت نوع‌پذیری را حفظ کند. این امر تضمین می‌کند که این ترجمه، هنگام ترکیب با ترجمه عبارات که در بخش بعدی معرفی می‌شود، انواع خوش‌تعریفی را به وجود می‌آورد. به یاد داشته باشید که λS^* همان نسخه غیروابسته λS است.

لم ۶.۳ (حفظ نوع). برای هر $0 \leq i \leq n$ ، اگر Γ یک زمینه و A و B عباراتی باشند که $A \in T_{\geq i+1}$ و $\Gamma \vdash_{\lambda S} A : B$ ، آنگاه

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)$$

اثبات. روی i استقرای معکوس می‌زنیم. حکم به وضوح برای ρ_n برقرار است. برای یک i ثابت، با استفاده از استقرا روی استنتاج نوع پیش می‌رویم.

• اصل

فرض کنید استنتاج به فرم

$$\vdash_{\lambda S} s_j : s_{j+1}$$

باشد که $j \geq i$. اگر $i \geq 1$ آنگاه حاصل ترجمه معادل است با

$$0 : s_1 \vdash_{\lambda S^*} s_i : s_{i+1}$$

و در غیر این صورت

$$0 : s_1 \vdash_{\lambda S^*} 0 : s_1$$

از آنجایی که هر سیستم کامل، شامل رابطه (s_1, s_2) است، این قضاوت قابل استنتاج است.

• شروع

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma' \vdash_{\lambda S} B : s_j}{\Gamma', {}^{s_j}x : B \vdash_{\lambda S} {}^{s_j}x : B}$$

باشد که $j \geq i + 2$ (پس $\deg({}^{s_j}x) \geq i + 1$). طبق فرض استقرار، برای هر $i \leq k \leq j - 1$ داریم

$$\rho_{k+1}(\Gamma') \vdash_{\lambda S^*} \rho_k(B) : s_{k+1}$$

و آنجا که زمینه $\rho_{i+1}(\Gamma')$ معتبر است، با استفاده از تضعیف، داریم

$$\rho_{i+1}(\Gamma') \vdash_{\lambda S^*} \rho_k(B) : s_{k+1}$$

اکنون با استفاده مکرر از تضعیف می‌توانیم بنویسیم

$$\rho_{i+1}(\Gamma'), {}^{s_j}x : \rho_{j-1}(B), \dots, {}^{s_{i+3}}x : \rho_{i+2}(B) \vdash_{\lambda S^*} \rho_{i+1}(B) : s_{i+2}$$

در نهایت با استفاده از قاعده شروع داریم

$$\rho_{i+1}(\Gamma'), {}^{s_j}x : \rho_{j-1}(B), \dots, {}^{s_{i+2}}x : \rho_{i+1}(B) \vdash_{\lambda S^*} {}^{s_{i+2}}x : \rho_{i+1}(B)$$

• تضعیف

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma' \vdash_{\lambda S} A : B \quad \Gamma' \vdash_{\lambda S} C : s_j}{\Gamma', {}^{s_j}x : C \vdash_{\lambda S} A : B}$$

باشد.

اگر $j < i + 2$ ، با اعمال فرض استقرا بر حکم مقدم سمت چپ داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)$$

دقت کنید که متغیر ${}^{s_j}x$ توسط ترجمه ρ_{i+1} حذف می‌شود.

اگر $j \geq i + 2$ ، از روشی مشابه آنچه در حالت شروع در بالا ذکر شد، استفاده می‌کنیم؛ یعنی برای هر k که $i \leq k \leq j - 1$ و سپس با استفاده مکرر از تضعیف

$$\rho_{i+1}(\Gamma'), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{i+2}}x : \rho_{i+1}(C) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)$$

• تولید

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma \vdash_{\lambda S} C : s_j \quad \Gamma, {}^{s_j}x : C \vdash_{\lambda S} D : s_k}{\Gamma \vdash_{\lambda S} (\Pi x^C.D) : s_k}$$

باشد که $k \geq i + 1$.

اگر $j < i + 1$ ، با اعمال فرض استقرا بر حکم مقدم سمت راست، داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(D) : s_{i+1}$$

اگر $j \geq i + 1$ ، برای هر k که $i \leq k \leq j - 1$ داریم

$$\rho_{k+1}(\Gamma) \vdash_{\lambda S^*} \rho_k(C) : s_{k+1}$$

و هر یک از این استنتاج‌ها را می‌توان به صورت زیر تضعیف کرد

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{k+2}}x : \rho_{k+1}(C) \vdash_{\lambda S^*} \rho_k(C) : s_{k+1}$$

سپس با استفاده مکرر از قاعده تولید به همراه

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{i+2}}x : \rho_{i+1}(C) \vdash_{\lambda S^*} \rho_i(D) : s_{i+1}$$

می‌توانیم بنویسیم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \Pi x^{\rho_{j-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}.(\rho_i(C) \rightarrow \rho_i(D)) : s_{i+1}$$

اینجا از این فرض استفاده می‌کنیم که λS^* برای هر $k \geq i+1$ دارای قوانین (s_k, s_{i+1}, s_{i+1}) است. همچنین در مورد استفاده از نمادگذاری تابع توجه کنید که عبارت $\rho_i(C)$ در زمینه پیشین ظاهر نشده است.

• انتزاع

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma, {}^{s_j}x : C \vdash_{\lambda S} M : D \quad \Gamma \vdash_{\lambda S} \Pi x^C.D : s_k}{\Gamma \vdash_{\lambda S} (\lambda x^C.M) : (\Pi x^C.D)}$$

باشد که $k \geq i+2$.

اگر $\deg C < i+2$ (یعنی اگر $j < i+2$)، از آنجا که متغیر ${}^{s_j}x$ توسط ترجمه ρ_{i+1} حذف می‌شود، استنتاج متناظر عبارت است از

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(B)$$

اگر $\deg C \geq i+2$ ، می‌توانیم بنویسیم

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{i+2}}x : \rho_{i+1}(C) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(D)$$

و

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \Pi x^{\rho_{j-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D) : s_{i+2}$$

بنابراین اگر $\deg C \geq i+3$ ، با استفاده مکرر از لم تولید (۴۵.۱) می‌توانیم برای هر $i+1 \leq k \leq j-2$ نتیجه بگیریم

$$\rho_{i+1}(\Gamma), {}^{s_j}x : \rho_{j-1}(C), \dots, {}^{s_{k+2}}x : \rho_{k+1}(C) \vdash_{\lambda S^*} \Pi x^{\rho_k(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D) : s_{i+2}$$

و بدین ترتیب، با استفاده مکرر از قاعده انتزاع، می‌توانیم استنتاج کنیم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} (\lambda x^{\rho_{j-1}(C)} \dots \lambda x^{\rho_{i+1}(C)}. \rho_i(M)) : (\Pi x^{\rho_{j-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D))$$

• کاربرد

فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma \vdash_{\lambda S} M : (\Pi x^C.D) \quad \Gamma \vdash_{\lambda S} N : C}{\Gamma \vdash_{\lambda S} MN : D[N/x]}$$

باشد.

چون $\deg(MN) = \deg M$ ، با اعمال فرض استقرا بر حکم مقدم سمت چپ داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(\Pi x^C.D)$$

اگر $\deg N < i + 1$ (و $\deg C < i + 2$)، آنگاه

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \rho_{i+1}(D)$$

اگر $\deg N \geq i + 1$ (و $\deg C \geq i + 2$)، آنگاه داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M) : \Pi x^{\rho_{\deg C-1}(C)} \dots \Pi x^{\rho_{i+1}(C)}. \rho_{i+1}(D)$$

و برای هر $i \leq k \leq \deg N - 1$

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_k(N) : \rho_{k+1}(C)$$

اولین کاربرد $\rho_i(M)\rho_{\deg N-1}(N)$ دارای نوع زیر است

$$\Pi x^{\rho_{\deg C-2}(C)[\rho_{\deg C-2}(N)/^{s_{\deg C} x}]} \dots \Pi x^{\rho_{i+1}(C)[\rho_{\deg C-2}(N)/^{s_{\deg C} x}]} . (\rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}])$$

عبارت $\rho_{\deg C-2}(C)$ متغیر $^{s_{\deg C} x}$ را ندارد، لذا این نوع ساده می شود به

$$\Pi x^{\rho_{\deg C-2}(C)} \dots \Pi x^{\rho_{i+1}(C)}. (\rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}])$$

با تکرار این فرآیند، داریم

$$\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(M)\rho_{j-1}(N) \dots \rho_i(N) : \rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}] \dots [\rho_i(N)/^{s_{i+2} x}]$$

از آنجایی که $^{s_{i+2} x}$ در $\rho_{i+1}(D)$ ظاهر نمی شود، نوع مذکور معادل است با

$$\rho_{i+1}(D)[\rho_{\deg N-1}(N)/^{s_{\deg C} x}] \dots [\rho_{i+1}(N)/^{s_{i+3} x}]$$

که طبق لم ۴.۳ معادل $\rho_{i+1}(D[N/x])$ است.

- تبدیل
فرض کنید آخرین استنتاج به فرم

$$\frac{\Gamma \vdash_{\lambda S} A : C \quad \Gamma \vdash_{\lambda S} B : s_k}{\Gamma \vdash_{\lambda S} A : B}$$

است که $k \geq i + 2$. آنگاه استنتاج متناظر به صورت زیر است

$$\frac{\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(C) \quad \frac{\rho_{i+2}(\Gamma) \vdash_{\lambda S^*} \rho_{i+1}(B) : s_{i+2}}{\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_{i+1}(B) : s_{i+2}}}{\rho_{i+1}(\Gamma) \vdash_{\lambda S^*} \rho_i(A) : \rho_{i+1}(B)}$$

که در آن طبق لم ۵.۳ داریم: $\rho_{i+1}(B) = \rho_{i+1}(C)$

□

۴.۳ ترجمه سطح عبارت

در ادامه تابع ترجمه $\gamma : T \rightarrow T_0$ را بر روی همه عبارات تعریف می‌کنیم. این ترجمه، یک تفاوت اصلی با ترجمه Geuvers-Nederhof [۹] دارد. در ترجمه آن‌ها، متغیرها گاهی از طریق افزودن عبارات $\perp$ به زمینه، با عبارت‌های ساختگی جایگزین می‌شدند. از آنجا که $(s_2, s_1) \in \mathcal{R}_{\lambda\omega}$ ، مورد مذکور شدنی است. استفاده مستقیم از این تکنیک، به قوانین برای (s_{i+1}, s_i) نیاز دارد. با این حال، به دلیل کامل بودن، $(s_j, s_k) \in \mathcal{R}_{\lambda S}$ نیازمند این است که برای $l \in [k]$ داشته باشیم $(s_l, s_1) \in \mathcal{R}_{\lambda S}$ ، که به ما اجازه استفاده از متغیر $\text{prod} : \Pi A^{s_1}.0 \rightarrow A \rightarrow 0$ که فقط نیازمند قانون (s_2, s_1) است را می‌دهد. ما فرض می‌کنیم که این قانون در λS موجود است.

تعریف ۷.۳ (تابع ترجمه سطح عبارت). تابع $\gamma : T \rightarrow T$ به صورت استقرایی زیر تعریف می‌شود.

$$\begin{aligned} \gamma(s_i) &= \bullet \\ \gamma(s^i x) &= s^1 x \\ \gamma(\Pi x^A.B) &= \text{prod}(\Pi x^{\rho_{deg A-1}(A)} \dots \Pi x^{\rho_0(A)}.0) \gamma(A) (\lambda x^{\rho_{deg A-1}(A)} \dots \lambda x^{\rho_0(A)}. \gamma(B)) \\ \gamma(\lambda x^A.M) &= (\lambda z^0. \lambda x^{\rho_{deg A-1}(A)} \dots \lambda x^{\rho_0(A)}. \gamma(M)) \gamma(A) \\ \gamma(MN) &= \gamma(M) \rho_{deg N-1}(N) \dots \rho_0(N) \gamma(N) \end{aligned}$$

که در آن $\{\text{prod}, \bullet, z\}$ متغیرهای منحصر به فردی هستند.

سه لمی که در بخش قبل بررسی کردیم، برای ترجمه سطح عبارت نیز قابل بیان هستند.

لم ۸.۳ (حفظ نوع). برای هر عبارت A و B ، اگر $\Gamma \vdash_{\lambda S} A : B$ ، آنگاه

$$0 : s_1, \bullet : 0, \text{prod} : \Pi A^{s_1}. 0 \rightarrow A \rightarrow 0, \rho_0(\Gamma) \vdash_{\lambda S^*} \gamma(A) : \rho_0(B)$$

اثبات. بر روی استنتاج‌ها استقرا می‌زنیم. مواردی که در آن‌ها آخرین گام استنتاج، شروع یا تضعیف است، مشابه موارد متناظر در لم ۶.۳ هستند. در ادامه، فرض کنید

$$\Delta := (0 : s_1, \bullet : 0, \text{prod} : \Pi A^{s_1}. 0 \rightarrow A \rightarrow 0)$$

• اصل

اگر استنتاج به فرم

$$\vdash s_i : s_{i+1}$$

باشد، آنگاه استنتاج ترجمه‌شده به شکل زیر است

$$\Delta \vdash \bullet : 0$$

که به وضوح برقرار است.

• تولید

فرض کنید آخرین استنتاج به شکل زیر باشد

$$\frac{\Gamma \vdash C : s_i \quad \Gamma, {}^{s_i}x : C \vdash D : s_j}{\Gamma \vdash (\Pi x^C. D) : s_j}$$

بنا بر فرض استقرا

$$\Delta, \rho_0(\Gamma) \vdash \gamma(C) : 0$$

$$\Delta, \rho_0(\Gamma), {}^{s_i}x : \rho_{i-1}(C), \dots {}^{s_1}x : \rho_0(C) \vdash \gamma(D) : 0$$

به کمک تضعیف، داریم

$$\Delta, \rho_0(\Gamma), {}^{s_i}x : \rho_{i-1}(C), \dots {}^{s_1}x : \rho_0(C) \vdash 0 : s_1$$

توجه داشته باشید که به دلیل کامل بودن، از آنجا که $(s_i, s_j) \in \mathcal{R}_{\lambda\mathcal{S}}$ ، آنگاه

$$\forall k \in [i], (s_k, s_1) \in \mathcal{R}_{\lambda\mathcal{S}}$$

بدین ترتیب، با استفاده مکرر از تولید، که همگی با استفاده از ترجمه‌های از پیش تعریف شده صورت می‌گیرد؛

$$\rho_{k+1}(\Gamma) \vdash \rho_k(C) : s_{k+1}$$

می‌توانیم برای هر $1 \leq k \leq i$ بنویسیم

$$\Delta, \rho_0(\Gamma), {}^{s_i}x : \rho_{i-1}(C), \dots {}^{s_{k+1}}x : \rho_k(C) \vdash \Pi {}^{s_k}x^{\rho_{k-1}(C)} \dots \Pi {}^{s_1}x^{\rho_0(C)}.0 : s_1$$

همچنین

$$\Delta, \rho_0(\Gamma) \vdash \lambda {}^{s_i}x^{\rho_{i-1}(C)} \dots \lambda {}^{s_1}x^{\rho_0(C)}. \gamma(D) : 0$$

در نهایت با دو کاربرد با استفاده از استنتاج‌های تعریف شده‌ی قبلی، داریم

$$\Delta, \rho_0(\Gamma) \vdash \text{prod}(\Pi {}^{s_i}x^{\rho_{i-1}(C)} \dots \Pi {}^{s_1}x^{\rho_0(C)}.0) \gamma(A) (\lambda {}^{s_i}x^{\rho_{i-1}(C)} \dots \lambda {}^{s_1}x^{\rho_0(C)}. \gamma(D)) : 0$$

• انتزاع

فرض کنید آخرین استنتاج به فرم زیر باشد

$$\frac{\Gamma, {}^{s_i}x : C \vdash M : D \quad \Gamma \vdash \Pi x^C.D : s_j}{\Gamma \vdash \lambda x^C.M : \Pi x^C.D}$$

مشابه مورد متناظر در اثبات لم ۶.۳، داریم

$$\rho_0(\Gamma) \vdash (\lambda x^{\rho_{i-1}(C)} \dots \lambda x^{\rho_0(C)}. \gamma(D)) : (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D))$$

و به کمک تضعیف، برای یک متغیر جدید z داریم

$$\rho_0(\Gamma), z : 0 \vdash (\lambda x^{\rho_{i-1}(C)} \dots \lambda x^{\rho_0(C)}. \gamma(D)) : (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D))$$

بنابراین

$$\frac{\rho_0(\Gamma) \vdash (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D)) : s_1 \quad \rho_0(\Gamma) \vdash 0 : s_1}{\rho_0(\Gamma) \vdash (0 \rightarrow \Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D)) : s_1}$$

پس با استفاده از انتزاع و کاربرد می‌توانیم بنویسیم

$$\rho_0(\Gamma) \vdash ((\lambda z^0. \lambda x^{\rho_{i-1}(C)} \dots \lambda x^{\rho_0(C)}. \gamma(D)) \gamma(C)) : (\Pi x^{\rho_{i-1}(C)} \dots \Pi x^{\rho_0(C)}. \rho_0(D))$$

• کاربرد

فرض کنید آخرین استنتاج به فرم زیر باشد

$$\frac{\Gamma \vdash M : (\Pi x^C. D) \quad \Gamma \vdash N : C}{\Gamma \vdash MN : D[N/s_{\deg C} x]}$$

اگر $\deg N = 0$ ، آنگاه داریم

$$\frac{\rho_0(\Gamma) \vdash \gamma(M) : \rho_0(C) \rightarrow \rho_0(D) \quad \rho_0(\Gamma) \vdash \gamma(N) : \rho_0(C)}{\rho_0(\Gamma) \vdash \gamma(M)\gamma(N) : \rho_0(D)}$$

توجه کنید که به کمک لم ۴.۳، از این نکته استفاده کردیم که $\rho_0(D[N/s_1 x]) = \rho_0(D)$ حالتی که $\deg N \geq 1$ باشد، مشابه حالت متناظر در اثبات لم ۶.۳ است.

• تبدیل

فرض کنید آخرین استنتاج به فرم زیر باشد

$$\frac{\Gamma \vdash A : C \quad \Gamma \vdash B : s_i}{\Gamma \vdash A : B}$$

طبق لم ۵.۳، می‌دانیم $\rho_0(C) = \rho_0(B)$. پس داریم

$$\frac{\rho_0(\Gamma) \vdash \gamma(A) : \rho_0(C) \quad \frac{\rho_1(\Gamma) \vdash \rho_0(B) : s_1}{\rho_0(\Gamma) \vdash \rho_0(B) : s_1}}{\rho_0(\Gamma) \vdash \gamma(A) : \rho_0(B)}$$

□

در ادامه، مجدد نشان می‌دهیم که عمل جانشینی با ترجمه جابه‌جا می‌شود. این نتیجه بیشتر جنبه کمکی دارد، چرا که برای لم بعدی که در مورد حفظ کاهش β است، به آن نیازمندیم.

لم ۹.۳ (ترجمه با جانشینی). برای هر $1 \leq j \leq n$ ، و هر عبارت A و B که $\deg(B) = j - 1$ ، داریم

$$\gamma(A[B/s_j^j x]) = \gamma(A)[\rho_{j-2}(B)/s_j^j x] \dots [\rho_0(B)/s_2^2 x][\gamma(B)/s_1^1 x]$$

اثبات. روی ساختار A استقرا می‌زنیم. حالاتی که در آن‌ها، A یک رده یا یک متغیر است، ساده هستند.

• عبارت Π

فرض کنید A به فرم $\Pi y^C.D$ باشد. در ادامه از نمادگذاری مشابه [۲] استفاده می‌کنیم:

$$M^+ \equiv M[B/s^j x]$$

داریم

$$\begin{aligned} & \gamma((\Pi y^C.D)^+) \\ &= \gamma(\Pi y^{C^+}.D^+) \\ &= \text{prod}(\Pi^{s_{\deg(C^+)}} x^{\rho_{\deg(C^+)-1}(C^+)} \dots \Pi^{s_1} x^{\rho_0(C^+)}.0) \\ & \quad \gamma(A)(\lambda^{s_{\deg(C^+)}} x^{\rho_{\deg(C^+)-1}(C^+)} \dots \lambda^{s_1} x^{\rho_0(C^+)}. \gamma(D^+)) \end{aligned}$$

از آنجا که تمام جانشینی‌ها جابه‌جا می‌شوند، چه بر اساس فرض استقرا و چه از طریق **لم ۴.۳**، حکم برقرار است.

• حالات عبارت λ و کاربرد، به شیوه مشابه اثبات می‌شوند.

□

لم ۱۰.۳ (حفظ مسیر کاهش). برای هر عبارت A و B ، اگر $A \rightarrow_\beta B$ ، آنگاه

$$\gamma(A) \twoheadrightarrow_\beta^+ \gamma(B)$$

اثبات. روی تعریف رابطه کاهش β چندگانه، استقرا می‌زنیم. درمورد کاهش به فرم

$$(\lambda x^A.M)N \rightarrow_\beta M[N/s^{\deg A} x]$$

اگر $\deg N = 0$ (و $\deg A = 1$)، آنگاه

$$\begin{aligned} \gamma((\lambda x^A.M)N) &= \gamma(\lambda x^A.M)\gamma(N) \\ &= (\lambda z^0.\lambda x^{\rho_0(A)}. \gamma(M))\gamma(A)\gamma(N) \\ &\rightarrow_\beta (\lambda x^{\rho_0(A)}. \gamma(M))\gamma(N) \\ &\rightarrow_\beta \gamma(M)[\gamma(N)/s^1 x] \\ &= \gamma(M[N/s^1 x]) \end{aligned}$$

اگر $\deg N \geq 1$ ، آنگاه

$$\begin{aligned}
 \gamma((\lambda x^A.M)N) &= \gamma(\lambda x^A.M)\rho_{\deg N-1}(N) \dots \rho_0(N)\gamma(N) \\
 &= (\lambda z^0.\lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_0(A)}.\gamma(M))\gamma(A)\rho_{\deg N-1}(N) \dots \rho_0(N)\gamma(N) \\
 &\rightarrow_{\beta} (\lambda x^{\rho_{\deg A-1}(A)} \dots \lambda x^{\rho_0(A)}.\gamma(M))\rho_{\deg N-1}(N) \dots \rho_0(N)\gamma(N) \\
 &\rightarrow_{\beta} \gamma(M)[\rho_{\deg N-1}(N)/^{s_{\deg A}}x] \dots [\rho_0(N)/^{s_2}x][\gamma(N)/^{s_1}x] \\
 &= \gamma(M[N/^{s_{\deg A}}x])
 \end{aligned}$$

مشابه اثبات لم ۵.۳، اثبات اینکه ترجمه، کاهش‌های β را از نظر سازگاری حفظ می‌کند، ساده و سرراست است. نکته کلیدی این است که γ بر تک تک زیرعبارت‌های عبارت پیش‌تر ترجمه‌شده اعمال می‌شود؛ بنابراین، سازگاری، کاهش‌های β غیرصفر را حفظ می‌کند.

□

۵.۳ نتیجه اصلی

قضیه ۱۱.۳. فرض کنید λS یک سیستم نوع محض n -لایه‌ای است که تا یک مقدار i کامل است ($i < n$). λS دارای خاصیت نرمال‌سازی قوی است، اگر و تنها اگر λS^* چنین باشد.

اثبات. از آنجایی که همه عبارات قابل استنتاج λS^* ، در λS نیز استنتاج می‌شوند، اگر λS دارای خاصیت نرمال‌سازی قوی باشد آنگاه λS^* نیز دارای این خاصیت خواهد بود. حال فرض کنید λS دارای خاصیت نرمال‌سازی قوی نباشد. پس یک دنباله کاهش نامتناهی مانند

$$M_1 \rightarrow_{\beta} M_2 \rightarrow_{\beta} \dots$$

در آن وجود دارد. طبق لم‌های ۸.۳ و ۱۰.۳، می‌توان گفت دنباله کاهش نامتناهی

$$\gamma(M_1) \twoheadrightarrow_{\beta}^+ \gamma(M_2) \twoheadrightarrow_{\beta}^+ \dots$$

در λS^* یافت می‌شود، که با فرض آنکه λS^* دارای خاصیت نرمال‌سازی قوی است، در تناقض است.

□

تنها سیستم واقعا جالبی که این قضیه بر آن قابل اعمال است، سیستم n -لایه‌ای با قوانین زیر است

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_1, s_k) \mid k \in [n]\} \cup \{(s_2, s_k) \mid k \in [n]\}$$

و همچنین هر زیرسیستمی از آن که دارای همان قوانین غیروابسته باشد. قضیه ۱۱.۳ بیان می‌کند که خاصیت نرمال‌سازی قوی برای این سیستم، معادل همین خاصیت برای سیستم با قوانین زیر است.

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_2, s_2)\}$$

هرچند می‌توان این نتیجه را به صورت صوری نیز اثبات کرد، بررسی اجمالی این روش نشان می‌دهد که از دیدگاه فرانظری، این روش ضعیف است و می‌توان آن را در حساب پئانو اجرا کرد. از این رو، با ترکیب این نتیجه با نتیجه‌ی [۸]، به حالت خاصی از حدس قوی BGK دست می‌یابیم.

نتیجه ۱۲.۳. در سیستم نوع محض n -لایه‌ای با قوانین

$$\{(s_k, s_1) \mid k \in [n]\} \cup \{(s_1, s_k) \mid k \in [n]\} \cup \{(s_2, s_k) \mid k \in [n]\}$$

نرمال‌سازی ضعیف مستلزم نرمال‌سازی قوی است. همچنین، این اثبات را می‌توان در حساب پئانو انجام داد.

بخش سوم

نتایج و پیشنهادات آینده

فصل ۴

نتیجه‌گیری و جمع‌بندی

۱.۴ جمع‌بندی

در این پایان‌نامه، سیستم‌های نوع محض و تعمیم لایه‌ای آن‌ها، یعنی سیستم‌های نوع محض لایه‌ای، بر اساس رساله دکتری Mull به‌طور کامل در اثبات‌یار Coq صوری‌سازی شدند. نحو عبارات، جانشینی، کاهش β ، نرمال‌سازی ضعیف و قوی، و استنتاج نوع برای هر دو سیستم به همراه لم‌های پایه‌ای مرتبط با آن‌ها صوری‌سازی و اثبات شدند. در بخش نهایی کار، که به اثبات لم‌های مربوط به ترجمه حذف‌کننده وابستگی اختصاص دارد، به دلیل محدودیت زمانی، برخی از گام‌های اثبات این لم‌ها به صورت Axiom در Coq فرض شده‌اند. تکمیل اثبات صوری این موارد، طبیعی‌ترین ادامه این کار است.

۲.۴ پیشنهادات

Mull در جمع‌بندی رساله خود [۱۰]، تعمیم ترجمه Geuvers–Nederhof [۹] به سیستم‌های ناپیش‌گویانه^۱ رده بالاتر را «یک آزمایش ناموفق در تعمیم‌دهی» می‌نامد. به گفته او، تمام تلاش‌ها برای تضعیف شرایط بخش غیروابسته سیستم پایه، معمولاً به دلیل عدم بسته بودن تحت جانشینی یا تساوی β با شکست مواجه شده‌اند. او این احتمال را مطرح می‌کند که شاید بتوان با تعدیلی جزئی در روش اثبات، همان نتیجه را برای سیستم‌های n -لایه‌ای با مجموعه قوانین

$$\{(s_1, s_k) \mid k \in [n]\} \cup \{(s_i, s_j) \mid 1 \leq i \leq j \leq n\}$$

به دست آورد؛ چرا که این مجموعه قوانین، نقش نتیجه Geuvers–Nederhof را بهتر بازتاب

^۱impredicative

می‌دهد. این پرسش، به‌عنوان یک مسئله باز، هنوز پاسخ داده نشده است و می‌تواند موضوع پژوهش‌های آتی باشد.
بر این اساس، به‌عنوان جهت‌های پژوهشی آتی پیشنهاد می‌شود:

- بررسی دقیق‌تر مجموعه قوانین پیشنهادی Mull $\{(s_1, s_k)\} \cup \{(s_i, s_j) \mid i \leq j\}$ و تلاش برای یافتن تعدیلی در اثبات ترجمه حذف وابستگی که بتواند نتیجه اصلی رساله را برای این مجموعه قوانین نیز اثبات کند.
- بررسی این‌که آیا استدلال فرانظری Mull درباره قابل‌انجام بودن اثبات قضیه ۱۱.۳ در حساب پنانو، می‌تواند به‌طور دقیق و صوری تنظیم و اثبات شود؛ و در صورت امکان، صوری‌سازی این استدلال در Coq.
- صوری‌سازی کامل نتیجه ۱۲.۳ (تناظر نرمال‌سازی ضعیف و قوی برای سیستم n -لایه با قوانین $\{(s_k, s_1)\} \cup \{(s_1, s_k)\} \cup \{(s_2, s_k)\}$ در Coq، به‌عنوان یک حالت خاص از حدس قوی BGK، به‌ویژه با تکیه بر صوری‌سازی موجود این پایان‌نامه از PTS و TPTS.
- تکمیل و جایگزینی Axiom های به‌کاررفته در این پایان‌نامه با اثبات‌های صوری کامل، به‌منظور دستیابی به یک صوری‌سازی کاملاً بدون فرض اضافی از نتایج Mull.

مراجع

- [1] H. Barendregt. Introduction to generalized type systems. *Journal of Functional Programming*, 1(2):125–154, 1991.
- [2] H. Barendregt. *Lambda Calculi with Types*. Cambridge University Press, 2013.
- [3] S. Berardi. Towards a mathematical analysis of the Coquand–Huet calculus of constructions and the other systems in Barendregt’s cube. PhD thesis, University of Torino and Carnegie Mellon University, 1988.
- [4] A. Church. A set of postulates for the foundation of logic. *Annals of Mathematics*, 33(2):346–366, 1932.
- [5] A. Church. A formulation of the simple theory of types. *Journal of Symbolic Logic*, 5(2):56–68, 1940.
- [6] T. Coquand and H. Herbelin. A-translation and looping combinators in pure type systems. *Journal of Functional Programming*, 4(1):77–88, 1994.

- [7] H. Geuvers. Logics and Type Systems. PhD thesis, University of Nijmegen, 1993.
- [8] M. H. S. Gilles Barthe, John Hatcliff. Weak normalization implies strong normalization in a class of non-dependent pure type systems. *Theoretical Computer Science*, 269(1–2):317–361, 2001.
- [9] M.-J. N. Herman Geuvers. Modular proof of strong normalization for the calculi of constructions. *Journal of Functional Programming*, 1(2):155–189, 1991.
- [10] N. Mull. Weak and String Normalization of Tiered Pure Type Systems via Type-Preserving Translation. PhD thesis, The University of Chicago, 2023.
- [11] M. H. A. Newman. On theories with a combinatorial definition of equivalence. *Annals of Mathematics*, 43(2):223–243, 1942.
- [12] F. H. Robert Harper and G. Plotkin. A framework for defining logics. *Journal of the ACM*, 40(1):143–184, 1993.
- [13] C. Roux and F. van Doorn. The structural theory of pure type systems. In *Rewriting and Typed Lambda Calculi*, pages 364–378. Springer, 2014.
- [14] J. Terlouw. Een nadere bewijstheoretische analyse van gsts. Technical report, University of Nijmegen, 1989.

پيوست: كدهای COQ

مشخصه سیستم‌های نوع محض

```
1 Module Type PTS_SIGNATURE.  
2   Parameter Sort : Type.  
3   Parameter A : Sort -> Sort -> Prop.  
4   Parameter R : Sort -> Sort -> Sort -> Prop.  
5 End PTS_SIGNATURE.
```

مشخصه سیستم‌های نوع محض لایه‌ای

```
1 Require Import Thesis.PTSSignature.  
2  
3 Module Type TPTS_SIGNATURE <: PTS_SIGNATURE.  
4   Include PTS_SIGNATURE.  
5  
6   Parameter n : nat.  
7   Parameter index_of : Sort -> nat.  
8  
9   Axiom n_range : n > 0.  
10  Axiom index_range : forall x, 0 < index_of x /\ index_of x < n + 1.  
11  Axiom index_inj : forall x y, index_of x = index_of y -> x = y.  
12  Axiom index_surj : forall i : nat, 0 < i /\ i < n + 1 -> exists x : Sort,  
    index_of x = i.  
13  
14  Axiom A_spec : forall x y, A x y <-> index_of x < n /\ index_of y = S  
    (index_of x).  
15  Axiom R_shape : forall x y z, R x y z -> y = z.  
16 End TPTS_SIGNATURE.
```

سیستم نوع محض و لم‌ها

```
1 From Stdlib Require Import List.  
2 From Stdlib Require Import Classical.
```

```

3  From Stdlib Require Import Logic.IndefiniteDescription.
4  From Stdlib.Program Require Import Program Wf.
5  From Stdlib Require Import Lia.
6  From Stdlib Require Import Arith.Arith.
7  From Stdlib Require Import ZArith.
8  Import ListNotations.
9
10 Require Import Thesis.PTSSignature.
11
12 Module PTS (Sig : PTS_SIGNATURE).
13
14   Import Sig.
15
16   (* ===== *)
17   (** * Syntax *)
18
19   Definition var := nat.
20
21   Definition eq_var_dec : forall x y : var, {x = y} + {x <> y} := Nat.eq_dec.
22
23   Inductive term : Type :=
24     | t_sort : Sort -> term
25     | t_bvar : Sort -> nat -> term
26     | t_fvar : Sort -> var -> term
27     | t_app  : term -> term -> term
28     | t_lam  : Sort -> term -> term -> term
29     | t_pi   : Sort -> term -> term -> term.
30
31   (* ===== *)
32   (** * Free variables *)
33
34   Fixpoint fv (t : term) : list var :=
35     match t with
36     | t_sort _    => []
37     | t_bvar _ _ => []
38     | t_fvar _ x  => [x]
39     | t_pi _ A B  => fv A ++ fv B
40     | t_lam _ A M => fv A ++ fv M
41     | t_app M N   => fv M ++ fv N
42   end.
43
44   (* ===== *)
45   (** * Opening *)
46
47   Fixpoint open_rec (k : nat) (u : term) (t : term) : term :=
48     match t with
49     | t_sort s    => t_sort s
50     | t_bvar s n  => if Nat.eqb n k then u else t_bvar s n
51     | t_fvar s x  => t_fvar s x
52     | t_app M N   => t_app (open_rec k u M) (open_rec k u N)

```

```

53 | t_pi s A B => t_pi s (open_rec k u A) (open_rec (S k) u B)
54 | t_lam s A M => t_lam s (open_rec k u A) (open_rec (S k) u M)
55 end.
56
57 Notation "t ^^ u" := (open_rec 0 u t) (at level 67).
58
59 Definition open_var (t : term) (s : Sort) (x : var) : term := open_rec 0 (t_fvar
s x) t.
60
61 (* ===== *)
62 (** * Closing *)
63
64 Fixpoint close_rec (k : nat) (x : var) (t : term) : term :=
65   match t with
66   | t_sort s    => t_sort s
67   | t_bvar s n  => t_bvar s n
68   | t_fvar s y  => if eq_var_dec y x then t_bvar s k else t_fvar s y
69   | t_app M N   => t_app (close_rec k x M) (close_rec k x N)
70   | t_pi s A B  => t_pi s (close_rec k x A) (close_rec (S k) x B)
71   | t_lam s A M => t_lam s (close_rec k x A) (close_rec (S k) x M)
72   end.
73
74 Definition close (x : var) (t : term) : term := close_rec 0 x t.
75
76 (* ===== *)
77 (** * Substitution for a free variable *)
78
79 Reserved Notation "M { x N }"
80   (at level 1, left associativity).
81
82 Fixpoint subst_term (x : var) (N : term) (t : term) : term :=
83   match t with
84   | t_sort s    => t_sort s
85   | t_bvar s n  => t_bvar s n
86   | t_fvar s y  => if eq_var_dec y x then N else t_fvar s y
87   | t_app P Q   => t_app (P {x N}) (Q {x N})
88   | t_pi s A B  => t_pi s (A {x N}) (B {x N})
89   | t_lam s A M => t_lam s (A {x N}) (M {x N})
90   end
91
92 where "M { x N }" := (subst_term x N M).
93
94 Lemma subst_sort : forall s x N,
95   (t_sort s) {x N} = t_sort s.
96 Proof. reflexivity. Qed.
97
98 Lemma subst_bvar : forall s n x N,
99   (t_bvar s n) {x N} = t_bvar s n.
100 Proof. reflexivity. Qed.
101

```

```

102 Lemma subst_var : forall s y x N,
103   (t_fvar s y){x N} = if eq_var_dec y x then N else t_fvar s y.
104 Proof. reflexivity. Qed.
105
106 Lemma subst_app : forall M1 M2 x N,
107   (t_app M1 M2){x N} = t_app (M1{x N}) (M2{x N}).
108 Proof. reflexivity. Qed.
109
110 Lemma subst_pi : forall s A B x N,
111   (t_pi s A B){x N} = t_pi s (A{x N}) (B{x N}).
112 Proof. reflexivity. Qed.
113
114 Lemma subst_lam : forall s A M x N,
115   (t_lam s A M){x N} = t_lam s (A{x N}) (M{x N}).
116 Proof. reflexivity. Qed.
117
118 (* ===== *)
119 (** * Local closure *)
120
121 Inductive lc : term -> Prop :=
122   | lc_sort : forall s, lc (t_sort s)
123   | lc_fvar : forall s x, lc (t_fvar s x)
124   | lc_app : forall M N, lc M -> lc N -> lc (t_app M N)
125   | lc_pi : forall s A B L,
126     lc A ->
127     (forall x, ~ In x L -> lc (open_var B s x)) ->
128     lc (t_pi s A B)
129   | lc_lam : forall s A M L,
130     lc A ->
131     (forall x, ~ In x L -> lc (open_var M s x)) ->
132     lc (t_lam s A M).
133
134 (* ===== *)
135 (** * Bound-variable tag well-formedness. *)
136 Fixpoint bvar_tag_ok (k : nat) (s : Sort) (t : term) : Prop :=
137   match t with
138   | t_sort _ => True
139   | t_bvar s' n => n = k -> s' = s
140   | t_fvar _ _ => True
141   | t_app M _ => bvar_tag_ok k s M
142   | t_lam _ _ M => bvar_tag_ok (S k) s M
143   | t_pi _ _ B => bvar_tag_ok (S k) s B
144   end.
145
146 (* ===== *)
147 (** * Freshness of atoms *)
148
149 Fixpoint max_var_idx (xs : list var) : nat :=
150   match xs with
151   | [] => 0

```



```

152 | v :: xs => Nat.max v (max_var_idx xs)
153 end.
154
155 Definition fresh (xs : list var) : var := S (max_var_idx xs).
156
157 Lemma max_var_idx_ge : forall xs v,
158   In v xs -> v <= max_var_idx xs.
159 Proof.
160   induction xs as [| a xs IH]; intros v Hin.
161   - contradiction.
162   - simpl in Hin. destruct Hin as [Heq | Hin].
163     + subst. simpl. lia.
164     + simpl. specialize (IH v Hin). lia.
165 Qed.
166
167 Lemma fresh_notin : forall xs, ~ In (fresh xs) xs.
168 Proof.
169   intros xs Hin.
170   apply max_var_idx_ge in Hin.
171   unfold fresh in Hin. lia.
172 Qed.
173
174 (* ===== *)
175 (** * The open/subst/lc toolkit *)
176
177 Lemma not_in_app : forall (x : var) l1 l2,
178   ~ In x (l1 ++ l2) -> ~ In x l1 /\ ~ In x l2.
179 Proof.
180   intros x l1 l2 H. split; intro Hc; apply H; apply in_or_app; auto.
181 Qed.
182
183 Lemma not_in_cons_elim : forall (x : var) L y,
184   ~ In y (x :: L) -> y <> x /\ ~ In y L.
185 Proof.
186   intros x L y H. split.
187   - intro Hc. apply H. left. symmetry. exact Hc.
188   - intro Hc. apply H. right. exact Hc.
189 Qed.
190
191 Lemma open_rec_lc_core : forall t j v i u,
192   i <> j ->
193   open_rec j v t = open_rec i u (open_rec j v t) ->
194   t = open_rec i u t.
195 Proof.
196   induction t as [s | sn n | sx x | M IHM N IHN | sl A IHA M IHM | sp A IHA B
197     IHB];
198   - intros j v i u Hne Heq.
199   - reflexivity.
200   - destruct (Nat.eq_dec n j) as [Hnj | Hnj].
    + subst n. simpl.

```

```

201     destruct (Nat.eq_dec j i) as [Hji | Hji].
202     * exfalso. apply Hne. symmetry. exact Hji.
203     * apply Nat.eqb_neq in Hji. rewrite Hji. reflexivity.
204 + simpl in Heq. apply Nat.eqb_neq in Hnj. rewrite Hnj in Heq.
205     simpl in Heq. exact Heq.
206 - reflexivity.
207 - simpl in Heq |- *.
208     injection Heq as HeqM HeqN. f_equal.
209 + apply (IHM j v i u Hne HeqM).
210 + apply (IHN j v i u Hne HeqN).
211 - simpl in Heq |- *.
212     injection Heq as HeqA HeqM. f_equal.
213 + apply (IHA j v i u Hne HeqA).
214 + apply (IHM (S j) v (S i) u (fun Hc => Hne (eq_add_S i j Hc)) HeqM).
215 - simpl in Heq |- *.
216     injection Heq as HeqA HeqB. f_equal.
217 + apply (IHA j v i u Hne HeqA).
218 + apply (IHB (S j) v (S i) u (fun Hc => Hne (eq_add_S i j Hc)) HeqB).
219 Qed.
220
221 Lemma subst_fresh : forall t x u,
222   ~ In x (fv t) -> t{x u} = t.
223 Proof.
224   induction t as [s | sn n | sy y | M IHM N IHN | s1 A IHA M IHM | sp A IHA B
225     IHB];
226     intros x u Hnotin; simpl in *.
227 - reflexivity.
228 - reflexivity.
229 - destruct (eq_var_dec y x) as [Heq | Hneq].
230 + subst y. exfalso. apply Hnotin. left. reflexivity.
231 + reflexivity.
232 - destruct (not_in_app x (fv M) (fv N) Hnotin) as [H1 H2]. f_equal; auto.
233 - destruct (not_in_app x (fv A) (fv M) Hnotin) as [H1 H2]. f_equal; auto.
234 - destruct (not_in_app x (fv A) (fv B) Hnotin) as [H1 H2]. f_equal; auto.
235
236 Lemma open_rec_lc : forall t, lc t -> forall k u, open_rec k u t = t.
237 Proof.
238   intros t H.
239   induction H as [ s | sx x | M N HM IHM HN IHN
240     | sp A B L HA IHA HB IHB
241     | s1 A M L HA IHA HM IHM ];
242     intros k u; simpl.
243 - reflexivity.
244 - reflexivity.
245 - f_equal; auto.
246 - f_equal.
247 + auto.
248 + remember (fresh L) as x0 eqn:Hx0def.
249     assert (Hx0 : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).

```

```

250     specialize (IHB x0 Hx0 (S k) u).
251     unfold open_var in IHB.
252     symmetry.
253     apply (open_rec_lc_core B 0 (t_fvar sp x0) (S k) u).
254     * lia.
255     * symmetry. exact IHB.
256 - f_equal.
257 + auto.
258 + remember (fresh L) as x0 eqn:Hx0def.
259     assert (Hx0 : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
260     specialize (IHM x0 Hx0 (S k) u).
261     unfold open_var in IHM.
262     symmetry.
263     apply (open_rec_lc_core M 0 (t_fvar sl x0) (S k) u).
264     * lia.
265     * symmetry. exact IHM.
266 Qed.
267
268 Lemma subst_open_rec : forall t1 t2 x u k,
269   lc u ->
270   (open_rec k t2 t1){x u} = open_rec k (t2{x u}) (t1{x u}).
271 Proof.
272   induction t1 as [s | sn n | sy y | M IHM N IHN | sl A IHA M IHM | sp A IHA B
273   IHB];
274   intros t2 x u k Hlc; simpl.
275   - reflexivity.
276   - destruct (Nat.eqb n k) eqn:E; reflexivity.
277   - destruct (eq_var_dec y x) as [Heq | Hneq].
278     + subst y. symmetry. apply open_rec_lc. exact Hlc.
279     + reflexivity.
280   - f_equal; auto.
281   - f_equal.
282     + auto.
283     + rewrite IHM. reflexivity. exact Hlc.
284   - f_equal.
285     + auto.
286     + rewrite IHB. reflexivity. exact Hlc.
287 Qed.
288
289 Lemma subst_open : forall t1 t2 x u,
290   lc u ->
291   (t1 ^^ t2){x u} = (t1{x u}) ^^ (t2{x u}).
292 Proof. intros. apply subst_open_rec. exact H. Qed.
293
294 Lemma subst_open_var : forall t s x y u,
295   x <> y ->
296   lc u ->
297   (open_var t s x){y u} = open_var (t{y u}) s x.
298 Proof.
299   intros t s x y u Hxy Hlc.

```

```

299   unfold open_var.
300   rewrite subst_open_rec by exact Hlc.
301   simpl. destruct (eq_var_dec x y) as [Heq | Hneq].
302   - exfalso. apply Hxy. exact Heq.
303   - reflexivity.
304 Qed.
305
306 Lemma subst_intro : forall s x t u,
307   ~ In x (fv t) ->
308   lc u ->
309   t ^^ u = (open_var t s x){x u}.
310 Proof.
311   intros s x t u Hnotin Hlc.
312   unfold open_var.
313   rewrite subst_open_rec by exact Hlc.
314   simpl. destruct (eq_var_dec x x) as [_ | Hne].
315   - rewrite (subst_fresh t x u Hnotin). reflexivity.
316   - exfalso. apply Hne. reflexivity.
317 Qed.
318
319 Lemma subst_lc : forall t x u,
320   lc t -> lc u -> lc (t{x u}).
321 Proof.
322   intros t x u Ht.
323   induction Ht as [ s | sy y | M N HM IHM HN IHN
324     | sp A B L HA IHA HB IHB
325     | sl A M L HA IHA HM IHM ]; intros Hu; simpl.
326   - constructor.
327   - destruct (eq_var_dec y x); [exact Hu | constructor].
328   - constructor; auto.
329   - apply (lc_pi sp (A{x u}) (B{x u}) (x :: L)).
330     + auto.
331     + intros y Hy.
332       destruct (not_in_cons_elim x L y Hy) as [Hyx HyL].
333       rewrite <- (subst_open_var B sp y x u Hyx Hu).
334       apply IHB; auto.
335   - apply (lc_lam sl (A{x u}) (M{x u}) (x :: L)).
336     + auto.
337     + intros y Hy.
338       destruct (not_in_cons_elim x L y Hy) as [Hyx HyL].
339       rewrite <- (subst_open_var M sl y x u Hyx Hu).
340       apply IHM; auto.
341 Qed.
342
343 Lemma lc_open_var_rename : forall t s x0 y,
344   x0 <> y ->
345   ~ In x0 (fv t) ->
346   lc (open_var t s x0) -> lc (open_var t s y).
347 Proof.
348   intros t s x0 y Hxy Hnotin Hlc.

```

```

349   assert (Heq : open_var t s y = (open_var t s x0){x0 t_fvar s y}).
350   { unfold open_var. apply subst_intro; auto. constructor. }
351   rewrite Heq. apply subst_lc; auto. constructor.
352 Qed.
353
354 Lemma fv_open_rec_incl : forall t k u,
355   incl (fv t) (fv (open_rec k u t)).
356 Proof.
357   induction t as [s | sn n | sy y | M IHM N IHN | sl A IHA M IHM | sp A IHA B
358     IHB];
359   intros k u; simpl.
360   - apply incl_refl.
361   - apply incl_nil_l.
362   - apply incl_refl.
363   - apply incl_app.
364     + apply incl_appl. auto.
365     + apply incl_appr. auto.
366   - apply incl_app.
367     + apply incl_appl. auto.
368     + apply incl_appr. auto.
369   - apply incl_app.
370     + apply incl_appl. auto.
371     + apply incl_appr. auto.
372 Qed.
373
374 Lemma fv_open_var_incl : forall t s x,
375   incl (fv t) (fv (open_var t s x)).
376 Proof. intros. apply fv_open_rec_incl. Qed.
377
378 Lemma fv_open_rec_upper : forall t k u,
379   incl (fv (open_rec k u t)) (fv t ++ fv u).
380 Proof.
381   induction t as [s | sn n | sy y | M IHM N IHN | sl A IHA M IHM | sp A IHA B
382     IHB];
383   intros k u; simpl.
384   - apply incl_nil_l.
385   - destruct (Nat.eqb n k); simpl.
386     + apply incl_refl.
387     + apply incl_nil_l.
388   - intros z Hz. simpl in Hz. destruct Hz as [Hz | []]. left. exact Hz.
389   - intros z Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
390     + apply (IHM k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
391       * apply in_or_app; left; apply in_or_app; left; exact Hz.
392       * apply in_or_app; right; exact Hz.
393     + apply (IHN k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
394       * apply in_or_app; left; apply in_or_app; right; exact Hz.
395       * apply in_or_app; right; exact Hz.
396   - intros z Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
397     + apply (IHA k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
398       * apply in_or_app; left; apply in_or_app; left; exact Hz.

```

```

397     * apply in_or_app; right; exact Hz.
398 + apply (IHM (S k) u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz |
    Hz].
399     * apply in_or_app; left; apply in_or_app; right; exact Hz.
400     * apply in_or_app; right; exact Hz.
401 - intros z Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
402 + apply (IHA k u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz | Hz].
403     * apply in_or_app; left; apply in_or_app; left; exact Hz.
404     * apply in_or_app; right; exact Hz.
405 + apply (IHB (S k) u) in Hz. apply in_app_or in Hz. destruct Hz as [Hz |
    Hz].
406     * apply in_or_app; left; apply in_or_app; right; exact Hz.
407     * apply in_or_app; right; exact Hz.
408 Qed.
409
410 Lemma subst_open_var_eq : forall t s x u,
411   ~ In x (fv t) ->
412   lc u ->
413   (open_var t s x) {x u} = t ^^ u.
414 Proof. intros. symmetry. apply subst_intro; auto. Qed.
415
416 (* ===== *)
417 (** * Beta reduction *)
418
419 Definition beta (M N : term) : Prop :=
420   exists s Dom Body Arg,
421     M = t_app (t_lam s Dom Body) Arg /\
422     N = Body ^^ Arg.
423
424 Reserved Notation "M ->b N" (at level 70, no associativity).
425
426 Inductive beta_red : term -> term -> Prop :=
427
428   | beta_base : forall s A M N,
429     t_app (t_lam s A M) N ->b (M ^^ N)
430
431   | beta_app_l : forall M M' N,
432     M ->b M' ->
433     t_app M N ->b t_app M' N
434
435   | beta_app_r : forall M N N',
436     N ->b N' ->
437     t_app M N ->b t_app M N'
438
439   | beta_lam_A : forall s A A' M,
440     A ->b A' ->
441     t_lam s A M ->b t_lam s A' M
442
443   | beta_lam_M : forall s A M M',
444     M ->b M' ->

```

```

445     t_lam s A M ->b t_lam s A M'
446
447   | beta_pi_A : forall s A A' B,
448     A ->b A' ->
449     t_pi s A B ->b t_pi s A' B
450
451   | beta_pi_B : forall s A B B',
452     B ->b B' ->
453     t_pi s A B ->b t_pi s A B'
454
455 where "M ->b N" := (beta_red M N).
456
457 Reserved Notation "M ->>b N" (at level 70, no associativity).
458
459 Inductive beta_trans : term -> term -> Prop :=
460   | bt_step : forall M N,
461     M ->b N ->
462     M ->>b N
463
464   | bt_trans : forall M N P,
465     M ->>b N ->
466     N ->>b P ->
467     M ->>b P
468
469 where "M ->>b N" := (beta_trans M N).
470
471 Reserved Notation "M ->>b N" (at level 70, no associativity).
472
473 Inductive beta_rtrans : term -> term -> Prop :=
474   | brt_refl : forall M,
475     M ->>b M
476
477   | brt_step : forall M N,
478     M ->b N ->
479     M ->>b N
480
481   | brt_trans : forall M N P,
482     M ->>b N ->
483     N ->>b P ->
484     M ->>b P
485
486 where "M ->>b N" := (beta_rtrans M N).
487
488 Reserved Notation "M =b N" (at level 70, no associativity).
489
490 Inductive beta_eq : term -> term -> Prop :=
491   | beq_refl : forall M,
492     M =b M
493
494   | beq_step : forall M N,

```

```

495     M ->b N ->
496     M =b N
497
498   | beq_sym : forall M N,
499     M =b N ->
500     N =b M
501
502   | beq_trans : forall M N P,
503     M =b N ->
504     N =b P ->
505     M =b P
506
507 where "M =b N" := (beta_eq M N).
508
509 Lemma brt_from_trans : forall M N,
510   M ->>b N -> M ->>b N.
511 Proof.
512   intros M N H.
513   induction H.
514   - apply brt_step; auto.
515   - apply brt_trans with N; auto.
516 Qed.
517
518 Lemma beq_from_rtrans : forall M N,
519   M ->>b N -> M =b N.
520 Proof.
521   intros M N H.
522   induction H.
523   - apply beq_refl.
524   - apply beq_step; auto.
525   - apply beq_trans with N; auto.
526 Qed.
527
528 Definition normal_form (M : term) : Prop :=
529   ~ exists N, M ->b N.
530
531 Fixpoint has_redex (M : term) : Prop :=
532   match M with
533   | t_sort _      => False
534   | t_bvar _ _    => False
535   | t_fvar _ _    => False
536   | t_app (t_lam _ _ _) _ => True
537   | t_app P Q      => has_redex P ∨ has_redex Q
538   | t_lam _ A M    => has_redex A ∨ has_redex M
539   | t_pi _ A B     => has_redex A ∨ has_redex B
540   end.
541
542 Definition normal_form' (M : term) : Prop :=
543   ~ has_redex M.
544

```



```

545 Definition weakly_normalizing (M : term) : Prop :=
546   exists N, M ->>b N /\ normal_form N.
547
548 Definition strongly_normalizing (M : term) : Prop :=
549   Acc (fun N M => M ->b N) M.
550
551 Lemma sn_implies_wn : forall M,
552   strongly_normalizing M -> weakly_normalizing M.
553 Proof.
554   intros M Hacc.
555   induction Hacc as [M _ IH].
556   destruct (classic (exists N, M ->b N)) as [[N Hstep] | Hnf].
557   - destruct (IH N Hstep) as [P [HredP HnfP]].
558     exists P. split.
559     + apply brt_trans with N.
560       * apply brt_step; auto.
561       * auto.
562     + auto.
563   - exists M. split.
564     + apply brt_refl.
565     + unfold normal_form. auto.
566 Qed.
567
568 Lemma nf_is_sn : forall M,
569   normal_form M -> strongly_normalizing M.
570 Proof.
571   intros M Hnf.
572   constructor.
573   intros N Hstep.
574   exfalso. apply Hnf.
575   exists N. auto.
576 Qed.
577
578 (* ===== *)
579 (** * Contexts *)
580
581 Definition context := list (var * (Sort * term)).
582
583 Fixpoint lookup (c : context) (x : var) : option (Sort * term) :=
584   match c with
585   | [] => None
586   | (y, sA) :: c' =>
587     if eq_var_dec x y
588     then Some sA
589     else lookup c' x
590   end.
591
592 Definition dom (Γ : context) : list var :=
593   map fst Γ.
594

```

```

595 Definition is_fresh (x : var) (Γ : context) :=
596   ~ In x (dom Γ).
597
598 (* ===== *)
599 (** * Establishing [bvar_tag_ok] from [lc] *)
600 Fixpoint closed_at (k : nat) (t : term) : Prop :=
601   match t with
602   | t_sort _      => True
603   | t_bvar _ n    => n < k
604   | t_fvar _ _    => True
605   | t_app M _     => closed_at k M
606   | t_lam _ _ M   => closed_at (S k) M
607   | t_pi _ _ B    => closed_at (S k) B
608   end.
609
610 Lemma closed_at_open_rec_inv : forall t k u,
611   closed_at k (open_rec k u t) ->
612   (forall s m, u <> t_bvar s m) ->
613   closed_at (S k) t.
614 Proof.
615   induction t as [s0 | sn n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A IHA B
616   IHB];
617   intros k u Hc Hu; simpl in *; auto.
618   - destruct (Nat.eqb n k) eqn:Heqb.
619     + apply Nat.eqb_eq in Heqb. lia.
620     + apply Nat.eqb_neq in Heqb.
621       simpl in Hc. lia.
622   - eapply IHP; eauto.
623   - eapply IHM with (k := S k); eauto.
624   - eapply IHB with (k := S k); eauto.
625 Qed.
626
627 Lemma lc_closed_at : forall t, lc t -> closed_at 0 t.
628 Proof.
629   intros t Hlc.
630   induction Hlc as [ s | x | M N HM IHM HN IHN
631   | s A B L HA IHA HB IHB
632   | s A M L HA IHA HM IHM ]; simpl; auto.
633   - remember (fresh L) as x0 eqn:Hx0def.
634     assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
635     specialize (IHB x0 Hx0L).
636     unfold open_var in IHB.
637     apply (closed_at_open_rec_inv B 0 (t_fvar s x0) IHB).
638     intros sv m Hcontra. discriminate.
639   - remember (fresh L) as x0 eqn:Hx0def.
640     assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
641     specialize (IHM x0 Hx0L).
642     unfold open_var in IHM.
643     apply (closed_at_open_rec_inv M 0 (t_fvar s x0) IHM).
644     intros sv m Hcontra. discriminate.

```

```

644 Qed.
645
646 Lemma closed_at_bvar_tag_ok : forall t m,
647   closed_at m t -> forall k s, m <= k -> bvar_tag_ok k s t.
648 Proof.
649   induction t as [s0 | sn n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A IHA B
650     IHB];
651     intros m Hclosed k s Hmk; simpl in *; auto.
652   - intro Heq. subst n. lia.
653   - eapply IHP; eauto.
654   - eapply IHM; eauto. lia.
655   - eapply IHB; eauto. lia.
656 Qed.
657
658 Lemma lc_bvar_tag_ok : forall t, lc t -> forall k s, bvar_tag_ok k s t.
659 Proof.
660   intros t Hlc k s.
661   apply (closed_at_bvar_tag_ok t 0); [apply lc_closed_at; exact Hlc | lia].
662 Qed.
663
664 Lemma bvar_tag_ok_subst : forall t k s x N,
665   lc N -> bvar_tag_ok k s t -> bvar_tag_ok k s (t { x N }).
666 Proof.
667   induction t as [s0 | s_n n | sy y | P IHP Q IHQ | s1 A IHA M IHM | s1 A IHA B
668     IHB];
669     intros k s x N HlcN Htag; simpl in *; auto.
670   - destruct (eq_var_dec y x); auto.
671   - apply lc_bvar_tag_ok. exact HlcN.
672 Qed.
673
674 (* ===== *)
675 (** * Typing *)
676
677 Reserved Notation "Γ M A" (at level 70, no associativity).
678
679 Inductive typing : context -> term -> term -> Prop :=
680   | typing_axiom : forall s s',
681     A s s' ->
682     [] t_sort s t_sort s'
683   | typing_var : forall Γ x A s,
684     is_fresh x Γ ->
685     Γ A t_sort s ->
686     Γ ++ [(x, (s, A))] t_fvar s x A
687   | typing_weak : forall Γ x B M A s,
688     is_fresh x Γ ->
689     Γ M A ->
690     Γ B t_sort s ->
691     Γ ++ [(x, (s, B))] M A

```

```

692
693 | typing_pi : forall Γ A B s1 s2 s3 L,
694   Γ A t_sort s1 ->
695   bvar_tag_ok 0 s1 B ->
696   (forall x, ~ In x L ->
697     Γ ++ [(x, (s1, A))] (open_var B s1 x) t_sort s2) ->
698   R s1 s2 s3 ->
699   Γ (t_pi s1 A B) (t_sort s3)
700
701 | typing_lam : forall Γ A M B s1 s3 L,
702   Γ A t_sort s1 ->
703   bvar_tag_ok 0 s1 M ->
704   (forall x, ~ In x L ->
705     Γ ++ [(x, (s1, A))] (open_var M s1 x) (open_var B s1 x)) ->
706   Γ (t_pi s1 A B) (t_sort s3) ->
707   Γ (t_lam s1 A M) (t_pi s1 A B)
708
709 | typing_app : forall Γ M N A B s1,
710   Γ M t_pi s1 A B ->
711   Γ N A ->
712   Γ t_app M N (B ^^ N)
713
714 | typing_conv : forall Γ M A B s,
715   Γ M A ->
716   A =b B ->
717   Γ B t_sort s ->
718   Γ M B
719
720 where "Γ M A" := (typing Γ M A).
721
722 Definition legal (Γ : context) : Prop :=
723   exists M N, Γ M N.
724
725 (* ===== *)
726 (** * Regularity *)
727
728 Lemma lc_pi_body : forall s A B,
729   lc (t_pi s A B) -> exists L, forall x, ~ In x L -> lc (open_var B s x).
730 Proof.
731   intros s A B H.
732   remember (t_pi s A B) as t eqn:Ht.
733   destruct H; try discriminate.
734   injection Ht as Hs HA HB. subst.
735   exists L. exact H0.
736 Qed.
737
738 Lemma lc_open_lc : forall B s x N,
739   ~ In x (fv B) -> lc (open_var B s x) -> lc N -> lc (B ^^ N).
740 Proof.
741   intros B s x N Hnotin Hlc HlcN.

```

```

742   rewrite (subst_intro s x B N Hnotin HlcN).
743   apply subst_lc; auto.
744 Qed.
745
746 Lemma typing_lc : forall  $\Gamma$  M A,  $\Gamma \vdash M : A \rightarrow \text{lc } M \wedge \text{lc } A$ .
747 Proof.
748   intros  $\Gamma$  M A H.
749   induction H as
750     [ s s' HA
751     |  $\Gamma_0$  x A0 s0 Hfresh HA IHA
752     |  $\Gamma_0$  x B0 M0 A0 s0 Hfresh HM IHM HB IHB
753     |  $\Gamma_0$  A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
754     |  $\Gamma_0$  A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
755     |  $\Gamma_0$  M0 N0 A0 B0 s1a HM IHM HN IHN
756     |  $\Gamma_0$  M0 A0 B0 s HM IHM Heq HB IHB ].
757   - split; constructor.
758   - split; [constructor | apply IHA].
759   - split; [apply IHM | apply IHM].
760   - split.
761   + remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
762     assert (Hx0L :  $\sim \text{In } x0 \text{ L}$ ).
763     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
764       apply in_or_app. left. exact Hc. }
765     assert (Hx0B :  $\sim \text{In } x0 \text{ (fv B0)}$ ).
766     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
767       apply in_or_app. right. exact Hc. }
768     destruct (IHB x0 Hx0L) as [Hlc_open _].
769     apply (lc_pi s1 A0 B0 (x0 :: nil)).
770     * apply IHA.
771     * intros y Hy.
772       assert (Hyx0 :  $y <> x0$ ) by (intro Hc; apply Hy; left; symmetry; exact
773         Hc).
774       apply (lc_open_var_rename B0 s1 x0 y (not_eq_sym Hyx0) Hx0B Hlc_open).
775   + constructor.
776   - split.
777   + remember (fresh (L ++ fv M0)) as x0 eqn:Hx0def.
778     assert (Hx0L :  $\sim \text{In } x0 \text{ L}$ ).
779     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
780       apply in_or_app. left. exact Hc. }
781     assert (Hx0M :  $\sim \text{In } x0 \text{ (fv M0)}$ ).
782     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
783       apply in_or_app. right. exact Hc. }
784     destruct (IHM x0 Hx0L) as [Hlc_open _].
785     apply (lc_lam s1 A0 M0 (x0 :: nil)).
786     * apply IHA.
787     * intros y Hy.
788       assert (Hyx0 :  $y <> x0$ ) by (intro Hc; apply Hy; left; symmetry; exact
789         Hc).
790       apply (lc_open_var_rename M0 s1 x0 y (not_eq_sym Hyx0) Hx0M Hlc_open).
791   + apply IHPi.

```

```

790 - split.
791 + constructor; [apply IHM | apply IHN].
792 + destruct (lc_pi_body s1a A0 B0 (proj2 IHM)) as [L HL].
793   remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
794   assert (Hx0L : ~ In x0 L).
795   { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
796     apply in_or_app. left. exact Hc. }
797   assert (Hx0B : ~ In x0 (fv B0)).
798   { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
799     apply in_or_app. right. exact Hc. }
800   apply (lc_open_lc B0 s1a x0 N0 Hx0B (HL x0 Hx0L) (proj1 IHN)).
801 - split; [apply IHM | apply IHB].
802 Qed.
803
804 Lemma typing_fv_subset : forall  $\Gamma$  M A,
805    $\Gamma$  M A  $\rightarrow$  incl (fv M) (dom  $\Gamma$ ) /\ incl (fv A) (dom  $\Gamma$ ).
806 Proof.
807   intros  $\Gamma$  M A H.
808   induction H as
809     [ s s' HA
810     |  $\Gamma$ 0 x A0 s0 Hfresh HA IHA
811     |  $\Gamma$ 0 x B0 M0 A0 s0 Hfresh HM IHM HB IHB
812     |  $\Gamma$ 0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
813     |  $\Gamma$ 0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
814     |  $\Gamma$ 0 M0 N0 A0 B0 s1a HM IHM HN IHN
815     |  $\Gamma$ 0 M0 A0 B0 s HM IHM Heq HB IHB ].
816 - split; apply incl_nil_1.
817 - split.
818 + intros z Hz. destruct Hz as [Hz | []]. subst z.
819   unfold dom. rewrite map_app. apply in_or_app. right. simpl. left.
820   reflexivity.
821 + unfold dom in *. rewrite map_app. apply incl_appl. apply IHA.
822 - split.
823 + unfold dom in *. rewrite map_app. apply incl_appl. apply IHM.
824 + unfold dom in *. rewrite map_app. apply incl_appl. apply IHB.
825 - split.
826 + simpl. apply incl_app.
827   * apply IHA.
828   * remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
829     assert (Hx0L : ~ In x0 L).
830     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
831       apply in_or_app. left. exact Hc. }
832     assert (Hx0B : ~ In x0 (fv B0)).
833     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
834       apply in_or_app. right. exact Hc. }
834     destruct (IHB x0 Hx0L) as [Hfv_open _].
835     intros z Hz.
836     assert (Hz' : In z (fv (open_var B0 s1 x0))) by (apply (fv_open_var_incl
837       B0 s1 x0); exact Hz).
837     unfold dom in Hfv_open. rewrite map_app in Hfv_open.

```

```

838     specialize (Hfv_open z Hz'). apply in_app_or in Hfv_open.
839     destruct Hfv_open as [Hin | Hin].
840     -- exact Hin.
841     -- simpl in Hin. destruct Hin as [Hin | []]. subst z. contradiction.
842 + apply incl_nil_l.
843 - split.
844 + simpl. apply incl_app.
845   * apply IHA.
846   * remember (fresh (L ++ fv M0)) as x0 eqn:Hx0def.
847     assert (Hx0L : ~ In x0 L).
848     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
849       apply in_or_app. left. exact Hc. }
850     assert (Hx0M : ~ In x0 (fv M0)).
851     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0)).
852       apply in_or_app. right. exact Hc. }
853     destruct (IHM x0 Hx0L) as [Hfv_open _].
854     intros z Hz.
855     assert (Hz' : In z (fv (open_var M0 s1 x0))) by (apply (fv_open_var_incl
856       M0 s1 x0); exact Hz).
857     unfold dom in Hfv_open. rewrite map_app in Hfv_open.
858     specialize (Hfv_open z Hz'). apply in_app_or in Hfv_open.
859     destruct Hfv_open as [Hin | Hin].
860     -- exact Hin.
861     -- simpl in Hin. destruct Hin as [Hin | []]. subst z. contradiction.
862 + apply IHPi.
863 - split.
864 + simpl. apply incl_app; [apply IHM | apply IHN].
865 + assert (HfvB0 : incl (fv B0) (dom Γ0)).
866   { intros z Hz. apply (proj2 IHM). simpl. apply in_or_app. right. exact Hz.
867   }
868   intros z Hz.
869   apply (fv_open_rec_upper B0 0 N0) in Hz.
870   apply in_app_or in Hz. destruct Hz as [Hz | Hz].
871   * apply HfvB0. exact Hz.
872   * apply (proj1 IHN). exact Hz.
873 - split; [apply IHM | apply IHB].
874 Qed.
875
876 Lemma subst_fresh_typing : forall Γ M A x N,
877   is_fresh x Γ ->
878   Γ M A ->
879   M { x N } = M /\ A { x N } = A.
880 Proof.
881   intros Γ M A x N Hfresh Htyp.
882   destruct (typing_fv_subset Γ M A Htyp) as [HfvM HfvA].
883   split.
884   - apply subst_fresh. intro Hc. apply Hfresh. apply HfvM. exact Hc.
885   - apply subst_fresh. intro Hc. apply Hfresh. apply HfvA. exact Hc.
886 Qed.

```

```

886 Lemma beta_red_subst : forall M N x u,
887   M ->b N -> lc u -> M{x u} ->b N{x u}.
888 Proof.
889   intros M N x u H.
890   induction H; intros Hlc; simpl.
891   - rewrite subst_open by exact Hlc. apply beta_base.
892   - apply beta_app_l. apply IHbeta_red; auto.
893   - apply beta_app_r. apply IHbeta_red; auto.
894   - apply beta_lam_A. apply IHbeta_red; auto.
895   - apply beta_lam_M. apply IHbeta_red; auto.
896   - apply beta_pi_A. apply IHbeta_red; auto.
897   - apply beta_pi_B. apply IHbeta_red; auto.
898 Qed.
899
900 Lemma beta_eq_subst : forall M N x u,
901   M =b N -> lc u -> M{x u} =b N{x u}.
902 Proof.
903   intros M N x u H.
904   induction H; intros Hlc.
905   - apply beq_refl.
906   - apply beq_step. apply beta_red_subst; auto.
907   - apply beq_sym. apply IHbeta_eq; auto.
908   - apply beq_trans with (N{x u}); auto.
909 Qed.
910
911 (* ===== *)
912 (** * Basic context lemmas *)
913
914 Lemma start_axiom :
915   forall Γ s s',
916     legal Γ ->
917     A s s' ->
918     Γ t_sort s t_sort s'.
919 Proof.
920   intros Γ s s' H1 Hs. destruct H1 as [M [N Htyp]].
921   induction Htyp; auto.
922   - apply typing_axiom. apply Hs.
923   - apply typing_weak.
924     + apply H.
925     + apply IHHtyp.
926     + apply Htyp.
927   - apply typing_weak.
928     + apply H.
929     + apply IHHtyp1.
930     + apply Htyp2.
931 Qed.
932
933 Lemma start_var :
934   forall Γ x s T,
935     legal Γ ->

```



```

936   In (x, (s, T))  $\Gamma \rightarrow$ 
937    $\Gamma$    t_fvar s x   T.
938 Proof.
939   intros  $\Gamma$  x s T H1 Hin. destruct H1 as [M [N Htyp]].
940   induction Htyp as
941   [ s' s'' HA
942   |  $\Gamma$ 0 x0 A0 s0 Hfresh0 HAO IHA0
943   |  $\Gamma$ 0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
944   |  $\Gamma$ 0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
945   |  $\Gamma$ 0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
946   |  $\Gamma$ 0 M0 N0 A0 B0 s1a HM IHM HN IHN
947   |  $\Gamma$ 0 M0 A0 B0 s3 HM IHM Heq HB IHB ].
948
949   - contradiction.
950
951   - apply in_app_or in Hin. destruct Hin as [Hin | Hin].
952     + apply typing_weak.
953       * apply Hfresh0.
954       * apply IHA0. apply Hin.
955       * apply HAO.
956     + destruct Hin as [Hin | []]. inversion Hin. subst.
957       apply typing_var.
958       * apply Hfresh0.
959       * apply HAO.
960
961   - apply in_app_or in Hin. destruct Hin as [Hin | Hin].
962     + apply typing_weak.
963       * apply Hfresh0.
964       * apply IHMO. apply Hin.
965       * apply HBO.
966     + destruct Hin as [Hin | []]. inversion Hin. subst.
967       apply typing_var.
968       * apply Hfresh0.
969       * apply HBO.
970
971   - auto.
972
973   - auto.
974
975   - auto.
976
977   - auto.
978 Qed.
979
980 Definition subcontext ( $\Gamma$   $\Delta$  : context) : Prop :=
981   forall x T,
982     In (x, T)  $\Gamma \rightarrow$ 
983     In (x, T)  $\Delta$ .
984
985 Notation " $\Gamma$   $\Delta$ " := (subcontext  $\Gamma$   $\Delta$ ) (at level 70, no associativity).

```

```

986
987 Lemma subcontext_extend : forall  $\Gamma$   $\Delta$  z A,
988    $\Gamma$   $\Delta$  ->
989   is_fresh z  $\Delta$  ->
990    $\Gamma$  ++ [(z, A)]  $\Delta$  ++ [(z, A)].
991 Proof.
992   intros  $\Gamma$   $\Delta$  z A Hsub Hfr y T Hin.
993   apply in_app_or in Hin.
994   destruct Hin.
995   - apply in_or_app. left. apply Hsub. assumption.
996   - simpl in H.
997     destruct H as [H | H].
998     + inversion H; subst.
999     apply in_or_app. right. left. reflexivity.
1000    + contradiction.
1001 Qed.
1002
1003 Lemma subcontext_extend_r : forall  $\Gamma$  x T,  $\Gamma$   $\Gamma$  ++ [(x, T)].
1004 Proof.
1005   intros.
1006   unfold subcontext. intros.
1007   apply in_or_app. left.
1008   apply H.
1009 Qed.
1010
1011 Lemma thinning :
1012   forall  $\Gamma$   $\Delta$  M N,
1013     legal  $\Delta$  ->
1014      $\Gamma$   $\Delta$  ->
1015      $\Gamma$  M N ->
1016      $\Delta$  M N.
1017 Proof.
1018   intros  $\Gamma$   $\Delta$  M N Hleg Hsub Htyp.
1019   generalize dependent  $\Delta$ .
1020   induction Htyp as
1021     [ s s' HA
1022     |  $\Gamma$ 0 x0 A0 s0 Hfresh0 HAO IHA0
1023     |  $\Gamma$ 0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
1024     |  $\Gamma$ 0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
1025     |  $\Gamma$ 0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
1026     |  $\Gamma$ 0 M0 N0 A0 B0 s1a HM IHM HN IHN
1027     |  $\Gamma$ 0 M0 A0 B0 s HM IHM Heq HB IHB ];
1028   intros  $\Delta$  Hleg Hsub.
1029
1030   - apply start_axiom.
1031     + apply Hleg.
1032     + apply HA.
1033
1034   - apply (start_var  $\Delta$  x0 s0 A0 Hleg).
1035     unfold subcontext in Hsub. apply Hsub. apply in_or_app. right.

```

```

1036     simpl. left. reflexivity.
1037
1038 - apply IHMO.
1039 + apply Hleg.
1040 + unfold subcontext in *. intros y T Hin.
1041     apply Hsub. apply in_or_app. left. apply Hin.
1042
1043 - apply (typing_pi Δ A0 B0 s1 s2 s3 (L ++ dom Δ)).
1044 + apply IHA; auto.
1045 + exact HTagB.
1046 + intros x Hx.
1047     assert (HxL : ~ In x L).
1048     { intro Hc. apply Hx. apply in_or_app. left. exact Hc. }
1049     assert (HxΔ : is_fresh x Δ).
1050     { intro Hc. apply Hx. apply in_or_app. right. exact Hc. }
1051     apply (IHB x HxL (Δ ++ [(x, (s1, A0))])).
1052     * exists (t_fvar s1 x). exists A0. apply typing_var.
1053         -- exact HxΔ.
1054         -- apply IHA; auto.
1055     * apply subcontext_extend; [exact Hsub | exact HxΔ].
1056 + exact HR.
1057
1058 - apply typing_lam with (s1 := s1) (s3 := s3) (L := L ++ dom Δ).
1059 + apply IHA; auto.
1060 + exact HTagM.
1061 + intros x Hx.
1062     assert (HxL : ~ In x L).
1063     { intro Hc. apply Hx. apply in_or_app. left. exact Hc. }
1064     assert (HxΔ : is_fresh x Δ).
1065     { intro Hc. apply Hx. apply in_or_app. right. exact Hc. }
1066     apply (IHM x HxL (Δ ++ [(x, (s1, A0))])).
1067     * exists (t_fvar s1 x). exists A0. apply typing_var.
1068         -- exact HxΔ.
1069         -- apply IHA; auto.
1070     * apply subcontext_extend; [exact Hsub | exact HxΔ].
1071 + apply IHPi; auto.
1072
1073 - apply typing_app with (A := A0) (s1 := s1a); auto.
1074
1075 - apply typing_conv with A0 s; auto.
1076 Qed.
1077
1078 (* ===== *)
1079 (** * Generation lemmas *)
1080
1081 Lemma app_singleton_injective :
1082   forall (Γ Δ : context)
1083     (x y : var)
1084     (M N : Sort * term),
1085   Γ ++ [(x, M)] = Δ ++ [(y, N)] ->

```

```

1086    $\Gamma = \Delta \wedge x = y \wedge M = N.$ 
1087 Proof.
1088   induction  $\Gamma$  as [| [v T]  $\Gamma$  IH].
1089   - intros  $\Delta$  x y M N H.
1090     destruct  $\Delta$  as [| [v' T']  $\Delta$ ].
1091     + simpl in H.
1092       inversion H. subst.
1093       repeat split; reflexivity.
1094     + simpl in H.
1095       destruct  $\Delta$ .
1096       * discriminate.
1097       * discriminate.
1098
1099   - intros  $\Delta$  x y M N H.
1100     destruct  $\Delta$  as [| [v' T']  $\Delta$ ].
1101     + simpl in H.
1102       destruct  $\Gamma$ .
1103       * discriminate.
1104       * discriminate.
1105
1106     + simpl in H.
1107       inversion H. subst.
1108       specialize (IH  $\Delta$  x y M N H3).
1109       destruct IH as [H $\Gamma$  [Hx HM]].
1110       subst.
1111       repeat split; reflexivity.
1112 Qed.
1113
1114 Lemma generation_sort :
1115   forall  $\Gamma$  s W,
1116      $\Gamma$  t_sort s W  $\rightarrow$ 
1117     exists s',
1118       W =b t_sort s' /\ Sig.A s s'.
1119 Proof.
1120   intros  $\Gamma$  s W H.
1121   remember (t_sort s) as T eqn:HeqT.
1122   induction H as
1123     [ sa sb HA
1124     |  $\Gamma$ 0 x0 A0 s0 Hfresh0 HAO IHA0
1125     |  $\Gamma$ 0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
1126     |  $\Gamma$ 0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1127     |  $\Gamma$ 0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1128     |  $\Gamma$ 0 M0 N0 A0 B0 s1a HM IHM HN IHN
1129     |  $\Gamma$ 0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1130   inversion HeqT; subst.
1131   - exists sb. split.
1132     + apply beq_refl.
1133     + apply HA.
1134   - apply IHMO. reflexivity.
1135   - destruct (IHM eq_refl) as [s' [HAeq HAx]]. exists s'. split.

```

```

1136   + apply beq_trans with (N := A0).
1137   * apply beq_sym. apply Heq.
1138   * apply HAeq.
1139   + apply HAx.
1140 Qed.
1141
1142 Lemma lookup_fresh :
1143   forall C y N, is_fresh y C -> lookup (C ++ [(y, N)]) y = Some N.
1144 Proof.
1145   intros C y N F. unfold is_fresh in F. induction C.
1146   - simpl. destruct (eq_var_dec y y).
1147     + reflexivity.
1148     + contradiction.
1149   - destruct a as (v, T). simpl in *. destruct (eq_var_dec y v) as [Heq | Hneq].
1150     + exfalso. apply F. simpl. left. f_equal. symmetry. exact Heq.
1151     + apply IHC. intro Hc. apply F. simpl. right. exact Hc.
1152 Qed.
1153
1154 Lemma lookup_extend :
1155   forall C x y M N, (lookup C x = Some M) -> (lookup (C ++ [(y, N)]) x = Some
1156     M).
1157 Proof.
1158   intros C x y M N H. induction C.
1159   - simpl in H. discriminate H.
1160   - destruct a as (v, T). simpl in *. destruct (eq_var_dec x v) as [Heq | Hneq].
1161     + apply H.
1162     + apply IHC. apply H.
1163 Qed.
1164
1165 Lemma generation_var :
1166   forall Γ x s N,
1167     Γ t_fvar s x N ->
1168     exists B,
1169       lookup Γ x = Some (s, B) /\
1170       Γ B t_sort s /\
1171       N =b B.
1172 Proof.
1173   intros Γ x s N H.
1174   remember (t_fvar s x) as T eqn:HeqT.
1175   induction H as
1176     [ sa sb HA
1177     | Γ0 x0 A0 s0 Hfresh0 HAO IHA0
1178     | Γ0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
1179     | Γ0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1180     | Γ0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1181     | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
1182     | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1183   inversion HeqT; subst.
1184   - exists A0. repeat split; auto.

```

```

1185 + apply lookup_fresh; auto.
1186 + apply typing_weak; auto.
1187 + apply beq_refl.
1188
1189 - destruct (IHMO eq_refl) as [B' [Q1 [Q2 Q3]]]. exists B'. repeat split; auto.
1190 + apply lookup_extend; auto.
1191 + apply typing_weak; auto.
1192
1193 - destruct (IHM eq_refl) as [B' [Q1 [Q2 Q3]]]. exists B'. repeat split; auto.
1194   apply beq_trans with A0; auto. apply beq_sym; auto.
1195 Qed.
1196
1197 Lemma generation_pi :
1198   forall  $\Gamma$  s0 B C W,
1199      $\Gamma$  t_pi s0 B C W ->
1200     exists s2 s3 L,
1201        $\Gamma$  B t_sort s0 /\
1202       bvar_tag_ok 0 s0 C /\
1203       (forall x, ~ In x L ->  $\Gamma$  ++ [(x, (s0, B))]) open_var C s0 x t_sort s2)
1204       /\
1205       Sig.R s0 s2 s3 /\
1206       W =b t_sort s3.
1207 Proof.
1208   intros  $\Gamma$  s0 B C W H.
1209   remember (t_pi s0 B C) as T eqn:HeqT.
1210   induction H as
1211     [ sa sb HA
1212     |  $\Gamma$ 0 x0 A0 s1 Hfresh0 HAO IHA0
1213     |  $\Gamma$ 0 x0 B0 M0 A0 s1 Hfresh0 HMO IHMO HBO IHBO
1214     |  $\Gamma$ 0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1215     |  $\Gamma$ 0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1216     |  $\Gamma$ 0 M0 N0 A0 B0 s1a HM IHM HN IHN
1217     |  $\Gamma$ 0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1218   inversion HeqT; subst.
1219 - destruct (IHMO eq_refl) as [s2 [s3 [L0 [HB1 [HTag1 [HB2 [HB3 HB4]]]]]]].
1220   assert (HBthin :  $\Gamma$ 0 ++ [(x0, (s1, B0))] B t_sort s0)
1221     by (apply typing_weak; auto).
1222   exists s2. exists s3. exists (L0 ++ dom ( $\Gamma$ 0 ++ [(x0, (s1, B0))])).
1223   repeat split; auto.
1224 + intros y Hy.
1225   assert (HyL0 : ~ In y L0).
1226   { intro Hc. apply Hy. apply in_or_app. left. exact Hc. }
1227   assert (Hy $\Gamma$  : is_fresh y ( $\Gamma$ 0 ++ [(x0, (s1, B0))])).
1228   { intro Hc. apply Hy. apply in_or_app. right. exact Hc. }
1229   apply (thinning ( $\Gamma$ 0 ++ [(y, (s0, B))]) (( $\Gamma$ 0 ++ [(x0, (s1, B0))]) ++ [(y,
1230     (s0, B))])).
1231   * exists (t_fvar s0 y). exists B.
1232     apply typing_var.
1233     -- exact Hy $\Gamma$ .

```

```

1233     -- exact HBthin.
1234 * apply subcontext_extend.
1235     -- apply subcontext_extend_r.
1236     -- exact HyΓ.
1237 * apply HB2; auto.
1238
1239 - exists sb. exists sc. exists L.
1240   repeat split; auto. apply beq_refl.
1241
1242 - destruct (IHM eq_refl) as [s2 [s3 [L0 [HB1 [HTag1 [HB2 [HB3 HB4]]]]]]].
1243   exists s2. exists s3. exists L0.
1244   repeat split; auto. apply beq_trans with A0; auto. apply beq_sym; auto.
1245 Qed.
1246
1247 Lemma pi_domain_valid : forall Γ s0 B C s',
1248   Γ t_pi s0 B C t_sort s' ->
1249   Γ B t_sort s0.
1250 Proof.
1251   intros Γ s0 B C s' H.
1252   destruct (generation_pi Γ s0 B C (t_sort s') H) as [s2 [s3 [L [HB _]]]].
1253   exact HB.
1254 Qed.
1255
1256 Lemma generation_lam :
1257   forall Γ s0 B M T,
1258     Γ t_lam s0 B M T ->
1259     exists C s3 L,
1260       Γ B t_sort s0 /\
1261       (forall x, ~ In x L -> Γ ++ [(x, (s0, B))] open_var M s0 x open_var C
1262         s0 x) /\
1263       Γ t_pi s0 B C t_sort s3 /\
1264       T =b t_pi s0 B C.
1265 Proof.
1266   intros Γ s0 B M T H.
1267   remember (t_lam s0 B M) as W eqn:HeqW.
1268   induction H as
1269     [ sa sb HA
1270     | Γ0 x0 A0 s1 Hfresh0 HAO IHA0
1271     | Γ0 x0 B0 M0 A0 s1 Hfresh0 HMO IHMO HBO IHBO
1272     | Γ0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1273     | Γ0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1274     | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
1275     | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1276   inversion HeqW; subst.
1277 - destruct (IHMO eq_refl) as [C [s3 [L0 [HB1 [HB2 [HB3 HB4]]]]]].
1278   assert (HBthin : Γ0 ++ [(x0, (s1, B0))] B t_sort s0)
1279     by (apply typing_weak; auto).
1280   exists C. exists s3. exists (L0 ++ dom (Γ0 ++ [(x0, (s1, B0))])).
1281   repeat split; auto.

```

```

1282 + intros y Hy.
1283 assert (HyL0 : ~ In y L0).
1284 { intro Hc. apply Hy. apply in_or_app. left. exact Hc. }
1285 assert (HyΓ : is_fresh y (Γ0 ++ [(x0, (s1, B0))])).
1286 { intro Hc. apply Hy. apply in_or_app. right. exact Hc. }
1287 apply (thinning (Γ0 ++ [(y, (s0, B))]) ((Γ0 ++ [(x0, (s1, B0))]) ++ [(y,
(s0, B))])).
1288 * exists (t_fvar s0 y). exists B.
1289 apply typing_var.
1290 -- exact HyΓ.
1291 -- exact HBthin.
1292 * apply subcontext_extend.
1293 -- apply subcontext_extend_r.
1294 -- exact HyΓ.
1295 * apply HB2; auto.
1296 + apply thinning with Γ0.
1297 * exists (t_fvar s1 x0). exists B0. apply typing_var; auto.
1298 * apply subcontext_extend_r.
1299 * exact HB3.
1300
1301 - exists B0. exists sc. exists L.
1302 repeat split; auto. apply beq_refl.
1303
1304 - destruct (IHM eq_refl) as [C [s3 [L0 [HB1 [HB2 [HB3 HB4]]]]]].
1305 exists C. exists s3. exists L0.
1306 repeat split; auto. apply beq_trans with A0; auto. apply beq_sym; auto.
1307 Qed.
1308
1309 Lemma generation_app :
1310 forall Γ M N T,
1311   Γ t_app M N T ->
1312   exists s0 A B,
1313     Γ M (t_pi s0 A B) /\
1314     Γ N A /\
1315     T =b B ^^ N.
1316 Proof.
1317   intros Γ M N T H.
1318   remember (t_app M N) as W eqn:HeqW.
1319   induction H as
1320     [ sa sb HA
1321     | Γ0 x0 A0 s1 Hfresh0 HAO IHA0
1322     | Γ0 x0 B0 M0 A0 s1 Hfresh0 HMO IHMO HBO IHBO
1323     | Γ0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1324     | Γ0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1325     | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
1326     | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ];
1327   inversion HeqW; subst.
1328
1329 - destruct (IHMO eq_refl) as [s0 [A' [B' [Q1 [Q2 Q3]]]]].
1330 exists s0. exists A'. exists B'. repeat split; auto.

```



```

1331   + apply thinning with  $\Gamma_0$ ; auto.
1332   * exists (t_fvar s1 x0). exists B0. apply typing_var; auto.
1333   * apply subcontext_extend_r.
1334   + apply thinning with  $\Gamma_0$ ; auto.
1335   * exists (t_fvar s1 x0). exists B0. apply typing_var; auto.
1336   * apply subcontext_extend_r.
1337
1338   - exists s1a. exists A0. exists B0. repeat split; auto. apply beq_refl.
1339
1340   - destruct (IHM eq_refl) as [s0 [A' [B' [Q1 [Q2 Q3]]]]].
1341     exists s0. exists A'. exists B'. repeat split; auto.
1342     apply beq_trans with A0; auto. apply beq_sym; auto.
1343 Qed.
1344
1345 (* ===== *)
1346 (** * Substitution lemma*)
1347
1348 Definition subst_decl (x : var) (N : term) '(y,(s,T)) : (var * (Sort * term)) :=
1349   (y, (s, T{x N})).
1350
1351 Definition subst_context (x : var) (N : term) ( $\Gamma$  : context) := map (subst_decl x
1352   N)  $\Gamma$ .
1353
1354 Notation " $\Gamma$  x N " := (subst_context x N  $\Gamma$ ) (at level 1, left associativity).
1355
1356 Lemma subst_context_snoc : forall x N  $\Delta$  y s A,
1357   ( $\Delta$  ++ [(y, (s, A))]) x N =  $\Delta$  x N ++ [(y, (s, A { x N }))].
1358 Proof.
1359   intros. unfold subst_context. rewrite map_app. reflexivity.
1360 Qed.
1361
1362 Lemma map_fst_subst_decl : forall x N  $\Delta$ ,
1363   map fst (map (subst_decl x N)  $\Delta$ ) = map fst  $\Delta$ .
1364 Proof.
1365   intros x N  $\Delta$ .
1366   induction  $\Delta$  as [| [v [s A]]  $\Delta_{tl}$  IH].
1367   - reflexivity.
1368   - cbn [map subst_decl fst]. f_equal. exact IH.
1369 Qed.
1370
1371 Lemma substitution :
1372   forall ( $\Gamma$  : context) ( $\Delta$  : context) x sx C M B N,
1373      $\Gamma$  ++ [(x,(sx,C))] ++  $\Delta$  M B ->
1374      $\Gamma$  N C ->
1375      $\Gamma$  ++ ( $\Delta$  x N) M{x N} B{x N}.
1376 Proof.
1377   intros  $\Gamma$   $\Delta$  x sx C M B N H1 H2.
1378   assert (H1cN : 1c N) by (apply (typing_1c  $\Gamma$  N C H2)).
1379   remember ( $\Gamma$  ++ [(x, (sx, C))] ++  $\Delta$ ) as Z eqn:HZ.
1380   generalize dependent  $\Delta$ .

```

```

1379 induction H1 as
1380   [ sa sb HA
1381   |  $\Gamma$ 0 x0 A0 s0 Hfresh0 HAO IHA0
1382   |  $\Gamma$ 0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
1383   |  $\Gamma$ 0 A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1384   |  $\Gamma$ 0 A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1385   |  $\Gamma$ 0 M0 N0 A0 B0 s1a HM IHM HN IHN
1386   |  $\Gamma$ 0 M0 A0 B0 sd HM IHM Heq HB IHB ].
1387
1388 - intros  $\Delta$  HZ. destruct  $\Gamma$ .
1389 + discriminate.
1390 + discriminate.
1391
1392 - intros  $\Delta$  HZ. destruct  $\Delta$  as [| (y, E)  $\Delta$ 1].
1393 + rewrite app_nil_r in HZ. apply app_singleton_injective in HZ.
1394   destruct HZ as [HZ1 [HZ2 HZ3]]. subst.
1395   inversion HZ3; subst.
1396   unfold subst_context. simpl map. rewrite app_nil_r.
1397   rewrite subst_var. destruct (eq_var_dec x x) as [_ | Hne]; [ |
     contradiction Hne; reflexivity].
1398   destruct (subst_fresh_typing  $\Gamma$  C (t_sort sx) x N Hfresh0 HAO) as [HC _].
1399   rewrite HC. exact H2.
1400 + assert (Hnonempty : (y, E) ::  $\Delta$ 1 <> []) by discriminate.
1401   destruct (exists_last Hnonempty) as [ $\Delta$ 1' [[y0 E0] Hlast]]. rewrite Hlast
     in *.
1402   rewrite app_assoc in HZ. rewrite app_assoc in HZ.
1403   apply app_singleton_injective in HZ. destruct HZ as [Q1 [Q2 Q3]]. subst.
1404   unfold subst_context in *.
1405   rewrite map_app in *.
1406   simpl map in *.
1407   rewrite subst_var. destruct (eq_var_dec y0 x).
1408   * subst. exfalso. apply Hfresh0.
1409     unfold dom. rewrite map_app. rewrite map_app. simpl.
1410     apply in_or_app. left. apply in_or_app. right. simpl. left. reflexivity.
1411   * rewrite app_assoc. apply typing_var.
1412     -- unfold is_fresh in *. intros Hin. apply Hfresh0.
1413       unfold dom in *. repeat rewrite map_app in *. simpl in *.
1414       rewrite map_fst_subst_decl in Hin.
1415       apply in_app_or in Hin. destruct Hin as [Hin | Hin].
1416       ++ apply in_or_app. left. apply in_or_app. left. exact Hin.
1417       ++ apply in_or_app. right. exact Hin.
1418     -- rewrite subst_sort in IHA0. apply IHA0. rewrite app_assoc.
       reflexivity.
1419
1420 - intros  $\Delta$  HZ. destruct  $\Delta$  as [| (y, E)  $\Delta$ 1].
1421 + rewrite app_nil_r in HZ. apply app_singleton_injective in HZ.
1422   destruct HZ as [HZ1 [HZ2 HZ3]]. subst.
1423   simpl in *. rewrite app_nil_r.
1424   destruct (subst_fresh_typing  $\Gamma$  M0 A0 x N Hfresh0 HMO) as [HM' HAO'].
1425   rewrite HM', HAO'. apply HMO.

```

```

1426 + assert (Hnonempty : (y, E) :: Δ1 <> []) by discriminate.
1427   destruct (exists_last Hnonempty) as [Δ1' [[y1 E1] Hlast]].
1428   rewrite Hlast in *.
1429   rewrite app_assoc in HZ. rewrite app_assoc in HZ.
1430   apply app_singleton_injective in HZ.
1431   destruct HZ as [Hf0 [Hy1 HE1]]. subst.
1432   unfold subst_context in *. rewrite map_app. simpl map.
1433   rewrite app_assoc.
1434   apply typing_weak.
1435   * unfold is_fresh in *. intro Hin. apply Hfresh0.
1436     unfold dom in *. repeat rewrite map_app in *.
1437     rewrite map_fst_subst_decl in Hin.
1438     apply in_app_or in Hin. destruct Hin as [Hin | Hin].
1439     -- apply in_or_app. left. apply in_or_app. left. exact Hin.
1440     -- apply in_or_app. right. exact Hin.
1441   * apply IHM0. rewrite <- app_assoc. reflexivity.
1442   * rewrite subst_sort in IHB0. apply IHB0. rewrite <- app_assoc.
    reflexivity.

1443
1444 - intros Δ HZ.
1445   apply (typing_pi (Γ ++ Δ x N) (A0 { x N }) (B0 { x N }) sa sb sc
1446     (x :: L ++ fv N)).
1447   + rewrite <- (subst_sort sa x N). apply IHA. exact HZ.
1448   + apply bvar_tag_ok_subst; [exact HlcN | exact HTagB].
1449   + intros y Hy.
1450     assert (Hyx : y <> x).
1451     { intro Heq. subst y. apply Hy. simpl. left. reflexivity. }
1452     assert (HyL' : ~ In y L).
1453     { intro Hc. apply Hy. simpl. right. apply in_or_app. left. exact Hc. }
1454     rewrite <- (subst_open_var B0 sa y x N Hyx HlcN).
1455     rewrite <- (subst_sort sb x N).
1456     rewrite <- app_assoc. rewrite <- subst_context_snoc.
1457     apply (IHB y HyL' (Δ ++ [(y, (sa, A0))])).
1458     rewrite HZ. rewrite <- app_assoc. reflexivity.
1459   + exact HR.

1460
1461 - intros Δ HZ.
1462   apply (typing_lam (Γ ++ Δ x N) (A0 { x N }) (M0 { x N }) (B0 { x N
1463     sa sc (x :: L ++ fv N)).
1464   + rewrite <- (subst_sort sa x N). apply IHA. exact HZ.
1465   + apply bvar_tag_ok_subst; [exact HlcN | exact HTagM].
1466   + intros y Hy.
1467     assert (Hyx : y <> x).
1468     { intro Heq. subst y. apply Hy. simpl. left. reflexivity. }
1469     assert (HyL' : ~ In y L).
1470     { intro Hc. apply Hy. simpl. right. apply in_or_app. left. exact Hc. }
1471     rewrite <- (subst_open_var M0 sa y x N Hyx HlcN).
1472     rewrite <- (subst_open_var B0 sa y x N Hyx HlcN).
1473     rewrite <- app_assoc. rewrite <- subst_context_snoc.

```

```

1474     apply (IHM y HyL' ( $\Delta$  ++ [(y, (sa, A0))])).
1475     rewrite HZ. rewrite <- app_assoc. reflexivity.
1476 + rewrite <- (subst_pi sa A0 B0 x N). apply IHPi. exact HZ.
1477
1478 - intros  $\Delta$  HZ.
1479     rewrite subst_open by exact HlcN.
1480     apply typing_app with (A := A0 { x N }) (s1 := s1a).
1481 + rewrite <- (subst_pi s1a A0 B0 x N). apply IHM. exact HZ.
1482 + apply IHN. exact HZ.
1483
1484 - intros  $\Delta$  HZ.
1485     apply typing_conv with (A0 { x N }) sd.
1486 + apply IHM. exact HZ.
1487 + apply beta_eq_subst; auto.
1488 + rewrite <- (subst_sort sd x N). apply IHB. exact HZ.
1489 Qed.
1490
1491 Lemma type_correctness:
1492   forall  $\Gamma$  M N,
1493      $\Gamma \vdash M \vdash N \rightarrow$ 
1494     (exists s, N = t_sort s) /\
1495     (exists s,  $\Gamma \vdash N$  (t_sort s)).
1496 Proof.
1497   intros  $\Gamma$  M N Htyp.
1498   induction Htyp as
1499     [ sa sb HA
1500     |  $\Gamma_0$  x0 A0 s0 Hfresh0 HAO IHA0
1501     |  $\Gamma_0$  x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
1502     |  $\Gamma_0$  A0 B0 sa sb sc L HA IHA HTagB HB IHB HR
1503     |  $\Gamma_0$  A0 M0 B0 sa sc L HA IHA HTagM HM IHM HPi IHPi
1504     |  $\Gamma_0$  M0 N0 A0 B0 s1a HM IHM HN IHN
1505     |  $\Gamma_0$  M0 A0 B0 sd HM IHM Heq HB IHB ].
1506
1507 - left. exists sb. reflexivity.
1508
1509 - right. exists s0. apply typing_weak; auto.
1510
1511 - destruct IHMO as [[s Hs] | [s Hs]].
1512 + left. exists s. apply Hs.
1513 + right. exists s. apply thinning with  $\Gamma_0$ ; auto.
1514   * exists (t_fvar s0 x0). exists B0. apply typing_var; auto.
1515   * apply subcontext_extend_r.
1516
1517 - left. exists sc. reflexivity.
1518
1519 - right. exists sc. apply HPi.
1520
1521 - destruct IHM as [[s Hs] | [s Hs]].
1522 + discriminate Hs.
1523 + right.

```

```

1524   destruct (generation_pi Γ0 s1a A0 B0 (t_sort s) Hs)
1525     as [s2 [s3 [L [HB1 [HTag1 [HB2 [HB3 HB4]]]]]]].
1526   exists s2.
1527   remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
1528   assert (Hx0L : ~ In x0 L).
1529   { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
1530     apply in_or_app. left. exact Hc. }
1531   assert (Hx0B : ~ In x0 (fv B0)).
1532   { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
1533     apply in_or_app. right. exact Hc. }
1534   assert (Hopen : Γ0 ++ [(x0, (s1a, A0))] ++ [] open_var B0 s1a x0 t_sort
1535     s2).
1536   { rewrite app_nil_r. apply HB2; auto. }
1537   pose proof (substitution Γ0 [] x0 s1a A0 (open_var B0 s1a x0) (t_sort s2)
1538     NO Hopen HN)
1539   as Hsub.
1540   simpl in Hsub.
1541   assert (Heq : (open_var B0 s1a x0) { x0 NO } = B0 ^^ NO).
1542   { symmetry. apply subst_intro; auto. apply (typing_lc Γ0 NO A0 HN). }
1543   rewrite Heq in Hsub. rewrite app_nil_r in Hsub. exact Hsub.
1544   - right. exists sd. exact HB.
1545   Qed.
1546   (* ===== *)
1547   (** * Sort-order metatheory *)
1548
1549   Axiom permutation :
1550     forall Γ Δ x y sx sy Tx Ty M C,
1551     ~ In x (fv Ty) ->
1552     Γ ++ [(x, (sx, Tx))] ++ [(y, (sy, Ty))] ++ Δ M C ->
1553     Γ ++ [(y, (sy, Ty))] ++ [(x, (sx, Tx))] ++ Δ M C.
1554
1555   Definition functional : Prop :=
1556     (forall s t t', A s t -> A s t' -> t = t') /\
1557     (forall s t u u', R s t u -> R s t u' -> u = u').
1558
1559   Axiom type_unicity :
1560     functional ->
1561     forall Γ M T1 T2,
1562     Γ M T1 ->
1563     Γ M T2 ->
1564     T1 = T2.
1565
1566   Definition top_sort (s : Sort) : Prop := ~ (exists s', A s s').
1567
1568   Definition bottom_sort (s : Sort) : Prop := ~ (exists s', A s' s).
1569
1570   Axiom top_sort_lemma :
1571     forall Γ x y M N s,

```

```

1572 top_sort s ->
1573 (~ (Γ t_sort s M)) /\
1574 (~ (Γ t_fvar x y t_sort s)) /\
1575 (~ (Γ t_app M N t_sort s)).
1576
1577 Definition persistent : Prop :=
1578 functional /\
1579 (forall s s' t , A s t -> A s' t -> s = s') /\
1580 (forall s t u, R s t u -> t = u).
1581
1582 Definition tiered (n : nat) : Prop :=
1583 exists index_of : Sort -> nat,
1584 (forall x : Sort, 0 < index_of x /\ index_of x < n + 1) /\
1585 (forall x y : Sort, index_of x = index_of y -> x = y) /\
1586 (forall i : nat, 0 < i /\ i < n + 1 -> exists x : Sort, index_of x = i) /\
1587 (forall x y : Sort, A x y <-> index_of x < n /\ index_of y = S (index_of x))
1588 /\
1589 (forall x y z : Sort, R x y z -> y = z).
1590
1591 Definition A_neighbor (s : Sort) (p : Sort * Sort) : Prop :=
1592 A (fst p) (snd p) /\ (fst p = s \/ snd p = s).
1593
1594 Lemma persistent_neighbor_bound :
1595 persistent ->
1596 forall s p1 p2 p3,
1597 A_neighbor s p1 -> A_neighbor s p2 -> A_neighbor s p3 ->
1598 p1 = p2 \/ p1 = p3 \/ p2 = p3.
1599
1600 Proof.
1601 intros [[Hfunc _] [Hpers _]] s [x1 y1] [x2 y2] [x3 y3]
1602 [HA1 [Hx1|Hy1]] [HA2 [Hx2|Hy2]] [HA3 [Hx3|Hy3]].
1603 - subst. simpl in *. subst. left. f_equal. eauto using Hfunc, Hpers.
1604 - subst. simpl in *. subst. left. f_equal. eauto using Hfunc, Hpers.
1605 - subst. simpl in *. subst. right. left. f_equal. eauto using Hfunc,
1606 Hpers.
1607 - subst. simpl in *. subst. right. right. f_equal. eauto using Hfunc,
1608 Hpers.
1609 - subst. simpl in *. subst. right. right. f_equal. eauto using Hfunc,
1610 Hpers.
1611 - subst. simpl in *. subst. right. left. f_equal. eauto using Hfunc,
1612 Hpers.
1613 - subst. simpl in *. subst. left. f_equal. eauto using Hfunc, Hpers.
1614 - subst. simpl in *. subst. left. f_equal. eauto using Hfunc, Hpers.
1615 Qed.
1616
1617 Reserved Notation "s <A t" (at level 70, no associativity).
1618
1619 Inductive A_lt : Sort -> Sort -> Prop :=
1620 | A_lt_step : forall s t, A s t -> s <A t
1621 | A_lt_trans : forall s t u, A s t -> t <A u -> s <A u

```

```

1616
1617 where "s <A t" := (A_lt s t).
1618
1619 Reserved Notation "s A t" (at level 70, no associativity).
1620
1621 Inductive A_le : Sort -> Sort -> Prop :=
1622   | A_le_refl : forall s, s A s
1623   | A_le_step : forall s t u, A s t -> t A u -> s A u
1624
1625 where "s A t" := (A_le s t).
1626
1627 Reserved Notation "s A t" (at level 70, no associativity).
1628
1629 Inductive A_eq : Sort -> Sort -> Prop :=
1630   | A_eq_refl : forall s, s A s
1631   | A_eq_step : forall s t, A s t -> s A t
1632   | A_eq_sym : forall s t, s A t -> t A s
1633   | A_eq_trans : forall s t u, s A t -> t A u -> s A u
1634
1635 where "s A t" := (A_eq s t).
1636
1637 Lemma A_le_of_lt : forall s t, s <A t -> s A t.
1638 Proof. induction 1; econstructor; eauto. apply A_le_refl. Qed.
1639
1640 Lemma A_lt_trans' : forall s t u, s <A t -> t <A u -> s <A u.
1641 Proof.
1642   intros s t u H1 H2.
1643   induction H1.
1644   - apply A_lt_trans with t; auto.
1645   - apply IHA_lt in H2. apply A_lt_trans with t; auto.
1646 Qed.
1647
1648 Fixpoint chain (c : list Sort) : Prop :=
1649   match c with
1650   | [] | [_] => True
1651   | t1 :: (t2 :: _) as rest => A t1 t2 /\ chain rest
1652   end.
1653
1654 Definition chain_from_to (s t : Sort) (c : list Sort) : Prop :=
1655   chain c /\ hd_error c = Some s /\ List.last c s = t.
1656
1657 Lemma last_default_irrelevant :
1658   forall (c : list Sort) (d1 d2 : Sort),
1659     c <> [] -> last c d1 = last c d2.
1660 Proof.
1661   induction c as [| a c IH]; intros d1 d2 Hne.
1662   - contradiction.
1663   - destruct c as [| b c'].
1664     + reflexivity.
1665     + simpl. apply IH. discriminate.

```

```

1666 Qed.
1667
1668 Lemma A_le_iff_chain : forall s t,
1669   s A t <=> exists c, chain_from_to s t c.
1670 Proof.
1671   intros s t. repeat split.
1672   - intros H. induction H as [| s t u H1 H2 [c IH]].
1673     + exists [s]. repeat split.
1674     + destruct IH as [Q1 [Q2 Q3]]. exists (s :: c). repeat split.
1675       * destruct c.
1676         ** reflexivity.
1677         ** repeat split; auto. simpl in Q2. injection Q2 as Q2. subst. apply H1.
1678       * destruct c as [| c0 c'] eqn:Hc.
1679         ** discriminate.
1680         ** simpl. simpl in Q3.
1681           injection Q2 as Q2. subst c0.
1682           destruct c' as [| c1 c''].
1683             *** exact Q3.
1684             *** rewrite (last_default_irrelevant (c1 :: c'') s t).
1685               **** exact Q3.
1686               **** discriminate.
1687
1688   - intros [c Hc0]. generalize dependent s.
1689     induction c as [| a c IH]; intros s [H1 [H2 H3]].
1690     + discriminate.
1691     + injection H2 as H2. subst a.
1692       destruct c as [| c0 c'].
1693       * simpl in H3. subst t. apply A_le_refl.
1694       * simpl in H1. destruct H1 as [HA Hchain]. apply A_le_step with (t := c0).
1695         -- exact HA.
1696         -- apply IH. repeat split.
1697           ++ exact Hchain.
1698           ++ destruct c' as [| c1 c''] eqn:E'.
1699             ** simpl in H3 |- *. exact H3.
1700             ** simpl in H3 |- *.
1701               destruct c'' as [| c2 c3].
1702               --- exact H3.
1703               --- rewrite (last_default_irrelevant (c2 :: c3) c0 s).
1704                 +++ exact H3.
1705                 +++ discriminate.
1706 Qed.
1707
1708 Lemma persistent_chain_unique_from :
1709   persistent ->
1710   forall c1 c2 s,
1711     chain c1 -> chain c2 ->
1712     hd_error c1 = Some s -> hd_error c2 = Some s ->
1713     length c1 = length c2 ->
1714     c1 = c2.
1715 Proof.

```



```

1716   intros Hpers c1.
1717   induction c1 as [| a1 c1 IH]; intros c2 s H1 H2 Hh1 Hh2 Hlen.
1718   - discriminate Hh1.
1719   - injection Hh1 as Hh1. subst a1. destruct c2 as [| a2 c2].
1720     + discriminate Hh2.
1721     + injection Hh2 as Hh2. subst a2. destruct c1 as [| b1 c1'].
1722       * destruct c2 as [| b2 c2']; auto. discriminate Hlen.
1723       * destruct c2 as [| b2 c2'].
1724         -- discriminate Hlen.
1725         -- simpl in H1, H2.
1726           destruct H1 as [HA1 Hc1'], H2 as [HA2 Hc2'].
1727           assert (Hb : b1 = b2).
1728           { destruct Hpers as [[Hfunc1 _] _].
1729             eapply Hfunc1; eauto. }
1730           subst b2.
1731           f_equal.
1732           apply IH with (s := b1); auto.
1733   Qed.
1734
1735   Lemma chain_last :
1736     forall c x y,
1737       chain (c ++ [x;y]) ->
1738       A x y.
1739   Proof.
1740     induction c as [|a c IH].
1741     - simpl. tauto.
1742     - destruct c.
1743       + simpl. tauto.
1744       + intros. destruct H as [_ H]; auto.
1745   Qed.
1746
1747   Lemma chain_prefix :
1748     forall c x y,
1749       chain (c ++ [x; y]) ->
1750       chain (c ++ [x]).
1751   Proof.
1752     induction c as [|a c IH].
1753     - reflexivity.
1754     - destruct c as [|b c].
1755       + simpl. tauto.
1756       + simpl. intros x y [Hab Hchain]. split; auto. apply IH with y; auto.
1757   Qed.
1758
1759   Lemma chain_snoc_inv : forall l x,
1760     chain (l ++ [x]) ->
1761     chain l /\ (l <> [] -> A (last l x) x).
1762   Proof.
1763     induction l as [| a l IH]; intros x Hc.
1764     - simpl. split; [exact I | intros []; reflexivity].
1765     - destruct l as [| b l'].

```

```

1766 + simpl in Hc. destruct Hc as [HA _].
1767     simpl. split; [exact I | intros _; simpl; exact HA].
1768 + simpl in Hc. destruct Hc as [HAab Hc'].
1769     destruct (IH x Hc') as [IHchain IHlast].
1770     split.
1771     * simpl. split; [exact HAab | exact IHchain].
1772     * intros _.
1773         specialize (IHlast ltac:(discriminate)).
1774         simpl in IHlast.
1775         simpl.
1776         exact IHlast.
1777 Qed.
1778
1779 Lemma persistent_chain_unique_to :
1780   persistent ->
1781   forall c1 t s1 c2 s2,
1782     chain_from_to s1 t c1 -> chain_from_to s2 t c2 ->
1783     length c1 = length c2 ->
1784     c1 = c2.
1785 Proof.
1786   intros Hpers c1.
1787   induction c1 as [| x l IH] using rev_ind; intros t s1 c2 s2 H1 H2 Hlen.
1788   - destruct H1 as [_ [Hh _]]. discriminate Hh.
1789   - destruct H1 as [Hc1 [Hh1 Hlast1]].
1790     rewrite last_last in Hlast1. subst x.
1791     destruct c2 as [| c2h c2t].
1792     + destruct H2 as [_ [Hh2 _]]. discriminate Hh2.
1793     + assert (Hne2 : c2h :: c2t <> []) by discriminate.
1794       destruct (exists_last Hne2) as [l2 [x2 Hl2eq]].
1795       rewrite Hl2eq in H2.
1796       destruct H2 as [Hc2 [Hh2 Hlast2]].
1797       rewrite last_last in Hlast2. subst x2.
1798       pose proof (chain_snoc_inv l t Hc1) as [Hc1 Hal].
1799       pose proof (chain_snoc_inv l2 t Hc2) as [Hc12 Hal2].
1800       destruct l as [| a l'].
1801       ++ destruct l2 as [| b l2'].
1802         * rewrite Hl2eq. reflexivity.
1803         * exfalso.
1804           assert (Hlen2 : length (c2h :: c2t) = length ((b :: l2') ++ [t])).
1805             { rewrite Hl2eq. reflexivity. }
1806           rewrite length_app in Hlen2.
1807           simpl in Hlen2, Hlen.
1808           lia.
1809       ++ destruct l2 as [| b l2'].
1810         * exfalso.
1811           assert (Hlen2 : length (c2h :: c2t) = length ([] ++ [t])).
1812             { rewrite Hl2eq. reflexivity. }
1813           simpl in Hlen2.
1814           rewrite length_app in Hlen.
1815           simpl in Hlen.

```

```

1816     lia.
1817 * assert (Hne : a :: l' <> []) by discriminate.
1818 assert (Hne2' : b :: l2' <> []) by discriminate.
1819 specialize (Hal Hne). specialize (Hal2 Hne2').
1820 assert (Ht' : last (a :: l') t = last (b :: l2') t).
1821 { destruct Hpers as [_ [Hpred _]]. exact (Hpred _ _ t Hal Hal2). }
1822 assert (Hlen' : length (a :: l') = length (b :: l2')).
1823 { simpl. rewrite Hl2eq, !length_app in Hlen. simpl in Hlen. lia. }
1824 assert (Heq : a :: l' = b :: l2').
1825 { apply (IH (last (a :: l') t) a (b :: l2') b).
1826   - split; [exact Hcl1|. split; [reflexivity|].
1827     apply (last_default_irrelevant (a :: l') a t). discriminate.
1828   - split; [exact Hcl2|. split; [reflexivity|].
1829     rewrite Ht'.
1830     apply (last_default_irrelevant (b :: l2') b t). discriminate.
1831   - exact Hlen'. }
1832 rewrite Hl2eq. f_equal. exact Heq.
1833 Qed.
1834
1835 Definition separable : Prop :=
1836   forall s s', R s s' s' -> s A s'.
1837
1838 Definition atomic : Prop :=
1839   forall s s', s A s'.
1840
1841 Definition ascending_chain_condition : Prop :=
1842   forall s : Sort, Acc (fun u v => u <A v) s.
1843
1844 Definition descending_chain_condition : Prop :=
1845   forall s : Sort, Acc (fun u v => v <A u) s.
1846
1847 Definition bounded : Prop :=
1848   ascending_chain_condition /\ descending_chain_condition.
1849
1850 Definition weakly_non_dependent : Prop :=
1851   forall s t u, R s t u -> u A t /\ t A s.
1852
1853 Definition stratified : Prop :=
1854   ascending_chain_condition /\ weakly_non_dependent.
1855
1856 Definition generalized_non_dependent : Prop :=
1857   stratified /\ persistent.
1858
1859 Definition bounded_non_dependent : Prop :=
1860   generalized_non_dependent /\ bounded.
1861
1862 Lemma wf_by_measure :
1863   forall (T : Type) (Rel : T -> T -> Prop) (f : T -> nat),
1864     (forall u v, Rel u v -> f u < f v) ->
1865     forall s, Acc (fun u v => Rel u v) s.

```

```

1866 Proof.
1867   intros T Rel f Hmono s.
1868   remember (f s) as k eqn:Hk.
1869   generalize dependent s.
1870   induction k as [k IH] using (well_founded_induction Wf_nat.lt_wf).
1871   intros s Hk.
1872   constructor.
1873   intros y Hy.
1874   apply (IH (f y)).
1875   - rewrite Hk. apply Hmono. exact Hy.
1876   - reflexivity.
1877 Qed.
1878
1879 Lemma acc_irrefl :
1880   forall (T : Type) (Rel : T -> T -> Prop) (x : T),
1881   Acc Rel x -> ~ Rel x x.
1882 Proof.
1883   intros T Rel x Hacc.
1884   induction Hacc as [x _ IH].
1885   intro Hxx.
1886   exact (IH x Hxx Hxx).
1887 Qed.
1888
1889 Lemma A_lt_irrefl : bounded -> forall s, ~ (s <A s).
1890 Proof.
1891   intros Hbnd s Hlt.
1892   destruct Hbnd as [Hasc _].
1893   exact (acc_irrefl Sort (fun u v => u <A v) s (Hasc s) Hlt).
1894 Qed.
1895
1896 Lemma chain_skipn : forall c k, chain c -> chain (skipn k c).
1897 Proof.
1898   induction c as [| a c IH]; intros k Hc.
1899   - destruct k; simpl; exact I.
1900   - destruct k as [| k].
1901     + simpl. exact Hc.
1902     + simpl. apply IH.
1903       destruct c as [| b c']; simpl in Hc.
1904       * exact I.
1905       * destruct Hc as [_ Hc]. exact Hc.
1906 Qed.
1907
1908 Lemma chain_firstn : forall c k, chain c -> chain (firstn k c).
1909 Proof.
1910   induction c as [| a c IH]; intros k Hc.
1911   - destruct k; simpl; exact I.
1912   - destruct k as [| k].
1913     + simpl. exact I.
1914     + destruct c as [| b c'].
1915       * simpl. destruct k; exact I.

```

```

1916      * simpl in Hc |- *. destruct Hc as [HAab Hc'].
1917      assert (Hfk : chain (firstn k (b :: c')) by (apply IH; simpl;
1918      assumption).
1919      destruct (firstn k (b :: c')) as [| t2 rest] eqn:Efk.
1920      -- exact I.
1921      -- split.
1922      ++ destruct k as [| k'].
1923      ** simpl in Efk. discriminate Efk.
1924      ** simpl in Efk. injection Efk as Ht2. subst t2. exact HAab.
1925      ++ exact Hfk.
1926
1927      Qed.
1928
1929      Lemma hd_error_skipn : forall c k a,
1930      nth_error c k = Some a -> hd_error (skipn k c : list Sort) = Some a.
1931      Proof.
1932      induction c as [| x c IH]; intros k a Hnth.
1933      - destruct k; discriminate.
1934      - destruct k as [| k].
1935      + simpl in Hnth |- *. exact Hnth.
1936      + simpl in Hnth |- *. apply IH. exact Hnth.
1937
1938      Qed.
1939
1940      Lemma last_skipn : forall c k d,
1941      k < length c -> last (skipn k c : list Sort) d = last c d.
1942      Proof.
1943      induction c as [| x c IH]; intros k d Hk.
1944      - simpl in Hk. lia.
1945      - destruct k as [| k].
1946      + reflexivity.
1947      + simpl. destruct c as [| y c'].
1948      * simpl in Hk. lia.
1949      * apply IH. simpl in *. lia.
1950
1951      Qed.
1952
1953      Lemma hd_error_firstn : forall c k a,
1954      hd_error c = Some a -> 0 < k -> hd_error (firstn k c : list Sort) = Some a.
1955      Proof.
1956      intros c k a Hhd Hk.
1957      destruct c as [| x c]; [discriminate |].
1958      simpl in Hhd. injection Hhd as Hhd. subst x.
1959      destruct k as [| k]; [lia |]. reflexivity.
1960
1961      Qed.
1962
1963      Lemma last_firstn_nth : forall c k a d,
1964      nth_error c k = Some a -> last (firstn (S k) c : list Sort) d = a.
1965      Proof.
1966      induction c as [| x c IH]; intros k a d Hnth.
1967      - destruct k as [| k']; simpl in Hnth; discriminate Hnth.
1968      - destruct k as [| k].
1969      + simpl in Hnth. injection Hnth as Hnth. subst a. simpl.

```

```

1965     destruct c; reflexivity.
1966 + simpl in Hnth. simpl.
1967     destruct c as [| y c'].
1968     * destruct k as [| k']; simpl in Hnth; discriminate Hnth.
1969     * apply (IH k a d Hnth).
1970 Qed.
1971
1972 Lemma length_skipn : forall (c : list Sort) k,
1973   k <= length c -> length (skipn k c) = length c - k.
1974 Proof. intros c k Hk. apply length_skipn. Qed.
1975
1976 Lemma length_firstn_le : forall (c : list Sort) k,
1977   k <= length c -> length (firstn k c) = k.
1978 Proof. intros c k Hk. rewrite length_firstn. lia. Qed.
1979
1980 Lemma chain_pos_length_A_lt :
1981   forall c a b, chain_from_to a b c -> 2 <= length c -> a <A b.
1982 Proof.
1983   induction c as [| x l IH] using rev_ind; intros a b [Hc [Hh Hl]] Hlen.
1984   - simpl in Hlen. lia.
1985   - rewrite last_last in Hl. subst x.
1986     destruct l as [| y l'].
1987     + simpl in Hlen. lia.
1988     + destruct (chain_snoc_inv (y :: l') b Hc) as [Hcl Hal].
1989       specialize (Hal ltac:(discriminate)).
1990       destruct l' as [| z l''].
1991       * simpl in Hh. injection Hh as Hh. subst y.
1992         apply A_lt_step. exact Hal.
1993       * apply A_lt_trans' with (last (y :: z :: l'') b).
1994         -- apply IH.
1995         ++ split; [exact Hcl |]. split; [exact Hh |].
1996           apply last_default_irrelevant. discriminate.
1997         ++ simpl. lia.
1998       -- apply A_lt_step. exact Hal.
1999 Qed.
2000
2001 Lemma nth_error_exists : forall (c : list Sort) k,
2002   k < length c -> exists a, nth_error c k = Some a.
2003 Proof.
2004   induction c as [| x c IH]; intros k Hk.
2005   - simpl in Hk. lia.
2006   - destruct k as [| k'].
2007     + exists x. reflexivity.
2008     + simpl. apply IH. simpl in Hk. lia.
2009 Qed.
2010
2011 Lemma chain_same_end_relates :
2012   persistent ->
2013   forall s u t c1 c2,
2014     chain_from_to s t c1 -> chain_from_to u t c2 ->

```

```

2015     s <A u \ / s = u \ / u <A s.
2016 Proof.
2017     intros Hpers s u t c1 c2 H1 H2.
2018     assert (Htri : length c1 < length c2 \ / length c1 = length c2 \ / length c2 <
length c1) by lia.
2019     destruct Htri as [Hlt | [Heq | Hgt]].
2020
2021     - (* c1 shorter: u <A s *)
2022       right. right.
2023       destruct H1 as [Hc1 [Hh1 Hl1]].
2024       destruct H2 as [Hc2 [Hh2 Hl2]].
2025       remember (length c2 - length c1) as k eqn: Hkeq.
2026       assert (Hc1ne : 1 <= length c1).
2027       { destruct c1 as [| ? ?]; simpl.
2028         - discriminate Hh1.
2029         - lia. }
2030       assert (Hk1 : 0 < k) by lia.
2031       assert (Hklt : k < length c2) by lia.
2032       destruct (nth_error_exists c2 k Hklt) as [a Hnth].
2033       pose proof (hd_error_skipn c2 k a Hnth) as Hheadskip.
2034       pose proof (chain_skipn c2 k Hc2) as Hchainskip.
2035       pose proof (last_skipn c2 k a Hklt) as Hlastskip.
2036       assert (Hlast_a_u : last c2 a = last c2 u).
2037       { apply last_default_irrelevant. destruct c2; [discriminate Hh2 |
discriminate]. }
2038       assert (Hlastskip' : last (skipn k c2) a = t).
2039       { rewrite Hlastskip, Hlast_a_u. exact Hl2. }
2040       assert (Hsuffix : chain_from_to a t (skipn k c2)).
2041       { split; [exact Hchainskip | split; [exact Hheadskip | exact Hlastskip']]. }
2042       assert (Hlensuf : length (skipn k c2) = length c1).
2043       { rewrite (length_skipn c2 k ltac:(lia)). lia. }
2044       assert (Heqc : c1 = skipn k c2).
2045       { apply (persistent_chain_unique_to Hpers c1 t s (skipn k c2) a).
2046         - split; [exact Hc1 | split; [exact Hh1 | exact Hl1]].
2047         - exact Hsuffix.
2048         - symmetry. exact Hlensuf. }
2049       assert (Hsa : s = a).
2050       { assert (Hh1' : hd_error c1 = Some s) by exact Hh1.
2051         rewrite Heqc in Hh1'. rewrite Hheadskip in Hh1'. injection Hh1' as Hh1'.
         symmetry. exact Hh1'. }
2052       assert (Hprefix : chain_from_to u s (firstn (S k) c2)).
2053       { split.
2054         - apply chain_firstn. exact Hc2.
2055         - split.
2056           + apply hd_error_firstn; [exact Hh2 | lia].
2057           + pose proof (last_firstn_nth c2 k a u Hnth) as Hlf.
2058             rewrite Hsa. exact Hlf. }
2059       assert (Hlenpref : length (firstn (S k) c2) = S k).
2060       { apply length_firstn_le. lia. }
2061       apply (chain_pos_length_A_lt (firstn (S k) c2) u s Hprefix).

```

```

2062     rewrite Hlenpref. lia.
2063
2064 - (* equal length: s = u *)
2065     right. left.
2066     destruct H1 as [Hc1 [Hh1 Hl1]].
2067     destruct H2 as [Hc2 [Hh2 Hl2]].
2068     assert (Heqc : c1 = c2).
2069     { apply (persistent_chain_unique_to Hpers c1 t s c2 u).
2070       - split; [exact Hc1 | split; [exact Hh1 | exact Hl1]].
2071       - split; [exact Hc2 | split; [exact Hh2 | exact Hl2]].
2072       - exact Heq. }
2073     rewrite Heqc in Hh1. rewrite Hh1 in Hh2. injection Hh2 as Hh2. exact Hh2.
2074
2075 - (* c2 shorter: s <A u, mirror of the first case *)
2076     left.
2077     destruct H1 as [Hc1 [Hh1 Hl1]].
2078     destruct H2 as [Hc2 [Hh2 Hl2]].
2079     remember (length c1 - length c2) as k eqn:Hkeq.
2080     assert (Hc2ne : 1 <= length c2).
2081     { destruct c2 as [| ? ?]; simpl.
2082       - discriminate Hh2.
2083       - lia. }
2084     assert (Hk1 : 0 < k) by lia.
2085     assert (Hklt : k < length c1) by lia.
2086     destruct (nth_error_exists c1 k Hklt) as [a Hnth].
2087     pose proof (hd_error_skipn c1 k a Hnth) as Hheadskip.
2088     pose proof (chain_skipn c1 k Hc1) as Hchainskip.
2089     pose proof (last_skipn c1 k a Hklt) as Hlastskip.
2090     assert (Hlast_a_s : last c1 a = last c1 s).
2091     { apply last_default_irrelevant. destruct c1; [discriminate Hh1 |
discriminate]. }
2092     assert (Hlastskip' : last (skipn k c1) a = t).
2093     { rewrite Hlastskip, Hlast_a_s. exact Hl1. }
2094     assert (Hsuffix : chain_from_to a t (skipn k c1)).
2095     { split; [exact Hchainskip | split; [exact Hheadskip | exact Hlastskip']]. }
2096     assert (Hlensuf : length (skipn k c1) = length c2).
2097     { rewrite (length_skipn c1 k ltac:(lia)). lia. }
2098     assert (Heqc : c2 = skipn k c1).
2099     { apply (persistent_chain_unique_to Hpers c2 t u (skipn k c1) a).
2100       - split; [exact Hc2 | split; [exact Hh2 | exact Hl2]].
2101       - exact Hsuffix.
2102       - symmetry. exact Hlensuf. }
2103     assert (Hua : u = a).
2104     { assert (Hh2' : hd_error c2 = Some u) by exact Hh2.
2105       rewrite Heqc in Hh2'. rewrite Hheadskip in Hh2'. injection Hh2' as Hh2'.
2106       symmetry. exact Hh2'. }
2107     assert (Hprefix : chain_from_to s u (firstn (S k) c1)).
2108     { split.
2109       - apply chain_firstn. exact Hc1.
2110       - split.

```



```

2110     + apply hd_error_firstn; [exact Hh1 | lia].
2111     + pose proof (last_firstn_nth c1 k a s Hnth) as Hlf.
2112     rewrite Hua. exact Hlf. }
2113 assert (Hlenpref : length (firstn (S k) c1) = S k).
2114 { apply length_firstn_le. lia. }
2115 apply (chain_pos_length_A_lt (firstn (S k) c1) s u Hprefix).
2116 rewrite Hlenpref. lia.
2117 Qed.
2118
2119 Lemma chain_same_start_relates :
2120   persistent ->
2121   forall s t u c1 c2,
2122     chain_from_to s t c1 -> chain_from_to s u c2 ->
2123     t <A u  $\vee$  t = u  $\vee$  u <A t.
2124 Proof.
2125   intros Hpers s t u c1 c2 H1 H2.
2126   assert (Htri : length c1 < length c2  $\vee$  length c1 = length c2  $\vee$  length c2 <
length c1) by lia.
2127   destruct Htri as [Hlt | [Heq | Hgt]].
2128
2129   - (* c1 shorter: t <A u *)
2130     left.
2131     destruct H1 as [Hc1 [Hh1 Hl1]].
2132     destruct H2 as [Hc2 [Hh2 Hl2]].
2133     assert (Hc1ne : 1 <= length c1).
2134     { destruct c1 as [| ? ?]; simpl.
2135       - discriminate Hh1.
2136       - lia. }
2137     remember (length c1 - 1) as j eqn:Hjeq.
2138     assert (Hjlt : j < length c2) by lia.
2139     destruct (nth_error_exists c2 j Hjlt) as [a Hnth].
2140     assert (Hlenpref : length (firstn (length c1) c2) = length c1).
2141     { apply length_firstn_le. lia. }
2142     assert (Hchainpref : chain (firstn (length c1) c2)).
2143     { apply chain_firstn. exact Hc2. }
2144     assert (Hheadpref : hd_error (firstn (length c1) c2) = Some s).
2145     { apply hd_error_firstn; [exact Hh2 | lia]. }
2146     assert (Heqc : c1 = firstn (length c1) c2).
2147     { apply (persistent_chain_unique_from Hpers c1 (firstn (length c1) c2) s).
2148       - exact Hc1.
2149       - exact Hchainpref.
2150       - exact Hh1.
2151       - exact Hheadpref.
2152       - symmetry. exact Hlenpref. }
2153     assert (Hat : a = t).
2154     { pose proof (last_firstn_nth c2 j a s Hnth) as Hlf.
2155       assert (HSj : S j = length c1) by lia.
2156       rewrite HSj in Hlf.
2157       rewrite <- Heqc in Hlf.
2158       rewrite Hl1 in Hlf.

```

```

2159     symmetry. exact H1f. }
2160 pose proof (hd_error_skipn c2 j a Hnth) as Hheadskip.
2161 pose proof (chain_skipn c2 j Hc2) as Hchainskip.
2162 pose proof (last_skipn c2 j a Hjlt) as Hlastskip.
2163 assert (Hlast_a_s : last c2 a = last c2 s).
2164 { apply last_default_irrelevant. destruct c2; [discriminate Hh2 |
discriminate]. }
2165 assert (Hlastskip' : last (skipn j c2) a = u).
2166 { rewrite Hlastskip, Hlast_a_s. exact H12. }
2167 assert (Hsuffix : chain_from_to t u (skipn j c2)).
2168 { rewrite <- Hat.
2169   split; [exact Hchainskip | split; [exact Hheadskip | exact Hlastskip']] . }
2170 assert (Hlensuf : length (skipn j c2) = length c2 - j).
2171 { apply length_skipn. lia. }
2172 apply (chain_pos_length_A_lt (skipn j c2) t u Hsuffix).
2173 rewrite Hlensuf. lia.
2174
2175 - (* equal length: t = u *)
2176 right. left.
2177 destruct H1 as [Hc1 [Hh1 H11]].
2178 destruct H2 as [Hc2 [Hh2 H12]].
2179 assert (Heqc : c1 = c2).
2180 { apply (persistent_chain_unique_from Hpers c1 c2 s).
2181   - exact Hc1.
2182   - exact Hc2.
2183   - exact Hh1.
2184   - exact Hh2.
2185   - exact Heq. }
2186 rewrite Heqc in H11.
2187 rewrite H11 in H12.
2188 exact H12.
2189
2190 - (* c2 shorter: u <A t, mirror of first case *)
2191 right. right.
2192 destruct H1 as [Hc1 [Hh1 H11]].
2193 destruct H2 as [Hc2 [Hh2 H12]].
2194 assert (Hc2ne : 1 <= length c2).
2195 { destruct c2 as [| ? ?]; simpl.
2196   - discriminate Hh2.
2197   - lia. }
2198 remember (length c2 - 1) as j eqn:Hjeq.
2199 assert (Hjlt : j < length c1) by lia.
2200 destruct (nth_error_exists c1 j Hjlt) as [a Hnth].
2201 assert (Hlenpref : length (firstn (length c2) c1) = length c2).
2202 { apply length_firstn_le. lia. }
2203 assert (Hchainpref : chain (firstn (length c2) c1)).
2204 { apply chain_firstn. exact Hc1. }
2205 assert (Hheadpref : hd_error (firstn (length c2) c1) = Some s).
2206 { apply hd_error_firstn; [exact Hh1 | lia]. }
2207 assert (Heqc : c2 = firstn (length c2) c1).

```

```

2208 { apply (persistent_chain_unique_from Hpers c2 (firstn (length c2) c1) s).
2209   - exact Hc2.
2210   - exact Hchainpref.
2211   - exact Hh2.
2212   - exact Hheadpref.
2213   - symmetry. exact Hlenpref. }
2214 assert (Hau : a = u).
2215 { pose proof (last_firstn_nth c1 j a s Hnth) as Hlf.
2216   assert (HSj : S j = length c2) by lia.
2217   rewrite HSj in Hlf.
2218   rewrite <- Heqc in Hlf.
2219   rewrite Hl2 in Hlf.
2220   symmetry. exact Hlf. }
2221 pose proof (hd_error_skipn c1 j a Hnth) as Hheadskip.
2222 pose proof (chain_skipn c1 j Hc1) as Hchainskip.
2223 pose proof (last_skipn c1 j a Hjlt) as Hlastskip.
2224 assert (Hlast_a_s : last c1 a = last c1 s).
2225 { apply last_default_irrelevant. destruct c1; [discriminate Hh1 |
discriminate]. }
2226 assert (Hlastskip' : last (skipn j c1) a = t).
2227 { rewrite Hlastskip, Hlast_a_s. exact Hl1. }
2228 assert (Hsuffix : chain_from_to u t (skipn j c1)).
2229 { rewrite <- Hau.
2230   split; [exact Hchainskip | split; [exact Hheadskip | exact Hlastskip']]. }
2231 assert (Hlensuf : length (skipn j c1) = length c1 - j).
2232 { apply length_skipn. lia. }
2233 apply (chain_pos_length_A_lt (skipn j c1) u t Hsuffix).
2234 rewrite Hlensuf. lia.
2235 Qed.
2236
2237 Lemma A_eq_lt_case :
2238   persistent -> bounded ->
2239   forall s t, s <A t -> s <A t \ / s = t \ / t <A s.
2240 Proof.
2241   intros Hpers Hbnd s t Heq.
2242   induction Heq as
2243     [ s
2244     | s t Hst
2245     | s t Heq IH
2246     | s t u Heq1 IH1 Heq2 IH2 ].
2247
2248   - right. left. reflexivity.
2249
2250   - left. apply A_lt_step. exact Hst.
2251
2252   - destruct IH as [IH | [IH | IH]].
2253     + right. right. exact IH.
2254     + right. left. symmetry. exact IH.
2255     + left. exact IH.
2256

```

```

2257 - destruct IH1 as [Hst | [Hst | Hst]]; destruct IH2 as [Htu | [Htu | Htu]].
2258 + left. apply A_lt_trans' with t; auto.
2259 + subst u. left. exact Hst.
2260 + destruct (A_le_iff_chain s t) as [Hfwd1 _].
2261   destruct (A_le_iff_chain u t) as [Hfwd2 _].
2262   assert (Hc1 : exists c, chain_from_to s t c) by (apply Hfwd1; apply
A_le_of_lt; exact Hst).
2263   assert (Hc2 : exists c, chain_from_to u t c) by (apply Hfwd2; apply
A_le_of_lt; exact Htu).
2264   destruct Hc1 as [c1 Hc1]. destruct Hc2 as [c2 Hc2].
2265   apply (chain_same_end_relates Hpers s u t c1 c2 Hc1 Hc2).
2266 + subst t. left. exact Htu.
2267 + subst t. subst u. right. left. reflexivity.
2268 + subst t. right. right. exact Htu.
2269 + destruct (A_le_iff_chain t s) as [Hfwd1 _].
2270   destruct (A_le_iff_chain t u) as [Hfwd2 _].
2271   assert (Hc1 : exists c, chain_from_to t s c) by (apply Hfwd1; apply
A_le_of_lt; exact Hst).
2272   assert (Hc2 : exists c, chain_from_to t u c) by (apply Hfwd2; apply
A_le_of_lt; exact Htu).
2273   destruct Hc1 as [c1 Hc1]. destruct Hc2 as [c2 Hc2].
2274   apply (chain_same_start_relates Hpers t s u c1 c2 Hc1 Hc2).
2275 + subst u. right. right. exact Hst.
2276 + right. right. apply A_lt_trans' with t; auto.
2277 Qed.
2278
2279 Lemma A_lt_total :
2280   persistent -> bounded -> atomic ->
2281   forall s t, s <A t \/ s = t \/ t <A s.
2282 Proof.
2283   intros Hpers Hbnd Hato s t.
2284   apply (A_eq_lt_case Hpers Hbnd s t (Hato s t)).
2285 Qed.
2286
2287 Axiom exists_top :
2288   bounded ->
2289   forall s0 : Sort, exists top, s0 <A top /\ ~ exists t, top <A t.
2290
2291 Axiom exists_bottom :
2292   bounded ->
2293   forall s0 : Sort, exists bottom, bottom <A s0 /\ ~ exists t, t <A bottom.
2294
2295 Lemma A_le_split : forall s t, s <A t -> s = t \/ s <A t.
2296 Proof.
2297   intros s t H. induction H as [s | s t u Hst H IH].
2298 - left. reflexivity.
2299 - destruct IH as [IH | IH].
2300 + subst. right. apply A_lt_step. exact Hst.
2301 + right. apply A_lt_trans' with t; [apply A_lt_step; exact Hst | exact IH].

```

```

2302 Qed.
2303
2304 Lemma A_le_lt_antisym :
2305   bounded ->
2306   forall s t, s A t -> t <A s -> False.
2307 Proof.
2308   intros Hbnd s t Hle Hlt.
2309   destruct (A_le_split s t Hle) as [Heq | Hlt2].
2310   - subst. exact (A_lt_irrefl Hbnd t Hlt).
2311   - exact (A_lt_irrefl Hbnd s (A_lt_trans' s t s Hlt2 Hlt)).
2312 Qed.
2313
2314 Lemma top_unique :
2315   persistent -> bounded -> atomic ->
2316   forall top1 top2,
2317   (forall s, s A top1) ->
2318   (forall s, s A top2) ->
2319   top1 = top2.
2320 Proof.
2321   intros Hpers Hbnd Hato top1 top2 H1 H2.
2322   destruct (A_lt_total Hpers Hbnd Hato top1 top2) as [Hlt | [Heq | Hgt]].
2323   - exfalso. apply (A_le_lt_antisym Hbnd top2 top1 (H1 top2) Hlt).
2324   - exact Heq.
2325   - exfalso. apply (A_le_lt_antisym Hbnd top1 top2 (H2 top1) Hgt).
2326 Qed.
2327
2328 Lemma bottom_unique :
2329   persistent -> bounded -> atomic ->
2330   forall bottom1 bottom2,
2331   (forall s, bottom1 A s) ->
2332   (forall s, bottom2 A s) ->
2333   bottom1 = bottom2.
2334 Proof.
2335   intros Hpers Hbnd Hato bottom1 bottom2 H1 H2.
2336   destruct (A_lt_total Hpers Hbnd Hato bottom1 bottom2) as [Hlt | [Heq | Hgt]].
2337   - exfalso. apply (A_le_lt_antisym Hbnd bottom2 bottom1 (H2 bottom1) Hlt).
2338   - exact Heq.
2339   - exfalso. apply (A_le_lt_antisym Hbnd bottom1 bottom2 (H1 bottom2) Hgt).
2340 Qed.
2341
2342 Lemma nth_error_lt_length : forall (l : list Sort) i a,
2343   nth_error l i = Some a -> i < length l.
2344 Proof.
2345   induction l as [| x l IH]; intros i a H.
2346   - destruct i; discriminate.
2347   - destruct i as [| i].
2348     + simpl. lia.
2349     + simpl in H. simpl. specialize (IH i a H). lia.
2350 Qed.
2351

```

```

2352 Lemma chain_nth_A : forall c i a b,
2353   chain c -> nth_error c i = Some a -> nth_error c (S i) = Some b -> A a b.
2354 Proof.
2355   induction c as [| x c IH]; intros i a b Hc Ha Hb.
2356   - destruct i as [| i]; simpl in Ha; discriminate Ha.
2357   - destruct i as [| i].
2358     + simpl in Ha. injection Ha as Ha. subst a.
2359       simpl in Hb.
2360       destruct c as [| y c'].
2361       * discriminate Hb.
2362       * simpl in Hc. destruct Hc as [HAxy _].
2363         injection Hb as Hb. subst b.
2364         exact HAxy.
2365     + simpl in Ha, Hb.
2366       destruct c as [| y c'].
2367       * destruct i as [| i']; simpl in Ha; discriminate Ha.
2368       * simpl in Hc. destruct Hc as [_ Hc'].
2369         apply (IH i a b Hc' Ha Hb).
2370 Qed.
2371
2372 Lemma last_nth_error : forall (l : list Sort) d,
2373   l <> [] -> nth_error l (length l - 1) = Some (last l d).
2374 Proof.
2375   induction l as [| x l IH]; intros d Hne.
2376   - contradiction.
2377   - destruct l as [| y l'].
2378     + reflexivity.
2379     + simpl. specialize (IH d ltac:(discriminate)). simpl in IH. rewrite <- IH.
2380       f_equal. lia.
2381 Qed.
2382
2383 Lemma chain_lt_A : forall c i j a b,
2384   chain c -> i < j -> nth_error c i = Some a -> nth_error c j = Some b -> a <A
2385     b.
2386 Proof.
2387   intros c i j a b Hc Hij.
2388   revert a b.
2389   induction j as [j IH] using (well_founded_induction Wf_nat.lt_wf).
2390   intros a b Ha Hb.
2391   destruct j as [| j'].
2392   - lia.
2393   - destruct (Nat.eq_dec i j') as [Heq | Hneq].
2394     + subst i. apply A_lt_step.
2395       apply (chain_nth_A c j' a b Hc Ha Hb).
2396     + assert (Hij' : i < j') by lia.
2397       destruct (nth_error_exists c j' ltac:(apply (nth_error_lt_length c (S j')
2398         b) in Hb; lia)) as [m Hm].
2399       apply A_lt_trans' with m.
2400       * apply (IH j' ltac:(lia) ltac:(lia) a m Ha Hm).
2401       * apply A_lt_step. apply (chain_nth_A c j' m b Hc Hm Hb).

```

```

2399 Qed.
2400
2401 Lemma chain_pos_unique :
2402   persistent -> bounded ->
2403   forall c i j s, chain c -> nth_error c i = Some s -> nth_error c j = Some s ->
2404     i = j.
2405 Proof.
2406   intros Hpers Hbnd c i j s Hc Hi Hj.
2407   destruct (Nat.lt_trichotomy i j) as [Hlt | [Heq | Hgt]]; auto.
2408   - exfalso. apply (A_lt_irrefl Hbnd s).
2409   - apply (chain_lt_A c i j s s Hc Hlt Hi Hj).
2410   - exfalso. apply (A_lt_irrefl Hbnd s).
2411   - apply (chain_lt_A c j i s s Hc Hgt Hj Hi).
2412 Qed.
2413
2414 Lemma persistent_chain_prefix :
2415   persistent ->
2416   forall c1 c2 s,
2417     chain c1 -> chain c2 ->
2418     hd_error c1 = Some s -> hd_error c2 = Some s ->
2419     length c1 <= length c2 ->
2420     c1 = firstn (length c1) c2.
2421 Proof.
2422   intros Hpers c1 c2 s Hc1 Hc2 Hh1 Hh2 Hlen.
2423   apply (persistent_chain_unique_from Hpers c1 (firstn (length c1) c2) s).
2424   - exact Hc1.
2425   - apply chain_firstn. exact Hc2.
2426   - exact Hh1.
2427   - apply hd_error_firstn; [exact Hh2 |].
2428   - destruct c1; [discriminate Hh1 | simpl; lia].
2429   - symmetry. apply length_firstn_le. exact Hlen.
2430 Qed.
2431
2432 Lemma app_nonempty_r : forall (l1 l2 : list Sort), l2 <> [] -> l1 ++ l2 <> [].
2433 Proof.
2434   intros l1 l2 Hne Heq.
2435   apply Hne. destruct l1; simpl in Heq; auto; discriminate.
2436 Qed.
2437
2438 Lemma last_app_r : forall (l1 l2 : list Sort) d, l2 <> [] -> last (l1 ++ l2) d =
2439   last l2 d.
2440 Proof.
2441   induction l1 as [| x l1 IH]; intros l2 d Hne.
2442   - reflexivity.
2443   - simpl. destruct (l1 ++ l2) as [| y ys] eqn:E.
2444     + exfalso. apply (app_nonempty_r l1 l2 Hne). exact E.
2445     + rewrite <- E. apply IH. exact Hne.
2446 Qed.
2447
2448 Lemma hd_error_app_l : forall (l1 l2 : list Sort) a,

```

```

2447   hd_error l1 = Some a -> hd_error (l1 ++ l2) = Some a.
2448 Proof.
2449   intros l1 l2 a H. destruct l1; simpl in H; [discriminate | simpl; exact H].
2450 Qed.
2451
2452 Lemma nth_error_app_l : forall (l1 l2 : list Sort) n,
2453   n < length l1 -> nth_error (l1 ++ l2) n = nth_error l1 n.
2454 Proof.
2455   induction l1 as [| x l1 IH]; intros l2 n Hn.
2456   - simpl in Hn. lia.
2457   - destruct n as [| n'].
2458     + reflexivity.
2459     + simpl. apply IH. simpl in Hn. lia.
2460 Qed.
2461
2462 Lemma nth_error_firstn : forall (l : list Sort) k i,
2463   i < k -> nth_error (firstn k l) i = nth_error l i.
2464 Proof.
2465   induction l as [| x l IH]; intros k i Hik.
2466   - destruct k; simpl; destruct i; reflexivity.
2467   - destruct k as [| k'].
2468     + lia.
2469     + destruct i as [| i'].
2470       * reflexivity.
2471       * simpl. apply IH. lia.
2472 Qed.
2473
2474 Lemma chain_app_general : forall (l1 l2 : list Sort) (m : Sort),
2475   chain (l1 ++ [m]) ->
2476   chain (m :: l2) ->
2477   chain (l1 ++ m :: l2).
2478 Proof.
2479   induction l1 as [| x l1 IH]; intros l2 m H1 H2.
2480   - simpl. exact H2.
2481   - simpl in H1 |- *.
2482     destruct l1 as [| y l1'].
2483     + simpl in H1. destruct H1 as [HAXm _].
2484       simpl. split; [exact HAXm | exact H2].
2485     + simpl in H1. destruct H1 as [HAXy Hrest].
2486       simpl. split; [exact HAXy | apply IH; [exact Hrest | exact H2]].
2487 Qed.
2488
2489 Lemma chain_from_to_app :
2490   forall c1 c2 b s t,
2491     chain_from_to b s c1 ->
2492     chain_from_to s t c2 ->
2493     chain_from_to b t (c1 ++ t1 c2).
2494 Proof.
2495   intros c1 c2 b s t [Hc1 [Hh1 Hl1]] [Hc2 [Hh2 Hl2]].
2496   destruct c2 as [| x2 c2'].

```



```

2497 - discriminate Hh2.
2498 - simpl in Hh2. injection Hh2 as Hh2. subst x2.
2499 simpl.
2500 assert (Hc1ne : c1 <> []).
2501 { destruct c1; [discriminate Hh1 | discriminate]. }
2502 destruct (exists_last Hc1ne) as [l1 [e Hc1eq]].
2503 assert (He : e = s).
2504 { assert (H1 : last c1 b = e).
2505   { rewrite Hc1eq. rewrite last_last. reflexivity. }
2506   rewrite H1 in H1. symmetry. exact H1. }
2507 subst e.
2508 destruct c2' as [| x2' c2''].
2509 + assert (Ht : t = s).
2510   { simpl in H12. symmetry. exact H12. }
2511   subst t.
2512   rewrite app_nil_r.
2513   split; [exact Hc1 | split; [exact Hh1 | exact H11]].
2514 + set (c2' := x2' :: c2'') in *.
2515   assert (Hc2'ne : c2' <> []) by (subst c2'; discriminate).
2516   split.
2517   * rewrite Hc1eq.
2518     rewrite <- app_assoc. simpl.
2519     apply (chain_app_general l1 c2' s).
2520     -- rewrite <- Hc1eq. exact Hc1.
2521     -- exact Hc2.
2522   * split.
2523     -- apply (hd_error_app_l c1 c2' b Hh1).
2524     -- rewrite (last_app_r c1 c2' b Hc2'ne).
2525       assert (H12' : last c2' s = t).
2526       { simpl in H12. exact H12. }
2527       rewrite <- H12'.
2528       apply last_default_irrelevant.
2529       exact Hc2'ne.
2530 Qed.
2531
2532 Lemma chain_length_eq_of_same_ends :
2533   persistent -> bounded ->
2534   forall e c b t,
2535     chain_from_to b t e -> chain_from_to b t c ->
2536     length e = length c.
2537 Proof.
2538   intros Hpers Hbnd e c b t He Hc.
2539   destruct He as [Hce [Hhe Hle]].
2540   destruct Hc as [Hcc [Hhc Hlc]].
2541   assert (Hene : e <> []) by (destruct e; [discriminate Hhe | discriminate]).
2542   assert (Hcne : c <> []) by (destruct c; [discriminate Hhc | discriminate]).
2543   assert (Hene1 : 1 <= length e) by (destruct e; [contradiction Hene;
2544     reflexivity | simpl; lia]).
2545   assert (Hcne1 : 1 <= length c) by (destruct c; [contradiction Hcne;
2546     reflexivity | simpl; lia]).

```

```

2545 destruct (Nat.lt_trichotomy (length e) (length c)) as [Hlt | [Heq | Hgt]];
2546 auto.
2547 - exfalso.
2548   assert (Hpre : e = firstn (length e) c).
2549   { apply (persistent_chain_prefix Hpers e c b); [exact Hce | exact Hcc |
2550     exact Hhe | exact Hhc | lia]. }
2551   assert (Hte : nth_error e (length e - 1) = Some t).
2552   { rewrite (last_nth_error e b Hene). rewrite Hle. reflexivity. }
2553   assert (Htc : nth_error c (length c - 1) = Some t).
2554   { rewrite (last_nth_error c b Hcne). rewrite Hlc. reflexivity. }
2555   assert (Hte' : nth_error (firstn (length e) c) (length e - 1) = Some t).
2556   { rewrite <- Hpre. exact Hte. }
2557   assert (Hte' : nth_error c (length e - 1) = Some t).
2558   { rewrite (nth_error_firstn c (length e) (length e - 1)) in Hte'.
2559     - exact Hte'.
2560     - lia. }
2561   assert (Heq2 : length e - 1 = length c - 1).
2562   { apply (chain_pos_unique Hpers Hbnd c (length e - 1) (length c - 1) t Hcc
2563     Hte' Htc). }
2564   lia.
2565 - exfalso.
2566   assert (Hpre : c = firstn (length c) e).
2567   { apply (persistent_chain_prefix Hpers c e b); [exact Hcc | exact Hce |
2568     exact Hhc | exact Hhe | lia]. }
2569   assert (Hte : nth_error e (length e - 1) = Some t).
2570   { rewrite (last_nth_error e b Hene). rewrite Hle. reflexivity. }
2571   assert (Htc : nth_error c (length c - 1) = Some t).
2572   { rewrite (last_nth_error c b Hcne). rewrite Hlc. reflexivity. }
2573   assert (Htc' : nth_error (firstn (length c) e) (length c - 1) = Some t).
2574   { rewrite <- Hpre. exact Htc. }
2575   assert (Htc' : nth_error e (length c - 1) = Some t).
2576   { rewrite (nth_error_firstn e (length c) (length c - 1)) in Htc'.
2577     - exact Htc'.
2578     - lia. }
2579   assert (Heq2 : length c - 1 = length e - 1).
2580   { apply (chain_pos_unique Hpers Hbnd e (length c - 1) (length e - 1) t Hce
2581     Htc' Hte). }
2582   lia.
2583 Qed.
2584
2585 Lemma tiered_iff_persistent_bounded_atomic :
2586   (exists n, tiered n) <-> persistent /\ bounded /\ atomic.
2587 Proof.
2588   split.
2589
2590   - intros [n [index_of [index_r [index_i [index_t [HA HR]]]]]]. split.
2591     + repeat split; auto.
2592     * intros s t u HA1 HA2.
2593     apply HA in HA1 as [_ HA1].

```

```

2589   apply HA in HA2 as [_ HA2].
2590   apply index_i. rewrite HA1, HA2. reflexivity.
2591   * intros s t u v HR1 HR2.
2592   apply HR in HR1. apply HR in HR2. rewrite <- HR1. exact HR2.
2593   * intros s t u HA1 HA2.
2594   apply HA in HA1 as [_ HA1].
2595   apply HA in HA2 as [_ HA2].
2596   apply index_i. apply eq_add_S. rewrite <- HA1. exact HA2.
2597
2598 + split.
2599
2600   * unfold bounded.
2601   assert (A_lt_index_1 : forall s t, s <A t -> index_of s < index_of t).
2602   intros s t H. induction H as [s t Hst | s t u Hst _ IH].
2603     apply HA in Hst as [_ Heq]. lia.
2604     apply HA in Hst as [_ Heq]. lia.
2605   assert (A_lt_index_2 : forall s t, t <A s -> n - index_of s < n - index_of
2606     t).
2607   intros t s H.
2608   induction H as [s t Hst | s t u Hst _ IH].
2609     apply HA in Hst as [_ Heq]. destruct (index_r s) as [_ Hsn], (index_r t)
2610     as [_ Htn]. lia.
2611     apply HA in Hst as [_ Heq]. destruct (index_r s) as [_ Hsn], (index_r t)
2612     as [_ Htn]. lia.
2613   split.
2614     ** unfold ascending_chain_condition. intros s.
2615     apply (wf_by_measure Sort (fun u v => u <A v) index_of A_lt_index_1).
2616     ** unfold descending_chain_condition. intros s.
2617     apply (wf_by_measure Sort (fun u v => v <A u) (fun s => n - index_of s)
2618       A_lt_index_2).
2619
2620   * assert (chain_eq : forall k i, i + k < n + 1 -> 0 < i ->
2621     forall x y, index_of x = i -> index_of y = i + k -> x A y).
2622   { induction k as [| k IH]; intros i Hbound Hipos x y Hx Hy.
2623     - assert (Hy0 : index_of y = i) by lia.
2624     assert (Heq : index_of x = index_of y) by lia.
2625     apply index_i in Heq. subst y. apply A_eq_refl.
2626     - assert (Hrange : 0 < i + k /\ i + k < n + 1) by lia.
2627     destruct (index_t (i + k) Hrange) as [z Hz].
2628     assert (Hxz : x A z).
2629     { apply IH with i; auto; lia. }
2630     assert (Hzy : z A y).
2631     { apply A_eq_step. apply HA. split.
2632       - rewrite Hz. lia.
2633       - rewrite Hz, Hy. lia. }
2634     apply A_eq_trans with z; auto. }
2635
2636   intros s s'.
2637   pose proof (index_r s) as [Hs1 Hs2].

```

```

2634 pose proof (index_r s') as [Hs1' Hs2'].
2635 assert (Hcase : index_of s <= index_of s' \ / index_of s' <= index_of s) by
lia.
2636 destruct Hcase as [Hle | Hge].
2637 ** assert (Hk : index_of s' = index_of s + (index_of s' - index_of s))
by lia.
2638 apply (chain_eq (index_of s' - index_of s) (index_of s)); lia || auto.
2639
2640 ** apply A_eq_sym.
2641 assert (Hk : index_of s = index_of s' + (index_of s - index_of s')) by
lia.
2642 apply (chain_eq (index_of s - index_of s') (index_of s')); lia || auto.
2643
- intros [Hpers [Hbnd Hato]].
2645 destruct (classic (exists s : Sort, True)) as [[s0 _] | Hempty].
2646 + destruct (classic (exists s1 s2 : Sort, s1 <> s2)) as [[s1 [s2 Hne]] |
Hone].
2647 * (* /S/ >= 2 *)
2648 destruct (exists_top Hbnd s1) as [top [_ Htopmax]].
2649 destruct (exists_bottom Hbnd s1) as [bot [_ Hbotmin]].
2650 assert (Htop : forall s, s A top).
2651 { intros s. destruct (A_lt_total Hpers Hbnd Hato s top) as [H|[H|H]].
2652 - apply A_le_of_lt; exact H.
2653 - subst; apply A_le_refl.
2654 - exfalse; apply Htopmax; exists s; exact H. }
2655 assert (Hbot : forall s, bot A s).
2656 { intros s. destruct (A_lt_total Hpers Hbnd Hato s bot) as [H|[H|H]].
2657 - exfalse; apply Hbotmin; exists s; exact H.
2658 - subst; apply A_le_refl.
2659 - apply A_le_of_lt; exact H. }
2660 destruct (A_le_iff_chain bot top) as [Hfwd _].
2661 destruct (Hfwd (Hbot top)) as [c Hc].
2662 exists (length c).
2663
2664 assert (Hpos : forall s : Sort, exists! i, nth_error c i = Some s).
2665 { intros s.
2666 pose proof (Hbot s) as Hbs.
2667 pose proof (Htop s) as Hst.
2668 destruct (proj1 (A_le_iff_chain bot s) Hbs) as [c1 Hc1].
2669 destruct (proj1 (A_le_iff_chain s top) Hst) as [c2 Hc2].
2670 assert (He : chain_from_to bot top (c1 ++ tl c2)).
2671 { apply (chain_from_to_app c1 c2 bot s top Hc1 Hc2). }
2672 assert (Hlen : length (c1 ++ tl c2) = length c).
2673 { apply (chain_length_eq_of_same_ends Hpers Hbnd (c1 ++ tl c2) c bot top
He Hc). }
2674 assert (Heqec : c1 ++ tl c2 = c).
2675 { apply (persistent_chain_unique_from Hpers (c1 ++ tl c2) c bot).
2676 - destruct He as [Hce _]. exact Hce.
2677 - destruct Hc as [Hcc _]. exact Hcc.

```

```

2678     - destruct He as [_ [Hhe _]]. exact Hhe.
2679     - destruct Hc as [_ [Hhc _]]. exact Hhc.
2680     - exact Hlen. }
2681 destruct Hc1 as [Hcc1 [Hhc1 Hlc1]].
2682 assert (Hc1ne : c1 <> []) by (destruct c1; [discriminate Hhc1 |
discriminate]).
2683 assert (Hpos_s : nth_error c1 (length c1 - 1) = Some s).
2684 { rewrite (last_nth_error c1 bot Hc1ne). rewrite Hlc1. reflexivity. }
2685 assert (Hc1ge1 : 1 <= length c1)
2686   by (destruct c1; [contradiction Hc1ne; reflexivity | simpl; lia]).
2687 assert (Hpos_e : nth_error (c1 ++ t1 c2) (length c1 - 1) = Some s).
2688 { rewrite (nth_error_app_1 c1 (t1 c2) (length c1 - 1)).
2689   - exact Hpos_s.
2690   - lia. }
2691 rewrite Heqec in Hpos_e.
2692 exists (length c1 - 1).
2693 split.
2694 - exact Hpos_e.
2695 - intros i' Hi'. symmetry.
2696   apply (chain_pos_unique Hpers Hbnd c i' (length c1 - 1) s).
2697   + destruct Hc as [Hcc _]; exact Hcc.
2698   + exact Hi'.
2699   + exact Hpos_e. }
2700 assert (Hchoice : forall s, {i | nth_error c i = Some s}).
2701 { intros s. apply constructive_indefinite_description.
2702   destruct (Hpos s) as [i [Hi _]]. exists i. exact Hi. }
2703 set (index_of := fun s => S (proj1_sig (Hchoice s))).
2704 exists index_of.
2705
2706 split.
2707
2708 (* range *)
2709 intros x. unfold index_of.
2710 pose proof (proj2_sig (Hchoice x)) as Hnth.
2711 pose proof (nth_error_lt_length c (proj1_sig (Hchoice x)) x Hnth) as Hlt.
2712 lia.
2713
2714 split.
2715
2716 (* injectivity *)
2717 intros x y Heq. unfold index_of in Heq.
2718 assert (Hixy : proj1_sig (Hchoice x) = proj1_sig (Hchoice y)) by lia.
2719 pose proof (proj2_sig (Hchoice x)) as Hx.
2720 pose proof (proj2_sig (Hchoice y)) as Hy.
2721 rewrite Hixy in Hx.
2722 assert (Hsome : Some x = Some y) by (rewrite <- Hx; exact Hy).
2723 injection Hsome as Hsome. exact Hsome.
2724
2725 split.
2726

```

```

2727   (* surjectivity *)
2728   intros i [Hi1 Hi2].
2729   assert (Hilt : i - 1 < length c) by lia.
2730   destruct (nth_error_exists c (i - 1) Hilt) as [x Hx].
2731   exists x.
2732   destruct (Hpos x) as [i0 [Hi0 Huniq]].
2733   assert (Heq0 : i0 = i - 1) by (apply Huniq; exact Hx).
2734   assert (Heqchoice : proj1_sig (Hchoice x) = i0).
2735   { symmetry. apply Huniq. exact (proj2_sig (Hchoice x)). }
2736   unfold index_of. rewrite Heqchoice. lia.
2737
2738   split.
2739
2740   (* A clause *)
2741   intros x y. split.
2742
2743   assert (Hcne : c <> []).
2744   { destruct Hc as [_ [Hh _]]. destruct c; [discriminate Hh | discriminate].
     }
2745
2746   (* -> *)
2747   intro HAx.
2748   assert (Hxlt : x <A y) by (apply A_lt_step; exact HAx).
2749   assert (Hxnetop : x <> top).
2750   { intro Heq. subst x. apply Htopmax. exists y. exact Hxlt. }
2751
2752   destruct (Hchoice x) as [ix Hix] eqn:Ex.
2753   assert (Hix_index : index_of x = S ix).
2754   { unfold index_of. simpl. rewrite Ex. reflexivity. }
2755   assert (Hixlt : ix < length c) by (apply (nth_error_lt_length c ix x);
     exact Hix).
2756
2757   assert (Hixne : ix <> length c - 1).
2758   { intro Heq.
     apply Hxnetop.
     assert (Hlast : nth_error c (length c - 1) = Some top).
     { destruct Hc as [_ [_ Hl]].
       rewrite (last_nth_error c bot Hcne). rewrite Hl. reflexivity. }
     clear Ex.
     rewrite Heq in Hix.
     rewrite Hix in Hlast.
     injection Hlast as Hlast.
     exact Hlast. }
2759
2760   assert (Hsix : S ix < length c) by lia.
2761   destruct (nth_error_exists c (S ix) Hsix) as [z Hz].
2762   assert (HAXz : A x z).
2763   { apply (chain_nth_A c ix x z);
     [destruct Hc as [Hcc _]; exact Hcc | exact Hix | exact Hz]. }
2764   assert (Hzy : z = y).

```

```

2775 { destruct Hpers as [[Hfunc1 _] _]. apply (Hfunc1 x); [exact HAxz | exact
2776 HAxxy]. }
2777 subst z.
2778
2779 destruct (Hchoice y) as [iy Hiy] eqn:Ey.
2780 assert (Hiy_index : index_of y = S iy).
2781 { unfold index_of. simpl. rewrite Ey. reflexivity. }
2782 assert (Hiyeq : iy = S ix).
2783 { destruct (Hpos y) as [i0 [Hi0 Huniq0]].
2784   assert (E1 : i0 = iy) by (apply Huniq0; exact Hiy).
2785   assert (E2 : i0 = S ix) by (apply Huniq0; exact Hz).
2786   lia. }
2787
2788 split; lia.
2789
2790 (* <- *)
2791 intros [Hlt Heqsucc].
2792 destruct (Hchoice x) as [ix Hix] eqn:Ex.
2793 assert (Hix_index : index_of x = S ix).
2794 { unfold index_of. simpl. rewrite Ex. reflexivity. }
2795 assert (Hsix : S ix < length c).
2796 { rewrite Hix_index in Hlt. exact Hlt. }
2797 destruct (nth_error_exists c (S ix) Hsix) as [z Hz].
2798 assert (HAXz : A x z).
2799 { apply (chain_nth_A c ix x z);
2800   [destruct Hc as [Hcc _]; exact Hcc | exact Hix | exact Hz]. }
2801
2802 destruct (Hchoice y) as [iy Hiy] eqn:Ey.
2803 assert (Hiy_index : index_of y = S iy).
2804 { unfold index_of. simpl. rewrite Ey. reflexivity. }
2805 assert (Hiyeq : iy = S ix).
2806 { rewrite Hix_index, Hiy_index in Heqsucc. lia. }
2807 assert (Hzy : z = y).
2808 { assert (Hiy' : nth_error c (S ix) = Some y).
2809   { rewrite <- Hiyeq. exact Hiy. }
2810   rewrite Hz in Hiy'.
2811   injection Hiy' as Hiy'.
2812   exact Hiy'. }
2813 subst z.
2814 exact HAXz.
2815
2816 (* R shape *)
2817 intros x y z HR. destruct Hpers as [_ [_ Hshape]]. eauto.
2818
2819 * (* |S| = 1 *)
2820 exists 1, (fun _ => 1).
2821 split. lia.
2822 split. intros x y _. destruct (classic (x = y)) as [|Hxy]; [auto|].
2823   exfalso. apply Hone. exists x, y. exact Hxy.

```

```

2824     split. intros i [Hi1 Hi2]. exists s0. lia.
2825     split. intros x y. split.
2826     ** intros HAx. exfalso. assert (Hxy : x = y).
2827     { destruct (classic (x = y)) as [|Hxy]; [auto|].
2828       exfalso. apply Hone. exists x, y. exact Hxy. }
2829     subst y. apply (A_lt_irrefl Hbnd x). apply A_lt_step. exact HAx.
2830     ** intros [Hlt _]. lia.
2831     ** intros x y z HR. destruct Hpers as [_ [_ Hshape]]. eauto.
2832 + (* |S| = 0 *)
2833     exists 0, (fun _ => 0). split.
2834     intros x. exfalso. apply Hempty. exists x. apply I. split.
2835     intros x y _. exfalso. apply Hempty. exists x. apply I. split.
2836     intros i [Hi1 Hi2]. lia. split.
2837     intros x y. split.
2838     intro HAx. exfalso. apply Hempty. exists x. apply I.
2839     intros [Hlt _]. lia.
2840     intros x y z HR. destruct Hpers as [_ [_ Hshape]]. eauto.
2841 Qed.
2842
2843 Definition disjoint_union_of_tiered : Prop :=
2844   exists (n_of : Sort -> nat) (index_of : Sort -> nat),
2845     (forall s t, A s t -> s A t) /\
2846     (forall s t u, R s t u -> s A t) /\
2847     (forall s t, s A t -> n_of s = n_of t) /\
2848     (forall s, 0 < index_of s /\ index_of s < n_of s + 1) /\
2849     (forall s t, s A t -> index_of s = index_of t -> s = t) /\
2850     (forall s i, 0 < i -> i <= n_of s -> exists t, t A s /\ index_of t = i) /\
2851     (forall s t, A s t <-> (s A t /\ index_of s < n_of s /\ index_of t = S
2852       (index_of s))) /\
2853     (forall s t u, R s t u -> t = u).
2854
2855 Lemma A_lt_in_class :
2856   forall (HAccl : forall s t, A s t -> s A t),
2857   forall u v, u <A v -> u A v.
2858 Proof.
2859   intros HAccl u v Huv.
2860   induction Huv as [u v Huv | u v w Huv _ IH].
2861   - apply A_eq_step. exact Huv.
2862   - apply A_eq_trans with v.
2863     + apply A_eq_step. exact Huv.
2864     + exact IH.
2865 Qed.
2866
2867 Lemma A_lt_index_incr :
2868   forall (index_of : Sort -> nat) (n_of : Sort -> nat),
2869   forall (HA : forall s t, A s t <-> (s A t /\ index_of s < n_of s /\ index_of t
2870     = S (index_of s))),
2871   forall u v, u <A v -> index_of u < index_of v.
2872 Proof.
2873   intros index_of n_of HA u v Huv.

```



```

2872 induction Huv as [u v Huv | u v w Huv _ IH].
2873 - apply HA in Huv as [_ [_ Heq]]. lia.
2874 - apply HA in Huv as [_ [_ Heq]]. lia.
2875 Qed.
2876
2877 Lemma A_lt_n_of_const :
2878   forall (n_of : Sort -> nat),
2879   forall (HAcClass : forall s t, A s t -> s A t)
2880     (Hnconst : forall s t, s A t -> n_of s = n_of t),
2881   forall u v, u <A v -> n_of u = n_of v.
2882 Proof.
2883   intros n_of HAcClass Hnconst u v Huv.
2884   apply Hnconst.
2885   apply (A_lt_in_class HAcClass u v Huv).
2886 Qed.
2887
2888 Lemma A_le_in_class : forall s t, s A t -> s A t.
2889 Proof.
2890   intros s t H. induction H as [s | s t u Hst H IH].
2891   - apply A_eq_refl.
2892   - apply A_eq_trans with t; [apply A_eq_step; exact Hst | exact IH].
2893 Qed.
2894
2895 Lemma chain_elem_in_class : forall c a,
2896   chain (a :: c) -> forall i x, nth_error (a :: c) i = Some x -> x A a.
2897 Proof.
2898   induction c as [| y c IH]; intros a Hc i x Hn.
2899   - destruct i as [| i'].
2900     + simpl in Hn. injection Hn as Hn. subst x. apply A_eq_refl.
2901     + destruct i' as [| i'']; simpl in Hn; discriminate Hn.
2902   - simpl in Hc. destruct Hc as [HAay Hc'].
2903     destruct i as [| i'].
2904     + simpl in Hn. injection Hn as Hn. subst x. apply A_eq_refl.
2905     + simpl in Hn.
2906       assert (Hxy : x A y) by (apply (IH y Hc' i' x Hn)).
2907       apply A_eq_trans with y. exact Hxy. apply A_eq_sym. apply A_eq_step. exact
        HAay.
2908 Qed.
2909
2910 Lemma chain_pos_in_class : forall c s0 bot,
2911   chain c -> hd_error c = Some bot -> bot A s0 ->
2912   forall i a, nth_error c i = Some a -> a A s0.
2913 Proof.
2914   intros c s0 bot Hc Hh Hbs i a Hn.
2915   destruct c as [| x c'].
2916   - discriminate Hh.
2917   - simpl in Hh. injection Hh as Hh. subst x.
2918     assert (Ha_bot : a A bot) by (apply (chain_elem_in_class c' bot Hc i a Hn)).
2919     apply A_eq_trans with bot; [exact Ha_bot | exact Hbs].
2920 Qed.

```

```

2921
2922 Lemma class_tiered :
2923   persistent -> bounded ->
2924   forall s0 : Sort,
2925   exists n index_of,
2926     (forall t, t A s0 -> 0 < index_of t /\ index_of t < n + 1) /\
2927     (forall t u, t A s0 -> u A s0 -> index_of t = index_of u -> t = u) /\
2928     (forall i, 0 < i -> i < n + 1 -> exists t, t A s0 /\ index_of t = i) /\
2929     (forall t u, t A s0 -> (A t u <-> (index_of t < n /\ index_of u = S
      (index_of t))))).
2930 Proof.
2931   intros Hpers Hbnd s0.
2932   destruct (exists_top Hbnd s0) as [top [Hs0top Htopmax]].
2933   destruct (exists_bottom Hbnd s0) as [bot [Hbots0 Hbotmin]].
2934   assert (Htops0 : top A s0) by (apply A_eq_sym; apply A_le_in_class; exact
      Hs0top).
2935   assert (Hbots0' : bot A s0) by (apply A_le_in_class; exact Hbots0).
2936
2937   assert (Htop : forall s, s A s0 -> s A top).
2938   { intros s Hs.
2939     assert (Hstop : s A top) by (apply A_eq_trans with s0; [exact Hs | apply
      A_eq_sym; exact Htops0]).
2940     destruct (A_eq_lt_case Hpers Hbnd s top Hstop) as [Hlt | [Heq | Hgt]].
2941     - apply A_le_of_lt; exact Hlt.
2942     - subst; apply A_le_refl.
2943     - exfalso; apply Htopmax; exists s; exact Hgt. }
2944
2945   assert (Hbot : forall s, s A s0 -> bot A s).
2946   { intros s Hs.
2947     assert (Hsbot : s A bot) by (apply A_eq_trans with s0; [exact Hs | apply
      A_eq_sym; exact Hbots0']).
2948     destruct (A_eq_lt_case Hpers Hbnd s bot Hsbot) as [Hlt | [Heq | Hgt]].
2949     - exfalso; apply Hbotmin; exists s; exact Hlt.
2950     - subst; apply A_le_refl.
2951     - apply A_le_of_lt; exact Hgt. }
2952
2953   destruct (A_le_iff_chain bot top) as [Hfwd _].
2954   assert (Hbotop : bot A top) by (apply Hbot; exact Htops0).
2955   destruct (Hfwd Hbotop) as [c Hc].
2956
2957   assert (Hcne : c <> []).
2958   { destruct Hc as [_ [Hh _]]. destruct c; [discriminate Hh | discriminate]. }
2959
2960   assert (Hpos : forall s : Sort, s A s0 -> exists! i, nth_error c i = Some s).
2961   { intros s Hs.
2962     pose proof (Hbot s Hs) as Hbs.
2963     pose proof (Htop s Hs) as Hst.
2964     destruct (proj1 (A_le_iff_chain bot s) Hbs) as [c1 Hc1].
2965     destruct (proj1 (A_le_iff_chain s top) Hst) as [c2 Hc2].

```

```

2966   assert (He : chain_from_to bot top (c1 ++ t1 c2)) by (apply
(chain_from_to_app c1 c2 bot s top Hc1 Hc2)).
2967   assert (Hlen : length (c1 ++ t1 c2) = length c)
2968     by (apply (chain_length_eq_of_same_ends Hpers Hbnd (c1 ++ t1 c2) c bot top
He Hc)).
2969   assert (Heqec : c1 ++ t1 c2 = c).
2970   { apply (persistent_chain_unique_from Hpers (c1 ++ t1 c2) c bot).
2971     - destruct He as [Hce _]; exact Hce.
2972     - destruct Hc as [Hcc _]; exact Hcc.
2973     - destruct He as [_ [Hhe _]]; exact Hhe.
2974     - destruct Hc as [_ [Hhc _]]; exact Hhc.
2975     - exact Hlen. }
2976   destruct Hc1 as [Hcc1 [Hhc1 Hlc1]].
2977   assert (Hc1ne : c1 <> []) by (destruct c1; [discriminate Hhc1 |
discriminate]).
2978   assert (Hpos_s : nth_error c1 (length c1 - 1) = Some s)
2979     by (rewrite (last_nth_error c1 bot Hc1ne); rewrite Hlc1; reflexivity).
2980   assert (Hc1ge1 : 1 <= length c1) by (destruct c1; [contradiction Hc1ne;
reflexivity | simpl; lia]).
2981   assert (Hpos_e : nth_error (c1 ++ t1 c2) (length c1 - 1) = Some s).
2982   { rewrite (nth_error_app_1 c1 (t1 c2) (length c1 - 1)); [exact Hpos_s |
lia]. }
2983   rewrite Heqec in Hpos_e.
2984   exists (length c1 - 1). split.
2985   - exact Hpos_e.
2986   - intros i' Hi'. symmetry.
2987     apply (chain_pos_unique Hpers Hbnd c i' (length c1 - 1) s).
2988     + destruct Hc as [Hcc _]; exact Hcc.
2989     + exact Hi'.
2990     + exact Hpos_e. }
2991
2992   assert (Hchoice : forall s, {i : nat |
(s A s0 -> nth_error c i = Some s) /\ (~ s A s0 -> i = 0)}).
2993   { intros s.
2994     apply constructive_indefinite_description.
2995     destruct (classic (s A s0)) as [Hs | Hs].
2996     - destruct (Hpos s Hs) as [i [Hi _]]. exists i.
2997       split; [intros _; exact Hi | intros Hcontra; contradiction].
2998     - exists 0. split; [intros Hcontra; contradiction | intros _; reflexivity].
2999   }
3000
3001   set (index_of := fun s => S (proj1_sig (Hchoice s))).
3002   exists (length c), index_of.
3003   split.
3004
3005   - (* range *)
3006     intros t Ht. unfold index_of.
3007     destruct (Hchoice t) as [it Hit] eqn:E. simpl.
3008     pose proof (proj1 Hit Ht) as Hnth.

```

```

3009 pose proof (nth_error_lt_length c it t Hnth) as Hlt.
3010 lia.
3011
3012 - split. (* injectivity *)
3013   intros t u Ht Hu Heq. unfold index_of in Heq.
3014   destruct (Hchoice t) as [it Hit] eqn:Et.
3015   destruct (Hchoice u) as [iu Hiu] eqn:Eu.
3016   simpl in Heq.
3017   assert (Hitu : it = iu) by lia.
3018   pose proof (proj1 Hit Ht) as Hntht.
3019   pose proof (proj1 Hiu Hu) as Hnthu.
3020   rewrite Hitu in Hntht.
3021   rewrite Hntht in Hnthu.
3022   injection Hnthu as Hnthu. exact Hnthu.
3023   split.
3024
3025   (* surjectivity *)
3026   intros i Hi1 Hi2.
3027   assert (Hilt : i - 1 < length c) by lia.
3028   destruct (nth_error_exists c (i - 1) Hilt) as [t Ht].
3029   assert (Hclass : t A s0).
3030   { destruct Hc as [Hcc [Hhc _]].
3031     apply (chain_pos_in_class c s0 bot Hcc Hhc Hbots0' (i - 1) t Ht). }
3032   exists t. split; [exact Hclass |].
3033   unfold index_of.
3034   destruct (Hchoice t) as [it Hit] eqn:E. simpl.
3035   destruct (Hpos t Hclass) as [i0 [Hi0 Huniq0]].
3036   assert (Heqchoice : it = i0) by (symmetry; apply Huniq0; exact (proj1 Hit
Hclass)).
3037   assert (Heq2 : i - 1 = i0) by (symmetry; apply Huniq0; exact Ht).
3038   lia.
3039
3040   (* A-iff *)
3041   intros t u Ht. split.
3042   + intro HAtu.
3043     assert (Htlt : t <A u) by (apply A_lt_step; exact HAtu).
3044     assert (Htnetop : t <> top).
3045     { intro Heq. subst t. apply Htopmax. exists u. exact Htlt. }
3046     destruct (Hchoice t) as [it Hit] eqn:Et.
3047     assert (Hit_index : index_of t = S it) by (unfold index_of; rewrite Et;
reflexivity).
3048     pose proof (proj1 Hit Ht) as Hntht.
3049     assert (Hitlt : it < length c) by (apply (nth_error_lt_length c it t);
exact Hntht).
3050     assert (Hitne : it <> length c - 1).
3051     { intro Heq. apply Htnetop.
3052       assert (Hlast : nth_error c (length c - 1) = Some top).
3053       { destruct Hc as [_ [_ Hl]]; rewrite (last_nth_error c bot Hcne);
rewrite Hl; reflexivity. }

```

```

3054     assert (Hit' : nth_error c (length c - 1) = Some t) by (rewrite <- Heq;
      exact Hntht).
3055     rewrite Hit' in Hlast. injection Hlast as Hlast. exact Hlast. }
3056   assert (Hsit : S it < length c) by lia.
3057   destruct (nth_error_exists c (S it) Hsit) as [z Hz].
3058   assert (HAtz : A t z).
3059   { apply (chain_nth_A c it t z); [destruct Hc as [Hcc _]; exact Hcc | exact
    Hntht | exact Hz]. }
3060   assert (Hzu : z = u).
3061   { destruct Hpers as [[Hfunc1 _] _]. apply (Hfunc1 t); [exact HAtz | exact
    HAtu]. }
3062   subst z.
3063   assert (Hu0 : u A s0).
3064   { apply A_eq_trans with t; [apply A_eq_sym; apply A_eq_step; exact HAtu |
    exact Ht]. }
3065   destruct (Hchoice u) as [iu Hiu] eqn:Eu.
3066   assert (Hiu_index : index_of u = S iu) by (unfold index_of; rewrite Eu;
    reflexivity).
3067   assert (Hiueq : iu = S it).
3068   { destruct (Hpos u Hu0) as [i0 [Hi0 Huniq0]].
    3069     assert (E1 : i0 = iu) by (apply Huniq0; exact (proj1 Hiu Hu0)).
    3070     assert (E2 : i0 = S it) by (apply Huniq0; exact Hz).
    3071     lia. }
3072   split; lia.
3073 + intros [Hlt Heqsucc].
3074   destruct (Hchoice t) as [it Hit] eqn:Et.
3075   assert (Hit_index : index_of t = S it) by (unfold index_of; rewrite Et;
    reflexivity).
3076   assert (Hsit : S it < length c) by (rewrite Hit_index in Hlt; exact Hlt).
3077   destruct (nth_error_exists c (S it) Hsit) as [z Hz].
3078   assert (HAtz : A t z).
3079   { apply (chain_nth_A c it t z); [destruct Hc as [Hcc _]; exact Hcc | exact
    (proj1 Hit Ht) | exact Hz]. }
3080   destruct (Hchoice u) as [iu Hiu] eqn:Eu.
3081   assert (Hiu_index : index_of u = S iu) by (unfold index_of; rewrite Eu;
    reflexivity).
3082   assert (Hiueq : iu = S it) by (rewrite Hit_index, Hiu_index in Heqsucc;
    lia).
3083   assert (Hu0 : u A s0).
3084   { destruct (classic (u A s0)) as [H | H]; [exact H | ].
    3085     exfalso.
    3086     assert (Hiu0 : iu = 0) by (apply (proj2 Hiu); exact H).
    3087     lia. }
3088   assert (Hzu : z = u).
3089   { assert (Hiu' : nth_error c (S it) = Some u) by (rewrite <- Hiueq; exact
    (proj1 Hiu Hu0)).
    3090     rewrite Hz in Hiu'. injection Hiu' as Hiu'. exact Hiu'. }
    3091   subst z. exact HAtz.
3092 Qed.

```

```

3093
3094 Lemma A_lt_pred : forall t u, t < A u -> exists w, A w u.
3095 Proof.
3096   intros t u H. induction H as [t u Htu | t v u Htv _ IH].
3097   - exists t. exact Htu.
3098   - exact IH.
3099 Qed.
3100
3101 Lemma no_pred_unique_in_class :
3102   persistent -> bounded ->
3103   forall s t u,
3104     t A s -> u A s ->
3105     (~ exists v, v A s /\ A v t) ->
3106     (~ exists v, v A s /\ A v u) ->
3107     t = u.
3108 Proof.
3109   intros Hpers Hbnd s t u Ht Hu Hnpt Hnpu.
3110   destruct (classic (t = u)) as [Heq | Hneq]; [exact Heq |].
3111   assert (Htu : t A u) by (apply A_eq_trans with s; [exact Ht | apply A_eq_sym;
3112   exact Hu]).
3112   destruct (A_eq_lt_case Hpers Hbnd t u Htu) as [Hlt | [Heq | Hgt]].
3113   - exfalso. destruct (A_lt_pred t u Hlt) as [w Hw].
3114     apply Hnpu. exists w. split; [| exact Hw].
3115     apply A_eq_trans with u; [apply A_eq_step; exact Hw | exact Hu].
3116   - exact Heq.
3117   - exfalso. destruct (A_lt_pred u t Hgt) as [w Hw].
3118     apply Hnpt. exists w. split; [| exact Hw].
3119     apply A_eq_trans with t; [apply A_eq_step; exact Hw | exact Ht].
3120 Qed.
3121
3122 Lemma class_tiered_unique :
3123   persistent -> bounded ->
3124   forall s n1 n2 f1 f2,
3125     (forall t, t A s -> 0 < f1 t /\ f1 t < n1 + 1) ->
3126     (forall t u, t A s -> u A s -> f1 t = f1 u -> t = u) ->
3127     (forall i, 0 < i -> i < n1 + 1 -> exists t, t A s /\ f1 t = i) ->
3128     (forall t u, t A s -> (A t u <-> (f1 t < n1 /\ f1 u = S (f1 t)))) ->
3129     (forall t, t A s -> 0 < f2 t /\ f2 t < n2 + 1) ->
3130     (forall t u, t A s -> u A s -> f2 t = f2 u -> t = u) ->
3131     (forall i, 0 < i -> i < n2 + 1 -> exists t, t A s /\ f2 t = i) ->
3132     (forall t u, t A s -> (A t u <-> (f2 t < n2 /\ f2 u = S (f2 t)))) ->
3133     n1 = n2 /\ forall t, t A s -> f1 t = f2 t.
3134 Proof.
3135   intros Hpers Hbnd s n1 n2 f1 f2
3136   Hr1 Hi1 Hs1 HA1 Hr2 Hi2 Hs2 HA2.
3137
3138   assert (Hnp1 : forall i, i = 1 -> forall t, t A s -> f1 t = i ->
3139     ~ exists v, v A s /\ A v t).
3140   { intros i Hileq t Ht Hf1t [v [Hv HAvt]].
3141     apply (HA1 v t Hv) in HAvt as [_ Heq].

```

```

3142     destruct (Hr1 v Hv) as [Hf1vpos _].
3143     lia. }
3144
3145 assert (Hmain : forall i, 0 < i -> i < n1 + 1 -> i < n2 + 1 ->
3146         forall t, t A s -> f1 t = i -> f2 t = i).
3147 { intros i.
3148   induction i as [i IH] using (well_founded_induction Wf_nat.lt_wf).
3149   intros Hi0 Hi1lt Hi2lt t Ht Hf1t.
3150   destruct i as [| i'].
3151   - lia.
3152   - destruct i' as [| i''].
3153     + (* i = 1: base case via no-predecessor uniqueness *)
3154       destruct (Hs2 1 ltac:(lia) Hi2lt) as [u [Hu Hf2u]].
3155       assert (Ht_np : ~ exists v, v A s /\ A v t).
3156       { intros [v [Hv HAvt]].
3157         apply (HA1 v t Hv) in HAvt as [_ Heq].
3158         destruct (Hr1 v Hv) as [Hf1vpos _].
3159         rewrite Hf1t in Heq. lia. }
3160       assert (Hu_np : ~ exists v, v A s /\ A v u).
3161       { intros [v [Hv HAvu]].
3162         apply (HA2 v u Hv) in HAvu as [_ Heq].
3163         destruct (Hr2 v Hv) as [Hf1vpos _].
3164         rewrite Hf2u in Heq.
3165         lia. }
3166       assert (Htu : t = u) by (apply (no_pred_unique_in_class Hpers Hbnd s t u
3167         Ht Hu Ht_np Hu_np)).
3168       rewrite Htu. exact Hf2u.
3169     + (* successor case *)
3170       destruct (Hs1 (S i'') ltac:(lia) ltac:(lia)) as [v [Hv Hf1v]].
3171       assert (HAvt : A v t).
3172       { apply (HA1 v t Hv). split; [lia | rewrite Hf1v, Hf1t; reflexivity]. }
3173       assert (Hf2v : f2 v = S i'') by (apply (IH (S i'') ltac:(lia) ltac:(lia)
3174         ltac:(lia) ltac:(lia) v Hv Hf1v)).
3175       apply (HA2 v t Hv) in HAvt as [_ Hf2t].
3176       rewrite Hf2v in Hf2t. exact Hf2t. }
3177
3178 assert (Hmain2 : forall i, 0 < i -> i < n1 + 1 -> i < n2 + 1 ->
3179         forall t, t A s -> f2 t = i -> f1 t = i).
3180 { intros i.
3181   induction i as [i IH] using (well_founded_induction Wf_nat.lt_wf).
3182   intros Hi0 Hi1lt Hi2lt t Ht Hf2t.
3183   destruct i as [| i'].
3184   - lia.
3185   - destruct i' as [| i''].
3186     + destruct (Hs1 1 ltac:(lia) Hi1lt) as [u [Hu Hf1u]].
3187       assert (Ht_np : ~ exists v, v A s /\ A v t).
3188       { intros [v [Hv HAvt]].
3189         apply (HA2 v t Hv) in HAvt as [_ Heq].
3190         destruct (Hr2 v Hv) as [Hf1vpos _].
3191         rewrite Hf2t in Heq.

```

```

3190     lia. }
3191 assert (Hu_np : ~ exists v, v A s /\ A v u).
3192 { intros [v [Hv HAvu]].
3193   apply (HA1 v u Hv) in HAvu as [_ Heq].
3194   destruct (Hr1 v Hv) as [Hf1vpos _].
3195   rewrite Hf1u in Heq.
3196   lia. }
3197 assert (Htu : t = u) by (apply (no_pred_unique_in_class Hpers Hbnd s t u
Ht Hu Ht_np Hu_np)).
3198 rewrite Htu. exact Hf1u.
3199 + destruct (Hs2 (S i'') ltac:(lia) ltac:(lia)) as [v [Hv Hf2v]].
3200 assert (HAvt : A v t).
3201 { apply (HA2 v t Hv). split; [lia | rewrite Hf2v, Hf2t; reflexivity]. }
3202 assert (Hf1v : f1 v = S i'') by (apply (IH (S i'') ltac:(lia) ltac:(lia)
ltac:(lia) ltac:(lia) v Hv Hf2v)).
3203 apply (HA1 v t Hv) in HAvt as [_ Hf1t].
3204 rewrite Hf1v in Hf1t. exact Hf1t. }
3205
3206 assert (Hn : n1 = n2).
3207 { destruct (Nat.lt_trichotomy n1 n2) as [Hlt | [Heq | Hgt]]; auto.
3208   - exfalso.
3209     destruct (Hs2 (n1+1) ltac:(lia) ltac:(lia)) as [u [Hu Hf2u]].
3210     destruct (Hr2 u Hu) as [_ Hf2u_lt].
3211     destruct (Nat.lt_trichotomy (f1 u) (n1+1)) as [Hc1 | [Hc2 | Hc3]].
3212     + assert (Hf1lt : f1 u < n1 + 1) by lia.
3213       destruct (Hr1 u Hu) as [Hf1pos _].
3214       assert (Hf2u' : f2 u = f1 u) by (apply (Hmain (f1 u) Hf1pos Hf1lt
ltac:(lia) u Hu eq_refl)).
3215       rewrite Hf2u in Hf2u'. lia.
3216     + destruct (Hr1 u Hu) as [_ Hf1u_lt]. lia.
3217     + destruct (Hr1 u Hu) as [_ Hf1u_lt]. lia.
3218   - exfalso.
3219     destruct (Hs1 (n2+1) ltac:(lia) ltac:(lia)) as [u [Hu Hf1u]].
3220     destruct (Hr1 u Hu) as [_ Hf1u_lt].
3221     destruct (Nat.lt_trichotomy (f2 u) (n2+1)) as [Hc1 | [Hc2 | Hc3]].
3222     + assert (Hf2lt : f2 u < n2 + 1) by lia.
3223       destruct (Hr2 u Hu) as [Hf2pos _].
3224       assert (Hf1u' : f1 u = f2 u) by (apply (Hmain2 (f2 u) Hf2pos ltac:(lia)
ltac:(lia) u Hu eq_refl)).
3225       rewrite Hf1u in Hf1u'. lia.
3226     + destruct (Hr2 u Hu) as [_ Hf2u_lt]. lia.
3227     + destruct (Hr2 u Hu) as [_ Hf2u_lt]. lia. }
3228
3229 split; [exact Hn |].
3230 intros t Ht.
3231 destruct (Hr1 t Ht) as [Hpos Hlt]. symmetry.
3232 apply (Hmain (f1 t) Hpos ltac:(lia) ltac:(lia) t Ht eq_refl).
3233 Qed.
3234

```



```

3235 Lemma persistent_bounded_seperable_iff_disjoint_union_of_tiered :
3236   (persistent /\ bounded /\ separable) <-> disjoint_union_of_tiered.
3237 Proof.
3238   split.
3239
3240 - intros [Hpers [Hbnd Hsep]].
3241   assert (Hwit : forall s0 : Sort, {p : nat * (Sort -> nat) |
3242     (forall t, t A s0 -> 0 < snd p t /\ snd p t < fst p + 1) /\
3243     (forall t u, t A s0 -> u A s0 -> snd p t = snd p u -> t = u) /\
3244     (forall i, 0 < i -> i < fst p + 1 -> exists t, t A s0 /\ snd p t = i) /\
3245     (forall t u, t A s0 -> (A t u <-> (snd p t < fst p /\ snd p u = S (snd p
3246       t))))).
3247   { intros s0.
3248     apply constructive_indefinite_description.
3249     destruct (class_tiered Hpers Hbnd s0) as [n [index_of H]].
3250     exists (n, index_of). exact H. }
3251
3252   set (n_of := fun s => fst (proj1_sig (Hwit s))).
3253   set (index_of := fun s => snd (proj1_sig (Hwit s)) s).
3254
3255   assert (Hprop : forall s,
3256     (forall t, t A s -> 0 < snd (proj1_sig (Hwit s)) t /\ snd (proj1_sig (Hwit
3257       s)) t < fst (proj1_sig (Hwit s)) + 1) /\
3258     (forall t u, t A s -> u A s -> snd (proj1_sig (Hwit s)) t = snd
3259       (proj1_sig (Hwit s)) u -> t = u) /\
3260     (forall i, 0 < i -> i < fst (proj1_sig (Hwit s)) + 1 -> exists t, t A s /\
3261       snd (proj1_sig (Hwit s)) t = i) /\
3262     (forall t u, t A s -> (A t u <-> (snd (proj1_sig (Hwit s)) t < fst
3263       (proj1_sig (Hwit s)) /\ snd (proj1_sig (Hwit s)) u = S (snd (proj1_sig
3264         (Hwit s)) t))))).
3265   { intros s. exact (proj2_sig (Hwit s)). }
3266
3267   assert (Hagree : forall s t, s A t -> forall u, u A s ->
3268     fst (proj1_sig (Hwit s)) = fst (proj1_sig (Hwit t)) /\
3269     snd (proj1_sig (Hwit s)) u = snd (proj1_sig (Hwit t)) u).
3270   { intros s t Hst u Hu.
3271     destruct (Hprop s) as [Hr1 [Hi1 [Hs1 HA1]]].
3272     destruct (Hprop t) as [Hr2 [Hi2 [Hs2 HA2]]].
3273     assert (Hr2' : forall v, v A s -> 0 < snd (proj1_sig (Hwit t)) v /\ snd
3274       (proj1_sig (Hwit t)) v < fst (proj1_sig (Hwit t)) + 1).
3275     { intros v Hv. apply Hr2. apply A_eq_trans with s; [exact Hv | exact Hst].
3276       }
3277     assert (Hi2' : forall v w, v A s -> w A s -> snd (proj1_sig (Hwit t)) v =
3278       snd (proj1_sig (Hwit t)) w -> v = w).
3279     { intros v w Hv Hw. apply Hi2; apply A_eq_trans with s; auto. }
3280     assert (Hs2' : forall i, 0 < i -> i < fst (proj1_sig (Hwit t)) + 1 ->
3281       exists v, v A s /\ snd (proj1_sig (Hwit t)) v = i).
3282     { intros i Hi0 Hilt. destruct (Hs2 i Hi0 Hilt) as [v [Hv Hvi]]. exists v.
3283       split; [| exact Hvi].

```

```

3273     apply A_eq_trans with t; [exact Hv | apply A_eq_sym; exact Hst]. }
3274   assert (HA2' : forall v w, v A s -> (A v w <-> (snd (proj1_sig (Hwit t)) v
    < fst (proj1_sig (Hwit t)) /\ snd (proj1_sig (Hwit t)) w = S (snd
    (proj1_sig (Hwit t)) v))))).
3275   { intros v w Hv. apply HA2. apply A_eq_trans with s; [exact Hv | exact
    Hst]. }
3276   destruct (class_tiered_unique Hpers Hbnd s
3277     (fst (proj1_sig (Hwit s))) (fst (proj1_sig (Hwit t)))
3278     (snd (proj1_sig (Hwit s))) (snd (proj1_sig (Hwit t)))
3279     Hr1 Hi1 Hs1 HA1 Hr2' Hi2' Hs2' HA2') as [Hn Hf].
3280   split; [exact Hn | apply Hf; exact Hu]. }
3281
3282   exists n_of, index_of.
3283   split.
3284
3285   (* A s t -> s A t *)
3286   intros s t HAs.
3287   destruct (Hprop s) as [_ [_ [_ HAiff]]].
3288   assert (Hss : s A s) by apply A_eq_refl.
3289   apply A_eq_step; exact HAs.
3290   split.
3291
3292   (* R s t u -> s A t *)
3293   intros s t u HR.
3294   destruct Hpers as [_ [_ Hshape]].
3295   assert (Htu : t = u) by (apply (Hshape s t u); exact HR).
3296   apply (Hsep s t). rewrite <- Htu in HR. exact HR.
3297   split.
3298
3299   (* n_of constant on class *)
3300   intros s t Hst. unfold n_of.
3301   destruct (Hagree s t Hst s) as [Hn _]; [apply A_eq_refl |]. exact Hn.
3302   split.
3303
3304   (* range *)
3305   intros s. unfold index_of.
3306   destruct (Hprop s) as [Hrange _].
3307   apply (Hrange s). apply A_eq_refl.
3308   split.
3309
3310   (* injectivity *)
3311   intros s t Hst Heq. unfold index_of, n_of in *.
3312   assert (Hs_ind : snd (proj1_sig (Hwit s)) s = snd (proj1_sig (Hwit t)) t) by
    exact Heq.
3313   destruct (Hagree s t Hst s) as [_ Hval]; [apply A_eq_refl |].
3314   rewrite Hval in Hs_ind.
3315   destruct (Hprop t) as [_ [Hinj _]].
3316   apply (Hinj s t).
3317   apply A_eq_trans with s; [apply A_eq_refl | exact Hst].

```

```

3318   apply A_eq_refl.
3319   exact Hs_ind.
3320   split.
3321
3322   (* surjectivity *)
3323   intros s i Hi1 Hi2. unfold n_of, index_of in *.
3324   destruct (Hprop s) as [_ [_ [Hsurj _]]].
3325   assert (Hi2' : i < fst (proj1_sig (Hwit s)) + 1) by lia.
3326   destruct (Hsurj i Hi1 Hi2') as [t [Ht Hti]].
3327   exists t. split; [exact Ht |]. apply A_eq_sym in Ht.
3328   destruct (Hagree s t Ht t) as [_ Hval]. apply A_eq_sym. apply Ht.
3329   rewrite <- Hval. exact Hti.
3330   split.
3331
3332   (* A iff *)
3333   intros s t. split.
3334
3335   intro HAs.
3336   assert (Hst_class : s A t) by (apply A_eq_step; exact HAs).
3337   destruct (Hprop s) as [_ [_ [_ HAiff]]].
3338   apply (HAiff s t (A_eq_refl s)) in HAs as [H1 H2].
3339   split; [exact Hst_class |].
3340   unfold index_of, n_of.
3341   destruct (Hagree s t Hst_class t) as [Hn Hval]. apply A_eq_sym. apply
Hst_class.
3342   split; [exact H1 | rewrite <- Hval; exact H2].
3343
3344   intros [Hclass [Hlt Heq]].
3345   unfold index_of, n_of in *.
3346   destruct (Hprop s) as [_ [_ [_ HAiff]]].
3347   apply (HAiff s t (A_eq_refl s)).
3348   destruct (Hagree s t Hclass t) as [Hn Hval]; [apply A_eq_sym; exact Hclass
|].
3349   split; [exact Hlt | rewrite Hval; exact Heq].
3350
3351   (* R shape *)
3352   intros s t u HR.
3353   destruct Hpers as [_ [_ Hshape]]. eauto.
3354
3355   - intros [n_of [index_of [HAcass [HRclass [Hnconst [Hrange [Hinj [Hsurj [HA
HR]]]]]]]]].
3356     split.
3357
3358   + (* persistent *)
3359     repeat split.
3360     * (* unique A-successor *)
3361       intros x y y' Hxy Hxy'.
3362       apply HA in Hxy as [Hxy1 [Hxy2 Hxy3]].
3363       apply HA in Hxy' as [Hxy1' [Hxy2' Hxy3']].
3364       apply Hinj.

```

```

3365 -- (* y A y' *)
3366 apply A_eq_trans with x.
3367 ++ apply A_eq_sym. apply A_eq_step. apply HA. auto.
3368 ++ apply A_eq_step. apply HA. auto.
3369 -- rewrite Hxy3, Hxy3'. reflexivity.
3370 * (* unique R-shape (functional's 2nd component) *)
3371 intros s t u u' Hu Hu'.
3372 rewrite <- (HR s t u Hu), (HR s t u' Hu'). reflexivity.
3373 * (* unique A-predecessor *)
3374 intros x x' y Hxy Hxy'.
3375 apply HA in Hxy as [Hxy1 [Hxy2 Hxy3]].
3376 apply HA in Hxy' as [Hxy1' [Hxy2' Hxy3']].
3377 apply Hinj.
3378 -- apply A_eq_trans with y.
3379 ++ apply A_eq_step. apply HA. auto.
3380 ++ apply A_eq_sym. apply A_eq_step. apply HA. auto.
3381 -- assert (Heq : index_of x = index_of x') by lia.
3382 exact Heq.
3383 * (* R shape *)
3384 intros x y z HRxyz. exact (HR x y z HRxyz).
3385
3386 + split. split.
3387
3388 (* ascending chain condition *)
3389 intro s.
3390 apply (wf_by_measure Sort (fun u v => u <A v) index_of).
3391 intros u v Huv.
3392 apply (A_lt_index_incr index_of n_of HA u v Huv).
3393
3394 (* descending chain condition *)
3395 intro s.
3396 apply (wf_by_measure Sort (fun u v => v <A u) (fun s => n_of s - index_of
s))).
3397 intros u v Huv.
3398 pose proof (A_lt_index_incr index_of n_of HA v u Huv) as Hidx.
3399 pose proof (A_lt_n_of_const n_of HAcClass Hnconst v u Huv) as Hn.
3400 pose proof (Hrange u) as [_ Hu2].
3401 pose proof (Hrange v) as [_ Hv2].
3402 lia.
3403
3404 (* separable *)
3405 intros s s' HR'.
3406 apply HRclass in HR' as Hclass.
3407 pose proof (HR s s' s' HR') as Ht.
3408 exact Hclass.
3409 Qed.
3410
3411 Definition non_dependent_tiered (n : nat) : Prop :=
3412   exists index_of : Sort -> nat,
3413   (forall x, 0 < index_of x /\ index_of x < n + 1) /\

```

```

3414 (forall x y, index_of x = index_of y -> x = y) /\
3415 (forall i, 0 < i -> i < n + 1 -> exists x, index_of x = i) /\
3416 (forall x y, A x y <-> index_of x < n /\ index_of y = S (index_of x)) /\
3417 (forall x y z, R x y z -> y = z /\ index_of y <= index_of x).
3418
3419 Definition disjoint_union_of_non_dependent_tiered : Prop :=
3420   exists (n_of : Sort -> nat) (index_of : Sort -> nat),
3421     (forall s t, A s t -> s A t) /\
3422     (forall s t u, R s t u -> s A t) /\
3423     (forall s t, s A t -> n_of s = n_of t) /\
3424     (forall s, 0 < index_of s /\ index_of s < n_of s + 1) /\
3425     (forall s t, s A t -> index_of s = index_of t -> s = t) /\
3426     (forall s i, 0 < i -> i <= n_of s -> exists t, t A s /\ index_of t = i) /\
3427     (forall s t, A s t <-> (s A t /\ index_of s < n_of s /\ index_of t = S
      (index_of s))) /\
3428     (forall s t u, R s t u -> t = u /\ index_of t <= index_of s).
3429
3430 (* Corollary 1 *)
3431 Axiom bounded_non_dependent_iff_disjoint_union_of_non_dependent_tiered :
3432   bounded_non_dependent <-> disjoint_union_of_non_dependent_tiered.
3433
3434 (* Proposition 2 *)
3435 Axiom weak_implies_strong_normalization_lift :
3436   (forall n, non_dependent_tiered n \ / tiered n ->
3437     (forall M, weakly_normalizing M -> strongly_normalizing M)) ->
3438   (persistent /\ bounded /\ separable ->
3439     (forall M, weakly_normalizing M -> strongly_normalizing M)).
3440
3441 Open Scope Z_scope.
3442
3443 Definition Zpos_tiered : Prop :=
3444   exists index_of : Sort -> Z,
3445     (forall x, 0 < index_of x) /\
3446     (forall x y, index_of x = index_of y -> x = y) /\
3447     (forall i : Z, 0 < i -> exists x, index_of x = i) /\
3448     (forall x y, A x y <-> index_of y = index_of x + 1) /\
3449     (forall x y z, R x y z -> y = z).
3450
3451 Definition Zneg_tiered : Prop :=
3452   exists index_of : Sort -> Z,
3453     (forall x, index_of x < 0) /\
3454     (forall x y, index_of x = index_of y -> x = y) /\
3455     (forall i : Z, i < 0 -> exists x, index_of x = i) /\
3456     (forall x y, A x y <-> index_of x <= -2 /\ index_of y = index_of x + 1) /\
3457     (forall x y z, R x y z -> y = z).
3458
3459 Definition Z_tiered : Prop :=
3460   exists index_of : Sort -> Z,
3461     (forall x y, index_of x = index_of y -> x = y) /\
3462     (forall i : Z, exists x, index_of x = i) /\

```

```

3463     (forall x y, A x y <-> index_of y = index_of x + 1) /\
3464     (forall x y z, R x y z -> y = z).
3465
3466 Close Scope Z_scope.
3467
3468 Axiom finite_or_Zneg_tiered_atomic_gen_non_dependent :
3469     forall n, tiered n \/ Zneg_tiered ->
3470         atomic /\ generalized_non_dependent.
3471
3472 Axiom weak_implies_strong_normalization_n_tiered_lift :
3473     (forall n, non_dependent_tiered n ->
3474         (forall M, weakly_normalizing M -> strongly_normalizing M)) ->
3475     (generalized_non_dependent ->
3476         (forall M, weakly_normalizing M -> strongly_normalizing M)).
3477
3478 Definition cyclic_tiered (n : nat) : Prop :=
3479     exists index_of : Sort -> nat,
3480     (forall x, 0 < index_of x /\ index_of x < n + 1) /\
3481     (forall x y, index_of x = index_of y -> x = y) /\
3482     (forall i, 0 < i -> i < n + 1 -> exists x, index_of x = i) /\
3483     (forall x y, A x y <->
3484         (index_of x < n /\ index_of y = S (index_of x)) \/
3485         (index_of x = n /\ index_of y = 1)) /\
3486     (forall x y z, R x y z -> y = z).
3487
3488 (* Proposition 4 *)
3489 Axiom weak_implies_strong_normalization_persistent_separable_lift :
3490     (forall n, (tiered n \/ cyclic_tiered n) ->
3491         (forall M, weakly_normalizing M -> strongly_normalizing M)) ->
3492     (persistent /\ separable ->
3493         (forall M, weakly_normalizing M -> strongly_normalizing M)).
3494
3495 End PTS.

```

سیستم نوع محض لایه‌ای و ترجمه حذف وابستگی

```

1  From Stdlib Require Import List.
2  From Stdlib Require Import Classical.
3  From Stdlib Require Import Logic.IndefiniteDescription.
4  From Stdlib.Program Require Import Program Wf.
5  From Stdlib Require Import Lia.
6  From Stdlib Require Import Arith.Arith.
7  From Stdlib Require Import ZArith.
8  Import ListNotations.
9
10 Require Import Thesis.TPTSSignature.
11 Require Import Thesis.PTS.

```

```

12
13 Module TPTS (Sig : TPTS_SIGNATURE).
14
15 Import Sig.
16
17 Module Base := PTS Sig.
18
19 Import Base.
20
21 (* ===== *)
22 (* Retag *)
23
24 Definition retag (x : var) (s : Sort) : var := x * (n + 2) + index_of s.
25
26 Lemma retag_inj : forall x1 s1 x2 s2,
27   retag x1 s1 = retag x2 s2 -> x1 = x2 /\ s1 = s2.
28 Proof.
29   intros x1 s1 x2 s2 H. unfold retag in H.
30   pose proof (index_range s1) as [Hs1a Hs1b].
31   pose proof (index_range s2) as [Hs2a Hs2b].
32   assert (Hx : x1 = x2).
33   { assert (E1 : (x1 * (n + 2) + index_of s1) / (n + 2) = x1).
34     { rewrite Nat.div_add_1 by lia. rewrite Nat.div_small by lia. lia. }
35     assert (E2 : (x2 * (n + 2) + index_of s2) / (n + 2) = x2).
36     { rewrite Nat.div_add_1 by lia. rewrite Nat.div_small by lia. lia. }
37     rewrite H in E1. rewrite E1 in E2. exact E2. }
38   subst x1.
39   assert (Hi : index_of s1 = index_of s2) by lia.
40   split; [reflexivity | apply index_inj; exact Hi].
41 Qed.
42
43 Lemma retag_neq_of_atom_neq : forall x1 s1 x2 s2,
44   x1 <> x2 -> retag x1 s1 <> retag x2 s2.
45 Proof. intros x1 s1 x2 s2 Hne Heq. apply retag_inj in Heq. tauto. Qed.
46
47 Lemma retag_neq_of_sort_neq : forall x s1 s2,
48   s1 <> s2 -> retag x s1 <> retag x s2.
49 Proof. intros x s1 s2 Hne Heq. apply retag_inj in Heq. tauto. Qed.
50
51 Lemma retag_neq_mult : forall k x s, retag x s <> k * (n + 2).
52 Proof.
53   intros k x s Heq. unfold retag in Heq.
54   pose proof (index_range s) as [Hlo Hhi].
55   assert (Hm : (x * (n + 2) + index_of s) mod (n + 2) = index_of s).
56   { rewrite Nat.add_comm. rewrite Nat.Div0.mod_add by lia. apply Nat.mod_small;
57     lia. }
57   rewrite Heq in Hm.
58   rewrite Nat.Div0.mod_mul in Hm by lia.
59   lia.
60 Qed.

```

```

61
62 (* ===== *)
63 (* Degree *)
64
65 Fixpoint deg (t : term) : nat :=
66   match t with
67   | t_sort s      => index_of s + 1
68   | t_bvar s _    => index_of s - 1
69   | t_fvar s _    => index_of s - 1
70   | t_app M _     => deg M
71   | t_lam _ _ M   => deg M
72   | t_pi _ _ B    => deg B
73   end.
74
75 Definition T_eq (j : nat) (M : term) : Prop := deg M = j.
76 Definition T_geq (j : nat) (M : term) : Prop := j <= deg M.
77
78 Notation "'T_[' j ]'" := (T_eq j) (at level 0).
79 Notation "'T_[' j ]'" := (T_geq j) (at level 0).
80
81 Definition derivable (M : term) : Prop := exists  $\Gamma$  N,  $\Gamma \vdash M$  N.
82
83 Fixpoint closed_at (k : nat) (t : term) : Prop :=
84   match t with
85   | t_sort _      => True
86   | t_bvar _ n    => n < k
87   | t_fvar _ _    => True
88   | t_app M _     => closed_at k M
89   | t_lam _ _ M   => closed_at (S k) M
90   | t_pi _ _ B    => closed_at (S k) B
91   end.
92
93 Lemma closed_at_open_rec_inv : forall t k u,
94   closed_at k (open_rec k u t) ->
95   (forall s m, u <> t_bvar s m) ->
96   closed_at (S k) t.
97 Proof.
98   induction t as [s0 | sn n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A IHA B
99   IHB];
100   intros k u Hc Hu; simpl in *; auto.
101   - destruct (Nat.eqb n k) eqn:Heqb.
102     + apply Nat.eqb_eq in Heqb. lia.
103     + apply Nat.eqb_neq in Heqb.
104       simpl in Hc. lia.
105   - eapply IHP; eauto.
106   - eapply IHM with (k := S k); eauto.
107   - eapply IHB with (k := S k); eauto.
108 Qed.
109
110 Lemma lc_closed_at : forall t, lc t -> closed_at 0 t.

```



```

110 Proof.
111   intros t Hlc.
112   induction Hlc as [ s | x | M N HM IHM HN IHN
113                     | s A B L HA IHA HB IHB
114                     | s A M L HA IHA HM IHM ]; simpl; auto.
115   - remember (fresh L) as x0 eqn:Hx0def.
116     assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
117     specialize (IHB x0 Hx0L).
118     unfold open_var in IHB.
119     apply (closed_at_open_rec_inv B 0 (t_fvar s x0) IHB).
120     intros sv m Hcontra. discriminate.
121   - remember (fresh L) as x0 eqn:Hx0def.
122     assert (Hx0L : ~ In x0 L) by (rewrite Hx0def; apply fresh_notin).
123     specialize (IHM x0 Hx0L).
124     unfold open_var in IHM.
125     apply (closed_at_open_rec_inv M 0 (t_fvar s x0) IHM).
126     intros sv m Hcontra. discriminate.
127 Qed.
128
129 Lemma deg_open_rec_gen : forall B k x s,
130   bvar_tag_ok k s B ->
131   deg B = deg (open_rec k (t_fvar s x) B).
132 Proof.
133   induction B as [s0 | s_n n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A IHA B
134                   IHB];
135   intros k x s Htag; simpl in *.
136   - reflexivity.
137   - destruct (Nat.eqb n k) eqn:Heqb.
138     + apply Nat.eqb_eq in Heqb. subst n.
139       specialize (Htag eq_refl). subst s_n.
140       reflexivity.
141     + reflexivity.
142   - reflexivity.
143   - apply (IHP k x s Htag).
144   - apply (IHM (S k) x s Htag).
145   - apply (IHB (S k) x s Htag).
146 Qed.
147
148 Lemma deg_open : forall B x s,
149   bvar_tag_ok 0 s B ->
150   deg B = deg (open_var B s x).
151 Proof.
152   intros B x s Htag.
153   unfold open_var.
154   apply deg_open_rec_gen. exact Htag.
155 Qed.
156
157 Lemma deg_open_rec_subst : forall B k u s,
158   bvar_tag_ok k s B ->
159   deg u = index_of s - 1 ->

```

```

159   deg (open_rec k u B) = deg B.
160 Proof.
161   induction B as [s0 | s_n n | y | P IHP Q IHQ | s1 A IHA M IHM | s1 A IHA B
    IHB];
162     intros k u s Htag Hdeg; simpl in *.
163   - reflexivity.
164   - destruct (Nat.eqb n k) eqn:Heqb.
165     + apply Nat.eqb_eq in Heqb. subst n.
166       specialize (Htag eq_refl). subst s_n.
167       exact Hdeg.
168     + reflexivity.
169   - reflexivity.
170   - apply (IHP k u s Htag Hdeg).
171   - apply (IHM (S k) u s Htag Hdeg).
172   - apply (IHB (S k) u s Htag Hdeg).
173 Qed.
174
175 Lemma deg_subst_open : forall B x s N,
176   ~ In x (fv B) ->
177   bvar_tag_ok 0 s B ->
178   lc N ->
179   deg N = index_of s - 1 ->
180   deg ((open_var B s x) { x N }) = deg (open_var B s x).
181 Proof.
182   intros B x s N HxB Htag HlcN HdegN.
183   rewrite (subst_open_var_eq B s x N HxB HlcN).
184   pose proof (deg_open_rec_subst B 0 N s Htag HdegN) as Hgen.
185   rewrite Hgen.
186   unfold open_var.
187   apply deg_open_rec_gen. exact Htag.
188 Qed.
189
190 Axiom deg_sort_typed : forall  $\Gamma$  A s,
191    $\Gamma$  A t_sort s -> deg A = index_of s.
192
193 Axiom deg_conv_invariant : forall  $\Gamma$  M A B s,
194    $\Gamma$  M A -> A =b B ->  $\Gamma$  B t_sort s -> deg A = deg B.
195
196 Lemma typing_pi_subject_bvar_tag_ok : forall  $\Gamma$  M T,
197    $\Gamma$  M T -> forall s A B, M = t_pi s A B -> bvar_tag_ok 0 s B.
198 Proof.
199   intros  $\Gamma$  M T Htyp.
200   induction Htyp as
201     [ sa sb HA
202     |  $\Gamma$ 0 x0 A0 s0 Hfresh0 HAO IHA0
203     |  $\Gamma$ 0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
204     |  $\Gamma$ 0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
205     |  $\Gamma$ 0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
206     |  $\Gamma$ 0 M0 N0 A0 B0 s1a HM IHM HN IHN
207     |  $\Gamma$ 0 M0 A0 B0 sd HM IHM Heq HB IHB ];

```

```

208   intros s A B HeqM; try discriminate.
209   - apply (IHMO s A B HeqM).
210   - injection HeqM as Hs HA' HB'. subst. exact HTagB.
211   - apply (IHM s A B HeqM).
212 Qed.
213
214 Lemma deg_typing_succ : forall  $\Gamma$  M N,  $\Gamma \vdash M \vdash N \rightarrow \text{deg } N = \text{deg } M + 1$ .
215 Proof.
216   intros  $\Gamma$  M N Htyp.
217   induction Htyp as
218     [ sa sb HA
219     |  $\Gamma_0$  x0 A0 s0 Hfresh0 HAO IHA0
220     |  $\Gamma_0$  x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
221     |  $\Gamma_0$  A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
222     |  $\Gamma_0$  A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
223     |  $\Gamma_0$  M0 N0 A0 B0 s1a HM IHM HN IHN
224     |  $\Gamma_0$  M0 A0 B0 sd HM IHM Heq HB IHB ];
225   simpl in *.
226
227   - (* axiom *)
228     apply A_spec in HA as [Hlt Heq2]. lia.
229
230   - (* var *)
231     unfold deg in IHA0. simpl in IHA0.
232     pose proof (index_range s0) as [Hr1 Hr2]. unfold deg. lia.
233
234   - (* weak *)
235     exact IHMO.
236
237   - (* pi *)
238     remember (fresh (L ++ fv B0)) as x0 eqn:Hx0def.
239     assert (Hx0L : ~ In x0 L).
240     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv B0)).
241       apply in_or_app. left. exact Hc. }
242     specialize (IHB x0 Hx0L). unfold deg in IHB. simpl in IHB.
243     rewrite (deg_open B0 x0 s1 HTagB).
244     pose proof (R_shape s1 s2 s3 HR) as Hshape. subst s3. unfold deg.
245     lia.
246
247   - (* lam *)
248     remember (fresh (L ++ fv M0 ++ fv B0)) as x0 eqn:Hx0def.
249     assert (Hx0L : ~ In x0 L).
250     { rewrite Hx0def. intro Hc. apply (fresh_notin (L ++ fv M0 ++ fv B0)).
251       apply in_or_app. left. exact Hc. }
252     assert (HtypM :  $\Gamma_0 \vdash [(x0, (s1, A0))] \text{ open\_var } M0 \text{ s1 } x0 \text{ open\_var } B0 \text{ s1 } x0$ )
253       by (apply HM; auto).
254     specialize (IHM x0 Hx0L). unfold deg in IHM. simpl in IHM.
255     rewrite (deg_open M0 x0 s1 HTagM).
256     assert (HTagB0 : bvar_tag_ok 0 s1 B0)

```

```

257   by (apply (typing_pi_subject_bvar_tag_ok  $\Gamma$ 0 (t_pi s1 A0 B0) (t_sort s3)
258       HPi
259       s1 A0 B0 eq_refl)).
259   rewrite (deg_open B0 x0 s1 HTagB0). unfold deg.
260   lia.
261
262 - (* app *)
263   pose proof (type_correctness  $\Gamma$ 0 M0 (t_pi s1a A0 B0) HM) as Hcor.
264   destruct Hcor as [[s Hs] | [s Hs]].
265   + discriminate Hs.
266   + destruct (generation_pi  $\Gamma$ 0 s1a A0 B0 (t_sort s) Hs)
267       as [s2 [s3 [L [HB1 [HTag1 [HB2 [HB3 HB4]]]]]]].
268   assert (Hdeg_A0 : deg A0 = index_of s1a) by (apply (deg_sort_typed  $\Gamma$ 0 A0
269       s1a HB1)).
269   assert (Hdeg_N0 : deg N0 = index_of s1a - 1) by (unfold deg in *; lia).
270   pose proof (typing_lc  $\Gamma$ 0 N0 A0 HN) as [HlcN0 _].
271   remember (fresh (fv B0)) as x0 eqn:Hx0def.
272   assert (HxOB : ~ In x0 (fv B0)).
273   { rewrite Hx0def. apply fresh_notin. }
274   assert (Heqterm : B0 ~ N0 = (open_var B0 s1a x0) { x0 N0 })
275       by (apply subst_intro; [exact HxOB | exact HlcN0]).
276   assert (Hdegsub : deg ((open_var B0 s1a x0) { x0 N0 }) = deg (open_var B0
277       s1a x0))
277       by (apply deg_subst_open; [exact HxOB | exact HTag1 | exact HlcN0 |
278       exact Hdeg_N0]).
279   assert (Hdegopen : deg B0 = deg (open_var B0 s1a x0))
280       by (apply deg_open; exact HTag1).
281   unfold deg. rewrite Heqterm.
282   unfold deg in Hdegsub, Hdegopen.
283   rewrite Hdegsub. rewrite <- Hdegopen.
284   exact IHM.
285
286 - (* conv *)
287   pose proof (deg_conv_invariant  $\Gamma$ 0 M0 A0 B0 sd HM Heq HB) as Hcv.
288   unfold deg in *. lia.
289   Qed.
290
291 Lemma deg_le_top : forall t, deg t <= n + 1.
292 Proof.
293   induction t as [s | s_n0 n0 | sv v | P IHP Q IHQ | s A IHA M IHM | s A IHA B
294       IHB];
295   simpl; auto.
296   - pose proof (index_range s) as [_ H]. lia.
297   - pose proof (index_range s_n0) as [_ H]. lia.
298   - pose proof (index_range sv) as [_ H]. lia.
299   Qed.
300
301 Lemma deg_top : forall M, (derivable M) ->
302   (deg M = n + 1 <-> exists s, M = t_sort s /\ index_of s = n).

```

```

301 Proof.
302   intros M [Γ [N Htyp]]. split.
303
304 - intros Hdeg. induction Htyp as
305   [ sa sb HA
306   | Γ0 x0 A0 s0 Hfresh0 HAO IHA0
307   | Γ0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
308   | Γ0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
309   | Γ0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
310   | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
311   | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ].
312 + exists sa. split; [reflexivity |]. unfold deg in Hdeg. simpl in Hdeg. lia.
313 + unfold deg in Hdeg. simpl in Hdeg.
314   pose proof (index_range s0) as [H1 H2]. lia.
315 + apply IHM0. exact Hdeg.
316 + exfalso. unfold deg in Hdeg. simpl in Hdeg.
317   assert (Hsucc : deg (t_sort s3) = deg (t_pi s1 A0 B0) + 1)
318     by (apply deg_typing_succ with Γ0;
319         apply (typing_pi Γ0 A0 B0 s1 s2 s3 L HA HTagB HB HR)).
320   unfold deg in Hsucc. simpl in Hsucc.
321   pose proof (index_range s3) as [_ Hs2]. lia.
322 + exfalso. unfold deg in Hdeg. simpl in Hdeg.
323   assert (Hsucc1 : deg (t_pi s1 A0 B0) = deg (t_lam s1 A0 M0) + 1)
324     by (apply deg_typing_succ with Γ0;
325         apply (typing_lam Γ0 A0 M0 B0 s1 s3 L HA HTagM HM HPi)).
326   unfold deg in Hsucc1. simpl in Hsucc1.
327   pose proof (deg_le_top B0) as HB'. unfold deg in *. lia.
328 + exfalso. unfold deg in Hdeg. simpl in Hdeg.
329   assert (Hsucc : deg (t_pi s1a A0 B0) = deg M0 + 1)
330     by (apply deg_typing_succ with Γ0; exact HM).
331   unfold deg in Hsucc. simpl in Hsucc.
332   pose proof (deg_le_top B0) as HB'. unfold deg in *. lia.
333 + apply IHM. exact Hdeg.
334
335 - intros [s [Heq Hidx]]. subst M. unfold deg. simpl. lia.
336 Qed.
337
338 Lemma deg_top_type : forall M, (derivable M) ->
339   deg M = n <-> exists Γ s, (Γ M t_sort s /\ index_of s = n).
340 Proof.
341   intros M [Γ [N Htyp]]. split.
342
343 - intros Hdeg. induction Htyp as
344   [ sa sb HA
345   | Γ0 x0 A0 s0 Hfresh0 HAO IHA0
346   | Γ0 x0 B0 M0 A0 s0 Hfresh0 HMO IHMO HBO IHBO
347   | Γ0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
348   | Γ0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
349   | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
350   | Γ0 M0 A0 B0 sd HM IHM Heq HB IHB ].

```

```

351
352 + exists [], sb. split.
353 * apply typing_axiom. exact HA.
354 * pose proof (A_spec sa sb) as [HAiff _]. apply HAiff in HA as [_ HAEq].
355   unfold deg in Hdeg. simpl in Hdeg. lia.
356
357 + pose proof (deg_typing_succ  $\Gamma$ 0 A0 (t_sort s0) HA0) as Hs.
358   unfold deg in *. simpl in *.
359   pose proof (index_range s0). lia.
360
361 + apply IHM0. apply Hdeg.
362
363 + exists  $\Gamma$ 0, s3. split.
364 * apply (typing_pi  $\Gamma$ 0 A0 B0 s1 s2 s3 L HA HTagB HB HR).
365 * pose proof (deg_typing_succ  $\Gamma$ 0 (t_pi s1 A0 B0) (t_sort s3)
366   (typing_pi  $\Gamma$ 0 A0 B0 s1 s2 s3 L HA HTagB HB HR)) as Hsucc.
367   unfold deg in Hsucc, Hdeg. simpl in Hsucc, Hdeg. lia.
368
369 + exfalso.
370   assert (Hsucc1 : deg (t_pi s1 A0 B0) = deg (t_lam s1 A0 M0) + 1)
371     by (apply deg_typing_succ with  $\Gamma$ 0;
372       apply (typing_lam  $\Gamma$ 0 A0 M0 B0 s1 s3 L HA HTagM HM HPi)).
373   unfold deg in Hsucc1, Hdeg. simpl in Hsucc1, Hdeg.
374   assert (Htop : deg (t_pi s1 A0 B0) = n + 1) by (unfold deg; simpl; lia).
375   assert (Hderiv : derivable (t_pi s1 A0 B0)).
376   { unfold derivable. exists  $\Gamma$ 0. exists (t_sort s3). exact HPi. }
377   destruct (proj1 (deg_top (t_pi s1 A0 B0) Hderiv) Htop) as [s' [Heqs' _]].
378   discriminate Heqs'.
379
380 + exfalso.
381   unfold deg in Hdeg. simpl in Hdeg.
382   assert (Hsucc : deg (t_pi s1a A0 B0) = deg M0 + 1)
383     by (apply deg_typing_succ with  $\Gamma$ 0; exact HM).
384   assert (Htop : deg (t_pi s1a A0 B0) = n + 1) by (unfold deg in *; simpl in
385     *; lia).
386   pose proof (type_correctness  $\Gamma$ 0 M0 (t_pi s1a A0 B0) HM) as Hcor.
387   destruct Hcor as [[s Hs] | [s Hs]].
388   * discriminate Hs.
389   * assert (Hderiv : derivable (t_pi s1a A0 B0)).
390     { unfold derivable. exists  $\Gamma$ 0. exists (t_sort s). exact Hs. }
391     destruct (proj1 (deg_top (t_pi s1a A0 B0) Hderiv) Htop) as [s' [Heqs'
392       _]].
393     discriminate Heqs'.
394
395 + apply IHM. exact Hdeg.
396
397 - intros [ $\Gamma'$  [s [H1 H2]]].
398   pose proof (deg_typing_succ  $\Gamma'$  M (t_sort s) H1). unfold deg in *. simpl in
399     *. lia.
400 Qed.

```

```

398
399 Lemma deg_typing : forall M, derivable M -> forall i, (i > 0) -> (i < n) ->
400 (deg M = i <-> (exists  $\Gamma$  N s, ( $\Gamma$  M N) /\ ( $\Gamma$  N t_sort s) /\ (index_of s =
401 i + 1))).
402 Proof.
403   intros M [ $\Gamma$  [T Htyp]] i Hi1 Hi2. split.
404   - intros Hdeg. pose proof (deg_typing_succ  $\Gamma$  M T Htyp).
405     pose proof (type_correctness  $\Gamma$  M T Htyp) as Hcor. destruct Hcor as [[s Hs] |
406 [s Hs]].
407     + subst T. unfold deg in *. simpl in *. pose proof (index_range s). assert
408 (Gi : index_of s = i) by lia.
409     assert (Hi : 0 < i + 1 < n + 1) by lia.
410     pose proof (index_surj (i + 1) Hi) as Hs'. destruct Hs' as [s' Hs'].
411     assert (Hss' : A s s') by (apply A_spec; lia).
412     exists  $\Gamma$ , (t_sort s), s'. repeat split; auto. apply start_axiom; auto.
413     exists M, (t_sort s); auto.
414     + pose proof (deg_typing_succ  $\Gamma$  T (t_sort s) Hs). unfold deg in *. simpl in
415 *.
416     exists  $\Gamma$ , T, s. repeat split; auto; lia.
417   - intros [ $\Gamma'$  [N [s [H1 [H2 H3]]]]].
418     pose proof (deg_typing_succ  $\Gamma'$  M N H1) as Hdeg1.
419     pose proof (deg_typing_succ  $\Gamma'$  N (t_sort s) H2) as Hdeg2.
420     unfold deg in *. simpl in *. lia.
421 Qed.
422
423 Axiom deg_beta : forall M N, (M ->>b N) -> (deg M = deg N).
424
425 (* ===== *)
426 (* Full System *)
427
428 Definition full_with (i j : nat) : Prop :=
429   forall l k : Sort,
430   index_of l <= i ->
431   index_of l <= index_of k ->
432   index_of k <= j ->
433   R l k k.
434
435 Definition full : Prop :=
436   forall s1 s2 s3, R s1 s2 s3 -> full_with (index_of s2) (index_of s1).
437
438 (* For the remainder of this section, fix a full n-tiered pure type system S. *)
439 Axiom is_full : full.
440
441 Definition R_star (s s' s'' : Sort) : Prop :=
442   R s s' s'' /\ index_of s' <= index_of s.
443
444 (* ===== *)

```

```

443 (* The non-dependent typing judgement *)
444
445 Reserved Notation " $\Gamma$  * M A" (at level 70, no associativity).
446
447 Inductive typing_star : context -> term -> term -> Prop :=
448   | typing_star_axiom : forall s s',
449     A s s' ->
450     [] * t_sort s t_sort s'
451
452   | typing_star_var : forall  $\Gamma$  x A s,
453     is_fresh x  $\Gamma$  ->
454      $\Gamma$  * A t_sort s ->
455      $\Gamma$  ++ [(x, (s, A))] * t_fvar s x A
456
457   | typing_star_weak : forall  $\Gamma$  x B M A s,
458     is_fresh x  $\Gamma$  ->
459      $\Gamma$  * M A ->
460      $\Gamma$  * B t_sort s ->
461      $\Gamma$  ++ [(x, (s, B))] * M A
462
463   | typing_star_pi : forall  $\Gamma$  A B s1 s2 s3 L,
464      $\Gamma$  * A t_sort s1 ->
465     (forall x, ~ In x L ->
466        $\Gamma$  ++ [(x, (s1, A))] * (open_var B s1 x) t_sort s2) ->
467     R_star s1 s2 s3 ->
468      $\Gamma$  * (t_pi s1 A B) (t_sort s3)
469
470   | typing_star_lam : forall  $\Gamma$  A M B s1 s3 L,
471      $\Gamma$  * A t_sort s1 ->
472     (forall x, ~ In x L ->
473        $\Gamma$  ++ [(x, (s1, A))] * (open_var M s1 x) (open_var B s1 x)) ->
474      $\Gamma$  * (t_pi s1 A B) (t_sort s3) ->
475      $\Gamma$  * (t_lam s1 A M) (t_pi s1 A B)
476
477   | typing_star_app : forall  $\Gamma$  M N A B s1,
478      $\Gamma$  * M t_pi s1 A B ->
479      $\Gamma$  * N A ->
480      $\Gamma$  * t_app M N (B ^^ N)
481
482   | typing_star_conv : forall  $\Gamma$  M A B s,
483      $\Gamma$  * M A ->
484     A =b B ->
485      $\Gamma$  * B t_sort s ->
486      $\Gamma$  * M B
487
488 where " $\Gamma$  * M A" := (typing_star  $\Gamma$  M A).
489
490 Definition legal_star ( $\Gamma$  : context) : Prop :=
491   exists M N,  $\Gamma$  * M N.
492

```



```

493 Definition derivable_star (M : term) : Prop :=
494   exists  $\Gamma$  N,  $\Gamma \vdash M \vdash N$ .
495
496 Definition partial_sort_of (i : nat) (Hi : 0 < i < n + 1) : Sort :=
497   proj1_sig (
498     constructive_indefinite_description
499       (fun s => index_of s = i)
500       (index_surj i Hi)
501   ).
502
503 Lemma one_in_range : 0 < 1 /\ 1 < n + 1.
504 Proof. pose proof n_range. lia. Qed.
505
506 Definition sort_of (i : nat) : Sort :=
507   match (lt_dec 0 i, lt_dec i (n+1)) with
508   | (left H1, left H2) => partial_sort_of i (conj H1 H2)
509   | _ => partial_sort_of 1 one_in_range
510   end.
511
512 (* ===== *)
513 (* Type-Level Translation *)
514 (*  $i : T^{i+1} \rightarrow T^{i+1}$  *)
515
516 Definition zero_var : var := 0.
517
518 Lemma zero_var_retag_ne : forall (x : var) (s : Sort),
519   retag x s <> zero_var.
520 Proof. intros x s. unfold zero_var. apply (retag_neq_mult 0). Qed.
521
522 Axiom typing_avoids_zero_var :
523   forall  $\Gamma$  M N,  $\Gamma \vdash M \vdash N \rightarrow$ 
524   forall x sA A, In (x, (sA, A))  $\Gamma \rightarrow x \neq$  zero_var.
525
526 Fixpoint rho_pi_tel (r : nat -> term -> term) (A B : term) (i k : nat) : term :=
527   match k with
528   | 0 => t_pi (sort_of i) (r i A) (r i B)
529   | S k' => t_pi (sort_of (i + S k')) (r (i + S k') A) (rho_pi_tel r A B i k')
530   end.
531
532 Fixpoint rho_lam_tel (r : nat -> term -> term) (A M' : term) (i k : nat) : term
533 :=
534   match k with
535   | 0 => t_lam (sort_of (i + 1)) (r (i + 1) A) (r i M')
536   | S k' => t_lam (sort_of (i + 1 + S k')) (r (i + 1 + S k') A) (rho_lam_tel r A
537     M' i k')
538   end.
539
540 Fixpoint rho_app_tel (r : nat -> term -> term) (M' N : term) (i k : nat) : term
541 :=
542   match k with

```

```

540 | 0 => t_app (r i M') (r i N)
541 | S k' => t_app (rho_app_tel r M' N i k') (r (i + S k') N)
542 end.
543
544 Fixpoint rho (i : nat) (M : term) : term :=
545   match M with
546   | t_sort s =>
547     match i with
548     | 0 => t_fvar (sort_of 2) zero_var
549     | _ => t_sort (sort_of i)
550     end
551   | t_bvar s0 k => t_bvar s0 k
552   | t_fvar s0 x =>
553     t_fvar (sort_of (i + 2)) (retag x (sort_of (i + 2)))
554   | t_pi s A B =>
555     match le_lt_dec (i + 1) (deg A) with
556     | left _ => rho_pi_tel rho A B i (deg A - 1 - i)
557     | right _ => rho i B
558     end
559   | t_lam s A M' =>
560     match le_lt_dec (i + 2) (deg A) with
561     | left _ => rho_lam_tel rho A M' i (deg A - 1 - (i + 1))
562     | right _ => rho i M'
563     end
564   | t_app M' N =>
565     match le_lt_dec (i + 1) (deg N) with
566     | left _ => rho_app_tel rho M' N i (deg N - 1 - i)
567     | right _ => rho i M'
568     end
569   end.
570
571 Lemma rho_pi_named : forall i s Dom B, i + 1 <= deg Dom ->
572   rho i (t_pi s Dom B) = rho_pi_tel rho Dom B i (deg Dom - 1 - i).
573 Proof.
574   intros i s Dom B Hdeg. simpl.
575   destruct (le_lt_dec (i + 1) (deg Dom)); [reflexivity | lia].
576 Qed.
577
578 Lemma rho_lam_named : forall i s Dom B, i + 2 <= deg Dom ->
579   rho i (t_lam s Dom B) = rho_lam_tel rho Dom B i (deg Dom - 1 - (i + 1)).
580 Proof.
581   intros i s Dom B Hdeg. simpl.
582   destruct (le_lt_dec (i + 2) (deg Dom)); [reflexivity | lia].
583 Qed.
584
585 Lemma rho_app_named : forall i P Q, i + 1 <= deg Q ->
586   rho i (t_app P Q) = rho_app_tel rho P Q i (deg Q - 1 - i).
587 Proof.
588   intros i P Q Hdeg. simpl.
589   destruct (le_lt_dec (i + 1) (deg Q)); [reflexivity | lia].

```

```

590 Qed.
591
592 Fixpoint rho_ctx_tel (x : var) (s : Sort) (T : term) (k : nat) : context :=
593   let j := index_of s in
594   match k with
595   | 0    => [(retag x (sort_of j), (sort_of j, rho (j - 1) T))]
596   | S k' => (rho_ctx_tel x s T k')
597             ++ [(retag x (sort_of (j - k)), (sort_of (j - k), rho (j - k - 1)
598               T))]
599
600   end.
601
602 Fixpoint rho_ctx (i : nat) (Γ : context) : context :=
603   match Γ with
604   | [] => [(zero_var, (sort_of 2, t_sort (sort_of 1)))]
605   | (x, (s, T)) :: Γ' => (rho_ctx_tel x s T (index_of s - i - 1)) ++ rho_ctx i
606     Γ'
607   end.
608
609 Lemma subcontext_append :
610   forall Γ1 Δ1 Γ2 Δ2,
611   (Γ1 Δ1 -> (Γ2 Δ2) -> ((Γ1 ++ Γ2) (Δ1 ++ Δ2))).
612 Proof.
613   intros Γ1 Δ1 Γ2 Δ2 H1 H2 x T Hin.
614   apply in_app_or in Hin. destruct Hin; apply in_or_app.
615   - left; auto.
616   - right; auto.
617 Qed.
618
619 Lemma rho_ctx_tel_step : forall x s T k,
620   rho_ctx_tel x s T k  rho_ctx_tel x s T (S k).
621 Proof.
622   intros x s T k y T' Hin.
623   simpl. apply in_or_app. left. exact Hin.
624 Qed.
625
626 Lemma rho_ctx_tel_sub :
627   forall x s T p q,
628   p < q -> rho_ctx_tel x s T p  rho_ctx_tel x s T q.
629 Proof.
630   intros x s T p q Hpq.
631   remember (q - p - 1) as d eqn:Hd.
632   assert (Hq : q = p + S d) by lia.
633   subst q. clear Hpq Hd.
634   induction d as [| d IH].
635   - replace (p + 1) with (S p) by lia.
636     apply rho_ctx_tel_step.
637   - replace (p + S (S d)) with (S (p + S d)) by lia.
638     intros y T' Hin.
639     apply rho_ctx_tel_step.
640     apply IH. exact Hin.

```

```

638 Qed.
639
640 Definition ctx_above (b : nat) (Γ : context) : Prop :=
641   forall x s T, In (x, (s, T)) Γ -> b < index_of s.
642
643 Lemma rho_ctx_sub : forall Γ i j,
644   i < j -> ctx_above j Γ -> rho_ctx j Γ  rho_ctx i Γ.
645 Proof.
646   induction Γ as [| [y [s T]] Γ' IH]; intros i j Hij Habove.
647   - simpl. unfold subcontext. auto.
648   - simpl. apply subcontext_append.
649     + apply rho_ctx_tel_sub.
650       assert (Hy : j < index_of s).
651       { apply Habove with y T. left. reflexivity. }
652       lia.
653     + apply IH; auto. intros x' s' T' Hin. apply Habove with x' T'. right. exact
        Hin.
654 Qed.
655
656 (* ===== *)
657 (* rho commutes with substitution *)
658
659 Fixpoint rho_subst_tel (M N : term) (varidx i k : nat) : term :=
660   match k with
661   | 0 => M { retag varidx (sort_of (i + 2))  rho i N }
662   | S k' => rho_subst_tel (M { retag varidx (sort_of (k + i + 2))  rho (k + i) N
663   }) N varidx i k'
664   end.
665
666 Definition rho_subst (M N : term) (x : var) (s : Sort) (i : nat) : term :=
667   let j := index_of s in
668   match lt_dec j (i + 2) with
669   | left _ => rho i M
670   | right _ => rho_subst_tel (rho i M) (N) (x) (i) (j - i - 2)
671   end.
672
673 Lemma sort_of_correct : forall i, 0 < i -> i < n + 1 -> index_of (sort_of i) =
674   i.
675 Proof.
676   intros i H1 H2. unfold sort_of.
677   destruct (lt_dec 0 i) as [Hd1|Hd1]; [| lia].
678   destruct (lt_dec i (n+1)) as [Hd2|Hd2]; [| lia].
679   unfold partial_sort_of.
680   destruct (constructive_indefinite_description (fun s => index_of s = i)
681     (index_surj i (conj Hd1 Hd2))) as [s Hs].
682   exact Hs.
683 Qed.
684
685 Lemma sort_of_inj : forall p q,
686   0 < p -> p < n + 1 -> 0 < q -> q < n + 1 ->

```

```

685   sort_of p = sort_of q -> p = q.
686 Proof.
687   intros p q Hp1 Hp2 Hq1 Hq2 Heq.
688   assert (Hp : index_of (sort_of p) = p) by (apply sort_of_correct; lia).
689   assert (Hq : index_of (sort_of q) = q) by (apply sort_of_correct; lia).
690   rewrite Heq in Hp. rewrite Hp in Hq. exact Hq.
691 Qed.
692
693 Lemma index_of_sort_of_any : forall p, index_of (sort_of p) = 1 \/ index_of
694 (sort_of p) = p.
695 Proof.
696   intros p. unfold sort_of.
697   destruct (lt_dec 0 p) as [H1 | H1].
698   - destruct (lt_dec p (n + 1)) as [H2 | H2].
699     + right. unfold partial_sort_of.
700       destruct (constructive_indefinite_description (fun s => index_of s = p)
701         (index_surj p (conj H1 H2))) as [s Hs].
702       exact Hs.
703     + left. unfold partial_sort_of.
704       destruct (constructive_indefinite_description (fun s => index_of s = 1)
705         (index_surj 1 one_in_range)) as [s Hs].
706       exact Hs.
707   - left. unfold partial_sort_of.
708     destruct (constructive_indefinite_description (fun s => index_of s = 1)
709       (index_surj 1 one_in_range)) as [s Hs].
710     exact Hs.
711 Qed.
712
713 Lemma sort_of_tier_neq : forall D m, m < D -> m + 2 <= n -> sort_of (D + 2) <>
714 sort_of (m + 2).
715 Proof.
716   intros D m Hmd Hmn Heq.
717   assert (Hidx : index_of (sort_of (D + 2)) = index_of (sort_of (m + 2))) by
718     (rewrite Heq; reflexivity).
719   rewrite (sort_of_correct (m + 2) ltac:(lia) ltac:(lia)) in Hidx.
720   destruct (index_of_sort_of_any (D + 2)) as [H | H]; lia.
721 Qed.
722
723 Lemma rho_min_tier_noop : forall M D m v N',
724   m < D -> m + 2 <= n ->
725   (rho D M) { retag v (sort_of (m + 2)) N' } = rho D M.
726 Proof.
727   induction M as [s | s_k k | sy y | P IHP Q IHQ | s A IHA M' IHM' | s A IHA B
728     IHB];
729   intros D m v N' Hmd Hmn.
730   - (* t_sort s *)
731     destruct D as [| D']; [lia | simpl; reflexivity].
732   - (* t_bvar k *)
733     simpl; reflexivity.

```

```

730 - (* t_fvar sy y *)
731   simpl.
732   destruct (eq_var_dec (retag y (sort_of (D + 2))) (retag v (sort_of (m +
733     as [Heq | Hne]; [ | reflexivity].
734   exfalse. apply retag_inj in Heq as [_ Hsort].
735   exact (sort_of_tier_neq D m Hmd Hmn Hsort).
736 - (* t_app P Q *)
737   simpl.
738   destruct (le_lt_dec (D + 1) (deg Q)) as [Hle | Hlt].
739   + generalize (deg Q - 1 - D) as k.
740     induction k as [| k' IHk]; intros.
741     * simpl. f_equal.
742       -- apply IHP; lia.
743       -- apply IHQ; lia.
744     * simpl. f_equal.
745       -- exact IHk.
746       -- apply IHQ; lia.
747   + apply IHP; lia.
748 - (* t_lam s A M' *)
749   simpl.
750   destruct (le_lt_dec (D + 2) (deg A)) as [Hle | Hlt].
751   + generalize (deg A - 1 - (D + 1)) as k.
752     induction k as [| k' IHk]; intros.
753     * simpl. f_equal.
754       -- apply IHA; lia.
755       -- apply IHM'; lia.
756     * simpl. f_equal.
757       -- apply IHA; lia.
758       -- exact IHk.
759   + apply IHM'; lia.
760 - (* t_pi s A B *)
761   simpl.
762   destruct (le_lt_dec (D + 1) (deg A)) as [Hle | Hlt].
763   + generalize (deg A - 1 - D) as k.
764     induction k as [| k' IHk]; intros.
765     * simpl. f_equal.
766       -- apply IHA; lia.
767       -- apply IHB; lia.
768     * simpl. f_equal.
769       -- apply IHA; lia.
770       -- exact IHk.
771   + apply IHB; lia.
772 Qed.
773
774 Lemma eq_var_dec_refl : forall v : var, exists e, eq_var_dec v v = left e.
775 Proof.
776   intros v. destruct (eq_var_dec v v) as [e | f].
777   - exists e. reflexivity.
778   - exfalse. apply f. reflexivity.

```

```

779 Qed.
780
781 Lemma subst_high_preserves_clean_low : forall P vhi R vlo,
782   vhi <> vlo ->
783   (forall NO, R { vlo NO } = R) ->
784   (forall NO, P { vlo NO } = P) ->
785   forall N1, (P { vhi R }) { vlo N1 } = P { vhi R }.
786 Proof.
787   induction P as [s | s_k k | sy y | P1 IHP1 P2 IHP2 | s A IHA M' IHM' | s A IHA
788   B IHB];
789   intros vhi R vlo Hne HRclean HPclean N1.
790   - reflexivity.
791   - reflexivity.
792   - rewrite subst_var. destruct (eq_var_dec y vhi) as [Heq | Hneq].
793     + apply HRclean.
794     + rewrite subst_var. destruct (eq_var_dec y vlo) as [Heq2 | Hneq2].
795       * subst y. specialize (HPclean N1). rewrite subst_var in HPclean.
796         destruct (eq_var_dec_refl vlo) as [e Heq]. rewrite Heq in HPclean.
797         destruct (eq_var_dec vlo vhi) as [Hc | Hc]; [congruence | exact
798         HPclean].
799       * reflexivity.
800   - assert (Hclean1 : forall NO, P1 { vlo NO } = P1).
801     { intros NO. specialize (HPclean NO). rewrite subst_app in HPclean.
802       injection HPclean as H1 H2. exact H1. }
803     assert (Hclean2 : forall NO, P2 { vlo NO } = P2).
804     { intros NO. specialize (HPclean NO). rewrite subst_app in HPclean.
805       injection HPclean as H1 H2. exact H2. }
806     rewrite subst_app. rewrite subst_app. f_equal.
807     + apply IHP1; auto.
808     + apply IHP2; auto.
809   - assert (Hclean1 : forall NO, A { vlo NO } = A).
810     { intros NO. specialize (HPclean NO). rewrite subst_lam in HPclean.
811       injection HPclean as H1 H2. exact H1. }
812     assert (Hclean2 : forall NO, M' { vlo NO } = M').
813     { intros NO. specialize (HPclean NO). rewrite subst_lam in HPclean.
814       injection HPclean as H1 H2. exact H2. }
815     rewrite subst_lam. rewrite subst_lam. f_equal.
816     + apply IHA; auto.
817     + apply IHM'; auto.
818   - assert (Hclean1 : forall NO, A { vlo NO } = A).
819     { intros NO. specialize (HPclean NO). rewrite subst_pi in HPclean. injection
820     HPclean as H1 H2. exact H1. }
821     assert (Hclean2 : forall NO, B { vlo NO } = B).
822     { intros NO. specialize (HPclean NO). rewrite subst_pi in HPclean. injection
823     HPclean as H1 H2. exact H2. }
824     rewrite subst_pi. rewrite subst_pi. f_equal.
825     + apply IHA; auto.
826     + apply IHB; auto.
827 Qed.

```

```

820
821 Fixpoint only_tagged (x : var) (s : Sort) (t : term) : Prop :=
822   match t with
823   | t_sort _      => True
824   | t_bvar _ _    => True
825   | t_fvar s0 y   => y = x -> s0 = s
826   | t_app P Q     => only_tagged x s P /\ only_tagged x s Q
827   | t_pi _ A B    => only_tagged x s A /\ only_tagged x s B
828   | t_lam _ A M   => only_tagged x s A /\ only_tagged x s M
829   end.
830
831 Lemma deg_subst_tagged : forall C x s N,
832   only_tagged x s C -> lc N -> deg N = index_of s - 1 ->
833   deg (C { x N }) = deg C.
834 Proof.
835   induction C as [s0 | s_k k | s0 y | P IHP Q IHQ | s1 A IHA M IHM | s1 A IHA B
836     IHB];
837   intros x s N Htag HlcN HdegN; simpl in *.
838   - reflexivity.
839   - reflexivity.
840   - destruct (eq_var_dec y x) as [Heq | Hneq].
841     + subst y. rewrite (Htag eq_refl). exact HdegN.
842     + reflexivity.
843   - destruct Htag as [HtagP _]. apply (IHP x s N HtagP HlcN HdegN).
844   - destruct Htag as [_ HtagM]. apply (IHM x s N HtagM HlcN HdegN).
845   - destruct Htag as [_ HtagB]. apply (IHB x s N HtagB HlcN HdegN).
846 Qed.
847
848 Lemma rho_subst_tel_noop_generic : forall P N varidx D i0 K,
849   i0 + K + 2 <= n ->
850   (forall m v NO, m < D -> m + 2 <= n -> P { retag v (sort_of (m + 2)) NO } =
851     P) ->
852   i0 + K + 1 <= D ->
853   rho_subst_tel P N varidx i0 K = P.
854 Proof.
855   intros P N varidx D i0 K HKn HcleanP.
856   revert i0 HKn HcleanP. induction K as [| K' IHK]; intros i0 HKn HcleanP Htop.
857   - simpl. apply HcleanP; lia.
858   - simpl.
859     assert (HS1 : S (K' + i0) = (S K' + i0)) by lia. rewrite HS1.
860     assert (HS2 : S (K' + i0 + 2) = S K' + i0 + 2) by lia. rewrite HS2.
861     rewrite (HcleanP (S K' + i0) varidx (rho (S K' + i0) N)); [| lia | lia].
862     apply IHK; [| lia | exact HcleanP | lia].
863 Qed.
864
865 Lemma rho_subst_tel_stays_clean : forall P N varidx D i0 K,
866   D <= i0 ->
867   (forall m v NO, m < D -> m + 2 <= n -> P { retag v (sort_of (m + 2)) NO } =
868     P) ->
869   forall m v NO, m < D -> m + 2 <= n ->

```



```

867   (rho_subst_tel P N varidx i0 K) { retag v (sort_of (m + 2)) NO } =
      rho_subst_tel P N varidx i0 K.
868 Proof.
869   intros P N varidx D i0 K. revert P.
870   induction K as [| K' IHK]; intros P Hle HcleanP m v NO Hmd Hmn.
871   - simpl. apply subst_high_preserves_clean_low.
872     + intro Hc. apply retag_inj in Hc as [_ Hsc]. apply (sort_of_tier_neq i0 m);
      [lia | lia | exact Hsc].
873     + intros N1. apply rho_min_tier_noop; lia.
874     + intros N1. apply HcleanP; auto.
875   - simpl. apply IHK.
876     + lia.
877     + intros m' v' NO' Hmd' Hmn'. apply subst_high_preserves_clean_low.
878       * intro Hc. apply retag_inj in Hc as [_ Hsc]. apply (sort_of_tier_neq (S
        K' + i0) m'); [lia | lia | exact Hsc].
879       * intros N1. apply rho_min_tier_noop; lia.
880       * intros N1. apply HcleanP; auto.
881     + exact Hmd.
882     + exact Hmn.
883 Qed.
884
885 Lemma rho_subst_tel_split : forall P N varidx i0 K D,
886   i0 < D -> D <= i0 + K ->
887   rho_subst_tel P N varidx i0 K
888   = rho_subst_tel (rho_subst_tel P N varidx D (i0 + K - D)) N varidx i0 (D - i0
    - 1).
889 Proof.
890   intros P N varidx i0 K. revert P i0.
891   induction K as [| K' IHK]; intros P i0 D Hlt Hle.
892   - lia.
893   - destruct (Nat.eq_dec D (i0 + S K')) as [Heq | Hneq].
894     + subst D.
895       replace (i0 + S K' - (i0 + S K')) with 0 by lia.
896       simpl.
897       replace (i0 + S K' - i0 - 1) with K' by lia.
898       replace (i0 + S K') with (S K' + i0) by lia.
899       reflexivity.
900     + assert (HleK' : D <= i0 + K') by lia.
901       simpl.
902       assert (HS1 : S (K' + i0 + 2) = S K' + i0 + 2) by lia. rewrite HS1.
903       assert (HS2 : S (K' + i0) = S K' + i0) by lia. rewrite HS2.
904       rewrite (IHK (P { retag varidx (sort_of (S K' + i0 + 2)) rho (S K' + i0)
        N }) i0 D Hlt HleK')).
905       replace (i0 + S K' - D) with (S (i0 + K' - D)) by lia.
906       simpl.
907       f_equal.
908       replace (i0 + K' - D + D + 2) with (K' + i0 + 2) by lia.
909       replace (i0 + K' - D + D) with (K' + i0) by lia.
910       reflexivity.

```

```

911 Qed.
912
913 Lemma rho_subst_tel_pi_commute : forall N s' A' B' varidx i k,
914   rho_subst_tel (t_pi s' A' B') N varidx i k
915   = t_pi s' (rho_subst_tel A' N varidx i k) (rho_subst_tel B' N varidx i k).
916 Proof.
917   intros N s' A' B' varidx i k.
918   revert A' B' s'.
919   induction k as [| k' IHk]; intros s' A' B'; simpl; auto.
920 Qed.
921
922 Lemma rho_subst_tel_pi_tel_commute : forall A B N varidx i0 k0 i1 k1,
923   rho_subst_tel (rho_pi_tel rho A B i1 k1) N varidx i0 k0
924   = rho_pi_tel (fun i' M => rho_subst_tel (rho i' M) N varidx i0 k0) A B i1 k1.
925 Proof.
926   intros A B N varidx i0 k0 i1.
927   induction k1 as [| k1' IHk1]; intros.
928   - simpl. apply rho_subst_tel_pi_commute.
929   - simpl. rewrite rho_subst_tel_pi_commute. f_equal. apply IHk1.
930 Qed.
931
932 Lemma rho_subst_tel_app_commute : forall N A' B' varidx i k,
933   rho_subst_tel (t_app A' B') N varidx i k
934   = t_app (rho_subst_tel A' N varidx i k) (rho_subst_tel B' N varidx i k).
935 Proof.
936   intros N A' B' varidx i k.
937   revert A' B'.
938   induction k as [| k' IHk]; intros A' B'; simpl; auto.
939 Qed.
940
941 Lemma rho_subst_tel_app_tel_commute : forall A B N varidx i0 k0 i1 k1,
942   rho_subst_tel (rho_app_tel rho A B i1 k1) N varidx i0 k0
943   = rho_app_tel (fun i' M => rho_subst_tel (rho i' M) N varidx i0 k0) A B i1 k1.
944 Proof.
945   intros A B N varidx i0 k0 i1.
946   induction k1 as [| k1' IHk1]; intros.
947   - simpl. apply rho_subst_tel_app_commute.
948   - simpl. rewrite rho_subst_tel_app_commute. f_equal. apply IHk1.
949 Qed.
950
951 Lemma rho_subst_tel_lam_commute : forall N s' A' B' varidx i k,
952   rho_subst_tel (t_lam s' A' B') N varidx i k
953   = t_lam s' (rho_subst_tel A' N varidx i k) (rho_subst_tel B' N varidx i k).
954 Proof.
955   intros N s' A' B' varidx i k.
956   revert A' B' s'.
957   induction k as [| k' IHk]; intros s' A' B'; simpl; auto.
958 Qed.
959
960 Lemma rho_subst_tel_lam_tel_commute : forall A B N varidx i0 k0 i1 k1,

```

```

961   rho_subst_tel (rho_lam_tel rho A B i1 k1) N varidx i0 k0
962   = rho_lam_tel (fun i' M => rho_subst_tel (rho i' M) N varidx i0 k0) A B i1 k1.
963 Proof.
964   intros A B N varidx i0 k0 i1.
965   induction k1 as [| k1' IHk1]; intros.
966   - simpl. apply rho_subst_tel_lam_commute.
967   - simpl. rewrite rho_subst_tel_lam_commute. f_equal. apply IHk1.
968 Qed.
969
970 Lemma rho_commutes_substitution :
971   forall i, 0 <= i <= n ->
972   forall x s, x <> zero_var -> 1 <= index_of s <= n ->
973   forall M, deg M >= i + 1 -> only_tagged x s M ->
974   forall N, deg N = index_of s - 1 -> lc N ->
975   rho i (M { x N }) = rho_subst M N x s i.
976 Proof.
977   intros i.
978   induction i as [i IHouter] using
979     (well_founded_induction (Wf_nat.well_founded_ltof nat (fun i => n - i))).
980   intros Hi x s Hxnz Hx M.
981   induction M as [s0 | s_m m | sy y | D IHD C IHC | s1 C IHC D IHD | s1 C IHC D
982     IHD];
983     intros Hdeg Honly N HdegN HlcN.
984   - (* M = t_sort s0 *)
985     rewrite subst_sort. unfold rho_subst.
986     destruct (lt_dec (index_of s) (i + 2)) as [E | E]; auto.
987     assert (Hconst : forall k M0,
988       k <= index_of s - i - 2 ->
989       (exists s0', M0 = t_sort s0') /\ (i = 0 /\ M0 = t_fvar
990         (sort_of 2) zero_var) ->
991       rho_subst_tel M0 N x i k = M0).
992   { induction k as [| k' IHk]; intros M0 Hkbound Hcase.
993     - simpl. destruct Hcase as [[s0' Heq] | [Hi0 Heq]]; subst M0.
994       + rewrite subst_sort. reflexivity.
995       + rewrite subst_var.
996         destruct (eq_var_dec zero_var (retag x (sort_of (i + 2)))) as [Heq2 |
997           Hne2]; auto.
998         exfalso. apply (zero_var_retag_ne x (sort_of (i + 2))). symmetry.
999         exact Heq2.
1000     - simpl. destruct Hcase as [[s0' Heq] | [Hi0 Heq]]; subst M0.
1001       + rewrite subst_sort. apply IHk; [lia | left; exists s0'; reflexivity].
1002       + rewrite subst_var.
1003         destruct (eq_var_dec zero_var (retag x (sort_of (k' + i + 2)))) as
1004           [Heq2 | Hne2].
1005         * exfalso. apply (zero_var_retag_ne x (sort_of (k' + i + 2))).
1006           symmetry. exact Heq2.
1007         * destruct (eq_var_dec zero_var (retag x (sort_of (S (k' + i + 2)))))
1008           as [Heq3 | Hne3].

```

```

1003         -- exfalso. apply (zero_var_retag_ne x (sort_of (S (k' + i + 2)))).
1004         symmetry. exact Heq3.
1005         -- apply IHk; [lia | right; split; [exact Hi0 | reflexivity]].
1006     }
1007     symmetry. apply Hconst; auto.
1008     destruct i as [| i'].
1009     + right. split; auto.
1010     + left. exists (sort_of (S i')). reflexivity.
1011
1012 - (* M = t_bvar m *)
1013     rewrite subst_bvar. unfold rho_subst.
1014     destruct (lt_dec (index_of s) (i + 2)) as [E | E].
1015     + reflexivity.
1016     + assert (Hconst : forall sy0 y0 k, t_bvar sy0 y0 = rho_subst_tel (t_bvar
1017       sy0 y0) N x i k)
1018       by (induction k as [| k' IHk]; auto).
1019     apply Hconst.
1020
1021 - (* M = t_fvar sy y *)
1022     cbn in Hdeg. unfold rho_subst.
1023     destruct (lt_dec (index_of s) (i + 2)) as [E | E].
1024     + rewrite subst_var. destruct (eq_var_dec y x) as [Heq | Hneq].
1025     * exfalso. subst y. simpl in Honly. specialize (Honly eq_refl). rewrite
1026       Honly in Hdeg.
1027     pose proof (index_range s) as [Hsr1 Hsr2]. lia.
1028     * reflexivity.
1029     + rewrite subst_var. destruct (eq_var_dec y x) as [Heq | Hneq].
1030     * subst y. remember (retag x (sort_of (i + 2))) as z eqn:Heqz.
1031     assert (Hz : rho i N = (t_fvar (sort_of (i + 2)) z) { z rho i N }).
1032     { rewrite subst_var. destruct (eq_var_dec z z) as [_ | Hc]; [reflexivity
1033       | exfalso; apply Hc; reflexivity]. }
1034     rewrite Hz. simpl. rewrite <- Heqz.
1035     assert (Hconst : forall k, k <= index_of s - i - 2 ->
1036       rho_subst_tel (t_fvar (sort_of (i + 2)) z) N x i k = (t_fvar (sort_of
1037         (i + 2)) z) { z rho i N }).
1038     { induction k as [| k' IHk]; intros Hk.
1039       - simpl. rewrite <- Heqz. reflexivity.
1040       - simpl.
1041         assert (Hlevel_ne : retag x (sort_of (S (k' + i + 2))) <> z).
1042         { rewrite Heqz. intro Hcontra.
1043           apply retag_inj in Hcontra as [_ Hsort].
1044           assert (Heq2 : S (k' + i + 2) = i + 2)
1045             by (apply (sort_of_inj (S (k' + i + 2)) (i + 2)); [lia | lia |
1046               lia | lia | exact Hsort]).
1047           lia. }
1048         destruct (eq_var_dec z (retag x (sort_of (S (k' + i + 2))))) as [Heq
1049           | Hne].
1050         + exfalso. apply Hlevel_ne. symmetry. exact Heq.
1051         + apply IHk. lia.

```

```

1045     }
1046     symmetry. apply Hconst. lia.
1047 * assert (Hconst : forall k, rho_subst_tel (t_fvar (sort_of (i + 2))
1048 (retag y (sort_of (i + 2)))) N x i k
1049 = t_fvar (sort_of (i + 2)) (retag y (sort_of (i +
1050 2)))).
1051 { induction k as [| k' IHk].
1052 - simpl. destruct (eq_var_dec (retag y (sort_of (i + 2))) (retag x
1053 (sort_of (i + 2)))) as [Heq | Hne].
1054 + exfalso. apply retag_inj in Heq as [Heqxy _]. apply Hneq. exact
1055 Heqxy.
1056 + reflexivity.
1057 - simpl. destruct (eq_var_dec (retag y (sort_of (i + 2))) (retag x
1058 (sort_of (S (k' + i + 2))))) as [Heq | Hne].
1059 + exfalso. apply retag_inj in Heq as [Heqxy _]. apply Hneq. exact
1060 Heqxy.
1061 + exact IHk.
1062 }
1063 symmetry. apply Hconst.
1064
1065 - (* M = t_app D C *)
1066 destruct Honly as [HonlyD HonlyC].
1067 assert (HdegEqC : deg (C { x N }) = deg C)
1068 by (apply (deg_subst_tagged C x s N); auto).
1069 assert (HdegEqD : deg (D { x N }) = deg D)
1070 by (apply (deg_subst_tagged D x s N); auto).
1071
1072 destruct (le_lt_dec (i + 1) (deg C)) as [HdegC | HdegC].
1073
1074 + (* deg C >= i+1 *)
1075 rewrite subst_app. unfold rho_subst in *.
1076 assert (E : i + 1 <= deg (C { x N })) by lia.
1077 rewrite (rho_app_named i (D{x N}) (C{x N}) E).
1078 rewrite (rho_app_named i D C HdegC).
1079 destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1080
1081 ++ (* j < i+2 *)
1082 rewrite HdegEqC. remember (deg C - 1 - i) as k.
1083 assert (Htel : forall k', k' <= k -> rho_app_tel rho (D{x N}) (C{x N})
1084 i k' = rho_app_tel rho D C i k').
1085 { induction k' as [| k'' IHk']; intros Hk'.
1086 - simpl. f_equal.
1087 + apply IHD; auto.
1088 + apply IHC; auto.
1089 - simpl. f_equal.
1090 + apply IHk'. lia.
1091 + assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1092 + assert (HIH : rho (i + S k'') (C{x N}) = rho_subst C N x s (i + S
1093 k''))

```

```

1086         by (apply IHouter; [unfold ltof; lia | lia | exact Hxnz |
            exact Hx | unfold deg in *; lia | exact HonlyC | exact HdegN |
            exact HlcN]).
1087     rewrite HIH. unfold rho_subst.
1088     destruct (lt_dec (index_of s) (i + S k'' + 2)) as [_ | Hcontra];
        [reflexivity | lia]. }
1089     apply Htel; lia.
1090
1091 ++ (* j >= i+2 *)
1092     rewrite HdegEqC.
1093     assert (FD : rho i (D { x N }) = rho_subst_tel (rho i D) N x i
        (index_of s - i - 2))
1094     by (apply IHD; auto).
1095     assert (FC : rho i (C { x N }) = rho_subst_tel (rho i C) N x i
        (index_of s - i - 2))
1096     by (apply IHC; auto).
1097
1098     remember (fun i0 M0 => rho_subst M0 N x s i0) as f.
1099     assert (Step1 : forall k, k <= deg C - 1 - i ->
1100         rho_app_tel rho (D{x N}) (C{x N}) i k = rho_app_tel f D C i
            k).
1101     { induction k as [| k' IHk]; intros Hk.
1102       - simpl. rewrite Heqf. f_equal.
1103       + unfold rho_subst.
1104         destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 | Hge0]; [lia |
            exact FD].
1105       + unfold rho_subst.
1106         destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 | Hge0]; [lia |
            exact FC].
1107       - simpl. rewrite Heqf. f_equal.
1108       + rewrite <- Heqf. apply IHk. lia.
1109       + assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1110         apply IHouter; [unfold ltof; lia | lia | exact Hxnz | exact Hx |
            unfold deg in *; lia | exact HonlyC | exact HdegN | exact HlcN].
1111     }
1112     rewrite Step1; auto.
1113
1114     assert (Step2 : forall k, k <= deg C - 1 - i ->
1115         rho_app_tel f D C i k = rho_subst_tel (rho_app_tel rho D C i k)
            N x i (index_of s - i - 2)).
1116     {
1117       intros k.
1118       rewrite (rho_subst_tel_app_tel_commute D C N x i (index_of s - i -
            2) i k).
1119       induction k as [| k' IHk]; intros Hk.
1120       - simpl. rewrite Heqf. simpl. f_equal.
1121       + unfold rho_subst. destruct (lt_dec (index_of s) (i + 2)) as
            [Hlt0 | Hge0]; [lia | reflexivity].

```

```

1122 + unfold rho_subst. destruct (lt_dec (index_of s) (i + 2)) as
1123 [Hlt0 | Hge0]; [lia | reflexivity].
1124 - simpl. f_equal.
1125 + apply IHk. lia.
1126 + rewrite Heqf. simpl. unfold rho_subst.
1127 destruct (lt_dec (index_of s) (i + S k' + 2)) as [HjA | HjB].
1128 * assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1129 symmetry.
1130 apply (rho_subst_tel_noop_generic (rho (i + S k') C) N x (i +
1131 S k') i (index_of s - i - 2)).
1132 -- lia.
1133 -- intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1134 -- lia.
1135 * assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1136 assert (Hsplit := rho_subst_tel_split (rho (i + S k') C) N x i
1137 (index_of s - i - 2) (i + S k'))
1138 ltac:(lia) ltac:(lia)).
1139 replace (i + (index_of s - i - 2) - (i + S k'))
1140 with (index_of s - (i + S k') - 2) in Hsplit by lia.
1141 rewrite Hsplit.
1142 replace (i + S k' - i - 1) with k' by lia.
1143 symmetry.
1144 apply rho_subst_tel_noop_generic with (D := i + S k').
1145 -- lia.
1146 -- apply rho_subst_tel_stays_clean with (D := i + S k').
1147 ++ lia.
1148 ++ intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1149 -- lia.
1150 }
1151 rewrite Step2; auto.
1152
1153 + (* deg C < i+1 *)
1154 assert (HdegC' : deg (C { x N }) < i + 1) by lia.
1155 simpl.
1156 destruct (le_lt_dec (i + 1) (deg (C { x N }))) as [E | E']. lia.
1157 unfold rho_subst in *.
1158 destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj'].
1159
1160 * (* j < i+2 *)
1161 simpl. destruct (le_lt_dec (i + 1) (deg C)) as [E' | E'']. lia.
1162 apply IHD; auto.
1163
1164 * (* j >= i+2 *)
1165 assert (F : rho i (D { x N }) = rho_subst_tel (rho i D) N x i (index_of
1166 s - i - 2))
1167 by (apply IHD; auto).
1168 rewrite F. f_equal. simpl.
1169 destruct (le_lt_dec (i + 1) (deg C)) as [E' | E'']. lia. auto.

```

```

1167 - (* M = t_lam s1 C D *)
1168 destruct Honly as [HonlyC HonlyD].
1169 assert (HdegEqC : deg (C { x N }) = deg C)
1170 by (apply (deg_subst_tagged C x s N); auto).
1171 destruct (le_lt_dec (i + 2) (deg C)) as [HdegC | HdegC].
1172
1173 + (* deg C >= i+2 *)
1174 rewrite subst_lam. unfold rho_subst in *.
1175 assert (E : i + 2 <= deg (C { x N })) by lia.
1176 assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1177 rewrite (rho_lam_named i s1 (C{xN}) (D{xN}) E).
1178 rewrite (rho_lam_named i s1 C D HdegC).
1179 destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1180
1181 ++ (* j < i+2 *)
1182 rewrite HdegEqC. remember (deg C - 1 - (i + 1)) as k.
1183 assert (Htel : forall k', k' <= k -> rho_lam_tel rho (C{xN}) (D{xN})
1184 i k' = rho_lam_tel rho C D i k').
1185 { induction k' as [| k'' IHk']; intros Hk'.
1186 - simpl. f_equal.
1187 + assert (HIH : rho (i + 1) (C{xN}) = rho_subst C N x s (i + 1))
1188 by (apply IHouter; [unfold lt_of; lia | lia | exact Hxnz |
1189 exact Hx | unfold deg in *; lia | exact HonlyC | exact HdegN |
1190 exact HlcN]).
1191 rewrite HIH. unfold rho_subst.
1192 destruct (lt_dec (index_of s) (i + 1 + 2)) as [_ | Hcontra];
1193 [reflexivity | lia].
1194 + apply IHD; auto.
1195 - simpl. f_equal.
1196 + assert (HIH : rho (i + 1 + S k'') (C{xN}) = rho_subst C N x s (i
1197 + 1 + S k''))
1198 by (apply IHouter; [unfold lt_of; lia | lia | exact Hxnz |
1199 exact Hx | unfold deg in *; lia | exact HonlyC | exact HdegN |
1200 exact HlcN]).
1201 rewrite HIH. unfold rho_subst.
1202 destruct (lt_dec (index_of s) (i + 1 + S k'' + 2)) as [_ |
1203 Hcontra]; [reflexivity | lia].
1204 + apply IHk'. lia. }
1205 apply Htel; lia.
1206
1207 ++ (* j >= i+2 *)
1208 rewrite HdegEqC.
1209 assert (FD : rho i (D { x N }) = rho_subst_tel (rho i D) N x i
1210 (index_of s - i - 2))
1211 by (apply IHD; auto).
1212
1213 remember (fun i0 M0 => rho_subst M0 N x s i0) as f.
1214 assert (Step1 : forall k, k <= deg C - 1 - (i + 1) ->
1215 rho_lam_tel rho (C{xN}) (D{xN}) i k = rho_lam_tel f C D i
1216 k).

```



```

1207 { induction k as [| k' IHk]; intros Hk.
1208 - simpl. rewrite Heqf. f_equal.
1209 + apply IHouter; [unfold lt_of; lia | lia | exact Hxnz | exact Hx |
    unfold deg in *; lia | exact HonlyC | exact HdegN | exact HlcN].
1210 + unfold rho_subst.
1211 destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 | Hge0]; [lia |
    exact FD].
1212 - simpl. rewrite Heqf. f_equal.
1213 + apply IHouter; [unfold lt_of; lia | lia | exact Hxnz | exact Hx |
    unfold deg in *; lia | exact HonlyC | exact HdegN | exact HlcN].
1214 + rewrite <- Heqf. apply IHk. lia.
1215 }
1216 rewrite Step1; auto.
1217
1218 assert (Step2 : forall k, k <= deg C - 1 - (i + 1) ->
1219 rho_lam_tel f C D i k = rho_subst_tel (rho_lam_tel rho C D i k)
    N x i (index_of s - i - 2)).
1220 {
1221 intros k.
1222 rewrite (rho_subst_tel_lam_tel_commute C D N x i (index_of s - i -
    2) i k).
1223 induction k as [| k' IHk]; intros Hk.
1224 - simpl. rewrite Heqf. simpl. f_equal.
1225 + unfold rho_subst.
1226 destruct (lt_dec (index_of s) (i + 1 + 2)) as [HjA | HjB].
1227 * symmetry.
1228 apply (rho_subst_tel_noop_generic (rho (i + 1) C) N x (i + 1)
    i (index_of s - i - 2)).
1229 -- lia.
1230 -- intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1231 -- lia.
1232 * assert (Hsplit := rho_subst_tel_split (rho (i + 1) C) N x i
    (index_of s - i - 2) (i + 1)
    ltac:(lia) ltac:(lia)).
1233 replace (i + (index_of s - i - 2) - (i + 1))
1234 with (index_of s - (i + 1) - 2) in Hsplit by lia.
1235 rewrite Hsplit.
1236 replace (i + 1 - i - 1) with 0 by lia.
1237 symmetry.
1238 apply rho_subst_tel_noop_generic with (D := i + 1).
1239 -- lia.
1240 -- apply rho_subst_tel_stays_clean with (D := i + 1).
1241 ++ lia.
1242 ++ intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1243 -- lia.
1244 + unfold rho_subst. destruct (lt_dec (index_of s) (i + 2)) as
    [Hlt0 | Hge0]; [lia | reflexivity].
1245 - simpl. f_equal.
1246 + rewrite Heqf. simpl. unfold rho_subst.

```

```

1248     destruct (lt_dec (index_of s) (i + 1 + S k' + 2)) as [HjA |
1249     HjB].
1250     * symmetry.
1251     apply (rho_subst_tel_noop_generic (rho (i + 1 + S k') C) N x
1252     (i + 1 + S k') i (index_of s - i - 2)).
1253     -- lia.
1254     -- intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1255     -- lia.
1256     * assert (Hsplit := rho_subst_tel_split (rho (i + 1 + S k') C) N
1257     x i (index_of s - i - 2) (i + 1 + S k'))
1258     ltac:(lia) ltac:(lia)).
1259     replace (i + (index_of s - i - 2) - (i + 1 + S k'))
1260     with (index_of s - (i + 1 + S k') - 2) in Hsplit by lia.
1261     rewrite Hsplit.
1262     replace (i + 1 + S k' - i - 1) with (S k') by lia.
1263     symmetry.
1264     apply rho_subst_tel_noop_generic with (D := i + 1 + S k').
1265     -- lia.
1266     -- apply rho_subst_tel_stays_clean with (D := i + 1 + S k').
1267     ++ lia.
1268     ++ intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1269     -- lia.
1270     + apply IHk. lia.
1271   }
1272   rewrite Step2; auto.
1273
1274 + (* deg C < i+2 *)
1275   assert (HdegC' : deg (C { x N }) < i + 2) by lia.
1276   simpl.
1277   destruct (le_lt_dec (i + 2) (deg (C { x N }))) as [E | E']. lia.
1278   unfold rho_subst in *.
1279   destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj'].
1280
1281   * (* j < i+2 *)
1282     simpl. destruct (le_lt_dec (i + 2) (deg C)) as [E' | E'']. lia.
1283     apply IHD; auto.
1284
1285   * (* j >= i+2 *)
1286     assert (F : rho i (D { x N }) = rho_subst_tel (rho i D) N x i (index_of
1287     s - i - 2))
1288     by (apply IHD; auto).
1289     rewrite F. f_equal. simpl.
1290     destruct (le_lt_dec (i + 2) (deg C)) as [E' | E'']. lia. auto.
1291
1292 - (* M = t_pi s1 C D *)
1293   destruct Honly as [HonlyC HonlyD].
1294   assert (HdegEqC : deg (C { x N }) = deg C)
1295   by (apply (deg_subst_tagged C x s N); auto).
1296   destruct (le_lt_dec (i + 1) (deg C)) as [HdegC | HdegC'].

```

```

1293
1294 + (* deg C >= i+1 *)
1295   rewrite subst_pi. unfold rho_subst in *.
1296   assert (E : i + 1 <= deg (C { x N })) by lia.
1297   rewrite (rho_pi_named i s1 (C{xN}) (D{xN}) E).
1298   rewrite (rho_pi_named i s1 C D HdegC).
1299   destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1300
1301 ++ (* j < i+2 *)
1302   rewrite HdegEqC. remember (deg C - 1 - i) as k.
1303   assert (Htel : forall k', k' <= k -> rho_pi_tel rho (C{xN}) (D{xN}) i
1304     k' = rho_pi_tel rho C D i k').
1305   { induction k' as [| k'' IHk']; intros Hk'.
1306     - simpl. f_equal.
1307       + apply IHC; auto.
1308       + apply IHD; auto.
1309     - simpl. f_equal.
1310       + assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1311         assert (HIH : rho (i + S k'') (C{xN}) = rho_subst C N x s (i + S
1312           k''))
1313           by (apply IHouter; [unfold lt_of; lia | lia | exact Hxnz |
1314             exact Hx | unfold deg in *; lia | exact HonlyC | exact HdegN |
1315             exact HlcN]).
1316         rewrite HIH. unfold rho_subst.
1317         destruct (lt_dec (index_of s) (i + S k'' + 2)) as [_ | Hcontra];
1318         [reflexivity | lia].
1319       + apply IHk'. lia. }
1320   apply Htel; lia.
1321
1322 ++ (* j >= i+2 *)
1323   rewrite HdegEqC.
1324   assert (FD : rho i (D { x N }) = rho_subst_tel (rho i D) N x i
1325     (index_of s - i - 2))
1326   by (apply IHD; auto).
1327   assert (FC : rho i (C { x N }) = rho_subst_tel (rho i C) N x i
1328     (index_of s - i - 2))
1329   by (apply IHC; auto).
1330
1331   remember (fun i0 M0 => rho_subst M0 N x s i0) as f.
1332   assert (Step1 : forall k, k <= deg C - 1 - i ->
1333     rho_pi_tel rho (C{xN}) (D{xN}) i k = rho_pi_tel f C D i k).
1334   { induction k as [| k' IHk]; intros Hk.
1335     - simpl. rewrite Heqf. f_equal.
1336       + unfold rho_subst.
1337         destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 | Hge0]; [lia |
1338           exact FC].
1339       + unfold rho_subst.
1340         destruct (lt_dec (index_of s) (i + 2)) as [Hlt0 | Hge0]; [lia |
1341           exact FD].

```

```

1333   - simpl. rewrite Heqf. f_equal.
1334   + assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1335     apply IHouter; [unfold ltof; lia | lia | exact Hxnz | exact Hx |
1336       unfold deg in *; lia | exact HonlyC | exact HdegN | exact HlcN].
1337   + rewrite <- Heqf. apply IHk. lia.
1338 }
1339 rewrite Step1; auto.
1340
1341 assert (Step2 : forall k, k <= deg C - 1 - i ->
1342   rho_pi_tel f C D i k = rho_subst_tel (rho_pi_tel rho C D i k) N
1343   x i (index_of s - i - 2)).
1344 {
1345   intros k.
1346   rewrite (rho_subst_tel_pi_tel_commute C D N x i (index_of s - i - 2)
1347     i k).
1348   induction k as [| k' IHk]; intros Hk.
1349   - simpl. rewrite Heqf. simpl. f_equal.
1350     + unfold rho_subst. destruct (lt_dec (index_of s) (i + 2)) as
1351       [Hlt0 | Hge0]; [lia | reflexivity].
1352     + unfold rho_subst. destruct (lt_dec (index_of s) (i + 2)) as
1353       [Hlt0 | Hge0]; [lia | reflexivity].
1354   - simpl. f_equal.
1355     + rewrite Heqf. simpl. unfold rho_subst.
1356       destruct (lt_dec (index_of s) (i + S k' + 2)) as [HjA | HjB].
1357       * assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1358         symmetry.
1359         apply (rho_subst_tel_noop_generic (rho (i + S k') C) N x (i +
1360           S k') i (index_of s - i - 2)).
1361         -- lia.
1362         -- intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1363         -- lia.
1364       * assert (HCle : deg C <= n + 1) by (apply deg_le_top).
1365         assert (Hsplit := rho_subst_tel_split (rho (i + S k') C) N x i
1366           (index_of s - i - 2) (i + S k'))
1367           ltac:(lia) ltac:(lia)).
1368         replace (i + (index_of s - i - 2) - (i + S k'))
1369           with (index_of s - (i + S k') - 2) in Hsplit by lia.
1370         rewrite Hsplit.
1371         replace (i + S k' - i - 1) with k' by lia.
1372         symmetry.
1373         apply rho_subst_tel_noop_generic with (D := i + S k').
1374         -- lia.
1375         -- apply rho_subst_tel_stays_clean with (D := i + S k').
1376         ++ lia.
1377         ++ intros m v NO Hm Hn. apply rho_min_tier_noop; lia.
1378         -- lia.
1379     + apply IHk. lia.
1380 }
1381 rewrite Step2; auto.

```

```

1375
1376 + (* deg C < i+1 *)
1377   assert (HdegC' : deg (C { x N }) < i + 1) by lia.
1378   simpl.
1379   destruct (le_lt_dec (i + 1) (deg (C { x N }))) as [E | E]. lia.
1380   unfold rho_subst in *.
1381   destruct (lt_dec (index_of s) (i + 2)) as [Hj | Hj].
1382
1383   * (* j < i+2 *)
1384     simpl. destruct (le_lt_dec (i + 1) (deg C)) as [E' | E']. lia.
1385     apply IHD; auto.
1386
1387   * (* j >= i+2 *)
1388     assert (F : rho i (D { x N }) = rho_subst_tel (rho i D) N x i (index_of
1389       s - i - 2))
1390     by (apply IHD; auto).
1391     rewrite F. f_equal. simpl.
1392     destruct (le_lt_dec (i + 1) (deg C)) as [E' | E']. lia. auto.
1393 Qed.
1394
1395 (* ===== *)
1396 (* rho commutes with beta *)
1397
1398 Lemma brt_pi_A : forall s A A' B, A ->>b A' -> t_pi s A B ->>b t_pi s A' B.
1399 Proof.
1400   intros s A A' B H.
1401   induction H as [A | A A' HA | A A' A'' H1 IH1 H2 IH2].
1402   - apply brt_refl.
1403   - apply brt_step. apply beta_pi_A. exact HA.
1404   - apply brt_trans with (t_pi s A' B); auto.
1405 Qed.
1406
1407 Lemma brt_pi_B : forall s A B B', B ->>b B' -> t_pi s A B ->>b t_pi s A B'.
1408 Proof.
1409   intros s A B B' H.
1410   induction H as [B | B B' HB | B B' B'' H1 IH1 H2 IH2].
1411   - apply brt_refl.
1412   - apply brt_step. apply beta_pi_B. exact HB.
1413   - apply brt_trans with (t_pi s A B'); auto.
1414 Qed.
1415
1416 Lemma brt_pi_congr : forall s A A' B B',
1417   A ->>b A' -> B ->>b B' -> t_pi s A B ->>b t_pi s A' B'.
1418 Proof.
1419   intros s A A' B B' HA HB.
1420   apply brt_trans with (t_pi s A' B).
1421   - apply brt_pi_A. exact HA.
1422   - apply brt_pi_B. exact HB.
1423 Qed.

```

```

1424 Lemma brt_lam_A : forall s A A' M, A ->>b A' -> t_lam s A M ->>b t_lam s A' M.
1425 Proof.
1426   intros s A A' M H.
1427   induction H as [A | A A' HA | A A' A'' H1 IH1 H2 IH2].
1428   - apply brt_refl.
1429   - apply brt_step. apply beta_lam_A. exact HA.
1430   - apply brt_trans with (t_lam s A' M); auto.
1431 Qed.
1432
1433 Lemma brt_lam_M : forall s A M M', M ->>b M' -> t_lam s A M ->>b t_lam s A M'.
1434 Proof.
1435   intros s A M M' H.
1436   induction H as [M | M M' HM | M M' M'' H1 IH1 H2 IH2].
1437   - apply brt_refl.
1438   - apply brt_step. apply beta_lam_M. exact HM.
1439   - apply brt_trans with (t_lam s A M'); auto.
1440 Qed.
1441
1442 Lemma brt_lam_congr : forall s A A' M M',
1443   A ->>b A' -> M ->>b M' -> t_lam s A M ->>b t_lam s A' M'.
1444 Proof.
1445   intros s A A' M M' HA HM.
1446   apply brt_trans with (t_lam s A' M).
1447   - apply brt_lam_A. exact HA.
1448   - apply brt_lam_M. exact HM.
1449 Qed.
1450
1451 Lemma brt_app_l : forall M M' N, M ->>b M' -> t_app M N ->>b t_app M' N.
1452 Proof.
1453   intros M M' N H.
1454   induction H as [M | M M' HM | M M' M'' H1 IH1 H2 IH2].
1455   - apply brt_refl.
1456   - apply brt_step. apply beta_app_l. exact HM.
1457   - apply brt_trans with (t_app M' N); auto.
1458 Qed.
1459
1460 Lemma brt_app_r : forall M N N', N ->>b N' -> t_app M N ->>b t_app M N'.
1461 Proof.
1462   intros M N N' H.
1463   induction H as [N | N N' HN | N N' N'' H1 IH1 H2 IH2].
1464   - apply brt_refl.
1465   - apply brt_step. apply beta_app_r. exact HN.
1466   - apply brt_trans with (t_app M N'); auto.
1467 Qed.
1468
1469 Lemma brt_app_congr : forall M M' N N',
1470   M ->>b M' -> N ->>b N' -> t_app M N ->>b t_app M' N'.
1471 Proof.
1472   intros M M' N N' HM HN.
1473   apply brt_trans with (t_app M' N).

```

```

1474   - apply brt_app_l. exact HM.
1475   - apply brt_app_r. exact HN.
1476 Qed.
1477
1478 Lemma rho_pi_tel_dom_congr : forall C C' D i k,
1479   (forall m, i <= m <= i + k -> rho m C ->>b rho m C') ->
1480   rho_pi_tel rho C D i k ->>b rho_pi_tel rho C' D i k.
1481 Proof.
1482   induction k as [| k' IHk]; intros Hall; simpl.
1483   - apply brt_pi_A. apply Hall. lia.
1484   - apply brt_pi_congr.
1485     + apply Hall. lia.
1486     + apply IHk. intros m Hm. apply Hall. lia.
1487 Qed.
1488
1489 Lemma rho_pi_tel_body_congr : forall A B B' i k,
1490   rho i B ->>b rho i B' ->
1491   rho_pi_tel rho A B i k ->>b rho_pi_tel rho A B' i k.
1492 Proof.
1493   induction k as [| k' IHk]; intros Hb; simpl.
1494   - apply brt_pi_B. exact Hb.
1495   - apply brt_pi_B. apply IHk. exact Hb.
1496 Qed.
1497
1498 Lemma rho_lam_tel_dom_congr : forall C C' M' i k,
1499   (forall m, i + 1 <= m <= i + 1 + k -> rho m C ->>b rho m C') ->
1500   rho_lam_tel rho C M' i k ->>b rho_lam_tel rho C' M' i k.
1501 Proof.
1502   induction k as [| k' IHk]; intros Hall; simpl.
1503   - apply brt_lam_A. apply Hall. lia.
1504   - apply brt_lam_congr.
1505     + apply Hall. lia.
1506     + apply IHk. intros m Hm. apply Hall. lia.
1507 Qed.
1508
1509 Lemma rho_lam_tel_body_congr : forall A M' M'' i k,
1510   rho i M' ->>b rho i M'' ->
1511   rho_lam_tel rho A M' i k ->>b rho_lam_tel rho A M'' i k.
1512 Proof.
1513   induction k as [| k' IHk]; intros Hb; simpl.
1514   - apply brt_lam_M. exact Hb.
1515   - apply brt_lam_M. apply IHk. exact Hb.
1516 Qed.
1517
1518 Lemma rho_app_tel_func_congr : forall P P' Q i k,
1519   rho i P ->>b rho i P' ->
1520   rho_app_tel rho P Q i k ->>b rho_app_tel rho P' Q i k.
1521 Proof.
1522   induction k as [| k' IHk]; intros Hp; simpl.
1523   - apply brt_app_l. exact Hp.

```

```

1524   - apply brt_app_l. apply IHk. exact Hp.
1525 Qed.
1526
1527 Lemma rho_app_tel_arg_congr : forall P Q Q' i k,
1528   (forall m, i <= m <= i + k -> rho m Q ->>b rho m Q') ->
1529   rho_app_tel rho P Q i k ->>b rho_app_tel rho P Q' i k.
1530 Proof.
1531   induction k as [| k' IHk]; intros Hall; simpl.
1532   - apply brt_app_r. apply Hall. lia.
1533   - apply brt_app_congr.
1534     + apply IHk. intros m Hm. apply Hall. lia.
1535     + apply Hall. lia.
1536 Qed.
1537
1538 Axiom rho_commutates_beta_base : forall i, 0 <= i <= n ->
1539   forall s A M Q, deg M >= i + 1 ->
1540   rho i (t_app (t_lam s A M) Q) ->>b rho i (M ^^ Q).
1541
1542 Lemma rho_commutates_beta_aux :
1543   forall i, 0 <= i <= n ->
1544   forall M, deg M >= i + 1 ->
1545   forall N, M ->b N ->
1546   rho i M ->>b rho i N.
1547 Proof.
1548   intros i.
1549   induction i as [i IHouter] using
1550     (well_founded_induction (Wf_nat.well_founded_ltof nat (fun i => n - i))).
1551   intros Hi M.
1552   induction M as [s | s_m m | y | P IHP Q IHQ | s A IHA M' IHM' | s A IHA B
1553     IHB];
1554
1555     intros Hdeg N Hstep.
1556
1557     - (* t_sort *) inversion Hstep.
1558     - (* t_bvar *) inversion Hstep.
1559     - (* t_fvar *) inversion Hstep.
1560
1561     - (* t_app P Q *)
1562       simpl in Hdeg.
1563       inversion Hstep; subst; clear Hstep.
1564
1565     + (* beta_base *)
1566       apply rho_commutates_beta_base; [exact Hi | ].
1567       simpl in Hdeg. exact Hdeg.
1568
1569     + (* beta_app_l *)
1570       match goal with
1571       | Hs : P ->b ?P' |- _ =>
1572         assert (HdegP : deg P >= i + 1) by exact Hdeg;
1573         destruct (le_lt_dec (i + 1) (deg Q)) as [E | E];
1574         [ rewrite (rho_app_named i P Q E); rewrite (rho_app_named i P' Q E);

```



```

1573     apply rho_app_tel_func_congr;
1574     apply IHP; auto
1575 | assert (Hrho1 : rho i (t_app P Q) = rho i P) by (simpl; destruct
    (le_lt_dec (i+1) (deg Q)); [lia | reflexivity]);
1576     assert (Hrho2 : rho i (t_app P' Q) = rho i P') by (simpl; destruct
    (le_lt_dec (i+1) (deg Q)); [lia | reflexivity]);
1577     rewrite Hrho1, Hrho2;
1578     apply IHP; auto ]
1579 end.
1580
1581 + (* beta_app_r *)
1582 match goal with
1583 | Hs : Q ->b ?Q' |- _ =>
1584     assert (HdegQeq : deg Q = deg Q') by (apply deg_beta; apply brt_step;
    exact Hs);
1585     destruct (le_lt_dec (i + 1) (deg Q)) as [E | E];
1586     [ assert (E' : i + 1 <= deg Q') by lia;
    1587       rewrite (rho_app_named i P Q E); rewrite (rho_app_named i P Q' E');
    1588       assert (Hk : deg Q - 1 - i = deg Q' - 1 - i) by lia;
    1589       rewrite Hk;
    1590       apply rho_app_tel_arg_congr;
    1591       intros mtier Hmtier;
    1592       assert (HQle : deg Q <= n + 1) by (apply deg_le_top);
    1593       assert (HdegQm : deg Q >= mtier + 1) by lia;
    1594       destruct (Nat.eq_dec mtier i) as [Heqm | Hneqm];
    1595       [ subst mtier; apply IHQ; [unfold deg in *; lia | exact Hs]
    1596         | apply IHouter; [unfold ltof; lia | lia | unfold deg in *; lia |
    exact Hs] ]
    1597 | assert (E' : ~ i + 1 <= deg Q') by lia;
    1598     assert (Hrho1 : rho i (t_app P Q) = rho i P) by (simpl; destruct
    (le_lt_dec (i+1) (deg Q)); [lia | reflexivity]);
    1599     assert (Hrho2 : rho i (t_app P Q') = rho i P) by (simpl; destruct
    (le_lt_dec (i+1) (deg Q')); [lia | reflexivity]);
    1600     rewrite Hrho1, Hrho2; apply brt_refl ]
    1601 end.
    1602
    1603 - (* t_lam s A M' *)
    1604 inversion Hstep; subst; clear Hstep.
    1605
    1606 + (* beta_lam_A *)
    1607 match goal with
    1608 | Hs : A ->b ?A1' |- _ =>
    1609     assert (HdegAA' : deg A = deg A1') by (apply deg_beta; apply brt_step;
    exact Hs);
    1610     destruct (le_lt_dec (i + 2) (deg A)) as [E | E];
    1611     [ assert (E' : i + 2 <= deg A1') by lia;
    1612       rewrite (rho_lam_named i s A M' E); rewrite (rho_lam_named i s A1' M'
    E');
    1613       assert (Hk : deg A - 1 - (i + 1) = deg A1' - 1 - (i + 1)) by lia;

```

```

1614     rewrite Hk;
1615     apply rho_lam_tel_dom_congr;
1616     intros mtier Hmtier;
1617     assert (Hale : deg A <= n + 1) by (apply deg_le_top);
1618     assert (HdegAm : deg A >= mtier + 1) by lia;
1619     apply IHouter; [unfold lt_of; lia | lia | unfold deg in *; lia | exact
1620       Hs]
1621   | assert (E' : ~ i + 2 <= deg A1') by lia;
1622     assert (Hrho1 : rho i (t_lam s A M') = rho i M') by (simpl; destruct
1623       (le_lt_dec (i+2) (deg A))); [lia | reflexivity]];
1624     assert (Hrho2 : rho i (t_lam s A1' M') = rho i M') by (simpl; destruct
1625       (le_lt_dec (i+2) (deg A1'))); [lia | reflexivity]];
1626     rewrite Hrho1, Hrho2; apply brt_refl ]
1627 end.
1628
1629 + (* beta_lam_M *)
1630 match goal with
1631 | Hs : M' -> b ?M1' |- _ =>
1632   assert (HdegM' : deg M' >= i + 1) by exact Hdeg;
1633   destruct (le_lt_dec (i + 2) (deg A)) as [E | E];
1634   [ rewrite (rho_lam_named i s A M' E); rewrite (rho_lam_named i s A M1'
1635     E);
1636     apply rho_lam_tel_body_congr;
1637     apply IHM'; auto
1638   | assert (Hrho1 : rho i (t_lam s A M') = rho i M') by (simpl; destruct
1639     (le_lt_dec (i+2) (deg A))); [lia | reflexivity]];
1640     assert (Hrho2 : rho i (t_lam s A M1') = rho i M1') by (simpl; destruct
1641     (le_lt_dec (i+2) (deg A))); [lia | reflexivity]];
1642     rewrite Hrho1, Hrho2;
1643     apply IHM'; auto ]
1644 end.
1645
1646 - (* t_pi s A B *)
1647 inversion Hstep; subst; clear Hstep.
1648
1649 + (* beta_pi_A *)
1650 match goal with
1651 | Hs : A -> b ?A1' |- _ =>
1652   assert (HdegAA' : deg A = deg A1') by (apply deg_beta; apply brt_step;
1653     exact Hs);
1654   destruct (le_lt_dec (i + 1) (deg A)) as [E | E];
1655   [ assert (E' : i + 1 <= deg A1') by lia;
1656     rewrite (rho_pi_named i s A B E); rewrite (rho_pi_named i s A1' B E');
1657     assert (Hk : deg A - 1 - i = deg A1' - 1 - i) by lia;
1658     rewrite Hk;
1659     apply rho_pi_tel_dom_congr;
1660     intros mtier Hmtier;
1661     assert (Hale : deg A <= n + 1) by (apply deg_le_top);
1662     assert (HdegAm : deg A >= mtier + 1) by lia;

```

```

1656     destruct (Nat.eq_dec mtier i) as [Heqm | Hneqm];
1657     [ subst mtier; apply IHA; [unfold deg in *; lia | exact Hs]
1658     | apply IHouter; [unfold ltof; lia | lia | unfold deg in *; lia |
      exact Hs] ]
1659   | assert (E' : ~ i + 1 <= deg A1') by lia;
1660     assert (Hrho1 : rho i (t_pi s A B) = rho i B) by (simpl; destruct
      (le_lt_dec (i+1) (deg A)); [lia | reflexivity]);
1661     assert (Hrho2 : rho i (t_pi s A1' B) = rho i B) by (simpl; destruct
      (le_lt_dec (i+1) (deg A1'))); [lia | reflexivity];
1662     rewrite Hrho1, Hrho2; apply brt_refl ]
1663   end.
1664
1665 + (* beta_pi_B *)
1666   match goal with
1667   | Hs : B -> b ?B1' |- _ =>
1668     assert (HdegB : deg B >= i + 1) by exact Hdeg;
1669     destruct (le_lt_dec (i + 1) (deg A)) as [E | E];
1670     [ rewrite (rho_pi_named i s A B E); rewrite (rho_pi_named i s A B1' E);
1671       apply rho_pi_tel_body_congr;
1672       apply IHB; auto
1673     | assert (Hrho1 : rho i (t_pi s A B) = rho i B) by (simpl; destruct
      (le_lt_dec (i+1) (deg A)); [lia | reflexivity]);
1674       assert (Hrho2 : rho i (t_pi s A B1') = rho i B1') by (simpl; destruct
      (le_lt_dec (i+1) (deg A)); [lia | reflexivity]);
1675       rewrite Hrho1, Hrho2;
1676       apply IHB; auto ]
1677   end.
1678 Qed.
1679
1680 Lemma rho_commutes_beta :
1681   forall i, 0 <= i <= n ->
1682   forall M, deg M >= i + 1 ->
1683   forall N, M ->>b N ->
1684   rho i M ->>b rho i N.
1685 Proof.
1686   intros i Hi M Hdeg N Hbeta.
1687   induction Hbeta as [M | M N Hstep | M N P Hstep1 IH1 Hstep2 IH2].
1688   - apply brt_refl.
1689   - apply (rho_commutes_beta_aux i Hi M Hdeg N Hstep).
1690   - apply brt_trans with (rho i N); auto.
1691     apply IH2; auto.
1692     rewrite <- (deg_beta M N Hstep1); auto.
1693 Qed.
1694
1695 (* ===== *)
1696 (* rho commutes with typing *)
1697
1698 Axiom n_at_least_two : n >= 2.
1699

```

```

1700 Axiom typing_star_weakening_incl : forall  $\Gamma$   $\Delta$  M A,
1701    $\Gamma \Delta \rightarrow \Gamma * M \ A \rightarrow \Delta * M \ A$ .
1702
1703 Lemma subcontext_app_same : forall  $\Gamma_1$   $\Gamma_2$   $\Delta$ ,
1704    $\Gamma_1 \Delta \rightarrow \Gamma_2 \Delta \rightarrow \Gamma_1 ++ \Gamma_2 \Delta$ .
1705 Proof.
1706   intros  $\Gamma_1$   $\Gamma_2$   $\Delta$  H1 H2 x T Hin.
1707   apply in_app_or in Hin. destruct Hin; [apply H1 | apply H2]; auto.
1708 Qed.
1709
1710 Lemma rho_ctx_tel_idx : forall y s T k v A,
1711   In (v, A) (rho_ctx_tel y s T k)  $\rightarrow$  exists s', v = retag y s'.
1712 Proof.
1713   intros y s T k.
1714   induction k as [| k' IHk]; intros v A Hin; simpl in Hin.
1715   - destruct Hin as [Heq | []]. injection Heq as Hv HA.
1716     exists (sort_of (index_of s)). symmetry. exact Hv.
1717   - apply in_app_or in Hin. destruct Hin as [Hin | Hin].
1718     + eapply IHk; eauto.
1719     + destruct Hin as [Heq | []]. injection Heq as Hv HA.
1720       exists (sort_of (index_of s - S k')). symmetry. exact Hv.
1721 Qed.
1722
1723 Lemma rho_ctx_tel_last : forall y s T k,
1724   In (retag y (sort_of (index_of s - k)),
1725     (sort_of (index_of s - k), rho (index_of s - k - 1) T))
1726     (rho_ctx_tel y s T k).
1727 Proof.
1728   intros y s T k.
1729   destruct k as [| k'].
1730   - simpl. left. replace (index_of s - 0) with (index_of s) by lia.
1731     replace (index_of s - 0 - 1) with (index_of s - 1) by lia. reflexivity.
1732   - simpl. apply in_or_app. right. left. reflexivity.
1733 Qed.
1734
1735 Lemma rho_ctx_incl_old : forall  $\Gamma_0$  i x s T,
1736   rho_ctx i  $\Gamma_0$  rho_ctx i ( $\Gamma_0 ++ [(x, (s, T))]$ ).
1737 Proof.
1738   induction  $\Gamma_0$  as [| [y [sy Ty]]  $\Gamma_0'$  IH]; intros i x s T p q Hp q.
1739   - simpl in Hp q. destruct Hp q as [Hp q | []]. inversion Hp q; subst.
1740     simpl. apply in_or_app. right. left. reflexivity.
1741   - simpl in Hp q |- *. apply in_app_or in Hp q. apply in_or_app.
1742     destruct Hp q as [Hp q | Hp q].
1743     + left. exact Hp q.
1744     + right. apply (IH i x s T). exact Hp q.
1745 Qed.
1746
1747 Lemma rho_ctx_incl_new : forall  $\Gamma_0$  i x s T,
1748   rho_ctx_tel x s T (index_of s - i - 1) rho_ctx i ( $\Gamma_0 ++ [(x, (s, T))]$ ).
1749 Proof.

```

```

1750 induction  $\Gamma_0$  as [| [y [sy Ty]]  $\Gamma_0'$  IH]; intros i x s T p q Hpq.
1751 - simpl. apply in_or_app. left. exact Hpq.
1752 - simpl. apply in_or_app. right. apply (IH i x s T). exact Hpq.
1753 Qed.
1754
1755 Lemma rho_ctx_fresh : forall  $\Gamma_0$  i x s,
1756   x <> zero_var -> is_fresh x  $\Gamma_0$  -> is_fresh (retag x s) (rho_ctx i  $\Gamma_0$ ).
1757 Proof.
1758   induction  $\Gamma_0$  as [| [y [sy Ty]]  $\Gamma_0'$  IH]; intros i x s Hxnz Hfresh.
1759   - simpl. unfold is_fresh, dom. simpl. intro Hin.
1760     destruct Hin as [Heq | []]. apply (zero_var_retag_ne x s). symmetry. exact
1761       Heq.
1762   - assert (Hfresh' : is_fresh x  $\Gamma_0'$ ) by (intros Hin; apply Hfresh; simpl;
1763     right; exact Hin).
1764     assert (Hxy : x <> y)
1765       by (intro Heq; apply Hfresh; simpl; left; symmetry; exact Heq).
1766     unfold is_fresh, dom in *. simpl.
1767     intro Hin. apply in_map_iff in Hin. destruct Hin as [[v A] [Hveq Hin']].
1768     simpl in Hveq. subst v.
1769     apply in_app_or in Hin'. destruct Hin' as [Hin' | Hin'].
1770     + pose proof (rho_ctx_tel_idx y sy Ty _ _ _ Hin') as [s' Hs'].
1771     exact (retag_neq_of_atom_neq x s y s' Hxy Hs').
1772     + apply (IH i x s Hxnz Hfresh').
1773     apply in_map_iff. exists (retag x s, A). split; [reflexivity | exact
1774       Hin'].
1775   Qed.
1776
1777 Lemma subcontext_trans : forall  $\Gamma_1$   $\Gamma_2$   $\Gamma_3$ ,  $\Gamma_1 \rightarrow \Gamma_2 \rightarrow \Gamma_2 \rightarrow \Gamma_3 \rightarrow \Gamma_1 \rightarrow \Gamma_3$ .
1778 Proof. intros  $\Gamma_1$   $\Gamma_2$   $\Gamma_3$  H12 H23 x T Hin. apply H23, H12, Hin. Qed.
1779
1780 Lemma rho_ctx_tel_mono : forall y s T k1 k2, k1 <= k2 ->
1781   rho_ctx_tel y s T k1 rho_ctx_tel y s T k2.
1782 Proof.
1783   intros y s T k1 k2 Hle.
1784   induction k2 as [| k2' IH].
1785   - assert (k1 = 0) by lia. subst. intros p q Hpq. exact Hpq.
1786   - destruct (Nat.eq_dec k1 (S k2')) as [Heq | Hneq].
1787     + subst. intros p q Hpq. exact Hpq.
1788     + assert (Hle' : k1 <= k2') by lia.
1789       intros p q Hpq. simpl. apply in_or_app. left. apply (IH Hle'). exact Hpq.
1790   Qed.
1791
1792 Lemma app_subset_app : forall ( $\Gamma_1$   $\Gamma_2$   $\Delta_1$   $\Delta_2$  : context),
1793    $\Gamma_1 \rightarrow \Delta_1 \rightarrow \Gamma_2 \rightarrow \Delta_2 \rightarrow \Gamma_1 ++ \Gamma_2 \rightarrow \Delta_1 ++ \Delta_2$ .
1794 Proof.
1795   intros  $\Gamma_1$   $\Gamma_2$   $\Delta_1$   $\Delta_2$  H1 H2 p q Hpq.
1796   apply in_app_or in Hpq. apply in_or_app.
1797   destruct Hpq as [Hpq | Hpq]; [left; apply H1 | right; apply H2]; exact Hpq.
1798   Qed.

```

```

1795
1796 Lemma rho_ctx_mono : forall  $\Gamma$  i i', i <= i' -> rho_ctx i'  $\Gamma$  rho_ctx i  $\Gamma$ .
1797 Proof.
1798   induction  $\Gamma$  as [| [y [sy Ty]]  $\Gamma'$  IH]; intros i i' Hle.
1799   - simpl. intros p q Hpq. exact Hpq.
1800   - simpl. apply app_subset_app.
1801     + apply rho_ctx_tel_mono. lia.
1802     + apply (IH i i' Hle).
1803 Qed.
1804
1805 Axiom rho_pi_domain_erased_below :
1806   forall  $\Gamma_0$  x A0 B0 s1 T i,
1807     is_fresh x  $\Gamma_0$  ->
1808      $\Gamma_0$  A0 t_sort s1 ->
1809     index_of s1 < i + 1 ->
1810     rho_ctx (i + 1) ( $\Gamma_0$  ++ [(x, (s1, A0))]) * rho i (open_var B0 s1 x) T ->
1811     rho_ctx (i + 1)  $\Gamma_0$  * rho i B0 T.
1812
1813 Axiom rho_pi_tower :
1814   forall  $\Gamma_0$  A0 B0 s1 i L,
1815      $\Gamma_0$  A0 t_sort s1 ->
1816     index_of s1 >= i + 1 ->
1817     (forall x, ~ In x L ->
1818       rho_ctx (i + 1) ( $\Gamma_0$  ++ [(x, (s1, A0))]) * rho i (open_var B0 s1 x)
1819         t_sort (sort_of (i + 1))) ->
1820     rho_ctx (i + 1)  $\Gamma_0$  * rho_pi_tel rho A0 B0 i (deg A0 - 1 - i) t_sort
1821       (sort_of (i + 1)).
1822
1823 Axiom rho_lam_domain_erased_below :
1824   forall  $\Gamma_0$  x A0 M0 B0 s1 i,
1825     is_fresh x  $\Gamma_0$  ->
1826      $\Gamma_0$  A0 t_sort s1 ->
1827     index_of s1 < i + 2 ->
1828     rho_ctx (i + 1) ( $\Gamma_0$  ++ [(x, (s1, A0))]) * rho i (open_var M0 s1 x) rho (i
1829       + 1) (open_var B0 s1 x) ->
1830     rho_ctx (i + 1)  $\Gamma_0$  * rho i M0 rho (i + 1) B0.
1831
1832 Axiom rho_lam_tower :
1833   forall  $\Gamma_0$  A0 M0 B0 s1 i L,
1834      $\Gamma_0$  A0 t_sort s1 ->
1835     index_of s1 >= i + 2 ->
1836     (forall x, ~ In x L ->
1837       rho_ctx (i + 1) ( $\Gamma_0$  ++ [(x, (s1, A0))]) * rho i (open_var M0 s1 x) rho
1838         (i + 1) (open_var B0 s1 x)) ->
1839     rho_ctx (i + 1)  $\Gamma_0$  * rho_lam_tel rho A0 M0 i (deg A0 - 1 - (i + 1))
1840       rho_pi_tel rho A0 B0 (i + 1) (deg A0 - 1 - (i + 1)).
1841
1842 Axiom rho_app_tower :
1843   forall  $\Gamma_0$  M0 N0 A0 B0 s1a i,

```

```

1840   Γ0  M0   t_pi s1a A0 B0 ->
1841   Γ0  N0   A0 ->
1842   deg N0 >= i + 1 ->
1843   rho_ctx (i + 1) Γ0 * rho i M0   rho (i + 1) (t_pi s1a A0 B0) ->
1844   rho_ctx (i + 1) Γ0 * rho i (t_app M0 N0)   rho (i + 1) (B0 ^^ N0).
1845
1846 Axiom rho_erased_subst_below : forall B N i,
1847   deg N < i + 1 -> rho (i + 1) (B ^^ N) = rho (i + 1) B.
1848
1849 Lemma deg_beq : forall M N, M =b N -> deg M = deg N.
1850 Proof.
1851   intros M N Heq.
1852   induction Heq as [M | M N Hstep | M N Heq IH | M N P Heq1 IH1 Heq2 IH2].
1853   - reflexivity.
1854   - apply deg_beta. apply brt_step. exact Hstep.
1855   - symmetry. exact IH.
1856   - rewrite IH1. exact IH2.
1857 Qed.
1858
1859 Lemma rho_commutes_beta_eq : forall i, 0 <= i <= n ->
1860   forall M N, deg M >= i + 1 -> M =b N -> rho i M =b rho i N.
1861 Proof.
1862   intros i Hi M N Hdeg Heq.
1863   revert Hdeg.
1864   induction Heq as [M | M N Hstep | M N Heq IH | M N P Heq1 IH1 Heq2 IH2];
1865   intros Hdeg.
1866   - apply beq_refl.
1867   - apply beq_from_rtrans. apply (rho_commutes_beta i Hi M Hdeg N (brt_step M N
1868     Hstep)).
1869   - assert (HdegM : deg M >= i + 1) by (rewrite (deg_beq M N Heq); exact Hdeg).
1870     apply beq_sym. apply (IH HdegM).
1871   - assert (HdegN : deg N >= i + 1) by (rewrite <- (deg_beq M N Heq1); exact
1872     Hdeg).
1873     apply beq_trans with (rho i N).
1874     + apply (IH1 Hdeg).
1875     + apply (IH2 HdegN).
1876 Qed.
1877
1878 Lemma rho_commutes_typing :
1879   forall i, 0 <= i <= n ->
1880   forall Γ M N, Γ ⊢ M ⊢ N ->
1881   deg M >= i + 1 ->
1882   rho_ctx (i + 1) Γ * (rho i M)   (rho (i + 1) N).
1883 Proof.
1884   intros i.
1885   induction i as [i IHouter] using
1886     (well_founded_induction (Wf_nat.well_founded_ltof nat (fun i => n - i))).
1887   intros Hi Γ M N Htype.
1888   induction Htype as
1889     [ s s' HA

```

```

1887 |  $\Gamma_0$  x A0 s0 Hfresh HA IHA
1888 |  $\Gamma_0$  x B0 M0 A0 s0 Hfresh HM IHM HB IHB
1889 |  $\Gamma_0$  A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
1890 |  $\Gamma_0$  A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
1891 |  $\Gamma_0$  M0 N0 A0 B0 s1a HM IHM HN IHN
1892 |  $\Gamma_0$  M0 A0 B0 s HM IHM Heq HB IHB ];
1893 intros Hdeg.
1894
1895 - (* typing_axiom *)
1896 simpl in Hdeg.
1897 apply A_spec in HA as [Hjn Hjs'].
1898 pose proof n_at_least_two as Hn2.
1899 assert (HzeroTyped : [] * t_sort (sort_of 1) t_sort (sort_of 2)).
1900 { apply typing_star_axiom. apply A_spec.
1901   rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
1902   rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
1903   lia. }
1904 destruct i as [| i'].
1905 + (* i = 0 *)
1906   simpl.
1907   apply typing_star_var with ( $\Gamma$  := []).
1908   * unfold is_fresh, dom. simpl. auto.
1909   * exact HzeroTyped.
1910 + (* i = S i' *)
1911   assert (HiSn : S i' < n) by lia.
1912   replace (S i' + 1) with (S (S i')) by lia.
1913   simpl.
1914   apply typing_star_weak with ( $\Gamma$  := []) (x := zero_var) (B := t_sort
1915     (sort_of 1)).
1916   * unfold is_fresh, dom. simpl. auto.
1917   * apply typing_star_axiom. apply A_spec.
1918     rewrite (sort_of_correct (S i') ltac:(lia) ltac:(lia)).
1919     rewrite (sort_of_correct (S (S i')) ltac:(lia) ltac:(lia)).
1920     lia.
1921   * exact HzeroTyped.
1922
1923 - (* typing_var *)
1924 simpl in Hdeg.
1925 pose proof (index_range s0) as [Hxlo Hxhi].
1926 assert (HdegA0 : deg A0 = index_of s0).
1927 { pose proof (deg_typing_succ  $\Gamma_0$  A0 (t_sort s0) HA) as Hds.
1928   unfold deg in *. simpl in Hds. lia. }
1929 assert (Hj2 : index_of s0 >= i + 2) by lia.
1930 assert (Hlt : ltof nat (fun k => n - k) (i + 1) i) by (unfold ltof; lia).
1931 assert (Hij : 0 <= i + 1 <= n) by (pose proof (index_range s0); lia).
1932 assert (HdegA0' : deg A0 >= (i + 1) + 1) by (lia).
1933 pose proof (IHouter (i + 1) Hlt Hij  $\Gamma_0$  A0 (t_sort s0) HA HdegA0') as HIH.
1934 replace (i + 1 + 1) with (S (S i)) in HIH by lia.
1935 simpl in HIH.
1936 replace (S (S i)) with (i + 2) in HIH by lia.

```



```

1936 pose (x' := retag x (sort_of (i + 2))).
1937 assert (Hxnz : x <> zero_var)
1938   by (apply (typing_avoids_zero_var (Γ0 ++ [(x, (s0, A0))]) (t_fvar s0 x) A0
1939         (typing_var Γ0 x A0 s0 Hfresh HA) x s0 A0);
1940       apply in_or_app; right; left; reflexivity).
1941 assert (Hfreshx' : is_fresh x' (rho_ctx (i + 2) Γ0))
1942   by (apply rho_ctx_fresh; [exact Hxnz | exact Hfresh]).
1943 assert (Hstep : rho_ctx (i + 2) Γ0 ++ [(x', (sort_of (i + 2), rho (i + 1)
1944   A0))]
1945         * t_fvar (sort_of (i + 2)) x' rho (i + 1) A0)
1946   by (apply typing_star_var; [exact Hfreshx' | exact HIH]).
1947 assert (Hmem : In (x', (sort_of (i + 2), rho (i + 1) A0))
1948         (rho_ctx_tel x s0 A0 (index_of s0 - (i + 1) - 1))).
1949 { replace (index_of s0 - (i + 1) - 1)
1950   with (index_of s0 - i - 2) by lia.
1951   pose proof (rho_ctx_tel_last x s0 A0 (index_of s0 - i - 2)) as Htel.
1952   replace (index_of s0 - (index_of s0 - i - 2) - 1)
1953     with (i + 1) in Htel by lia.
1954   replace (index_of s0 - (index_of s0 - i - 2))
1955     with (i + 2) in Htel by lia.
1956   unfold x'. exact Htel.
1957 }
1958 assert (Hincl : rho_ctx (i + 2) Γ0 ++ [(x', (sort_of (i + 2), rho (i + 1)
1959   A0))]
1960         rho_ctx (i + 1) (Γ0 ++ [(x, (s0, A0))])).
1961 { apply subcontext_app_same.
1962   - apply (subcontext_trans _ (rho_ctx (i + 1) Γ0)).
1963     + apply rho_ctx_mono. lia.
1964     + apply rho_ctx_incl_old.
1965   - intros p q Hpq. destruct Hpq as [Heq | []]. inversion Heq; subst.
1966     apply (rho_ctx_incl_new Γ0 (i + 1) x s0 A0). exact Hmem.
1967 }
1968 simpl. unfold x' in Hstep, Hinc1.
1969 apply (typing_star_weakening_incl _ _ _ _ Hinc1 Hstep).
1970
1971 - (* typing_weak *)
1972   exact (typing_star_weakening_incl _ _ _ _ (rho_ctx_incl_old Γ0 (i + 1) x s0
1973     B0)
1974         (IHM Hdeg)).
1975
1976 - (* typing_pi *)
1977   simpl in Hdeg.
1978   assert (Hs32 : s3 = s2) by (symmetry; apply (R_shape s1 s2 s3 HR)).
1979   subst s3.
1980   assert (HdegA0 : deg A0 = index_of s1) by (apply (deg_sort_typed Γ0 A0 s1
1981     HA)).
1982   set (x0 := fresh (dom Γ0 ++ L)).
1983   assert (Hx0dom : is_fresh x0 Γ0).
1984   { intro Hc. apply (fresh_notin (dom Γ0 ++ L)).

```

```

1981   apply in_or_app. left. exact Hc. }
1982 assert (HxOL : ~ In x0 L).
1983 { intro Hc. apply (fresh_notin (dom  $\Gamma_0$  ++ L)).
1984   apply in_or_app. right. exact Hc. }
1985 assert (HBx0 :  $\Gamma_0$  ++ [(x0, (s1, A0))] open_var B0 s1 x0 t_sort s2) by
  (apply HB; exact HxOL).
1986 assert (Hdegopen : deg (open_var B0 s1 x0) >= i + 1)
1987   by (rewrite <- (deg_open B0 x0 s1 HTagB); exact Hdeg).
1988 assert (Hcast : rho (i + 1) (t_sort s2) = t_sort (sort_of (i + 1))).
1989 { replace (i + 1) with (S i) by lia. reflexivity. }
1990 destruct (le_lt_dec (i + 1) (deg A0)) as [HJge | HJlt].
1991
1992 + (* j >= i+1 *)
1993   rewrite (rho_pi_named i s1 A0 B0 HJge).
1994   rewrite Hcast.
1995   apply (rho_pi_tower  $\Gamma_0$  A0 B0 s1 i L HA).
1996   * lia.
1997   * intros x Hx.
1998   assert (HBx :  $\Gamma_0$  ++ [(x, (s1, A0))] open_var B0 s1 x t_sort s2) by
    (apply HB; exact Hx).
1999   assert (Hd : deg (open_var B0 s1 x) >= i + 1)
2000     by (rewrite <- (deg_open B0 x s1 HTagB); exact Hdeg).
2001   pose proof (IHB x Hx Hd) as HIH.
2002   rewrite Hcast in HIH. exact HIH.
2003
2004 + (* j < i+1 *)
2005   assert (Hcollapse : rho i (t_pi s1 A0 B0) = rho i B0)
2006     by (simpl; destruct (le_lt_dec (i + 1) (deg A0)); [lia | reflexivity]).
2007   rewrite Hcollapse, Hcast.
2008   assert (Hjlt' : index_of s1 < i + 1) by lia.
2009   apply (rho_pi_domain_erased_below  $\Gamma_0$  x0 A0 B0 s1 (t_sort (sort_of (i +
    1))) i
2010     HxOdom HA Hjlt').
2011   pose proof (IHB x0 HxOL Hdegopen) as HIH.
2012   rewrite Hcast in HIH. exact HIH.
2013
2014 - (* typing_lam *)
2015   simpl in Hdeg.
2016   assert (HdegA0 : deg A0 = index_of s1) by (apply (deg_sort_typed  $\Gamma_0$  A0 s1
    HA)).
2017   set (x0 := fresh (dom  $\Gamma_0$  ++ L)).
2018   assert (HxOdom : is_fresh x0  $\Gamma_0$ ).
2019   { intro Hc. apply (fresh_notin (dom  $\Gamma_0$  ++ L)).
2020     apply in_or_app. left. exact Hc. }
2021   assert (HxOL : ~ In x0 L).
2022   { intro Hc. apply (fresh_notin (dom  $\Gamma_0$  ++ L)).
2023     apply in_or_app. right. exact Hc. }
2024   assert (HMx0 :  $\Gamma_0$  ++ [(x0, (s1, A0))] open_var M0 s1 x0 open_var B0 s1
    x0) by (apply HM; exact HxOL).

```

```

2025  assert (Hdegopen : deg (open_var M0 s1 x0) >= i + 1)
2026    by (rewrite <- (deg_open M0 x0 s1 HTagM); exact Hdeg).
2027  destruct (le_lt_dec (i + 2) (deg A0)) as [HJge | HJlt].
2028
2029  + (* deg C >= i+2 *)
2030    assert (Hlam : rho i (t_lam s1 A0 M0) = rho_lam_tel rho A0 M0 i (deg A0 -
2031      1 - (i + 1)))
2032      by (apply (rho_lam_named i s1 A0 M0 HJge)).
2033    assert (HJge' : i + 1 + 1 <= deg A0) by lia.
2034    assert (Hpi : rho (i + 1) (t_pi s1 A0 B0) = rho_pi_tel rho A0 B0 (i + 1)
2035      (deg A0 - 1 - (i + 1)))
2036      by (apply (rho_pi_named (i + 1) s1 A0 B0 HJge')).
2037    rewrite Hlam, Hpi.
2038    apply (rho_lam_tower Γ0 A0 M0 B0 s1 i L HA).
2039    * lia.
2040    * intros x Hx.
2041    assert (HMx : Γ0 ++ [(x, (s1, A0))] open_var M0 s1 x open_var B0 s1
2042      x) by (apply HM; exact Hx).
2043    assert (Hd : deg (open_var M0 s1 x) >= i + 1)
2044      by (rewrite <- (deg_open M0 x s1 HTagM); exact Hdeg).
2045    apply (IHM x Hx Hd).
2046
2047  + (* deg C < i+2 *)
2048    assert (Hcollapse1 : rho i (t_lam s1 A0 M0) = rho i M0)
2049      by (simpl; destruct (le_lt_dec (i + 2) (deg A0)); [lia | reflexivity]).
2050    assert (Hcollapse2 : rho (i + 1) (t_pi s1 A0 B0) = rho (i + 1) B0)
2051      by (simpl; destruct (le_lt_dec (i + 1 + 1) (deg A0)); [lia |
2052        reflexivity]).
2053    rewrite Hcollapse1, Hcollapse2.
2054    assert (HJlt' : index_of s1 < i + 2) by lia.
2055    apply (rho_lam_domain_erased_below Γ0 x0 A0 M0 B0 s1 i HxOdom HA HJlt').
2056    apply (IHM x0 HxOL Hdegopen).
2057
2058  - (* typing_app *)
2059    simpl in Hdeg.
2060    assert (HdegC : deg A0 = deg N0 + 1)
2061      by (pose proof (deg_typing_succ Γ0 N0 A0 HN) as Hh; lia).
2062    destruct (le_lt_dec (i + 1) (deg N0)) as [HNge | HNlt].
2063
2064  + (* deg N >= i+1 *)
2065    apply (rho_app_tower Γ0 M0 N0 A0 B0 s1a i HM HN HNge).
2066    exact (IHM Hdeg).
2067
2068  + (* deg N < i+1 *)
2069    assert (Hcollapse_app : rho i (t_app M0 N0) = rho i M0)
2070      by (simpl; destruct (le_lt_dec (i + 1) (deg N0)); [lia | reflexivity]).
2071    rewrite Hcollapse_app.
2072    assert (Hcol2 : rho (i + 1) (t_pi s1a A0 B0) = rho (i + 1) B0)
2073      by (simpl; destruct (le_lt_dec (i + 1 + 1) (deg A0)); [lia |
2074        reflexivity]).

```

```

2070     assert (Htgt : rho (i + 1) (B0 ^^ N0) = rho (i + 1) (t_pi s1a A0 B0)).
2071     { rewrite Hcol2. apply (rho_erased_subst_below B0 N0 i HNlt). }
2072     rewrite Htgt.
2073     exact (IHM Hdeg).
2074
2075 - (* typing_conv *)
2076     simpl in Hdeg.
2077     destruct (typing_lc _ _ _ HM) as [HlcM _].
2078     assert (HdegM0 : deg M0 >= i + 1) by (lia).
2079     assert (HdegA0 : deg A0 = deg M0 + 1)
2080       by (pose proof (deg_typing_succ Γ0 M0 A0 HM); lia).
2081     assert (HdegAB : deg A0 = deg B0)
2082       by (apply (deg_conv_invariant Γ0 M0 A0 B0 s HM Heq HB)).
2083     assert (HdegBs : deg B0 = index_of s) by (apply (deg_sort_typed Γ0 B0 s
2084       HB)).
2085     pose proof (index_range s) as [Hslo Hshi].
2086     assert (HsGe : index_of s >= i + 2) by lia.
2087     assert (Hlt : lt_of nat (fun k => n - k) (i + 1) i) by (unfold lt_of; lia).
2088     assert (Hij : 0 <= i + 1 <= n) by lia.
2089     assert (HdegA0' : deg A0 >= (i + 1) + 1) by lia.
2090     assert (HeqRho : rho (i + 1) A0 =b rho (i + 1) B0)
2091       by (apply (rho_commutes_beta_eq (i + 1) Hij A0 B0 HdegA0' Heq)).
2092     assert (HIH1 : rho_ctx (i + 1) Γ0 * rho i M0 rho (i + 1) A0)
2093       by (exact (IHM Hdeg)).
2094     assert (HdegB0' : deg B0 >= (i + 1) + 1) by (lia).
2095     pose proof (IHouter (i + 1) Hlt Hij Γ0 B0 (t_sort s) HB HdegB0') as HIH2.
2096     replace (i + 1 + 1) with (S (S i)) in HIH2 by lia.
2097     simpl in HIH2.
2098     replace (S (S i)) with (i + 2) in HIH2 by lia.
2099     assert (HIH2' : rho_ctx (i + 1) Γ0 * rho (i + 1) B0 t_sort (sort_of (i +
2100       2))).
2101     { apply (typing_star_weakening_incl (rho_ctx (i + 2) Γ0) _ _ _
2102       (rho_ctx_mono Γ0 (i + 1) (i + 2) ltac:(lia)) HIH2). }
2103     apply (typing_star_conv (rho_ctx (i + 1) Γ0) (rho i M0) (rho (i + 1) A0)
2104       (rho (i + 1) B0) (sort_of (i + 2)) HIH1 HeqRho HIH2').
2105   Qed.
2106
2107   (* ===== *)
2108   (* Term-Level Translation *)
2109   (* : T → T 0 *)
2110
2111   Definition bullet_var : var := n + 2.
2112   Definition prod_var : var := 2 * (n + 2).
2113
2114   Lemma bullet_var_retag_ne : forall x s, retag x s <> bullet_var.
2115   Proof.
2116     intros x s. unfold bullet_var. replace (n + 2) with (1 * (n + 2)) by lia.
2117     apply (retag_neq_mult 1).
2118   Qed.

```

```

2118 Lemma prod_var_retag_ne : forall x s, retag x s <> prod_var.
2119 Proof. intros x s. unfold prod_var. apply (retag_neq_mult 2). Qed.
2120
2121 Lemma prod_var_ne_bullet_var : prod_var <> bullet_var.
2122 Proof. unfold prod_var, bullet_var. pose proof n_range. lia. Qed.
2123
2124 Lemma prod_var_ne_zero_var : prod_var <> zero_var.
2125 Proof. unfold prod_var, zero_var. pose proof n_range. lia. Qed.
2126
2127 Lemma bullet_var_ne_zero_var : bullet_var <> zero_var.
2128 Proof. unfold bullet_var, zero_var. pose proof n_range. lia. Qed.
2129
2130 Fixpoint gamma_pi_tel (A body : term) (k : nat) : term :=
2131   match k with
2132   | 0    => t_pi (sort_of 0) (rho 0 A) body
2133   | S k' => t_pi (sort_of (S k')) (rho (S k') A) (gamma_pi_tel A body k')
2134   end.
2135
2136 Fixpoint gamma_lam_tel (A body : term) (k : nat) : term :=
2137   match k with
2138   | 0    => t_lam (sort_of 0) (rho 0 A) body
2139   | S k' => t_lam (sort_of (S k')) (rho (S k') A) (gamma_lam_tel A body k')
2140   end.
2141
2142 Fixpoint gamma_app_tel (M' N : term) (k : nat) : term :=
2143   match k with
2144   | 0    => t_app M' (rho 0 N)
2145   | S k' => gamma_app_tel (t_app M' (rho (S k') N)) N k'
2146   end.
2147
2148 Fixpoint gamma (M : term) : term :=
2149   match M with
2150   | t_sort s    => t_fvar (sort_of 1) bullet_var
2151   | t_bvar s0 k => t_bvar s0 k
2152   | t_fvar s0 x => t_fvar (sort_of 1) (retag x (sort_of 1))
2153   | t_pi s A B =>
2154     t_app (t_app (t_app (t_fvar (sort_of 1) prod_var)
2155       (gamma_pi_tel A (t_fvar (sort_of 2) zero_var) (deg A -
2156         1))))
2157       (gamma A))
2158       (gamma_lam_tel A (gamma B) (deg A - 1))
2159   | t_lam s A M' =>
2160     t_app (t_lam (sort_of 1) (t_fvar (sort_of 2) zero_var)
2161       (gamma_lam_tel A (gamma M') (deg A - 1)))
2162       (gamma A)
2163   | t_app M' N =>
2164     t_app (gamma_app_tel (gamma M') N (deg N - 1)) (gamma N)
2165   end.
2166 (* ===== *)

```

```

2167 (** * gamma commutes with typing *)
2168
2169 (* prod :  $\Pi A \ s1 . 0 \rightarrow A \rightarrow 0$ , requires the rule (s2,s1), which we assume appear
    in S. *)
2170 Axiom R_s2_s1 : R (sort_of 2) (sort_of 1) (sort_of 1).
2171
2172 Lemma sort_of_1_ne_2 : sort_of 1 <> sort_of 2.
2173 Proof.
2174   pose proof n_at_least_two as Hn2.
2175   intro Heq.
2176   assert (H1 : index_of (sort_of 1) = 1) by (apply sort_of_correct; lia).
2177   assert (H2 : index_of (sort_of 2) = 2) by (apply sort_of_correct; lia).
2178   rewrite Heq in H1. lia.
2179 Qed.
2180
2181 Lemma R_s1_s1_s1 : R (sort_of 1) (sort_of 1) (sort_of 1).
2182 Proof.
2183   pose proof n_at_least_two as Hn2.
2184   pose proof (is_full _ _ _ R_s2_s1) as Hfw.
2185   unfold full_with in Hfw.
2186   apply Hfw.
2187   - rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2188   - rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2189   - rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
2190     rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2191     lia.
2192 Qed.
2193
2194 Definition T_prod : term :=
2195   t_pi (sort_of 2) (t_sort (sort_of 1))
2196     (t_pi (sort_of 1) (t_fvar (sort_of 2) zero_var)
2197       (t_pi (sort_of 1) (t_bvar (sort_of 2) 1) (t_fvar (sort_of 2) zero_var))).
2198
2199 Lemma subcontext_app_l : forall  $\Gamma \ \Delta'$ ,  $\Gamma \ \Gamma \ ++ \ \Delta'$ .
2200 Proof. intros  $\Gamma \ \Delta'$  x T Hin. apply in_or_app. left. exact Hin. Qed.
2201
2202 Lemma subcontext_app_r : forall  $\Gamma \ \Delta'$ ,  $\Gamma \ \Delta' \ ++ \ \Gamma$ .
2203 Proof. intros  $\Gamma \ \Delta'$  x T Hin. apply in_or_app. right. exact Hin. Qed.
2204
2205 Lemma zero_var_typed :
2206   [(zero_var, (sort_of 2, t_sort (sort_of 1)))] * t_fvar (sort_of 2) zero_var
    t_sort (sort_of 1).
2207 Proof.
2208   pose proof n_at_least_two as Hn2.
2209   apply (typing_star_var [] zero_var (t_sort (sort_of 1)) (sort_of 2)).
2210   - unfold is_fresh, dom. simpl. auto.
2211   - apply typing_star_axiom. apply A_spec.
2212     rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
2213     rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2214     lia.

```

```

2215 Qed.
2216
2217 Lemma Tprod_typed :
2218   [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var, (sort_of 1, t_fvar
2219     (sort_of 2) zero_var))]
2219   * T_prod t_sort (sort_of 1).
2220 Proof.
2221   pose proof n_at_least_two as Hn2.
2222   pose proof zero_var_typed as Hz1.
2223   assert (HtsortA : [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2224     (sort_of 1, t_fvar (sort_of 2) zero_var))]
2225     * t_sort (sort_of 1) t_sort (sort_of 2)).
2226   { apply (typing_star_weakening_incl []).
2227     - intros x T [].
2228     - apply typing_star_axiom. apply A_spec.
2229       rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
2230       rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2231       lia. }
2232   apply (typing_star_pi
2233     [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var, (sort_of
2234       1, t_fvar (sort_of 2) zero_var))]
2235     (t_sort (sort_of 1))
2236     (t_pi (sort_of 1) (t_fvar (sort_of 2) zero_var)
2237       (t_pi (sort_of 1) (t_bvar (sort_of 2) 1) (t_fvar (sort_of 2)
2238         zero_var)))
2239     (sort_of 2) (sort_of 1) (sort_of 1)
2240     (dom [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2241       (sort_of 1, t_fvar (sort_of 2) zero_var))])).
2242   - exact HtsortA.
2243   - intros x Hx.
2244     unfold open_var. simpl.
2245     assert (Hx_typed :
2246       [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2247         (sort_of 1, t_fvar (sort_of 2) zero_var));
2248         (x, (sort_of 2, t_sort (sort_of 1)))]
2249       * t_fvar (sort_of 2) x t_sort (sort_of 1)).
2250     { apply (typing_star_var
2251       [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2252         (sort_of 1, t_fvar (sort_of 2) zero_var))]
2253       x (t_sort (sort_of 1)) (sort_of 2)).
2254       - exact Hx.
2255       - exact HtsortA. }
2256   apply (typing_star_pi
2257     [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var, (sort_of
2258       1, t_fvar (sort_of 2) zero_var));
2259     (x, (sort_of 2, t_sort (sort_of 1)))]
2260     (t_fvar (sort_of 2) zero_var)
2261     (t_pi (sort_of 1) (t_fvar (sort_of 2) x) (t_fvar (sort_of 2)
2262       zero_var))

```

```

2255         (sort_of 1) (sort_of 1) (sort_of 1) []).
2256 + apply (typing_star_weakening_incl [(zero_var, (sort_of 2, t_sort (sort_of
2257   * apply (subcontext_app_1 [(zero_var, (sort_of 2, t_sort (sort_of 1))))].
2258   * exact Hz1.
2259 + intros y Hy.
2260   unfold open_var. simpl.
2261   apply (typing_star_pi
2262     [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2263       (sort_of 1, t_fvar (sort_of 2) zero_var));
2264       (x, (sort_of 2, t_sort (sort_of 1))); (y, (sort_of 1, t_fvar
2265         (sort_of 2) zero_var))]
2266       (t_fvar (sort_of 2) x) (t_fvar (sort_of 2) zero_var) (sort_of 1)
2267       (sort_of 1) (sort_of 1) []).
2268   * apply (typing_star_weakening_incl
2269     [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2270       (sort_of 1, t_fvar (sort_of 2) zero_var));
2271       (x, (sort_of 2, t_sort (sort_of 1)))]).
2272   -- apply (subcontext_app_1
2273     [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2274       (sort_of 1, t_fvar (sort_of 2) zero_var));
2275       (x, (sort_of 2, t_sort (sort_of 1)))]).
2276   -- exact Hx_typed.
2277   * intros z Hz.
2278   unfold open_var. simpl.
2279   apply (typing_star_weakening_incl [(zero_var, (sort_of 2, t_sort
2280     (sort_of 1)))]).
2281   -- apply (subcontext_app_1 [(zero_var, (sort_of 2, t_sort (sort_of
2282     1)))]).
2283   -- exact Hz1.
2284   * split; [exact R_s1_s1_s1 |].
2285     rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2286 + split; [exact R_s1_s1_s1 |].
2287     rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)). lia.
2288 - split; [exact R_s2_s1 |].
2289     rewrite (sort_of_correct 1 ltac:(lia) ltac:(lia)).
2290     rewrite (sort_of_correct 2 ltac:(lia) ltac:(lia)).
2291     lia.
2292 Qed.
2293
2294 Lemma bullet_typed :
2295   [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2296   (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2297   (prod_var, (sort_of 1, T_prod))] * t_fvar (sort_of 1) bullet_var t_fvar
2298   (sort_of 2) zero_var.
2299
2300 Proof.
2301   pose proof n_at_least_two as Hn2.
2302   pose proof zero_var_typed as Hz1.
2303   assert (Hb1 : [(zero_var, (sort_of 2, t_sort (sort_of 1))); (bullet_var,
2304     (sort_of 1, t_fvar (sort_of 2) zero_var))]

```



```

2295         * t_fvar (sort_of 1) bullet_var t_fvar (sort_of 2) zero_var).
2296 { apply (typing_star_var [(zero_var, (sort_of 2, t_sort (sort_of 1)))]
bullet_var (t_fvar (sort_of 2) zero_var) (sort_of 1)).
2297   - unfold is_fresh, dom. simpl.
2298     intros [Heq | []].
2299     apply bullet_var_ne_zero_var. symmetry. exact Heq.
2300   - exact Hz1. }
2301 apply (typing_star_weak
2302   [(zero_var, (sort_of 2, t_sort (sort_of 1))]; (bullet_var, (sort_of
2303     1, t_fvar (sort_of 2) zero_var))]
prod_var T_prod (t_fvar (sort_of 1) bullet_var) (t_fvar (sort_of 2)
zero_var) (sort_of 1)).
2304 - unfold is_fresh, dom. simpl.
2305   intros [Heq | [Heq | []]].
2306   + apply prod_var_ne_zero_var. symmetry. exact Heq.
2307   + apply prod_var_ne_bullet_var. symmetry. exact Heq.
2308 - exact Hb1.
2309 - exact Tprod_typed.
2310 Qed.
2311
2312 Lemma zero_var_typed_Δ :
2313   [(zero_var, (sort_of 2, t_sort (sort_of 1))];
2314   (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2315   (prod_var, (sort_of 1, T_prod))] * t_fvar (sort_of 2) zero_var t_sort
(sort_of 1).
2316 Proof.
2317   apply (typing_star_weakening_incl [(zero_var, (sort_of 2, t_sort (sort_of
2318     1)))]).
2319   - apply (subcontext_app_1 [(zero_var, (sort_of 2, t_sort (sort_of 1)))]).
2320   - exact zero_var_typed.
2321 Qed.
2322 Lemma prod_var_typed_Δ :
2323   [(zero_var, (sort_of 2, t_sort (sort_of 1))];
2324   (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2325   (prod_var, (sort_of 1, T_prod))] * t_fvar (sort_of 1) prod_var T_prod.
2326 Proof.
2327   apply (typing_star_var [(zero_var, (sort_of 2, t_sort (sort_of 1))];
2328     (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var))]
prod_var T_prod (sort_of 1)).
2329   - unfold is_fresh, dom. simpl.
2330     intros [Heq | [Heq | []]].
2331     + apply prod_var_ne_zero_var. symmetry. exact Heq.
2332     + apply prod_var_ne_bullet_var. symmetry. exact Heq.
2333   - exact Tprod_typed.
2334 Qed.
2335
2336 Axiom gamma_pi_tower :
2337   forall Δ0 Γ0 A0 s1,

```

```

2338   Δ0 * t_fvar (sort_of 2) zero_var t_sort (sort_of 1) ->
2339   Γ0 A0 t_sort s1 ->
2340   Δ0 ++ rho_ctx 0 Γ0 * gamma_pi_tel A0 (t_fvar (sort_of 2) zero_var) (deg A0 -
    1) t_sort (sort_of 1).
2341
2342 Axiom gamma_lam_tower :
2343   forall Δ0 Γ0 A0 B0 s1 L,
2344     Δ0 * t_fvar (sort_of 2) zero_var t_sort (sort_of 1) ->
2345     Γ0 A0 t_sort s1 ->
2346     (forall x, ~ In x L ->
2347       Δ0 ++ rho_ctx 0 (Γ0 ++ [(x, (s1, A0))]) * gamma (open_var B0 s1 x)
2348       t_fvar (sort_of 2) zero_var) ->
2349     Δ0 ++ rho_ctx 0 Γ0 * gamma_lam_tel A0 (gamma B0) (deg A0 - 1)
2350     gamma_pi_tel A0 (t_fvar (sort_of 2) zero_var) (deg A0 -
2351     1).
2352
2353 Lemma gamma_pi_tel_eq_rho_pi_tel : forall A B k,
2354   gamma_pi_tel A (rho 0 B) k = rho_pi_tel rho A B 0 k.
2355 Proof.
2356   intros A B k. induction k as [| k' IH].
2357   - reflexivity.
2358   - simpl. rewrite IH. reflexivity.
2359 Qed.
2360
2361 Lemma rho0_pi_eq_gamma_pi_tel : forall Γ0 A0 B0 s1,
2362   Γ0 A0 t_sort s1 ->
2363   rho 0 (t_pi s1 A0 B0) = gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1).
2364 Proof.
2365   intros Γ0 A0 B0 s1 HA.
2366   pose proof (deg_sort_typed Γ0 A0 s1 HA) as HdegA0.
2367   pose proof (index_range s1) as [Hlo Hhi].
2368   assert (Hge : 0 + 1 <= deg A0) by lia.
2369   rewrite (rho_pi_named 0 s1 A0 B0 Hge).
2370   replace (deg A0 - 1 - 0) with (deg A0 - 1) by lia.
2371   symmetry. apply gamma_pi_tel_eq_rho_pi_tel.
2372 Qed.
2373
2374 Axiom gamma_lam_tower_D :
2375   forall Δ0 Γ0 A0 M0 B0 s1 L,
2376     Γ0 A0 t_sort s1 ->
2377     (forall x, ~ In x L ->
2378       Δ0 ++ rho_ctx 0 (Γ0 ++ [(x, (s1, A0))]) * gamma (open_var M0 s1 x) rho
2379       0 (open_var B0 s1 x)) ->
2380     Δ0 ++ rho_ctx 0 Γ0 * gamma_lam_tel A0 (gamma M0) (deg A0 - 1)
2381     gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1).
2382
2383 Axiom gamma_lam_applied :
2384   forall Δ0 Γ0 A0 lamTerm piType,
2385     Δ0 ++ rho_ctx 0 Γ0 * lamTerm piType ->

```

```

2383      Δ0 ++ rho_ctx 0 Γ0 * piType   t_sort (sort_of 1) ->
2384      Δ0 ++ rho_ctx 0 Γ0 * gamma A0   t_fvar (sort_of 2) zero_var ->
2385      Δ0 ++ rho_ctx 0 Γ0 *
2386      t_app (t_lam (sort_of 1) (t_fvar (sort_of 2) zero_var) lamTerm) (gamma A0)
        piType.

2387
2388  Axiom gamma_app_tower :
2389    forall Δ0 Γ0 M0 N0 A0 B0 s1a,
2390      Γ0   M0   t_pi s1a A0 B0 ->
2391      Γ0   N0   A0 ->
2392      Δ0 ++ rho_ctx 0 Γ0 * gamma M0   rho 0 (t_pi s1a A0 B0) ->
2393      Δ0 ++ rho_ctx 0 Γ0 * gamma N0   rho 0 A0 ->
2394      Δ0 ++ rho_ctx 0 Γ0 * t_app (gamma_app_tel (gamma M0) N0 (deg N0 - 1)) (gamma
        N0)
        rho 0 (B0 ^^ N0).

2395
2396
2397  Axiom gamma_prod_applied :
2398    forall Δ0 Γ0 A0 lamterm,
2399      Δ0 ++ rho_ctx 0 Γ0 * t_fvar (sort_of 1) prod_var   T_prod ->
2400      Δ0 ++ rho_ctx 0 Γ0 * gamma_pi_tel A0 (t_fvar (sort_of 2) zero_var) (deg A0 -
        1)   t_sort (sort_of 1) ->
2401      Δ0 ++ rho_ctx 0 Γ0 * gamma A0   t_fvar (sort_of 2) zero_var ->
2402      Δ0 ++ rho_ctx 0 Γ0 * lamterm   gamma_pi_tel A0 (t_fvar (sort_of 2) zero_var)
        (deg A0 - 1) ->
2403      Δ0 ++ rho_ctx 0 Γ0 *
2404      t_app (t_app (t_app (t_fvar (sort_of 1) prod_var) (gamma_pi_tel A0 (t_fvar
        (sort_of 2) zero_var) (deg A0 - 1)))
        (gamma A0))
        lamterm
        t_fvar (sort_of 2) zero_var.

2408
2409  Lemma gamma_commutes_typing :
2410    forall Γ M N, Γ   M   N ->
2411    [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2412     (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2413     (prod_var, (sort_of 1, T_prod))] ++ rho_ctx 0 Γ * gamma M   rho 0 N.
2414  Proof.
2415    intros Γ M N Htype.
2416    remember [(zero_var, (sort_of 2, t_sort (sort_of 1)));
2417              (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
2418              (prod_var, (sort_of 1, T_prod))]] as Δ.
2419    induction Htype as
2420      [ s s' HA
2421      | Γ0 x A0 s0 Hfresh HA IHA
2422      | Γ0 x B0 M0 A0 s0 Hfresh HM IHM HB IHB
2423      | Γ0 A0 B0 s1 s2 s3 L HA IHA HTagB HB IHB HR
2424      | Γ0 A0 M0 B0 s1 s3 L HA IHA HTagM HM IHM HPi IHPi
2425      | Γ0 M0 N0 A0 B0 s1a HM IHM HN IHN
2426      | Γ0 M0 A0 B0 s HM IHM Heq HB IHB ].

```

```

2427
2428 - (* typing_axiom *)
2429   simpl.
2430   apply (typing_star_weakening_incl Δ).
2431   + apply (subcontext_app_l Δ).
2432   + rewrite HeqΔ. exact bullet_typed.
2433
2434 - (* typing_var *)
2435   pose proof (index_range s0) as [Hxlo Hxhi].
2436   pose proof (deg_sort_typed Γ0 A0 s0 HA) as HdegA0.
2437   assert (Hdeg1 : deg A0 >= 0 + 1) by lia.
2438   pose proof (rho_commutes_typing 0 (ltac:(lia)) Γ0 A0 (t_sort s0) HA Hdeg1)
   as HB1.
2439   simpl in HB1.
2440   assert (HB0 : rho_ctx 0 Γ0 * rho 0 A0 t_sort (sort_of 1))
2441     by (apply (typing_star_weakening_incl (rho_ctx 1 Γ0) (rho_ctx 0 Γ0));
2442         [apply rho_ctx_mono; lia | exact HB1])).
2443   pose (x' := retag x (sort_of 1)).
2444   assert (Hxnz : x <> zero_var)
2445     by (apply (typing_avoids_zero_var (Γ0 ++ [(x, (s0, A0))]) (t_fvar s0 x) A0
2446         (typing_var Γ0 x A0 s0 Hfresh HA) x s0 A0);
2447         apply in_or_app; right; left; reflexivity).
2448   assert (Hfreshx' : is_fresh x' (rho_ctx 0 Γ0))
2449     by (apply rho_ctx_fresh; [exact Hxnz | exact Hfresh]).
2450   assert (Hstep : rho_ctx 0 Γ0 ++ [(x', (sort_of 1, rho 0 A0))] * t_fvar
   (sort_of 1) x' rho 0 A0)
2451     by (apply typing_star_var; [exact Hfreshx' | exact HB0])).
2452   assert (Hmem : In (x', (sort_of 1, rho 0 A0))
2453     (rho_ctx_tel x s0 A0 (index_of s0 - 0 - 1))).
2454   { pose proof (rho_ctx_tel_last x s0 A0 (index_of s0 - 0 - 1)) as Htel.
2455     replace (index_of s0 - (index_of s0 - 0 - 1))
2456       with 1 in Htel by lia.
2457     simpl in Htel.
2458     unfold x'. exact Htel.
2459   }
2460   assert (Hincl : rho_ctx 0 Γ0 ++ [(x', (sort_of 1, rho 0 A0))]
2461     rho_ctx 0 (Γ0 ++ [(x, (s0, A0))])).
2462   { apply subcontext_app_same.
2463     - apply rho_ctx_incl_old.
2464     - intros p q Hpq. destruct Hpq as [Heq | []]. inversion Heq; subst.
2465       apply (rho_ctx_incl_new Γ0 0 x s0 A0). exact Hmem.
2466   }
2467   assert (Hfinal : rho_ctx 0 (Γ0 ++ [(x, (s0, A0))]) * t_fvar (sort_of 1) x'
   rho 0 A0)
2468     by (apply (typing_star_weakening_incl _ _ _ Hincl Hstep)).
2469   simpl. unfold x' in Hfinal.
2470   apply (typing_star_weakening_incl (rho_ctx 0 (Γ0 ++ [(x, (s0, A0))]))
2471     (Δ ++ rho_ctx 0 (Γ0 ++ [(x, (s0, A0))]))).
2472   + apply (subcontext_app_r (rho_ctx 0 (Γ0 ++ [(x, (s0, A0))])) Δ).
2473   + exact Hfinal.

```

```

2474
2475 - (* typing_weak *)
2476   apply (typing_star_weakening_incl (Δ ++ rho_ctx 0 Γ0)
2477     (Δ ++ rho_ctx 0 (Γ0 ++ [(x, (s0, B0))])))).
2478 + apply app_subset_app.
2479   * intros p q Hpq. exact Hpq.
2480   * apply rho_ctx_incl_old.
2481 + exact IHM.
2482
2483 - (* typing_pi *)
2484   simpl.
2485   assert (HzeroΔ0 : Δ * t_fvar (sort_of 2) zero_var t_sort (sort_of 1))
2486     by (rewrite HeqΔ; exact zero_var_typed_Δ).
2487   assert (HProdTyped : Δ ++ rho_ctx 0 Γ0 * t_fvar (sort_of 1) prod_var
2488     T_prod).
2489   { apply (typing_star_weakening_incl Δ).
2490     - apply (subcontext_app_1 Δ).
2491     - rewrite HeqΔ. exact prod_var_typed_Δ. }
2492   assert (HpiTel : Δ ++ rho_ctx 0 Γ0 * gamma_pi_tel A0 (t_fvar (sort_of 2)
2493     zero_var) (deg A0 - 1)
2494     t_sort (sort_of 1))
2495     by (apply (gamma_pi_tower Δ Γ0 A0 s1 HzeroΔ0 HA)).
2496   assert (HgammaA : Δ ++ rho_ctx 0 Γ0 * gamma A0 t_fvar (sort_of 2)
2497     zero_var)
2498     by (simpl in IHA; exact IHA).
2499   assert (HBcofin : forall x, ~ In x L ->
2500     Δ ++ rho_ctx 0 (Γ0 ++ [(x, (s1, A0))])) * gamma (open_var B0 s1 x)
2501     t_fvar (sort_of 2) zero_var).
2502   { intros x Hx. pose proof (IHB x Hx) as H. simpl in H. exact H. }
2503   assert (HlamTel : Δ ++ rho_ctx 0 Γ0 * gamma_lam_tel A0 (gamma B0) (deg A0 -
2504     1)
2505     gamma_pi_tel A0 (t_fvar (sort_of 2) zero_var) (deg A0 -
2506     1))
2507     by (apply (gamma_lam_tower Δ Γ0 A0 B0 s1 L HzeroΔ0 HA HBcofin)).
2508   apply (gamma_prod_applied Δ Γ0 A0 (gamma_lam_tel A0 (gamma B0) (deg A0 - 1))
2509     HProdTyped HpiTel HgammaA HlamTel).
2510
2511 - (* typing_lam *)
2512   rewrite (rho0_pi_eq_gamma_pi_tel Γ0 A0 B0 s1 HA).
2513   assert (HdegPi1 : deg (t_pi s1 A0 B0) >= 0 + 1)
2514     by (pose proof (index_range s3) as [Hlo Hhi];
2515       pose proof (deg_sort_typed Γ0 (t_pi s1 A0 B0) s3 HPi); lia).
2516   pose proof (rho_commutes_typing 0 (ltac:(lia)) Γ0 (t_pi s1 A0 B0) (t_sort
2517     s3) HPi HdegPi1)
2518     as HpiTelD1.
2519   simpl in HpiTelD1.
2520   assert (HpiTelD0 : rho_ctx 0 Γ0 * rho 0 (t_pi s1 A0 B0) t_sort (sort_of
2521     1))
2522     by (apply (typing_star_weakening_incl (rho_ctx 1 Γ0) (rho_ctx 0 Γ0));

```

```

2515         [apply rho_ctx_mono; lia | exact HpiTelD1]].
2516 rewrite (rho0_pi_eq_gamma_pi_tel Γ0 A0 B0 s1 HA) in HpiTelD0.
2517 assert (HpiTelD : Δ ++ rho_ctx 0 Γ0 * gamma_pi_tel A0 (rho 0 B0) (deg A0 -
1)
2518         t_sort (sort_of 1))
2519   by (apply (typing_star_weakening_incl (rho_ctx 0 Γ0) (Δ ++ rho_ctx 0 Γ0));
2520       [apply (subcontext_app_r (rho_ctx 0 Γ0) Δ) | exact HpiTelD0]).
2521 assert (HgammaA : Δ ++ rho_ctx 0 Γ0 * gamma A0 t_fvar (sort_of 2)
zero_var)
2522   by (simpl in IHA; exact IHA).
2523 assert (HMcofin : forall x, ~ In x L ->
2524         Δ ++ rho_ctx 0 (Γ0 ++ [(x, (s1, A0))]) * gamma (open_var M0 s1 x)
rho 0 (open_var B0 s1 x)).
2525 { intros x Hx. exact (IHM x Hx). }
2526 assert (HlamTelD : Δ ++ rho_ctx 0 Γ0 * gamma_lam_tel A0 (gamma M0) (deg A0 -
1)
2527         gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1))
2528   by (apply (gamma_lam_tower_D Δ Γ0 A0 M0 B0 s1 L HA HMcofin)).
2529 apply (gamma_lam_applied Δ Γ0 A0
2530       (gamma_lam_tel A0 (gamma M0) (deg A0 - 1))
2531       (gamma_pi_tel A0 (rho 0 B0) (deg A0 - 1))
2532       HlamTelD HpiTelD HgammaA).
2533
2534 - (* typing_app *)
2535 apply (gamma_app_tower Δ Γ0 M0 N0 A0 B0 s1a HM HN IHM IHN).
2536
2537 - (* typing_conv *)
2538 assert (HdegA0 : deg A0 >= 0 + 1)
2539   by (pose proof (deg_typing_succ Γ0 M0 A0 HM); lia).
2540 assert (HeqRho : rho 0 A0 =b rho 0 B0)
2541   by (apply (rho_commutes_beta_eq 0 (ltac:(lia)) A0 B0 HdegA0 Heq)).
2542 assert (HdegB1 : deg B0 >= 0 + 1)
2543   by (pose proof (index_range s) as [Hlo Hhi];
2544       pose proof (deg_sort_typed Γ0 B0 s HB); lia).
2545 pose proof (rho_commutes_typing 0 (ltac:(lia)) Γ0 B0 (t_sort s) HB HdegB1)
as HrhoB1.
2546 simpl in HrhoB1.
2547 assert (HrhoB0 : rho_ctx 0 Γ0 * rho 0 B0 t_sort (sort_of 1))
2548   by (apply (typing_star_weakening_incl (rho_ctx 1 Γ0) (rho_ctx 0 Γ0));
2549       [apply rho_ctx_mono; lia | exact HrhoB1]).
2550 assert (HrhoB0Δ : Δ ++ rho_ctx 0 Γ0 * rho 0 B0 t_sort (sort_of 1))
2551   by (apply (typing_star_weakening_incl (rho_ctx 0 Γ0) (Δ ++ rho_ctx 0 Γ0));
2552       [apply (subcontext_app_r (rho_ctx 0 Γ0) Δ) | exact HrhoB0]).
2553 apply (typing_star_conv (Δ ++ rho_ctx 0 Γ0) (gamma M0) (rho 0 A0) (rho 0 B0)
2554       (sort_of 1) IHM HeqRho HrhoB0Δ).
2555 Qed.
2556
2557 (* ===== *)
2558 (* gamma commutes with substitution *)

```

```

2559
2560 Fixpoint gamma_subst_tel (M B : term) (x : var) (K : nat) : term :=
2561   match K with
2562   | 0      => M
2563   | S K' => gamma_subst_tel (M { retag x (sort_of (K' + 2))   rho K' B }) B x K'
2564   end.
2565
2566 Definition gamma_subst (A B : term) (x : var) (s : Sort) : term :=
2567   (gamma_subst_tel (gamma A) B x (index_of s - 1)) { retag x (sort_of 1)   gamma
2568   B }.
2569
2570 Lemma sort_of_1_ne_sort_of_m : forall m, 2 <= m -> m < n + 1 -> sort_of 1 <>
2571 sort_of m.
2572 Proof.
2573   intros m Hm2 Hmn Heq.
2574   assert (H1 : index_of (sort_of 1) = 1) by (apply sort_of_correct; lia).
2575   assert (H2 : index_of (sort_of m) = m) by (apply sort_of_correct; lia).
2576   rewrite Heq in H1. lia.
2577 Qed.
2578
2579 Lemma gamma_subst_tel_fvar_const : forall sC C x B K,
2580   (forall m, 2 <= m -> m <= K + 1 -> retag x (sort_of m) <> C) ->
2581   gamma_subst_tel (t_fvar sC C) B x K = t_fvar sC C.
2582 Proof.
2583   intros sC C x B K.
2584   induction K as [| K' IH]; intros Hm.
2585   - reflexivity.
2586   - simpl.
2587     destruct (eq_var_dec C (retag x (sort_of (K' + 2)))) as [Heq | Hneq].
2588     + ex falso. apply (Hm (K' + 2) ltac:(lia) ltac:(lia)). symmetry. exact Heq.
2589     + apply IH. intros m Hm2 Hm3. apply Hm; lia.
2590 Qed.
2591
2592 Lemma gamma_subst_tel_bvar_const : forall s0 k0 x B K,
2593   gamma_subst_tel (t_bvar s0 k0) B x K = t_bvar s0 k0.
2594 Proof.
2595   intros s0 k0 x B K.
2596   induction K as [| K' IH].
2597   - reflexivity.
2598   - simpl. exact IH.
2599 Qed.
2600
2601 Lemma gamma_subst_tel_app_commute : forall P Q x B K,
2602   gamma_subst_tel (t_app P Q) B x K
2603   = t_app (gamma_subst_tel P B x K) (gamma_subst_tel Q B x K).
2604 Proof.
2605   intros P Q x B K. revert P Q.
2606   induction K as [| K' IH]; intros P Q.
2607   - reflexivity.
2608   - simpl. apply IH.

```



```

2607 Qed.
2608
2609 Lemma gamma_subst_tel_lam_commute : forall s' A' M' x B K,
2610   gamma_subst_tel (t_lam s' A' M') B x K
2611   = t_lam s' (gamma_subst_tel A' B x K) (gamma_subst_tel M' B x K).
2612 Proof.
2613   intros s' A' M' x B K. revert A' M'.
2614   induction K as [| K' IH]; intros A' M'.
2615   - reflexivity.
2616   - simpl. apply IH.
2617 Qed.
2618
2619 Lemma gamma_subst_sentinel_const : forall sC C B x s,
2620   (forall y sy, retag y sy <> C) ->
2621   (gamma_subst_tel (t_fvar sC C) B x (index_of s - 1))
2622   { retag x (sort_of 1) gamma B }
2623   = t_fvar sC C.
2624 Proof.
2625   intros sC C B x s Hsent.
2626   assert (Hconst : gamma_subst_tel (t_fvar sC C) B x (index_of s - 1) = t_fvar
2627     sC C).
2628   { apply gamma_subst_tel_fvar_const.
2629     intros m Hm2 Hm3 Heq. apply (Hsent x (sort_of m)). exact Heq. }
2629   rewrite Hconst. simpl.
2630   destruct (eq_var_dec C (retag x (sort_of 1))) as [Heq | Hneq].
2631   - exfalso. apply (Hsent x (sort_of 1)). symmetry. exact Heq.
2632   - reflexivity.
2633 Qed.
2634
2635 Axiom gamma_pi_tel_commutates_subst : forall C B x s KC,
2636   x <> zero_var -> 1 <= index_of s <= n ->
2637   deg B = index_of s - 1 -> lc B -> only_tagged x s C ->
2638   (gamma_subst_tel (gamma_pi_tel C (t_fvar (sort_of 2) zero_var) KC) B x
2639     (index_of s - 1))
2639     { retag x (sort_of 1) gamma B }
2640   = gamma_pi_tel (C { x B }) (t_fvar (sort_of 2) zero_var) KC.
2641
2642 Axiom gamma_lam_tel_commutates_subst : forall C D B x s KC,
2643   x <> zero_var -> 1 <= index_of s <= n ->
2644   deg B = index_of s - 1 -> lc B -> only_tagged x s C ->
2645   (gamma_subst_tel (gamma_lam_tel C D KC) B x (index_of s - 1))
2646   { retag x (sort_of 1) gamma B }
2647   = gamma_lam_tel (C { x B })
2648     ((gamma_subst_tel D B x (index_of s - 1))
2649      { retag x (sort_of 1) gamma B })
2650   KC.
2651
2652 Axiom gamma_app_tel_commutates_subst : forall MG C B x s KC,
2653   x <> zero_var -> 1 <= index_of s <= n ->
2654   deg B = index_of s - 1 -> lc B -> only_tagged x s C ->

```



```

2655 (gamma_subst_tel (gamma_app_tel MG C KC) B x (index_of s - 1))
2656 { retag x (sort_of 1) gamma B }
2657 = gamma_app_tel
2658 ((gamma_subst_tel MG B x (index_of s - 1))
2659 { retag x (sort_of 1) gamma B })
2660 (C { x B }) KC.
2661
2662 Lemma gamma_commutes_substitution :
2663   forall x s, x <> bullet_var -> x <> zero_var -> x <> prod_var ->
2664   1 <= index_of s <= n ->
2665   forall A, only_tagged x s A ->
2666   forall B, deg B = index_of s - 1 -> lc B ->
2667   gamma (A { x B }) = gamma_subst A B x s.
2668 Proof.
2669   intros x s Hxb Hxz Hxp Hxrange A.
2670   unfold gamma_subst.
2671   induction A as [s0 | s_m m | sy y | D IHD C IHC | s1 C IHC D IHD | s1 C IHC D
IHD];
2672     intros HonlyA B HdegB HlcB.
2673
2674   - (* A = t_sort s0 *)
2675     rewrite subst_sort. simpl.
2676     rewrite (gamma_subst_sentinel_const (sort_of 1) bullet_var B x s
bullet_var_retag_ne).
2677     reflexivity.
2678
2679   - (* A = t_bvar s_m m *)
2680     rewrite subst_bvar. simpl.
2681     rewrite gamma_subst_tel_bvar_const, subst_bvar.
2682     reflexivity.
2683
2684   - (* A = t_fvar sy y *)
2685     rewrite subst_var.
2686     destruct (eq_var_dec y x) as [Heq | Hneq].
2687     + subst y.
2688       simpl.
2689       assert (Hconst : gamma_subst_tel (t_fvar (sort_of 1) (retag x (sort_of
1))) B x (index_of s - 1)
= t_fvar (sort_of 1) (retag x (sort_of 1))).
2690       { apply gamma_subst_tel_fvar_const.
2691         intros m Hm2 Hm3 Heq2.
2692         apply retag_inj in Heq2 as [_ Hsc].
2693         apply (sort_of_1_ne_sort_of_m m Hm2 ltac:(lia)).
2694         symmetry. exact Hsc. }
2695       rewrite Hconst. simpl.
2696       destruct (eq_var_dec (retag x (sort_of 1)) (retag x (sort_of 1))) as [_ |
Hne2].
2697       * reflexivity.
2698       * exfalso. apply Hne2. reflexivity.

```

```

2700 + simpl.
2701   assert (Hconst : gamma_subst_tel (t_fvar (sort_of 1) (retag y (sort_of
2702     1))) B x (index_of s - 1)
2703     = t_fvar (sort_of 1) (retag y (sort_of 1))).
2704   { apply gamma_subst_tel_fvar_const.
2705     intros m Hm2 Hm3 Heq2.
2706     apply retag_inj in Heq2 as [Hxeq _]. apply Hneq. symmetry. exact Hxeq. }
2707   rewrite Hconst. simpl.
2708   destruct (eq_var_dec (retag y (sort_of 1)) (retag x (sort_of 1))) as [Heq2
2709     | Hneq2].
2710   * exfalso. apply retag_inj in Heq2 as [Hc _]. apply Hneq. exact Hc.
2711   * reflexivity.
2712
2713 - (* A = t_app D C *)
2714   simpl in HonlyA. destruct HonlyA as [HonlyD HonlyC].
2715   assert (HdegC : deg (C { x B }) = deg C)
2716   by (apply (deg_subst_tagged C x s B HonlyC HlcB HdegB)).
2717   rewrite subst_app. simpl. rewrite HdegC.
2718   rewrite gamma_subst_tel_app_commute, subst_app.
2719   rewrite (gamma_app_tel_commutes_subst (gamma D) C B x s (deg C - 1) Hxz
2720     Hxrange HdegB HlcB HonlyC).
2721   rewrite <- (IHD HonlyD B HdegB HlcB), <- (IHC HonlyC B HdegB HlcB).
2722   reflexivity.
2723
2724 - (* A = t_lam s1 C D *)
2725   simpl in HonlyA. destruct HonlyA as [HonlyC HonlyD].
2726   assert (HdegC : deg (C { x B }) = deg C)
2727   by (apply (deg_subst_tagged C x s B HonlyC HlcB HdegB)).
2728   rewrite subst_lam. simpl. rewrite HdegC.
2729   rewrite gamma_subst_tel_app_commute, subst_app.
2730   rewrite gamma_subst_tel_lam_commute, subst_lam.
2731   rewrite (gamma_subst_sentinel_const (sort_of 2) zero_var B x s
2732     zero_var_retag_ne).
2733   rewrite (gamma_lam_tel_commutes_subst C (gamma D) B x s (deg C - 1) Hxz
2734     Hxrange HdegB HlcB HonlyC).
2735   rewrite <- (IHD HonlyD B HdegB HlcB), <- (IHC HonlyC B HdegB HlcB).
2736   reflexivity.
2737
2738 - (* A = t_pi s1 C D *)
2739   simpl in HonlyA. destruct HonlyA as [HonlyC HonlyD].
2740   assert (HdegC : deg (C { x B }) = deg C)
2741   by (apply (deg_subst_tagged C x s B HonlyC HlcB HdegB)).
2742   rewrite subst_pi. simpl. rewrite HdegC.
2743   rewrite gamma_subst_tel_app_commute, subst_app.
2744   rewrite gamma_subst_tel_app_commute, subst_app.
2745   rewrite gamma_subst_tel_app_commute, subst_app.
2746   rewrite (gamma_subst_sentinel_const (sort_of 1) prod_var B x s
2747     prod_var_retag_ne).
2748   rewrite (gamma_pi_tel_commutes_subst C B x s (deg C - 1) Hxz Hxrange HdegB
2749     HlcB HonlyC).

```

```

2743     rewrite (gamma_lam_tel_commutates_subst C (gamma D) B x s (deg C - 1) Hxz
2744           Hxrange HdegB HlcB HonlyC).
2745     rewrite <- (IHD HonlyD B HdegB HlcB), <- (IHC HonlyC B HdegB HlcB).
2746     reflexivity.
2747 Qed.
2748
2749 (* ===== *)
2750 (* gamma commutes with beta *)
2751
2752 Lemma gamma_pi_tel_dom_congr : forall C C' body k,
2753   (forall m, m <= k -> rho m C ->>b rho m C') ->
2754   gamma_pi_tel C body k ->>b gamma_pi_tel C' body k.
2755 Proof.
2756   induction k as [| k' IHk]; intros Hall; simpl.
2757   - apply brt_pi_A. apply Hall. lia.
2758   - apply brt_pi_congr.
2759     + apply Hall. lia.
2760     + apply IHk. intros m Hm. apply Hall. lia.
2761 Qed.
2762
2763 Lemma gamma_lam_tel_dom_congr : forall C C' body k,
2764   (forall m, m <= k -> rho m C ->>b rho m C') ->
2765   gamma_lam_tel C body k ->>b gamma_lam_tel C' body k.
2766 Proof.
2767   induction k as [| k' IHk]; intros Hall; simpl.
2768   - apply brt_lam_A. apply Hall. lia.
2769   - apply brt_lam_congr.
2770     + apply Hall. lia.
2771     + apply IHk. intros m Hm. apply Hall. lia.
2772 Qed.
2773
2774 Lemma gamma_lam_tel_body_congr : forall C body body' k,
2775   body ->>b body' -> gamma_lam_tel C body k ->>b gamma_lam_tel C body' k.
2776 Proof.
2777   induction k as [| k' IHk]; intros Hb; simpl.
2778   - apply brt_lam_M. exact Hb.
2779   - apply brt_lam_M. apply IHk. exact Hb.
2780 Qed.
2781
2782 Lemma gamma_app_tel_func_congr : forall M M' N k,
2783   M ->>b M' -> gamma_app_tel M N k ->>b gamma_app_tel M' N k.
2784 Proof.
2785   intros M M' N k. revert M M'.
2786   induction k as [| k' IH]; intros M M' Hstep; simpl.
2787   - apply brt_app_1. exact Hstep.
2788   - apply IH. apply brt_app_1. exact Hstep.
2789 Qed.
2790
2791 Lemma gamma_app_tel_arg_congr : forall MG N N' k,

```

```

2792   (forall j, j <= k -> rho j N ->>b rho j N') ->
2793   gamma_app_tel MG N k ->>b gamma_app_tel MG N' k.
2794 Proof.
2795   intros MG N N' k. revert MG.
2796   induction k as [| k' IH]; intros MG Hall; simpl.
2797   - apply brt_app_r. apply Hall. lia.
2798   - apply brt_trans with (gamma_app_tel (t_app MG (rho (S k') N'))) N k').
2799     + apply gamma_app_tel_func_congr. apply brt_app_r. apply Hall. lia.
2800     + apply IH. intros j Hj. apply Hall. lia.
2801 Qed.
2802
2803 Axiom rho_commutes_beta_below : forall i, 0 <= i <= n ->
2804   forall M N, deg M < i + 1 -> M ->>b N -> rho i M ->>b rho i N.
2805
2806 Lemma rho_commutes_beta_total :
2807   forall i, 0 <= i <= n -> forall M N, M ->>b N -> rho i M ->>b rho i N.
2808 Proof.
2809   intros i Hi M N Hbeta.
2810   destruct (le_lt_dec (i + 1) (deg M)) as [E | E].
2811   - apply rho_commutes_beta; [lia | lia | exact Hbeta].
2812   - apply rho_commutes_beta_below; [lia | lia | exact Hbeta].
2813 Qed.
2814
2815 Lemma bt_pi_A : forall s A A' B, A ->>b A' -> t_pi s A B ->>b t_pi s A' B.
2816 Proof.
2817   intros s A A' B H.
2818   induction H as [A A' Hs | A A'' A' H1 IH1 H2 IH2].
2819   - apply bt_step. apply beta_pi_A. exact Hs.
2820   - apply bt_trans with (t_pi s A'' B); auto.
2821 Qed.
2822
2823 Lemma bt_pi_B : forall s A B B', B ->>b B' -> t_pi s A B ->>b t_pi s A B'.
2824 Proof.
2825   intros s A B B' H.
2826   induction H as [B B' Hs | B B'' B' H1 IH1 H2 IH2].
2827   - apply bt_step. apply beta_pi_B. exact Hs.
2828   - apply bt_trans with (t_pi s A B''); auto.
2829 Qed.
2830
2831 Lemma bt_lam_A : forall s A A' M, A ->>b A' -> t_lam s A M ->>b t_lam s A' M.
2832 Proof.
2833   intros s A A' M H.
2834   induction H as [A A' Hs | A A'' A' H1 IH1 H2 IH2].
2835   - apply bt_step. apply beta_lam_A. exact Hs.
2836   - apply bt_trans with (t_lam s A'' M); auto.
2837 Qed.
2838
2839 Lemma bt_lam_M : forall s A M M', M ->>b M' -> t_lam s A M ->>b t_lam s A M'.
2840 Proof.
2841   intros s A M M' H.

```

```

2842   induction H as [M M' Hs | M M'' M' H1 IH1 H2 IH2].
2843   - apply bt_step. apply beta_lam_M. exact Hs.
2844   - apply bt_trans with (t_lam s A M''); auto.
2845 Qed.
2846
2847 Lemma bt_app_l : forall M M' N, M ->>+b M' -> t_app M N ->>+b t_app M' N.
2848 Proof.
2849   intros M M' N H.
2850   induction H as [M M' Hs | M M'' M' H1 IH1 H2 IH2].
2851   - apply bt_step. apply beta_app_l. exact Hs.
2852   - apply bt_trans with (t_app M'' N); auto.
2853 Qed.
2854
2855 Lemma bt_app_r : forall M N N', N ->>+b N' -> t_app M N ->>+b t_app M N'.
2856 Proof.
2857   intros M N N' H.
2858   induction H as [N N' Hs | N N'' N' H1 IH1 H2 IH2].
2859   - apply bt_step. apply beta_app_r. exact Hs.
2860   - apply bt_trans with (t_app M N''); auto.
2861 Qed.
2862
2863 Lemma brt_bt_trans : forall M K N, M ->>b K -> K ->>+b N -> M ->>+b N.
2864 Proof.
2865   intros M K N Hmk.
2866   induction Hmk as [M | M K Hs | M Q K H1 IH1 H2 IH2]; intros Hkn.
2867   - exact Hkn.
2868   - apply bt_trans with K; [apply bt_step; exact Hs | exact Hkn].
2869   - apply IH1. apply IH2. exact Hkn.
2870 Qed.
2871
2872 Lemma bt_brt_trans : forall M K N, K ->>b N -> M ->>+b K -> M ->>+b N.
2873 Proof.
2874   intros M K N Hkn.
2875   induction Hkn as [K | K N Hs | K Q N H1 IH1 H2 IH2]; intros Hmk.
2876   - exact Hmk.
2877   - apply bt_trans with K; [exact Hmk | apply bt_step; exact Hs].
2878   - apply IH2. apply IH1. exact Hmk.
2879 Qed.
2880
2881 Lemma bt_app_congr_L : forall M M' N N',
2882   M ->>+b M' -> N ->>b N' -> t_app M N ->>+b t_app M' N'.
2883 Proof.
2884   intros M M' N N' HM HN.
2885   apply (bt_brt_trans (t_app M N) (t_app M' N) (t_app M' N')).
2886   - apply brt_app_r. exact HN.
2887   - apply bt_app_l. exact HM.
2888 Qed.
2889
2890 Lemma bt_app_congr_R : forall M M' N N',
2891   M ->>b M' -> N ->>+b N' -> t_app M N ->>+b t_app M' N'.

```

```

2892 Proof.
2893   intros M M' N N' HM HN.
2894   apply (brt_bt_trans (t_app M N) (t_app M' N) (t_app M' N')).
2895   - apply brt_app_l. exact HM.
2896   - apply bt_app_r. exact HN.
2897 Qed.
2898
2899 Lemma gamma_app_tel_func_congr_plus : forall M M' N k,
2900   M ->>+b M' -> gamma_app_tel M N k ->>+b gamma_app_tel M' N k.
2901 Proof.
2902   intros M M' N k. revert M M'.
2903   induction k as [| k' IH]; intros M M' Hstep; simpl.
2904   - apply bt_app_l. exact Hstep.
2905   - apply IH. apply bt_app_l. exact Hstep.
2906 Qed.
2907
2908 Lemma gamma_lam_tel_body_congr_plus : forall C body body' k,
2909   body ->>+b body' -> gamma_lam_tel C body k ->>+b gamma_lam_tel C body' k.
2910 Proof.
2911   induction k as [| k' IHk]; intros Hb; simpl.
2912   - apply bt_lam_M. exact Hb.
2913   - apply bt_lam_M. apply IHk. exact Hb.
2914 Qed.
2915
2916 Axiom gamma_commutes_beta_base : forall s A M N,
2917   gamma (t_app (t_lam s A M) N) ->>+b gamma (M ^^ N).
2918
2919 Lemma gamma_commutes_beta_aux :
2920   forall M N, M ->b N -> gamma M ->>+b gamma N.
2921 Proof.
2922   induction M as [s | s_m m | y | P IHP Q IHQ | s A IHA M' IHM' | s A IHA B
2923     IHB];
2924     intros N Hstep.
2925   - (* t_sort *) inversion Hstep.
2926   - (* t_bvar *) inversion Hstep.
2927   - (* t_fvar *) inversion Hstep.
2928
2929   - (* t_app P Q *)
2930     inversion Hstep; subst; clear Hstep.
2931
2932   + (* beta_base *)
2933     apply gamma_commutes_beta_base.
2934
2935   + (* beta_app_l *)
2936     match goal with
2937     | Hs : P ->b ?P' |- _ =>
2938       simpl; apply bt_app_l; apply gamma_app_tel_func_congr_plus; apply IHP;
2939       exact Hs
2940     end.

```

```

2940
2941 + (* beta_app_r *)
2942 match goal with
2943 | Hs : Q ->b ?Q' |- _ =>
2944   assert (HdegQeq : deg Q = deg Q') by (apply deg_beta; apply brt_step;
     exact Hs);
2945   assert (HQle : deg Q <= n + 1) by (apply deg_le_top);
2946   simpl; rewrite <- HdegQeq;
2947   apply bt_app_congr_R;
2948   [ apply gamma_app_tel_arg_congr;
2949     intros j Hj; apply rho_commutes_beta_total; [lia | apply brt_step;
       exact Hs]
2950   | apply IHQ; exact Hs ]
2951 end.
2952
2953 - (* t_lam s A M' *)
2954 inversion Hstep; subst; clear Hstep.
2955
2956 + (* beta_lam_A *)
2957 match goal with
2958 | Hs : A ->b ?A1' |- _ =>
2959   assert (HdegAeq : deg A = deg A1') by (apply deg_beta; apply brt_step;
     exact Hs);
2960   assert (HAle : deg A <= n + 1) by (apply deg_le_top);
2961   simpl; rewrite <- HdegAeq;
2962   apply bt_app_congr_R;
2963   [ apply brt_lam_M; apply gamma_lam_tel_dom_congr;
2964     intros mtier Hmtier; apply rho_commutes_beta_total; [lia | apply
       brt_step; exact Hs]
2965   | apply IHA; exact Hs ]
2966 end.
2967
2968 + (* beta_lam_M *)
2969 match goal with
2970 | Hs : M' ->b ?M1' |- _ =>
2971   simpl; apply bt_app_l; apply bt_lam_M; apply
     gamma_lam_tel_body_congr_plus;
2972   apply IHM'; exact Hs
2973 end.
2974
2975 - (* t_pi s A B *)
2976 inversion Hstep; subst; clear Hstep.
2977
2978 + (* beta_pi_A *)
2979 match goal with
2980 | Hs : A ->b ?A1' |- _ =>
2981   assert (HdegAeq : deg A = deg A1') by (apply deg_beta; apply brt_step;
     exact Hs);
2982   assert (HAle : deg A <= n + 1) by (apply deg_le_top);

```

```

2983     simpl; rewrite <- HdegAeq;
2984     apply bt_app_congr_L;
2985     [ apply bt_app_congr_R;
2986       [ apply brt_app_r; apply gamma_pi_tel_dom_congr;
2987         intros mtier Hmtier; apply rho_commutes_beta_total; [lia | apply
2988           brt_step; exact Hs]
2989         | apply IHA; exact Hs ]
2990     | apply gamma_lam_tel_dom_congr;
2991     intros mtier Hmtier; apply rho_commutes_beta_total; [lia | apply
2992       brt_step; exact Hs] ]
2993   end.
2994
2995 + (* beta_pi_B *)
2996   match goal with
2997   | Hs : B ->b ?B1' |- _ =>
2998     simpl; apply bt_app_r; apply gamma_lam_tel_body_congr_plus; apply IHB;
2999     exact Hs
3000   end.
3001 Qed.
3002
3003 Lemma gamma_commutes_beta :
3004   forall M N, M ->>b N -> gamma M ->>b gamma N.
3005 Proof.
3006   intros M N Hbeta.
3007   induction Hbeta as [M | M N Hstep | M N P Hstep1 IH1 Hstep2 IH2].
3008   - apply brt_refl.
3009   - apply brt_from_trans. apply (gamma_commutes_beta_aux M N Hstep).
3010   - apply brt_trans with (gamma N); auto.
3011 Qed.
3012
3013 (* ===== *)
3014 (** Main Theorem: S is strongly normalizing iff S* is *)
3015
3016 Lemma bt_forward_acc : forall X Y, X ->>+b Y ->
3017   (forall Z, X ->b Z -> Acc (fun N M => M ->>+b N) Z) ->
3018   Acc (fun N M => M ->>+b N) Y.
3019 Proof.
3020   intros X Y HXY.
3021   induction HXY as [X Y Hstep | X Z Y H1 IH1 H2 IH2]; intros Hbase.
3022   - apply Hbase. exact Hstep.
3023   - apply IH2.
3024     intros Z0 HstepZ.
3025     destruct (IH1 Hbase) as [f].
3026     apply f. apply bt_step. exact HstepZ.
3027 Qed.
3028
3029 Lemma Acc_atomic_implies_Acc_plus : forall X,
3030   strongly_normalizing X -> Acc (fun N M => M ->>+b N) X.
3031 Proof.
3032   intros X HX.

```



```

3030 induction HX as [X _ IH].
3031 constructor.
3032 intros Y HXY.
3033 exact (bt_forward_acc X Y HXY IH).
3034 Qed.
3035
3036 Lemma sn_reflection_core :
3037   forall b, Acc (fun N M => M ->>+b N) b ->
3038   forall M, b = gamma M -> strongly_normalizing M.
3039 Proof.
3040   intros b Hacc.
3041   induction Hacc as [b _ IH].
3042   intros M Heq.
3043   constructor.
3044   intros M' HstepM.
3045   apply (IH (gamma M')).
3046   - rewrite Heq. apply gamma_commutes_beta_aux. exact HstepM.
3047   - reflexivity.
3048 Qed.
3049
3050 Lemma sn_reflection : forall M,
3051   strongly_normalizing (gamma M) -> strongly_normalizing M.
3052 Proof.
3053   intros M Hsn.
3054   exact (sn_reflection_core (gamma M) (Acc_atomic_implies_Acc_plus (gamma M)
3055     Hsn) M eq_refl).
3056 Qed.
3057
3058 Lemma gamma_commutes_typing_derivable : forall M,
3059   derivable M -> derivable_star (gamma M).
3060 Proof.
3061   intros M [Γ [N HMN]].
3062   exists ([ (zero_var, (sort_of 2, t_sort (sort_of 1)));
3063     (bullet_var, (sort_of 1, t_fvar (sort_of 2) zero_var));
3064     (prod_var, (sort_of 1, T_prod))] ++ rho_ctx 0 Γ), (rho 0 N).
3065   exact (gamma_commutes_typing Γ M N HMN).
3066 Qed.
3067
3068 Axiom derivable_star_implies_derivable : forall M,
3069   derivable_star M -> derivable M.
3070
3071 Definition system_strongly_normalizing (P : term -> Prop) : Prop :=
3072   forall M, P M -> strongly_normalizing M.
3073
3074 Theorem lambdaS_SN_iff_lambdaS_star_SN :
3075   system_strongly_normalizing derivable <-> system_strongly_normalizing
3076   derivable_star.
3077 Proof.
3078   split.

```

```

3078 - (* S SN -> S* SN *)
3079   intros HSN M HMstar.
3080   apply HSN.
3081   apply derivable_star_implies_derivable.
3082   exact HMstar.
3083
3084 - (* S* SN -> S SN *)
3085   intros HSNstar M HM.
3086   apply sn_reflection.
3087   apply HSNstar.
3088   apply gamma_commutes_typing_derivable.
3089   exact HM.
3090 Qed.
3091
3092 End TPTS.

```

Abstract

In this thesis, the dependency-erasure translation introduced by Nathan Mull for Tiered Pure Type Systems [10] is fully formalized and mechanically verified in the proof assistant Coq. The translation functions, together with all intermediate lemmas—including the substitution lemma, preservation of reduction paths, and lemmas related to injectivity and image—are proved in a fully machine-checked manner. The main theorem of Mull, stating that weak normalization implies strong normalization for all tiered pure type systems with specific rules, is mechanically verified for the first time.

As a result, an open and extensible library is developed, providing a solid foundation for extending this translation to more expressive dependent systems such as the Calculus of Constructions. The results of this work constitute a significant step toward the proof of the BGK conjecture¹ [7] in dependent type systems.

Keywords: type theory, tiered pure type systems, dependency-erasure translation, strong normalization, mechanized proof, Coq, BGK conjecture

¹Barendregt–Geuvers–Klop



University of Tehran
College of Science
School of Mathematics, Statistics, and Computer Science

Formalization of the Dependency-Eliminating Translation in Tiered Pure Type Systems

By
Hossein Madadi

Supervisor
Majid Alizadeh

A thesis submitted to graduate studies office in partial fulfillment of the
requirements for the degree of master of science in Computer Science

Summer 2026